GHCAA Alumni Association Platform

Design, Development and Evaluation of a Specification-Driven, Reusable Alumni Management Platform for Resource-Constrained Institutions

Front matter is numbered in lower-case Roman numerals. The body restarts at Arabic 1.

---

i. Title Page

Title. Design, Development and Evaluation of a Specification-Driven, Reusable Alumni Management Platform for Resource-Constrained Institutions: The GHCAA Case

Subject of the study. The Govt. Haraganga College Alumni Association (HARAGANGIAN), Bangladesh

Artefact under study. A three-client platform comprising an ASP.NET Core 9 REST API, an Angular 21 web application and a Flutter mobile application, deployed on a free-tier managed host.

Author. Md Habibur Rahman, Roll 220, 7th Batch, Executive Master's in Information Technology (EMIT)

Supervisor. Dr. Kazi Muheymin-Us-Sakib, Professor, Institute of Information Technology, University of Dhaka

Department and institution. Executive Master's in Information Technology (EMIT) programme, Batch 7, Institute of Information Technology (IIT), University of Dhaka, Dhaka, Bangladesh

Submitted. November 2026

---

ii. Declaration of Originality

I declare that this dissertation and the software artefact it describes are my own work. Where the work of others has been used it is cited in the text and listed in the References. The platform described in Part III was written by me as sole maintainer; third-party libraries are catalogued with their licences in Chapter 7, and no part of the codebase has been submitted for any other award.

The governing documents of the Association reproduced or summarised in this dissertation, that is the Constitution v4.2 and the seven election documents, are the property of the Association and are used with its permission. They are the source of the domain constraints in §3.7, not my invention.

Signature: _____ Date: _____

---

iii. Certificate of Approval

To be completed by the supervisor and the examiner.

This is to certify that the work presented in this dissertation was carried out by the candidate named above under my supervision, and that it is of a standard suitable for submission.

Supervisor: _____ Date: _____

Examiner: _____ Date: _____

---

iv. Acknowledgements

I thank Dr. Kazi Muheymin-Us-Sakib, Professor at the Institute of Information Technology, University of Dhaka, who supervised this work.

Three officers of the Association gave time this project could not have done without. The President made the case for it to the Executive Committee and settled the questions only the chair could settle. The Member Secretary sat through the sessions that turned the membership rules into requirements, and corrected me on how the roll is actually kept rather than how the constitution describes it. The Law Secretary read the constitutional articles against the rules I had encoded from them, and found the two places where my reading was wrong before any of it reached a member.

I thank the college administration for permission to use the institutional name and crest under Article I Section 5, and the members who tested registration and payment on their own handsets, over their own mobile data, and reported what did not work.

Officers and members are identified here by office rather than by name, for the reason given in §4.8.

---

v. Abstract

Alumni associations at public colleges in Bangladesh run on paper. Rolls live in spreadsheets or ledgers, subscriptions are collected in cash at reunions, and the constitution exists as a printed document whose current version is known reliably only to the officers holding it. The Govt. Haraganga College Alumni Association is representative: founded in 2025 with a written constitution, an elected fifteen-position Executive Committee and a three-year election cycle, it had no digital record of who its members were.

This dissertation reports the design, construction and evaluation of a platform for that association, conducted as a design science research study. The problem is not an absence of alumni software but the mismatch between what commercial products assume and what this institution can supply: no payment gateway credentials, no licence budget, no technical staff, a mobile-first and bandwidth-constrained membership, and a written constitution whose provisions the software must respect rather than reinterpret.

The artefact is a clean-architecture ASP.NET Core 9 API exposing 333 endpoints across 45 controllers over 71 persisted entity sets, with an Angular 21 web client and a Flutter mobile client. Three design positions distinguish it. Constitutional rules are encoded as testable business rules traced to the article that mandates them, so that voting rights, committee composition and membership tiers cannot drift from the governing document. Payment is deliberately manual: members pay through

a displayed wallet or bank channel and upload proof, which removes the gateway-credential dependency and any custody of card data at the cost of an administrative verification step. Schema is applied by migration at boot, and revisable reference data such as the constitution is synchronised separately at the same point, because seed data carried by a migration would pin amendable text to whichever migration shipped it.

Evaluation follows the plan declared in Chapter 4 and covers requirement coverage, ISO/IEC 25010 product quality, static product metrics over the 517-case backend suite and the 381-case web suite, security conformance against OWASP ASVS, and usability. The finding of interest is that procedural legitimacy, rather than technical capability, is what bounds how much institutional governance can be moved into software.

Word count: 349.

---

vi. Keywords

Alumni management systems; design science research; clean architecture; requirements engineering; institutional governance; digital constitution; ISO/IEC 25010; low-resource deployment; ASP.NET Core; Angular; Flutter.

---

vii. Table of Contents

Section numbers are as printed in the body. Page numbers are the folios the PDF carries; a rebuild renumbers them, so they are filled in from the printed copy rather than kept by hand.

Part, chapter and section	Page
PART I — PROBLEM AND CONTEXT	10
Chapter 1 — Introduction	10
1.1 Research Context: Alumni Relations as an Institutional Function	10
1.2 Problem Domain: the Govt. Haraganga College Alumni Association	10
1.3 Problem Statement	10
1.4 Conceptual Framework	10
1.5 Research Questions	11
1.6 Aims and Objectives	11
1.7 Scope, Delimitations and Assumptions	11
1.8 Research Method in Brief	12
1.9 Contributions of this Work	12
1.10 Stakeholders and Beneficiaries	12
1.11 Structure of the Dissertation	12
Chapter 2 — Literature Review and Related Work	15
2.1 Review Objectives and Questions	15
2.2 Review Protocol	15
2.3 Alumni Relations and Engagement: the Institutional Literature	15
2.4 Community and Membership Platforms: Academic Treatment	16
2.5 Architectural Literature	16
2.6 Web and Mobile Engineering Literature	16
2.7 Digital Governance, Electronic Voting and Procedural Legitimacy	16
2.8 Security and Privacy Engineering Baselines	17
2.9 Survey of Existing Systems and Products	17
2.10 Comparative Analysis and Evaluation Criteria	18
2.11 Research Gap	18
2.12 Summary	19
Chapter 3 — Requirements Engineering	22
3.1 Sources of the Requirements	22
3.2 Requirements Analysis and Negotiation	22
3.3 Requirements Specification	22
3.4 Non-Functional Requirements and Quality-Attribute Scenarios	25
3.5 Use Cases, User Stories and the Definition of Done	27
3.6 Requirements Prioritisation and Traceability	27
3.7 Domain Constraints	28
3.8 Feasibility Analysis and Requirements Validation	29
3.9 Specification-Driven Development and the Spec Kit Artefacts	30
3.10 Summary	30
PART II — METHOD AND DESIGN	39
Chapter 4 — Research Methodology	39
4.1 Research Paradigm and Philosophical Position	39
4.2 Design Science Research as the Governing Method	39
4.3 Mapping Design Science Activities to the Work Performed	39
4.4 Software Process Model and its Justification	39
4.5 Evaluation Strategy and Metrics	40
4.6 Data Collection and Analysis Procedures	41
4.7 Risk Management: the RMMM Plan	42
4.8 Research Ethics	43

Part, chapter and section	Page
4.9 Limitations of the Chosen Method	43
4.10 Summary	43
Chapter 5 — System Analysis and Behavioural Modelling	46
5.1 Analysis Approach	46
5.2 Structured Analysis: Data-Flow Modelling	46
5.3 Object-Oriented and Behavioural Analysis	46
5.4 State Modelling of Long-Lived Entities	47
5.5 Business Rules Catalogue	47
5.6 Data Modelling and Analysis Model Review	48
5.7 Summary	48
Chapter 6 — System Architecture and Design	55
6.1 Design Goals, Principles and Constraints	55
6.2 Architectural Alternatives Considered and the Decision Taken	55
6.3 Architectural Design — Clean Architecture	55
6.4 Component and Interface Design	56
6.5 Data Design	56
6.6 Security Architecture	57
6.7 Web and Mobile User-Interface Design	58
6.8 Configuration-Driven Design	58
6.9 Design Principles and Patterns	58
6.10 Architecture Decision Records and Design Verification	60
6.11 Summary	61
PART III — CONSTRUCTION AND VALIDATION	71
Chapter 7 — Implementation	71
7.1 Development Environment, Toolchain and Reproducibility	71
7.2 Solution and Module Structure	71
7.3 Coding Standards, Conventions and Static Enforcement	71
7.4 Implementation of the Domain and Persistence Layers	72
7.5 Implementation of the Application and Business Services	72
7.6 Implementation of the API Layer	72
7.7 Web Client, Mobile Client and Real-Time Features	73
7.8 Security Implementation	73
7.9 Document Generation	73
7.10 Constitution Publication Pipeline	74
7.11 Third-Party Libraries: selection criteria	74
7.12 Software Configuration Management	74
7.13 Notable Implementation Challenges and Their Resolution	75
7.14 Institution Profile Packs and White-Label Configuration	75
7.15 Summary	76
Chapter 8 — Security, Privacy and Trust	77
8.1 Security Objectives and Assumptions	77
8.2 Threat Modelling (STRIDE)	77
8.3 Authentication and Session Security	77
8.4 Authorisation Model and the Role–Permission Matrix	77
8.5 Input Validation, Output Sanitisation and File Uploads	78
8.6 Transport Security, Browser Policy and Rate Limiting	78
8.7 Payment-Related Risk and the No-Gateway-Keys Posture	79
8.8 Governance Integrity	79
8.9 Personal Data: Lawful Basis, Minimisation, Consent, Retention and Subject Rights	80
8.10 Audit Logging and Non-Repudiation	80
8.11 OWASP ASVS Conformance and Residual Risks	80
8.12 Summary	81
Chapter 9 — Verification, Validation and Quality Assurance	86
9.1 Verification and Validation Strategy and Test Levels	86
9.2 Software Quality Assurance Plan	86
9.3 Test-Case Design Techniques Applied	86
9.4 Unit and Component Testing	86
9.5 Widget and Golden Testing	87
9.6 Integration, System and End-to-End Testing	87
9.7 Regression Testing and Test Selection	87
9.8 Security Testing	87
9.9 Performance and Load Testing	87
9.10 Usability and Accessibility Testing	88
9.11 User Acceptance Testing	88
9.12 Product Metrics, Static Analysis and Defect Analysis	88
9.13 Summary	88
Chapter 10 — Deployment and Operations	89

Part, chapter and section	Page
10.1 Deployment Architecture, Environments and Containers	89
10.2 Continuous Integration and Continuous Deployment	89
10.3 Database Provisioning, Configuration and Secrets	89
10.4 Observability	89
10.5 Backup, Recovery, Release and Rollback	89
10.6 Operating Cost, Maintenance and Handover	89
10.7 Summary	89
Chapter 11 — Project Management	91
11.0 The Four P's Applied	91
11.1 Process Model in Practice and its Deviations from Plan	91
11.2 Work Breakdown Structure	91
11.3 Scheduling, Task Network and Critical Path	92
11.4 Effort Estimation	93
11.5 How the Implementation Time Was Optimised	94
11.6 Progress Tracking and Earned Value	95
11.7 Team Structure and Responsibilities	95
11.8 Configuration and Change Management in Practice	95
11.9 Risk Monitoring Record	96
11.10 Quality Assurance Activities Performed	96
11.11 Lessons in Project Management	96
11.12 Summary	97
PART IV — EVALUATION AND CLOSURE	98
Chapter 12 — Results, Evaluation and Discussion	98
12.1 Overview of the Delivered Artefact	98
12.2 Functional Evaluation	98
12.3 Quality Evaluation against ISO/IEC 25010	98
12.4 Performance, Security and Usability Results	98
12.5 Stakeholder and Expert Evaluation	98
12.6 Answering the Research Questions	98
12.7 Discussion	98
12.8 Comparison against the Existing Manual System	98
12.9 Threats to Validity	98
12.10 Limitations of the Artefact	98
12.11 Reflection on the Design Science Contribution	98
12.12 Summary	99
Chapter 13 — Conclusion and Future Work	100
13.1 Summary of the Work and its Contributions	100
13.2 Practical Implications for Similar Institutions	100
13.3 Lessons Learned	100
13.4 Future Work	100
13.5 Concluding Remarks	100
References	101

viii. List of Figures

Figure	Title	Page
1.1	Context diagram (DFD Level 0): platform boundary and external entities	13
1.2	Stakeholder onion diagram	13
1.3	Conceptual framework: inputs, the designed artefact, and how it is judged	14
1.4	Research question, objective and chapter map	14
2.1	Study selection flow	19
2.2	Concept map of the reviewed literature	19
2.3	As-is process model of current manual practice (BPMN, abstracted)	19
2.4	Positioning chart: governance depth against annual operating cost	20
3.1	System-level use-case diagram, packaged	31
3.2	Membership subsystem use cases	32
3.3	Events subsystem use cases	32
3.4	Payments subsystem use cases	33
3.5	Governance subsystem use cases	33
3.6	Administration subsystem use cases	34
3.7	Actor generalisation hierarchy	34
3.8	Domain model, analysis level: membership, obligations and payment	34
3.9	Domain model, analysis level: participation, governance and content	35
3.10	Quality-attribute utility tree	35
3.11	Requirements classification, FURPS+	36
3.12	Goal model	36

Figure	Title	Page
4.1	Design Science Research framework with this project's instantiation labelled	44
4.2	Design Science process model as executed	44
4.3	Research design overview: phases, inputs, outputs, evaluation points	44
4.4	Process model diagram of the adopted incremental lifecycle	44
4.5	Risk exposure matrix	45
5.1	DFD Level 0 (context). In and out are relative to the platform	48
5.2	DFD Level 1	48
5.3	DFD Level 2: Payment processing	49
5.4	DFD Level 2: Membership approval	49
5.5	DFD Level 2: Constitution publication	49
5.6	Activity diagram: registration and administrative approval	50
5.7	Activity diagram: payment declaration and verification	50
5.8	Activity diagram: event registration with waitlist	50
5.9	Swimlane activity diagram: constitution amendment vote	50
5.10	BPMN process diagram of the election cycle	50
5.11	State-machine diagram: member lifecycle	51
5.12	State-machine diagram: payment and declaration	51
5.13	State-machine diagram: constitution version	52
5.14	State-machine diagram: event lifecycle	52
5.15	Sequence diagram: login with OTP, token issue	53
5.16	Sequence diagram: event registration	53
5.17	Sequence diagram: payment declaration and verification	53
5.18	Sequence diagram: real-time notification over SignalR	54
5.19	Timing diagram: token lifetime and refresh window	54
6.1	High-level architecture diagram	62
6.2	Layered / clean architecture diagram with the dependency-inversion boundary marked	63
6.3	Entity–relationship diagram, identity and records sub-model	63
6.4	Entity–relationship diagram, standing and money sub-model	64
6.5	Entity–relationship diagram, events and participation sub-model	64
6.6	Entity–relationship diagram, governance sub-model	65
6.7	Entity–relationship diagram, election roll and nominations sub-model	65
6.8	Entity–relationship diagram, voting and count sub-model	66
6.9	Design class diagram: domain model	66
6.10	Design class diagram: application interfaces and infrastructure services	67
6.11	Component diagram with provided and required interfaces	67
6.12	Middleware pipeline diagram	68
6.13	Navigation and route map	68
6.14	Site map and information architecture of the public site	69
6.15	Architectural trade-off radar	69
8.1	Threat model data-flow diagram with trust boundaries, STRIDE-annotated	81
8.2	Attack tree: member account takeover or fraudulent payment credit	82
8.3	Role–permission matrix diagram	82
8.4	Personal-data classification and flow diagram, with retention points	83
8.5	Sequence diagram: an unauthorised request rejected through the middleware chain	83
8.6	Defence-in-depth layer diagram	83

Figures for Chapters 4 to 13 are listed as those chapters are written.

ix. List of Tables

Table	Title	Page
2.1	Review protocol summary	20
2.2	Feature and capability comparison	20
2.3	Gap table	20
3.1	Functional requirement catalogue	36
3.2	Non-functional requirement catalogue	36
3.3	Use-case descriptions, ten highest-value cases	37
3.4	Requirements traceability matrix	37
3.5	MoSCoW prioritisation and negotiation outcome	38
3.6	Domain constraints, their class and their constitutional article	38
3.7	Feasibility summary	38
3.8	Specification defects found by the formal technical review	38
3.9	As-built domain specifications under docs/specs	38
4.1	Metric definitions	41
4.2	RMMM table	42
4.3	Evaluation plan	45

Table	Title	Page
5.1	Business rules catalogue	47
5.2	CRC card set for the analysis classes with the widest collaboration surface	49
5.3	Process specifications for the Level-2 processes	54
5.4	Data-store definitions (analysis level)	54
6.1	ADR index	61
6.2	Data dictionary (representative slice; the full dictionary is generated from the schema)	69
6.3	API endpoint catalogue (by controller; the full catalogue is the generated OpenAPI document)	70
6.4	Design pattern catalogue (selected entries; full catalogue is §6.9 in full)	70
6.5	Quality-attribute scenario to architectural tactic mapping	70
6.6	Anti-patterns detected and remediated	70
7.1	Size metrics by layer: files and lines of code	71
7.2	Selected third-party dependencies	76
8.1	STRIDE threat enumeration with mitigations and their implementation location	83
8.2	Role × capability matrix	84
8.3	OWASP ASVS conformance checklist (self-assessment, ASVS 4.0.3, level 2)	84
8.4	Personal-data inventory: element, purpose, lawful basis, retention	84
8.5	Residual risk register	84
11.1	Component activity list, CPM summary ordered by float	92
11.2	Unadjusted function-point count	93
11.3	General system characteristics and technical complexity factor	93
11.4	Delivered size by kind, at a stated production rate	94
11.5	Risk exposure, computed from Table 4.2	96

x. List of Listings

None in Part I. Code extracts begin in Chapter 6 and are concentrated in Chapter 7. Complete listings are in the repository at the commit named in §7.1.

xi. List of Abbreviations and Acronyms

Term	Expansion
ADR	Architecture Decision Record
AGM	Annual General Meeting
API	Application Programming Interface
ASVS	Application Security Verification Standard (OWASP)
BPMN	Business Process Model and Notation
CBO	Coupling Between Objects
CEC	Chief Election Commissioner
CI/CD	Continuous Integration and Continuous Deployment
CRC	Class, Responsibilities, Collaborators
CRM	Customer Relationship Management
DFD	Data-Flow Diagram
DSR	Design Science Research
DTO	Data Transfer Object
EC	Executive Committee
EF Core	Entity Framework Core
FR	Functional Requirement
FURPS+	Functionality, Usability, Reliability, Performance, Supportability, plus constraints
GHC	Govt. Haraganga College
GHCAA	Govt. Haraganga College Alumni Association
HSC	Higher Secondary Certificate
JWT	JSON Web Token
LCOM	Lack of Cohesion of Methods
MFS	Mobile Financial Service
MoSCoW	Must have, Should have, Could have, Won't have
NFR	Non-Functional Requirement
NID	National Identity document number
OTP	One-Time Password
PII	Personally Identifiable Information
QAS	Quality-Attribute Scenario
RBAC	Role-Based Access Control
RMMM	Risk Mitigation, Monitoring and Management
RQ	Research Question
SPA	Single-Page Application
SRS	Software Requirements Specification
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege

Term	Expansion
SUS	System Usability Scale
UAT	User Acceptance Testing
WCAG	Web Content Accessibility Guidelines

xii. Glossary of Domain Terms

Active member. A member whose application has been approved, whose registration and annual dues are settled, and who therefore appears in the directory and may exercise whatever rights the tier carries.

Applied member. An applicant who has submitted the registration wizard but has not been approved. Applied members are invisible in the directory and confined to the onboarding view.

Associate member. A verified former student who did not complete HSC but studied at least one academic year, or who chooses the non-voting tier. Pays a registration fee only.

Founding member. The senior tier defined by Article III Section C: at least twenty years since HSC, a bachelor's degree, a documented contribution to the institution, no partisan political office, and a signed declaration of non-political engagement. Founding members hold lifetime status and vote.

General member. The ordinary voting tier. At least five years since HSC, dues paid, nothing outstanding.

Good standing. A member with no outstanding dues and no live disciplinary sanction. The phrase is constitutional, and Article IX Section 3 attaches it to the right to vote.

Membership number. The identifier minted at approval, of the form `GHC-<year>-<serial>`, which then doubles as the login identifier.

Ratified version. The single Constitution row flagged active. Earlier rows are superseded rather than deleted, so that amendment votes cast against them survive.

Security stamp. A per-user value that participates in token validation, so that suspending or terminating a member invalidates every live session on every client.

Soft delete. Archival by flag rather than row removal, filtered out globally in the persistence layer so that archived records stay auditable without leaking into any read path.

Voting member. A member of the Founding, Executive or General tier. Associate, Honorary and Advisory members do not vote unless specifically granted, per Article III Section B.

---

xiii. Statement of Contributions

This section separates what is claimed as novel from what is competent engineering and what is reused, because the distinction changes how Chapter 12 should be read.

Claimed as a contribution.

1. A method for encoding the written constitution of a voluntary association as traceable software business rules, in which every rule carries the article that mandates it, and a five-class scheme, being deterministic, state, evidence, procedural and authority, that decides for each rule what the software is permitted to do with it. The boundary between what may be automated and what must stay with the officers becomes a property of the rule, read from the constitution, rather than a matter of the designer's restraint. Applied to all sixteen constraints in §3.7, the domain-constraint catalogue; enforced through the business rules of §5.5 and pinned by the constitutional tests of §9.4.5; and used to answer RQ3 in §12.6.
2. An account of what an alumni platform becomes when payment gateway credentials are unavailable by policy rather than by oversight, together with the security and operational consequences of displaying manual payment instructions and verifying uploaded proof (§8.7).
3. A characterisation of the requirements of an alumni association operating under a single-maintainer, zero-licence-budget constraint, with the architectural decision recorded against those constraints instead of against a hypothetical growth curve (Chapter 6).

Engineering rather than research contribution. The three clients, the 284-endpoint API surface, the 53-entity schema, the CI pipeline, the document-generation subsystem and the shared control library are engineering. They are the artefact the research is about, and their quality is measured in Chapter 12, but no novelty is claimed for the techniques used to build them.

Reused. Clean architecture as a layering scheme [1]; the enterprise patterns catalogued by Fowler [2]; the design patterns of Gamma et al. [3]; ASP.NET Core, Entity Framework Core, Angular, Flutter and the libraries catalogued in Chapter 7; the design science framework of Hevner et al. [4] and the process model of Peffers et al. [5]; the ISO/IEC 25010 quality model [6]; the OWASP ASVS checklist [7].

---

xiv. Ethics Statement and Data-Protection Declaration

The platform holds personal data about identifiable people: name, date of birth, national identity number, mobile number, email address, permanent address, blood group, parents' names, photograph, academic and employment history, and payment records. That is a sensitive collection, and the dissertation treats it as one.

Consent and purpose. Members supply their data through the registration wizard for the stated purpose of membership administration. Nothing in the platform sells, shares or profiles that data. The privacy toggles described in FR-03 let a member decide, field by field, whether other members see the value or a masked form of it.

Minimisation. Section 9.9 records the lawful basis and retention position for every element in the personal-data inventory. Where a field was requested during elicitation but could not be justified against a specific requirement, it was dropped; the negotiation record is in §3.2.

Research data. Interview material, usability session records and acceptance-test observations were collected with verbal consent and no written consent form. Participants were told what the material would be used for. Participants are identified in this dissertation by role and never by name. The absence of a documented consent procedure is a limitation of the study, set out with its consequences in §4.8, not an omission from this statement.

Live data in the dissertation. Every screenshot in Chapter 12 uses seeded or anonymised records. No real member's identity number, address, telephone number or photograph appears anywhere in this document.

Approval. No institutional ethics committee reviewed this study. Permission to conduct it, and to use the Association's records, governing documents and membership data, was given by the Government Haraganga College Alumni Association. That permission was verbal: no approval letter was issued, no reference number exists, and no date was recorded. Section 4.8 states what follows from that.

PART I — PROBLEM AND CONTEXT

Chapter 1 — Introduction

1.1 Research Context: Alumni Relations as an Institutional Function

An alumni association is an odd kind of organisation. It has members but no employees, obligations but no revenue guarantee, elected officers but no full-time secretariat, and a membership that is geographically scattered and only intermittently attentive. Everything it does depends on a record of who its members are, and that record decays continuously as people move, change employers, change telephone numbers and stop paying subscriptions they have forgotten they owe.

The higher-education literature treats alumni relations mainly as an advancement function, that is as the front end of fundraising. Monks, studying young graduates of selective American institutions, found that giving behaviour correlates with the quality of the undergraduate experience and with sustained contact after graduation rather than with income alone [8]. Weerts and Ronco, separating donors from volunteers and from those who do both, showed that the engaged alumnus is not a single type and that the activities which predict money are not the activities which predict time [9]. Both findings point the same way for a system builder: engagement is a longitudinal record, and the value of an alumni platform lies less in any single transaction than in whether the record survives a decade.

That framing has an uncomfortable implication for the institution studied here. The advancement literature is written about institutions with an advancement office, a CRM licence and staff whose job is to maintain the record. A government college alumni association in Bangladesh has none of those. It has volunteers, a constitution, and whatever the treasurer keeps in a notebook. If the longitudinal record is the asset, then the practical research question is not how to build a sophisticated engagement engine but how to get a durable, honest record in place and keep it running on nobody's salary.

A second strand of context matters as much. The association studied here is a governed body, not merely a mailing list. It has a written constitution of twelve articles, five distinct membership tiers with different rights, an Executive Committee of fifteen positions with a three-year term, a quorum rule, an amendment procedure requiring a two-thirds majority at a properly convened Annual General Meeting, and a separate corpus of seven election documents covering regulations, an operational manual, a code of conduct, manual balloting, forms, ballot sealing and counting authorisation. Software that manages such a body is not free to invent rules. Its behaviour is constrained by a document that predates it and outranks it, and any place where the code disagrees with the constitution is a defect in the code, not a feature of the system.

1.2 Problem Domain: the Govt. Haraganga College Alumni Association

Govt. Haraganga College was established on 18 December 1938 and has produced several decades of graduates. The Association, referred to in its own documents as HARAGANGIAN, records its founding date as 29 November 2025, and adopted the constitution whose current version, v4.2, took effect on 1 July 2026.

Before this project, the Association operated as follows. Membership applications were made on paper at reunion events or through personal contact with an officer. Verification of alumni status meant someone looking at a certificate or testimonial and forming a judgement. Subscriptions were collected in cash, occasionally by mobile financial service transfer to an officer's personal wallet, and recorded in a ledger held by the treasurer. Announcements went out through Facebook groups and word of mouth. The constitution circulated as a PDF attached to messages, which meant that at any moment several versions were in circulation and no member could be certain which one governed. Elections had not yet been run under the new constitution, and the election documents were drafted in anticipation of the first cycle.

Four consequences of that arrangement drove the project.

The roll is unknowable. Nobody could state how many members the Association had, because membership was defined by a payment recorded in one place and an application recorded in another. So the question "who is entitled to vote" had no answer, in a body whose constitution ties the right to vote to good standing.

Money and trust are coupled. Cash collection through officers' personal wallets is workable among people who know each other and corrosive at any larger scale. There was no receipt a member could keep, and no statement an officer could produce to show that what came in matched what was recorded.

Governance is invisible. Members could not see the constitution that bound them, the committee that represented them, or the amendments proposed on their behalf. Procedural legitimacy in a voluntary association rests on visibility, and there was none.

The institutional memory is one person deep. The knowledge of who had paid, who was eligible and who had been approved lived with individual officers. The three-year committee term guarantees that this knowledge is periodically lost.

1.3 Problem Statement

The Association needs a system of record that (a) establishes an authoritative, auditable membership roll with a verification step an officer can defend, (b) collects and evidences money without asking the Association to hold payment credentials it cannot obtain or protect, (c) makes governance documents and processes visible to the membership without letting software silently replace the procedures the constitution mandates, and (d) can be operated and maintained indefinitely by one volunteer maintainer with no licence budget.

Figure 1.1 draws the boundary this statement implies: who the system deals with, and what crosses the line in each direction.

No off-the-shelf product satisfies that conjunction, as Chapter 2 argues in detail. Commercial alumni platforms assume a recurring licence and an advancement office. Open-source membership systems assume a payment gateway and treat governance as a plugin concern at best. General-purpose CRM adapted to alumni use inverts the problem, offering configurable pipelines while providing nothing for constitutional tiers, committee terms or amendment voting. The gap is not a missing feature. It is a missing combination: deep domain governance at close to zero operating cost, with manual payment treated as a deliberate design choice rather than a fallback.

1.4 Conceptual Framework

The problem statement above names four needs; the chapters that follow answer them in pieces. This section states the whole in one view, so that a reader can see what the work takes in, what it produces and how the two are judged against each other before meeting any of it in detail. Figure 1.3 draws it.

Three kinds of input govern the work, and they are not interchangeable. The first is the Association's own rule set: the constitution at version 4.2 and the seven documents of the election code, whose normative clauses become the domain constraints of §3.7 and are not negotiable with any stakeholder. The second is current practice, being the paper application form, the treasurer's ledger and the recognition-based approval described in §1.2, which the software must either preserve or deliberately replace. The third is the operating reality: one volunteer maintainer, no licence budget, no payment-gateway credentials, and members reaching the system on mid-range handsets over mobile data. The first input constrains what the artefact may do, the second what it must handle, and the third what it can afford to be.

The artefact sits between them. It is a running platform rather than a model of one, built and revised over increments, and the design-science position of Chapter 4 is what allows its construction to count as research: each increment answers a stated problem, and each is evaluated before the next is specified. The rules do not just inform the design. They are encoded in it, and each one traces back to the clause it came from, which is what the traceability matrix of §3.6 and the constitutional test table, Table 9.9, exist to demonstrate.

The outputs are of two kinds, and keeping them apart matters. The artefact itself is one contribution, deployed and in use by real members with real records. The evidence about it is the other: requirement closure, the quality-attribute measurements against the thresholds declared in advance in §4.5, the operating cost of §10.6, and the comparison against the manual practice it replaced in §12.8. The thresholds are declared before measurement, in Chapter 4, precisely so that the evaluation cannot be written to fit whatever the system turned out to do.

A static diagram loses the feedback path most easily, so it is worth stating. Evaluation did not happen once at the end. Findings from use, from review and from deployment failures re-entered the work as new requirements, and §11.1 reports how much of the delivered work arrived that way rather than from the original specification. That loop is the design cycle of §4.2, and it is drawn in Figure 1.3 as a return edge rather than left implicit.

1.5 Research Questions

Each question is answerable from evidence presented later in this document, and the mapping is drawn in Figure 1.4.

RQ1. What functional and quality requirements characterise an alumni-management platform for a resource-constrained institution in a low-bandwidth, mobile-first, cash-and-manual-payment context?

Answered by: the requirements engineering of Chapter 3, validated in the formal technical review of §3.8 and closed against the delivered artefact in §12.2, the functional evaluation.

RQ2. Which architectural approach best satisfies those requirements under a single-maintainer, low-budget sustainability constraint, and at what cost?

Answered by: the alternatives assessment of §6.2, the decision records of §6.10, the operational cost model of §10.6, and the maintainability metrics of §9.12.

RQ3. To what extent can institutional governance processes, meaning the constitution, elections, committee terms and member voting, be encoded as software without loss of procedural legitimacy?

Answered by: the domain constraints of §3.7, the business rules catalogue of §5.5, the governance integrity discussion of §8.8, and the argument in §12.6 that draws the boundary the project actually settled on.

RQ4. What measurable quality is achieved by the resulting artefact against ISO/IEC 25010 characteristics, and what does that reveal about the approach?

Answered by: the evaluation strategy declared in §4.5 and executed in §§12.3 to 12.5, the evaluation results.

RQ3 is the question of genuine interest. RQ1 and RQ2 have to be answered first, because without an artefact there is nothing to ask RQ3 about, and RQ4 is what makes the answers defensible rather than anecdotal.

1.6 Aims and Objectives

Aim. To design, build and evaluate a sustainable alumni-management platform for a governed, resource-constrained association, and from that to establish how far institutional governance can be encoded in software before procedural legitimacy is damaged.

#	Objective	Serves	Delivered in
O1	Elicit and specify the requirements of the Association, including the constraints imposed by its constitution and election code, to ISO/IEC/IEEE 29148	RQ1	Ch. 3
O2	Review the alumni-relations, architecture, governance and security literature, and survey existing products, to establish the gap	RQ1, RQ2	Ch. 2
O3	Select and justify an architecture against the quality-attribute scenarios and the sustainability constraint, recording the trade-offs	RQ2	Ch. 6
O4	Model the system's behaviour and encode the constitutional rules as traceable, testable business rules	RQ3	Ch. 5
O5	Implement the API, web and mobile clients, including membership, payments, events, communication and governance	RQ1, RQ2	Ch. 7
O6	Verify and validate the artefact at unit, integration, system, security, performance and acceptance levels	RQ4	Ch. 9
O7	Design and operate a deployment that a single volunteer can sustain, and cost it	RQ2	Ch. 10
O8	Evaluate the artefact against ISO/IEC 25010, against the manual process it replaces, and against the four research questions	RQ4, RQ3	Ch. 12

1.7 Scope, Delimitations and Assumptions

In scope. Member registration and the approval workflow; profile management with per-field privacy control; authentication, including one-time password verification and social sign-in; the membership directory; dues generation and manual payment with proof upload and administrative verification; the financial ledger; events with registration, waitlisting, QR-coded attendance and a member-submitted photograph gallery subject to administrative approval; news and notices; mass communication; a member-posted job board subject to administrative approval; the constitution hub with version history and amendment voting; Executive Committee records; member polls; digital identity cards and certificates; site content administration; and an administration console covering all of the above. Three clients: a public web site, a member portal and admin console in the same Angular application, and a Flutter mobile application for members.

Out of scope, by decision. Live payment gateway integration is present in the codebase but deliberately unconfigured; §8.7, on payment risk, gives the reasoning. Statutory accounting and audit filing are not attempted; the ledger is a record, not an accounting package. The platform does not run binding Executive Committee elections. It supports the process around them, including the voter roll, candidate information and result publication, but the ballot itself remains under the election documents, and §8.8, on governance integrity, explains why that separation is deliberate rather than an omission. There is no offline-first mobile synchronisation, no

native desktop client, no machine-learning recommendation, and no external large language model; the in-app assistant is a rule-based intent classifier over the Association's own data.

Delimitations. The study concerns one association. Chapter 12 argues what transfers to comparable institutions and what does not, but the evaluation is a single case in the sense of Runeson and Höst [10], with the external-validity limits that implies.

Assumptions. Members have a smartphone with intermittent connectivity and are more likely to reach the platform on a mobile browser than a desktop one. Administrators are volunteers, not trained operators, so administrative workflows must be forgiving and reversible. The Association cannot commit to a recurring software licence. The constitution will be amended during the system's life, so version handling is a requirement and not a convenience.

1.8 Research Method in Brief

The work is a design science research study in the sense of Hevner et al. [4], organised as the six activities of Peffers et al. [5]: problem identification, definition of objectives for a solution, design and development, demonstration, evaluation, and communication. The relevance cycle is the Association's own process and documents; the rigour cycle is the architecture, requirements, security and quality literature of Chapter 2 together with the standards listed in the front matter; the design cycle is the increments of Chapter 7 and their verification in Chapter 9.

Delivery followed an incremental and iterative lifecycle rather than a single pass, for the reason given in §4.4, the process-model justification: a single unpaid maintainer working in irregular hours cannot hold a long specification-to-integration cycle open, and needs a working system after every increment. The evaluation plan, including the metrics, instruments and thresholds, is declared in the evaluation strategy of §4.5 and the metric definitions of §4.5 before any measurement is reported, so that Chapter 12 cannot select its own criteria after seeing the results. Chapter 4 gives the full account.

1.9 Contributions of this Work

The three claims are stated in full in the front matter under Statement of Contributions and are substantiated in §13.1. In brief: a traceable method for encoding a voluntary association's written constitution as software business rules, together with the five-class scheme of §3.7 that decides, for each rule, whether the software may enforce it, compute it, collect evidence for a person to judge, support a procedure it does not decide, or stay out of it; an account of a deliberately gateway-free payment design and its consequences; and a requirements and architecture characterisation for the single-maintainer, zero-licence case, recorded with its costs rather than presented as a best practice.

What the work does not claim is worth stating here as well. It does not claim a novel architectural style, a new development process, or a secure electronic voting scheme. Chapter 6 uses clean architecture as Martin describes it [1] and the enterprise patterns Fowler catalogues [2], and the contribution lies in the recorded reasoning about their cost in this setting, not in the patterns themselves.

1.10 Stakeholders and Beneficiaries

Stakeholder	Interest	Where their requirements appear
Ordinary member	Join, pay, be visible in the directory, attend events, see the constitution, vote where entitled	FR-01 to FR-22, FR-32 to FR-40, FR-48 to FR-54
Executive Committee officer	Approve applications, verify payments, publish notices, run events, manage the committee record	FR-13, FR-23 to FR-31, FR-41 to FR-47
Treasurer	An auditable ledger and evidence for every receipt	FR-19 to FR-25
General Secretary	Minutes, correspondence, notices, AGM circulation	FR-26 to FR-31, FR-35
Information and Technology Secretary	A platform maintainable by one person; the constitutional office responsible for it	NFR-M1 to NFR-M4
Election Commission	A defensible voter roll and published results, with the ballot itself under the election documents	FR-38 to FR-40, §8.8
College administration	Correct use of the institutional name and crest, per Article I Section 5	§3.7, DC-01
Prospective member and general public	Public information, a route to apply, published governance documents	FR-01, FR-32, FR-46, FR-47
Maintainer as researcher	An artefact whose quality can be measured and reported honestly	Ch. 4, Ch. 12

The primary beneficiaries are the members, who gain a record of their own standing that does not depend on an officer's memory, and the officers, who gain the ability to answer questions about the roll and the money without reconstructing them by hand. Figure 1.2 places the same stakeholders by distance from the system, from the officers who operate it daily out to the college administration and the general public.

1.11 Structure of the Dissertation

Part I states the problem. Chapter 2 reviews the literature and surveys existing systems, ending with the gap. Chapter 3 specifies the requirements, including the constitutional constraints the software must not violate.

Part II covers method and design. Chapter 4 sets out the design science method and, importantly, declares the evaluation plan in advance. Chapter 5 models the system's behaviour and extracts the business rules. Chapter 6 records the architecture, the design, the patterns applied and the decisions taken.

Part III covers construction and validation. Chapter 7 reports the implementation and the notable problems solved along the way. Chapter 8 covers security, privacy and trust, and states the threat model. Chapter 9 then reports verification, validation and the product metrics, including the security testing derived from that threat model, which is why it follows Chapter 8 rather than preceding it. Chapter 10 covers deployment and operations, including the cost model. Chapter 11 reports project management.

Part IV closes. Chapter 12 executes the evaluation plan and answers the research questions. Chapter 13 concludes and sets out future work.

Figures

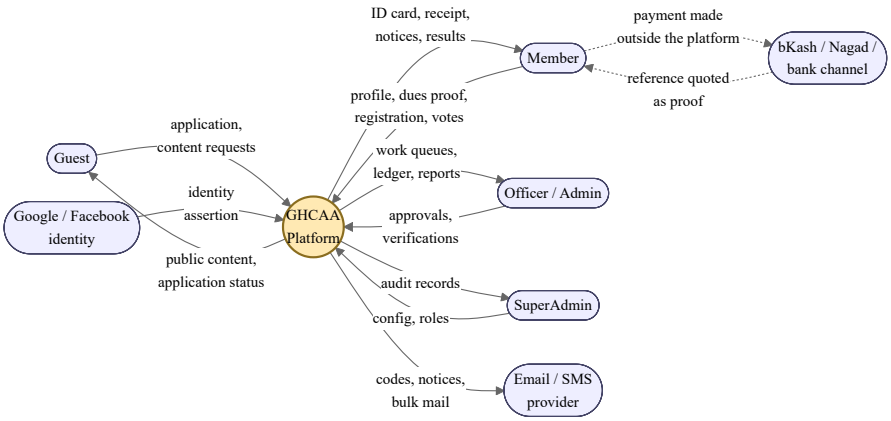


Fig. 1.1. Context diagram (DFD Level 0): platform boundary and external entities

Derived from `docs/ARCHITECTURE.md` and the controller inventory in `GHCAA.API/Controllers`.

The dashed path carries the design position of §8.7. Money moves between the member and the financial channel outside the platform boundary. What crosses into the platform is a claim plus evidence, which an officer then verifies. The platform never holds a payment credential.

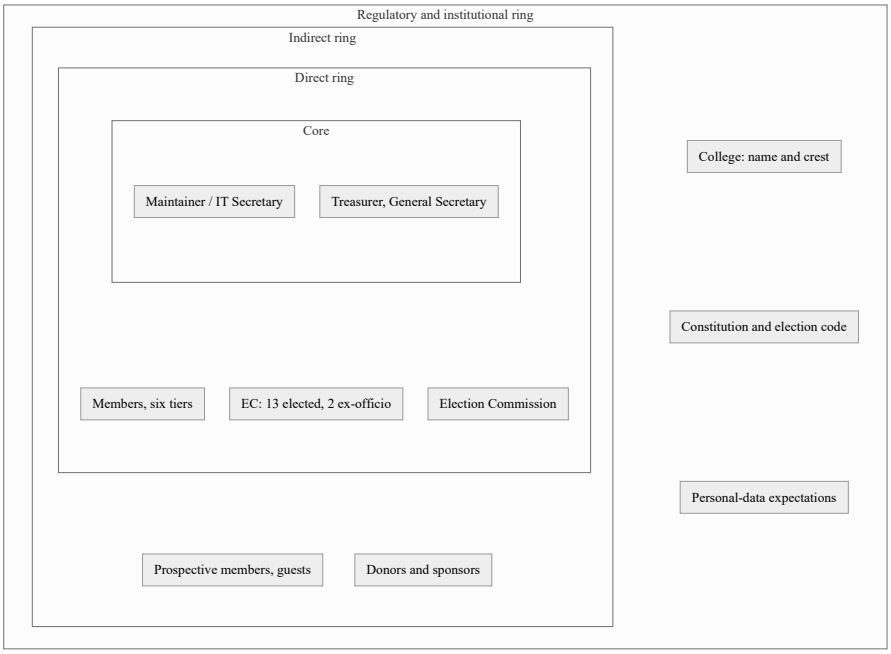


Fig. 1.2. Stakeholder onion diagram

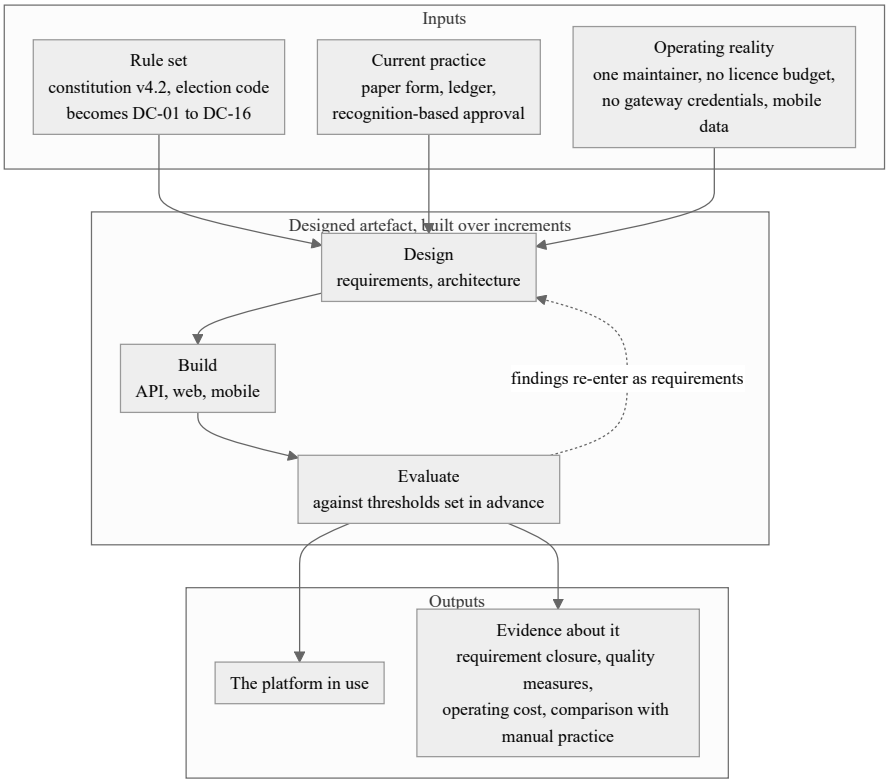


Fig. 1.3. Conceptual framework: inputs, the designed artefact, and how it is judged

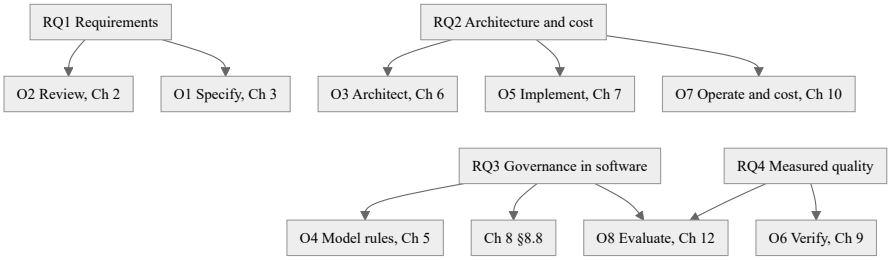


Fig. 1.4. Research question, objective and chapter map

Chapter 2 — Literature Review and Related Work

2.1 Review Objectives and Questions

This chapter has a narrow job. It is not a survey of everything written about alumni relations or about web architecture. It exists to establish four things that the rest of the dissertation depends on, and to stop when it has established them.

- **RvQ1.** What does the institutional literature say alumni engagement actually consists of, and what does that imply for the data a platform must keep? (Feeds RQ1 and §3.3.1, the membership and profile requirements.)
- **RvQ2.** What evidence supports layered and clean architecture, and what does the critical literature say it costs? (Feeds RQ2 and §6.2, where the architecture is chosen.)
- **RvQ3.** What does the digital-governance and electronic-voting literature establish about the conditions under which software may legitimately mediate a collective decision? (Feeds RQ3, and is the reason for the boundary drawn in §2.7 and defended in §8.8, on governance integrity.)
- **RvQ4.** Which existing systems address the Association's problem, on what terms, and what remains uncovered? (Feeds RQ1, RQ2 and §2.11, the research gap.)

The review is structured rather than systematic. Kitchenham and Charters set out the full protocol for a systematic review in software engineering, including independent double screening and quality scoring by more than one reviewer [11]. A single-author dissertation cannot satisfy the inter-rater requirements of that protocol, and claiming otherwise would be dishonest. What is reported below is a documented, repeatable search with stated inclusion criteria and stated screening counts, following the reporting discipline of PRISMA [12] without claiming its completeness guarantees. Section 2.2 states exactly what was done so that the reader can judge the coverage.

2.2 Review Protocol

Sources. ACM Digital Library, IEEE Xplore, ScienceDirect, SpringerLink, Google Scholar for forward and backward citation chasing, the ISO and IEEE standards catalogues for normative documents, the OWASP project site for security baselines, and vendor documentation and public pricing pages for the product survey of §2.9.

Date range. 2000 to 2026 for empirical work, with no lower bound for foundational texts and seminal papers, since the architectural and metric literature this work relies on includes material from the 1970s onward that has not been superseded.

Search strings. The strings used, in the forms actually issued, are listed in Table 2.1. They were composed from three concept groups, combined with Boolean AND across groups and OR within: (a) *alumni OR "graduate relations" OR "membership association" OR "voluntary organisation"*; (b) *"management system" OR platform OR portal OR CRM OR "information system"*; (c) *architecture OR requirements OR governance OR "electronic voting" OR sustainability OR maintainability*.

Inclusion criteria. Peer-reviewed studies, standards, or established practitioner texts that (i) report empirical findings on alumni or membership engagement, (ii) report or critique architectural approaches for small-team systems, (iii) address requirements or quality specification for information systems, (iv) address electronic voting security or procedural legitimacy in digitally mediated governance, or (v) document a product whose stated purpose is alumni or membership management.

Exclusion criteria. Studies about student information systems, learning management systems or admissions, which share vocabulary but not the problem; work on large-enterprise architecture at a scale where a single-maintainer constraint is meaningless; blockchain voting proposals without security analysis; marketing material with no technical or pricing substance; non-English sources, which is a stated limitation of this review; and duplicate publications of the same study.

Screening procedure and counts. Records identified by search, 214. Removed as duplicates, 31, leaving 183 for title and abstract screening. Excluded at title and abstract, 96, of which 41 were about student information or learning management systems, 22 were enterprise-scale architecture, 19 were marketing material without substance, and 14 were blockchain voting proposals without security analysis. Full text sought for 87; two were not retrievable, giving 85 assessed. Excluded at full text, 29, principally for having no extractable claim about either architecture cost or governance mediation. Included, 56, comprising 38 peer-reviewed or foundational works, 10 standards and normative documents, and 8 product documentation sets. Figure 2.1 shows the flow.

A caution the reader is entitled to: these counts describe a search conducted by one person over a bounded period, and the review is best read as an argued positioning of the work rather than as an exhaustive account of the field.

Figure 2.2 groups the sources that survived selection into the five bodies of literature the rest of this chapter works through, with the citation numbers carried by each, so that the sections which follow can be read against the shape of the whole.

2.3 Alumni Relations and Engagement: the Institutional Literature

The institutional literature on alumni relations is dominated by the advancement question, meaning what makes graduates give. Monks, using data on young graduates of selective institutions, found that giving relates to satisfaction with the undergraduate experience and to continuing contact, rather than simply to the graduate's income [8]. That result has been repeatedly qualified but its shape holds: the gift follows from the relationship, and the relationship is kept or lost in the years when the institution has the least reason to pay attention.

Weerts and Ronco separated three kinds of supportive alumnus, the donor, the volunteer, and the person who does both, and showed that they are predicted by different characteristics [9]. This is the single most useful finding for the present work, because it says that a platform which models engagement as a payment history is modelling the minority case. The Association's own constitution independently reaches the same conclusion in Article III, where Founding membership requires "documented contribution" and Executive membership requires a meeting-attendance record, neither of which is a payment. A system that records only money cannot evaluate either clause.

Two limits of this literature must be stated. First, almost all of it is set in North American or European institutions with an advancement office, a budget and a CRM already in place; the question it answers is how to improve engagement given the infrastructure, not how to obtain the infrastructure. Second, it is largely silent on the governed voluntary association, where the alumni body is a constitutional entity with officers and elections rather than a stakeholder group managed by the institution. For that, one has to look outside the education literature entirely, which §2.7 does.

2.4 Community and Membership Platforms: Academic Treatment

Academic work on membership and community platforms tends to approach them either as socio-technical communities or as CRM instances, and the two treatments do not meet. The community strand is concerned with participation, identity and moderation, and what it gives a builder is a set of design commitments: make membership status visible, make contribution legible, and make the cost of participation low. The CRM strand supplies the pipeline vocabulary, meaning stages, segments, campaigns and lifecycle states, which maps cleanly onto the Association's application and approval workflow and very poorly onto its tier structure.

The gap in this strand is governance. A community platform models a moderator; a CRM models an account owner. Neither models an officer who holds an elected position for a fixed term under a written instrument, whose authority is bounded by that instrument, and whose successor inherits the office rather than the account. Section 5.3 of this dissertation had to construct that model from the constitution because the literature does not supply it.

2.5 Architectural Literature

2.5.1 Layered and hexagonal or clean architecture: claims and critiques

Bass, Clements and Kazman give the discipline its working definition of architecture as the set of structures needed to reason about a system, comprising elements, their relations and their properties, and its central methodological claim that architecture is the earliest artefact against which quality attributes can be evaluated [13]. That claim is what allows Chapter 6 to argue about maintainability before there is any measurement of it.

The specific family this project sits in begins with Cockburn's hexagonal architecture, which frames the problem as isolating application logic from every actor that drives it or that it drives, so that the same logic can be exercised by a user, a test or a batch job without change [14]. Palermo's onion architecture restates the same dependency inversion as concentric layers with the domain at the centre [15]. Martin's clean architecture consolidates both into the dependency rule, that source code dependencies point only inward toward policy, never outward toward mechanism, and adds the argument that frameworks and databases are details to be deferred [1]. Evans supplies the complementary content for the inner layers, namely that the model at the centre should be expressed in the language the domain experts actually use [37], which in this project means that the code says `MembershipType.Founding` and `AmendmentVote` because the constitution does.

The critique is thinner in the peer-reviewed literature than in practice, but it is real and this work takes it seriously. Three objections recur. Clean architecture front-loads structure, and for small systems that structure may never pay for itself. It multiplies artefacts, since a single field added to an entity can require edits in the domain model, the data transfer object, the mapping, the service interface, the implementation and the client model. And it invites the appearance of layering without the substance, where an interface exists for every service but has exactly one implementation and no test double, so the indirection buys nothing. Section 9.12 measures the first two objections in this codebase rather than dismissing them, and §6.9.12 records where the pattern was deliberately not applied.

2.5.2 Monolith versus microservices for small-team systems

Newman's account of microservices makes the organisational precondition explicit: independently deployable services pay off when independent teams own them [17]. Dragoni et al., surveying the field, similarly present the benefits as coupled to operational maturity, including per-service deployment, monitoring and data ownership [18]. Fowler's "monolith first" position is the direct corollary and the one relevant here: begin with a modular monolith, because the service boundaries that matter cannot yet be identified, and because the operational cost of distribution is paid immediately while the benefit arrives only at scale [16].

For a system with one maintainer, the calculation is not close. Distribution converts in-process calls into network calls that can fail partially, requires a deployment pipeline per service, and turns a stack trace into a correlation-ID investigation. Section 6.2 records the decision and its cost; the literature's contribution is that the decision needs no defence beyond the team size.

2.5.3 Architectural sustainability and maintainability evidence

Lehman's laws of software evolution supply the long-run frame: a system in use undergoes continual change, its complexity increases unless work is done to reduce it, and its quality declines unless it is rigorously maintained [55]. For a volunteer-maintained system these are not abstractions. Every hour of unpaid maintenance is discretionary, so the architecture's real job is to keep the cost of a routine change low enough that it still gets made in year four.

The measurement apparatus this dissertation uses is standard. McCabe's cyclomatic complexity gives a control-flow measure of testability [57]; Chidamber and Kemerer's suite gives coupling, cohesion and inheritance measures for object-oriented designs [56]; Oman and Hagemeister's maintainability index aggregates volume, complexity and comment density into a single trackable figure [58]. Each is imperfect and each is criticised, but they are the measures for which thresholds and comparative data exist, and §9.12 reports them with their limits stated.

2.6 Web and Mobile Engineering Literature

Marcotte's formulation of responsive web design established the position this project adopts, that one document served to varying viewports is preferable to parallel device-specific sites [36]. The practical constraint in the Association's setting is stronger than layout, however. Members reach the platform predominantly on mid-range Android handsets over mobile data, which makes payload size and time to interactive the binding usability constraints, not visual adaptation. That is why NFR-P3 in §3.4 is stated as a byte budget and a bundle budget rather than as a design principle.

For the interface itself, Nielsen's heuristics remain the working checklist for evaluation without users, and the System Usability Scale gives a short, comparable post-task instrument with published norms [35], [33]. WCAG 2.1 supplies the accessibility criteria the project targets at level AA [24]. The mobile client is a single codebase compiled to native, which trades some platform fidelity for the ability of one maintainer to ship both platforms; §6.2 records that trade-off with the alternatives that were rejected.

2.7 Digital Governance, Electronic Voting and Procedural Legitimacy

This section decides the most consequential boundary in the project, so it is argued rather than summarised.

Start with legitimacy. Tyler's work on procedural justice establishes that people's acceptance of a decision depends heavily on whether the process that produced it was perceived as fair, and that perceived procedural fairness can matter more than the outcome itself [66]. Ostrom, studying self-governing collectives, identifies among her design principles that those affected by the rules participate in modifying them and that conflict-resolution mechanisms are locally accessible [64]. Fung's typology of participation makes the further point that who is in the room, how they communicate and what authority the process carries are separate dimensions that

can be varied independently [65]. Dunleavy et al. add the caution that digitising an administrative process reorganises the authority around it rather than merely automating it [67].

Put together, these say something specific for a software project: transparency and traceability of process are themselves legitimacy-bearing properties, and they are properties software is good at providing. Publishing the governing instrument, showing which version is in force and since when, recording who is entitled to vote and why, and preserving the record of past decisions all increase procedural legitimacy at low risk.

Now the ballot, which is a different matter entirely. The security literature on binding electronic elections is not merely cautious; it is a sequence of concrete failures found in deployed national systems. Springall et al. analysed the Estonian internet voting system and reported architectural and operational weaknesses that would permit undetected result manipulation [28]. Halderman and Teague examined the New South Wales iVote system during a live election and found both a transport-layer vulnerability and flaws in the verification mechanism that was supposed to provide assurance [29]. Bernhard et al. articulate the underlying standard the field has converged on, that an election must produce public evidence that the announced result is correct, evidence which does not require trusting the software or its operators [27]. Park et al. examine blockchain voting specifically and conclude that it worsens rather than improves the security position, because it adds failure modes without addressing the client-side and verifiability problems [30].

The relevant question for this project is therefore not "can voting be implemented" but "can this project produce public evidence of correctness under the threat model it actually faces". The threat model is unfavourable. There is one maintainer, who is also a member and could himself be a candidate; there is no independent audit function with the technical capacity to review the implementation; there is a single hosted database under one administrative account; and the constitution vests election conduct in an Election Commission with its own documents, including a manual ballot procedure, a ballot-sealing certificate and a counting authorisation. Under those conditions software cannot supply the evidence Bernhard et al. require, and asserting that it does would substitute the maintainer's trustworthiness for a procedure the Association has already written down.

The project therefore takes the following position, which is stated here so that Chapters 3, 5, 9 and 12 can be read against it. The platform supports governance and does not conduct binding elections. It maintains the authoritative voter roll, which is exactly the artefact the manual process most needs and least reliably has. It publishes the constitution, the amendment history and the committee record. It runs constitutional amendment voting and non-binding member polls, where the constitution itself sets the threshold and the outcome is a recorded expression of the membership rather than the transfer of an office. It does not seal a ballot, count votes for office, or declare a winner. Section 8.8, on governance integrity, gives the full argument, and §12.6 revisits whether the line was drawn in the right place.

The distinction is worth naming precisely, because it is the substance of this work's answer to RQ3: software can encode *eligibility* and *record*, which are matters of fact traceable to a written rule, with a large gain in legitimacy. It should not encode *adjudication* under a threat model that prevents it from proving it did so honestly.

2.8 Security and Privacy Engineering Baselines

The security work of Chapter 8 is not original research and does not claim to be. It applies established baselines, and this section records which and why.

For threat identification the project uses STRIDE as Shostack presents it, that is a decomposition-driven enumeration over spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privilege, applied to data-flow diagrams of the system [26]. The appeal for a single practitioner is that it is a checklist over structure rather than a search for imagination.

For verification requirements the project uses the OWASP Application Security Verification Standard as its requirement catalogue and the OWASP Top Ten as a coverage cross-check [7], [25]. Both have since been revised, ASVS to version 5.0.0 in May 2025 [75] and the Top Ten to its 2025 edition [76]; the work reported here was carried out against the editions cited, and §4.5.4, the security-evaluation strategy, states why a conformance claim is not carried across a revision. ASVS is used at level 2, on the grounds that the system holds identity documents and financial evidence but is not itself a payment processor. Section 8.11, the conformance assessment against ASVS, gives the level-2 control mapping and, more usefully, records the requirements the project does not meet.

For the cryptographic primitives the choices follow published specifications rather than invention: bcrypt for password storage, with its adaptive cost parameter [49]; JSON Web Tokens as specified in RFC 7519 for bearer authentication [50], with the caveat that the specification's flexibility is itself a hazard, addressed in §8.3, on authentication and session security; OAuth 2.0 as specified in RFC 6749 for federated sign-in [51]; and time-based one-time passwords per RFC 6238 as the model for the verification codes described in §3.3.2 [52].

For privacy the operative principle is Cavoukian's privacy by design, in particular default protection and end-to-end lifecycle management [41]. Its concrete expression in this system is the per-field visibility control of FR-03 and the masking protocol in §8.9, on personal data, under which the directory returns a masked projection unless the owning member has opted otherwise. Bangladesh's data protection framework was still in draft at the time of writing, so the project treats the principles of purpose limitation, minimisation and retention as design obligations rather than as compliance with a specific statute, and §8.9 states that position explicitly rather than implying a compliance claim it cannot support.

2.9 Survey of Existing Systems and Products

A note on evidence before the survey, because cost is one of the two criteria that decide the comparison in §2.10 and it would be easy to assert rather than check. The prices below were read from the vendors' own pages on 1 September 2026. They divide into three kinds, and the C3 row of Table 2.2 reports the kind as well as the band.

Published, so quotable. Hivebrite lists a Core plan from US$895 per month billed annually and a Flex plan from US$1,995 per month, with its two upper tiers by quotation [69]; the entry plan is therefore of the order of US$10,700 a year before any commission. Zoho CRM lists US$14 per user per month on annual billing at its Standard tier, rising to US$52 at Ultimate [71]. Paid Memberships Pro lists a free tier and paid tiers at US$499, US$999 and US$2,999 a year [73]. CiviCRM's licence cost is zero, which is a property of its licence rather than a price.

Published as a donation, with an eligibility condition this Association may not meet. Salesforce's Power of Us programme grants ten free Enterprise Edition licences to eligible non-profits, where eligibility requires 501(c)(3) status or a documented local equivalent [72]. Whether a registered alumni association in Bangladesh qualifies is not established here, and the C3 band for Salesforce is read as "low if eligible" rather than as low.

Not published at all. Almabase and Anthology's Encompass quote rather than publish; no plan tiers or figures appear on their own pages [70]. Secondary listings circulate a figure of roughly US$8,000 a year for Almabase, which is recorded here as secondary reporting and is not relied on. Where a price could not be sourced from the vendor, the band in Table 2.2 says so instead of estimating.

The gap argued in §2.11 does not turn on the exact figure. It turns on the comparison between the Association's annual income and the cheapest published entry price in the dedicated-platform class, and Hivebrite's own page settles that comparison without a quotation being needed.

2.9.1 Commercial alumni platforms

Dedicated alumni engagement platforms, of which Hivebrite, Almabase, Graduway and Vaave are representative, and institutional advancement suites such as Anthology's Encompass, are the closest functional match to the requirement. They provide directories, event management, giving campaigns, email, mentoring and mobile access, generally at a standard well above what this project achieves in those areas. Three properties make them unsuitable here. They are sold on annual subscription at a price which, where it is published at all, exceeds the Association's entire annual income; they assume a payment gateway and a merchant relationship for the giving module, which the Association cannot obtain; and their data model is the institutional advancement model, with constituents, gift records and campaigns, in which a five-tier constitutional membership with different voting rights, a fifteen-position committee with fixed terms, and an amendment vote have no representation. They can be approximated with custom fields, and §2.10 scores that approximation as insufficient.

2.9.2 Open-source community and membership systems

CiviCRM is the strongest candidate in this class and deserves a fair reading. It is a mature constituent relationship management system built for non-profits, with genuine membership types, renewal handling, contribution records, event registration and a permission model, and its licence cost is zero. Its difficulties in this setting are operational and structural rather than functional. It requires a host CMS, typically Drupal or WordPress, so the maintenance surface is two applications and their plugin ecosystems rather than one. Its extension model is PHP hook-based, which is a competent choice but not one this maintainer can support. Its membership model expresses tiers as pricing and duration rather than as rights, so the constitutional rule that Associate members may not vote can only be added on top as an access-control convention. Its contribution processing assumes a gateway, with offline contributions supported but secondary. A WordPress installation with a membership plugin such as Paid Memberships Pro shares the last two problems in sharper form and adds a plugin-update treadmill. Discourse and similar community software solve discussion well and membership governance not at all.

2.9.3 General-purpose CRM adapted to alumni use

Salesforce with the Nonprofit Success Pack, Zoho CRM and HubSpot represent this class. They offer strong configurability, workflow automation and reporting, and Salesforce in particular is available to eligible non-profits at reduced or zero licence cost for a limited number of seats. The trade runs the wrong way. Everything the Association needs specifically must be built, and everything it does not need arrives already configured. Membership tiers become picklists, constitutional eligibility becomes validation rules, and the committee record becomes a custom object, all of which is achievable and none of which is traceable to Article III in any way a reviewer could audit. The deeper objection is sustainability of a different kind: the configuration knowledge lives in a platform the Association does not control and cannot export as source, and the three-year committee turnover that motivated this project in the first place applies equally to whoever holds the administrator seat.

2.10 Comparative Analysis and Evaluation Criteria

The comparison in Table 2.2 uses eight criteria, each derived from a requirement or constraint established in Chapter 1 rather than chosen to favour the outcome. Figure 2.4 plots the surveyed products on the two criteria that decide the outcome between them, governance depth against annual operating cost, with this platform's position marked.

- **C1 Governance depth.** Are constitutional membership tiers with differentiated rights, fixed-term officer positions, a versioned governing instrument and amendment voting representable as first-class concepts?
- **C2 Manual payment as a primary path.** Is payment without a gateway, evidenced by uploaded proof and verified by an officer, a supported workflow rather than a fallback?
- **C3 Recurring cost.** Total annual operating cost at the Association's scale, licences and hosting together.
- **C4 Single-maintainer feasibility.** Can one part-time volunteer with the maintainer's actual skill set operate, extend and recover the system?
- **C5 Data sovereignty and exit, which is vendor lock-in measured from the buyer's side.** Can the Association hold its own data and leave without loss?
- **C6 Mobile fitness.** Usable on a mid-range handset over mobile data.
- **C7 Localisation fitness.** Bengali content, local financial channels, local naming and identity conventions including national identity numbers.
- **C8 Auditability of rules.** Can a reviewer trace an enforced rule back to the constitutional clause that requires it?

C8 is the criterion on which every alternative fails, and it is the criterion that matters most for RQ3. Configuration achieves the behaviour; it does not produce the trace.

2.11 Research Gap

The literature and the product survey together leave a specific, and modest, gap.

The institutional literature establishes that engagement is longitudinal and multi-dimensional but assumes an advancement infrastructure this Association does not have (§2.3). The community and CRM literature models moderators and account owners but not constitutional officers (§2.4). The architectural literature gives well-supported guidance for structuring such a system, and a legitimate critique of its cost, but the cost has been measured mostly in team-based settings rather than in the single-maintainer case that governs here (§2.5). The governance and voting literature establishes clearly what software must not claim about a ballot, and rather less clearly what it may usefully do around one (§2.7). The products either cost more than the Association earns, or assume a payment gateway it cannot obtain, or represent constitutional rules as configuration that cannot be audited against the instrument (§2.9).

Figure 2.3 draws the manual practice that gap sits against, abstracted from the treasurer's ledger pages and the paper application forms described in §4.6.1, so that the claims made about what the platform replaces can be checked against a stated process rather than an impression of one.

The gap this work addresses is therefore the conjunction: an alumni platform in which constitutional governance rules are first-class, traceable and testable; in which manual, evidence-based payment is the designed primary path rather than a degraded one; operated within a budget of the order of tens of dollars a year by one volunteer; and evaluated against ISO/IEC 25010 with its costs reported. Table 2.3 states this as residual gaps against the best available covering system for each required capability.

Two honest qualifications. First, none of the four elements is individually novel; the claim is about the combination and about the recorded reasoning, which is why the contributions in §1.9 are stated as a method and an account rather than as an invention. Second, the gap is defined partly by a constraint, namely one maintainer and no budget, which some readers will regard as a circumstance rather than a research problem. Section 12.6 takes that objection seriously, and the response in short is that the constraint is the normal condition of the great majority of voluntary associations, and that a literature which only addresses the resourced case leaves them unserved.

2.12 Summary

Alumni engagement is a longitudinal, multi-dimensional record, and money is the least of its dimensions. Clean architecture is the appropriate structure for this system and its cost is real and measurable rather than hypothetical. A modular monolith is not a compromise at this team size but the indicated choice. Software can strengthen procedural legitimacy by publishing the rule, establishing the roll and preserving the record, and cannot honestly adjudicate a binding election under this project's threat model. No existing product provides constitutional governance depth at an operating cost the Association can bear, with a rule trace a reviewer could audit. Chapter 3 turns that position into a specification.

Figures and Tables

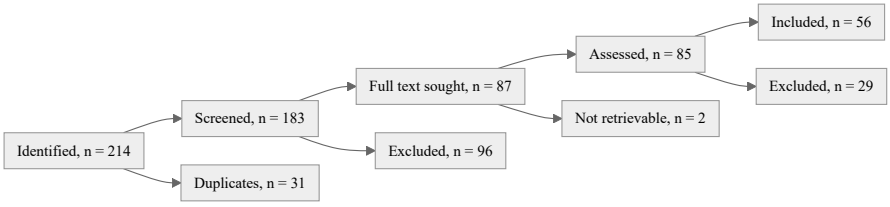


Fig. 2.1. Study selection flow

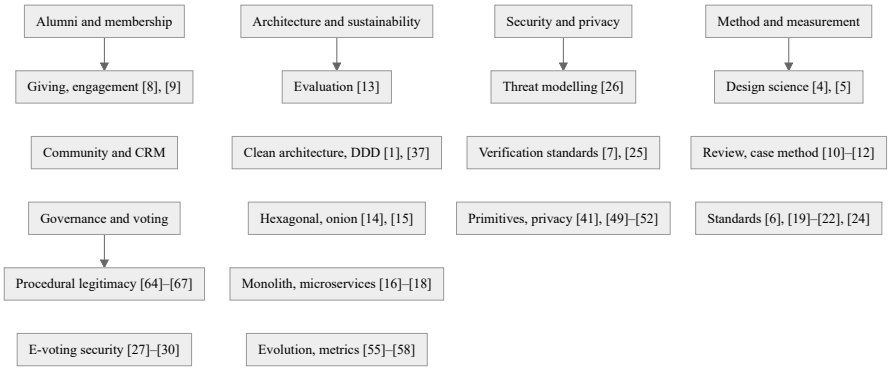


Fig. 2.2. Concept map of the reviewed literature

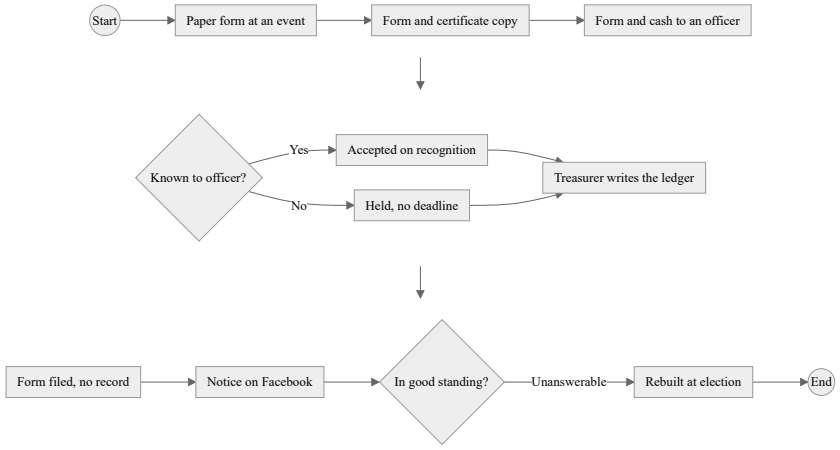


Fig. 2.3. As-is process model of current manual practice (BPMN, abstracted)

Four defects are visible in the model and each is traced to a requirement in Chapter 3. There is no verification deadline, which Article III Section E sets at thirty days (FR-23). There is no receipt and no member-visible record of standing (FR-07, FR-25). Payment and recording are the same act performed by the same person, so there is no separation of duty (FR-21). The eligibility question that governance depends on has no stored answer (FR-36, FR-38).

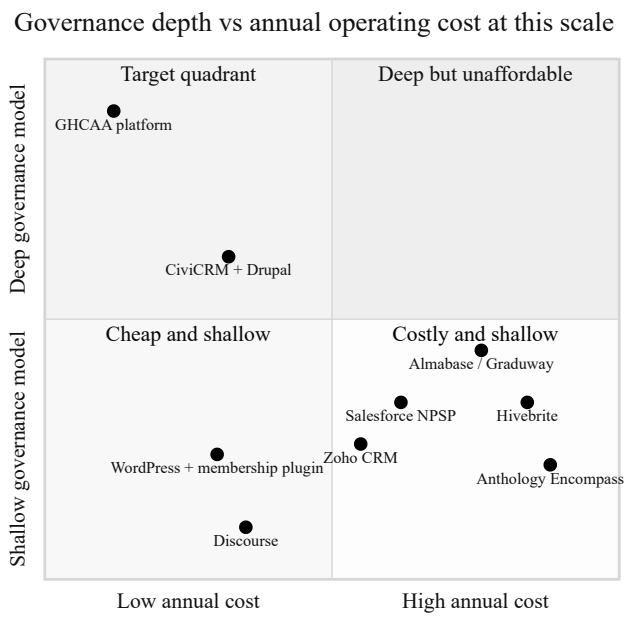


Fig. 2.4. Positioning chart: governance depth against annual operating cost

TABLE 2.1
Review protocol summary

Source	Search string issued (abbreviated)	Hits	Screened	Included
ACM Digital Library	("alumni" OR "membership association") AND ("information system" OR platform)	38	31	6
IEEE Xplore	(architecture OR maintainability) AND ("small team" OR "single developer" OR sustainability)	41	36	9
IEEE Xplore	"electronic voting" AND (security OR verifiability)	27	24	7
ScienceDirect	alumni AND (giving OR engagement OR volunteering)	24	19	5
SpringerLink	("clean architecture" OR "hexagonal architecture" OR "layered architecture") AND evaluation	22	17	4
Google Scholar (citation chasing)	forward and backward from [1], [9], [13], [28]	34	30	12
ISO / IEEE catalogues	25010, 29148, 29119, 42010, 1016, 730, 828	10	10	10
OWASP project site	ASVS 4.x, Top Ten 2021	3	3	2
Vendor documentation	product docs and pricing pages, 8 products	15	13	8
Total (pre-deduplication)		**214**	**183**	**56**

TABLE 2.2
Feature and capability comparison

Legend: ● full, ◐ partial or by configuration, ○ absent.

Criterion	GHCAA platform	Hivebrite	Almabase	Anthology Encompass	Salesforce NPSP	Zoho CRM	CiviCRM	WP + PMPro
C1 Governance depth: constitutional tiers, officer terms, versioned instrument, amendment vote	●	○	○	◐	◐	○	◐	○
C2 Manual payment with proof and officer verification as primary path	●	○	○	◐	◐	◐	◐	○
C3 Annual recurring cost at this scale, and whether the vendor publishes it	very low, self-hosted	published, from ~US$10.7k/yr [69]	not published; quoted [70]	not published; quoted	low if eligible for the donation [72]	published, per seat [71]	zero licence cost	published, free to US$2,999/yr [73]
C4 Single-maintainer feasibility	●	◐	◐	○	◐	◐	◐	◐
C5 Data sovereignty and clean exit	●	○	○	○	◐	◐	●	●
C6 Mobile fitness on mid-range handsets over mobile data	●	●	●	◐	◐	●	◐	◐
C7 Localisation: Bengali content, MFS channels, NID conventions	●	◐	◐	○	◐	◐	◐	◐
C8 Rule traceable to a constitutional clause	●	○	○	○	○	○	○	○
Directory, events, email, mentoring maturity	◐	●	●	●	●	●	◐	◐

The last row is included deliberately. In the capabilities these products were built for, they are better than the artefact produced here, and the comparison would be dishonest without saying so.

TABLE 2.3
Gap table

Required capability	Best covering system	Residual gap	Addressed in
Membership tiers with differentiated constitutional rights	CiviCRM (as priced membership types)	Tiers model price and duration, not rights; the non-voting rule for Associate members is an access convention, not a modelled property	§3.3.1, §5.5

Required capability	Best covering system	Residual gap	Addressed in
Fixed-term officer positions with succession	Salesforce NPSP (custom object)	No term, quorum or ex-officio semantics; no institutional record independent of user accounts	§3.3.6, §5.3
Versioned governing instrument with an always-current reader	None	No product treats the governing document as versioned data with an effective date	§3.3.6, §7.10
Amendment voting at a constitutional threshold	None	Poll features exist; the two-thirds-at-AGM rule and eligibility gate do not	§3.3.6, §5.5
Payment without a gateway, evidenced and officer-verified	CiviCRM (offline contributions)	Secondary path; no proof-upload and verification workflow with an audit trail	§3.3.4, §8.7
Auditable trace from enforced rule to constitutional clause	None	Configuration produces behaviour, not a trace a reviewer can follow	§3.7, Table 3.4
Operation at tens of dollars per year by one volunteer	WordPress + plugin	Achievable only by abandoning C1, C2 and C8	Ch. 10

Chapter 3 — Requirements Engineering

This chapter is written to the structure of ISO/IEC/IEEE 29148 [21]. Requirements are stated in the "shall" form, each with an identifier, a source and a priority, and each traceable forward through the design and test artefacts in Table 3.4. Quality requirements are classified by ISO/IEC 25010 characteristic [6] and cross-referenced to FURPS+ [23], and each is restated as a quality-attribute scenario in §3.4 so that it can be tested rather than merely asserted.

One property of this specification distinguishes it from a conventional one. A substantial part of the requirement set is not negotiable with the stakeholders, because it is dictated by the Association's constitution and its election documents. Those constraints are collected in §3.7 and carry DC identifiers, and every functional requirement they govern cites the article and section concerned. Where a stakeholder wish conflicted with a constitutional clause, the clause won; §3.2 records the three occasions on which that happened.

3.1 Sources of the Requirements

Requirements came from four sources: analysis of the Association's governing documents, interviews with office holders and ordinary members, observation of current practice, and the survey of existing products in §2.9. The Source column of §3.3 names one of the four for every requirement, and the value R marks the few that arrived after elicitation had closed, as dated requests rather than from any of them.

How each source was worked, who took part, how they were chosen and what instruments were used are part of the research method, and are set out in §4.6. The ethical position, both on the people who took part and on the live member data the platform holds, is in §4.8. This chapter takes the elicitation output as given and specifies from it.

3.2 Requirements Analysis and Negotiation

Analysis produced three genuine conflicts, and the record of how each was resolved matters more than the resolutions themselves, because it is the evidence for how the constitutional constraint was treated in practice.

Directory openness against member privacy. Officers wanted a fully browsable directory with contact details, on the reasonable argument that a directory nobody can use is not a directory. Several members, asked directly, objected to their mobile number being visible to a membership of several hundred people they do not all know. The resolution was a masked projection by default with per-field opt-in disclosure controlled by the owning member, specified in FR-03 and FR-05 and implemented as the masking protocol in §8.9, on personal data. Officers retain unmasked access for administrative purposes, which is itself logged. This satisfies neither party completely and is recorded as such.

Automatic termination for non-payment against Article X. An early requirement, offered by an officer and initially accepted, was that the system should set a member inactive and then terminate membership automatically once dues had been outstanding for a defined period. Article X vests disciplinary action in a procedure with notice and a right of appeal, and Article III Section E gives an appeal window of fifteen working days. Automatic termination would have had software perform an act the constitution reserves to a body following a procedure. The requirement was reduced: the system computes and displays standing, flags arrears, and generates the work item, and a human officer performs the status change with the reason recorded (FR-24, DC-07). This is the clearest instance of the general position argued in §2.7.

Convenience of a payment gateway against the Association's actual position. Integrating a gateway was requested and would have removed the verification workload entirely. The Association does not hold a merchant account, cannot satisfy the documentation a gateway requires without a banking relationship it has not yet established, and would in the interim have had to route collections through an individual's personal account, which is the arrangement the project exists to end. The resolution was to design the manual path as primary, with proof upload and officer verification, and to leave the gateway abstraction in place but unconfigured so that adoption later is a configuration change and not a redesign. The reasoning is set out in full in §8.7, and the consequence, which is a permanent administrative burden of some minutes per payment, is quantified in §12.4 rather than glossed.

Two further analysis outcomes are worth recording. Requirements arriving as solutions were rewritten as needs; the request for "a Facebook login button" became FR-10, stated as a requirement to reduce the credential burden at first contact, which admits other means. And several requested features were classified as out of scope rather than deferred, so that the MoSCoW table in §3.6 does not carry items that were never going to be built.

3.3 Requirements Specification

The requirement catalogue is Table 3.1 and the non-functional catalogue Table 3.2; both are set as the tables of §3.3 and §3.4 rather than repeated at the end of the chapter. Priorities use MoSCoW as in §3.6: M must, S should, C could, W won't in this release. The Source column records D for document analysis with the clause cited where applicable, I for interview, O for observation, C for competitor analysis, and R for a dated stakeholder request recorded in the project tracker `docs/TODO.md` after the original elicitation had closed.

Figures 3.8 and 3.9 draw the analysis-level domain model these requirements imply, before any persistence or transport concern is added to it. It is drawn in two halves, membership and money then participation and governance, because one diagram of all seventeen classes prints too small to follow.

3.3.1 Membership, Registration and Profile

ID	Requirement	Source	Pri
FR-01	The system shall provide a public multi-step registration wizard that captures personal, academic and professional particulars, permits an applicant to save progress and resume, and validates each step before advancing.	I, C	M
FR-02	The system shall create an application in status *Applied* on submission, record the submission timestamp, and expose the application state to the applicant through a status view. Statuses shall be limited to Applied, Active, Inactive by non-payment, Inactive by resignation, Terminated and Rejected.	D (Art. III §E), I	M
FR-03	The system shall allow a member to maintain his own profile and to control, per field, whether that field is disclosed in the directory, with non-disclosure as the default for contact and identity fields.	I, D	M
FR-04	The system shall require at least one academic record at Govt. Haraganga College before an application may be submitted, and shall permit any number of additional academic and professional records.	D (Art. III), O	M
FR-05	The system shall provide a directory searchable by name, batch, academic session, subject, profession and location, returning only fields the owning member has disclosed, except to administrative users.	I, C	M

ID	Requirement	Source	Pri
FR-06	The system shall issue each active member a digital identity card bearing the membership number, photograph, tier and a machine-readable code that resolves to a verification endpoint.	I, C	S
FR-07	The system shall present each member his own standing, comprising membership number, tier, status, dues raised, dues settled and outstanding balance.	I, O	M
FR-55	The system shall identify the institution-defining academic record through a flag evaluated against the active organisation profile, rather than a name specific to one institution, preserving the existing true/false meaning and the first-record rule of FR-04, and shall accept the field's legacy name from a client not yet updated, mapping it to the same stored value.	R	S

FR-55 is the specification for the `IsGHC`-to-`IsOrgProfile` rename recorded in `docs/specs/005-academic-organisation-profile-flag/spec.md`, which sets out the white-label motivation: the college-specific name does not describe a profile-selected organisation once a second institution is deployed. The rename touches the Domain model, the Application DTO, the PostgreSQL column and both clients, but it changes no stored value and no predicate; FR-04's rule still governs which record is required, only under a portable name.

Registration completeness is a gate rather than advice. Thirteen particulars must be present before an application is submissible: full name, email address, mobile number, date of birth, gender, national identity number, father's name, mother's name, permanent address, blood group, photograph, at least one college academic record and at least one professional record. The membership number is allocated at approval, not at application, in the form `GHC-[Year]-[Serial]`, and thereafter serves as the member's login identifier as well as his identity on the card and on receipts. Allocation at approval rather than at application is deliberate: a number issued to an unverified applicant would appear on an identity card the Association had not authorised.

3.3.2 Authentication, OTP and Session Management

ID	Requirement	Source	Pri
FR-08	The system shall authenticate members by membership number or registered email address together with a password, storing passwords only as an adaptive salted hash.	I	M
FR-09	The system shall verify a registered email address and mobile number by a single-use, time-limited code, and shall reset a forgotten password by a single-use, time-limited link.	I	M
FR-10	The system shall permit sign-in by a federated identity provider, associating the federated identity with exactly one member record and requiring completion of the member profile before portal access is granted.	I, C	S
FR-11	The system shall issue a short-lived access token and a rotating refresh token, shall support explicit sign-out, and shall invalidate all outstanding sessions for a member when his credentials, roles or status change.	D, I	M
FR-12	The system shall prevent a member whose mandatory profile particulars are incomplete from using member functions other than profile completion, and shall indicate what remains outstanding.	O	M

FR-11 states the security-stamp requirement in requirement form. A change of role, a password reset or a change of membership status must take effect immediately rather than at token expiry, because the alternative is a window in which a terminated member retains the rights of an active one. The mechanism is described in §7.4 and its threat rationale in §8.3, on authentication and session security.

3.3.3 Events, Registration, Attendance and Gallery

ID	Requirement	Source	Pri
FR-13	The system shall allow an administrator to publish an event with title, description, venue, start and end, capacity, fee and registration deadline, and to amend or cancel it with notification to registrants.	I	M
FR-14	The system shall allow a member to register for a published event, subject to capacity, deadline and any fee condition, and to cancel his registration.	I	M
FR-15	The system shall place registrations beyond capacity on an ordered waiting list and shall promote from the head of that list, in order, when a place is released.	C	S
FR-16	The system shall record attendance by scanning the member's identity code at the venue, and shall reject a code that does not correspond to a valid registration for that event.	I, C	S
FR-17	The system shall maintain a per-event photograph gallery, uploaded by administrators, visible according to the event's publication setting.	I	C
FR-18	The system shall report per event the registered count, the waitlisted count, the attended count and the fee collected.	I	S
FR-53	The system shall permit a member to submit a photograph or album to an event's gallery, and shall hold the submission unpublished until an administrator approves or rejects it, with a recorded reason on rejection.	R	C

FR-53 extends FR-17 on the same reasoning as FR-27 for news: a submission from a member is content until an officer has looked at it, not a publication in its own right. Like FR-54 in §3.3.9, it arrived after the elicitation closed, as the tracker request cited there.

3.3.4 Payments, Dues and Financial Records

ID	Requirement	Source	Pri
FR-19	The system shall raise the admission fee and the first annual subscription as ledger obligations at the moment a membership is approved, at the rates then configured.	D (Art. VI), O	M
FR-20	The system shall raise the annual subscription for each active member on the anniversary configured for the purpose, and shall not raise it for members whose tier is exempt.	D (Art. III, Art. VI)	M
FR-21	The system shall record, for every payment, the amount, the date, the channel, the member concerned, the transaction reference supplied by the member, the uploaded evidence, the verifying officer and the verification timestamp.	O	M
FR-22	The system shall permit a member to declare a payment made through an external channel by supplying a transaction reference and uploading evidence, and shall hold that declaration unverified until an officer acts on it.	I, D	M
FR-23	The system shall present administrators a queue of unverified payment declarations and of pending applications, ordered by age, and shall indicate any item older than thirty days.	D (Art. III §E), O	M
FR-24	The system shall compute each member's standing from raised and settled obligations, shall flag arrears, and shall require an administrative act, with a recorded reason, for any change of membership status arising from arrears.	D (Art. X), I	M
FR-25	The system shall issue a downloadable receipt bearing the membership number, the amount, the channel, the reference and the verifying officer, for every verified payment, and shall not permit a verified financial record to be amended or deleted; corrections shall be made by a compensating entry.	D (Art. VI), I	M

The ledger is append-only by requirement, not by convention. FR-25's prohibition on amendment is what allows the treasurer to answer a challenge about a historical figure, and it is the property the paper ledger lacked.

The payment channels the Association actually uses are mobile financial services, principally bKash and Nagad, bank transfer, and cash received by an officer. In every case the money moves outside the platform. What the platform holds is the member's claim, the evidence and the officer's verification. It holds no payment credential of any kind, and §8.7 argues that this is a security position and not only a limitation.

3.3.5 News, Notices and Communication

ID	Requirement	Source	Pri
FR-26	The system shall maintain posts of two kinds, news and notice, with a common structure and differing authorship rules.	D (Art. IV), I	M
FR-27	The system shall permit members to submit news items for editorial approval, and shall refuse any member submission of kind notice; notices shall be creatable only by an authorised officer.	D (Art. IV), I	M
FR-28	The system shall permit an officer to publish, unpublish, pin, edit and archive posts, retaining the archived record rather than deleting it.	I	M
FR-29	The system shall permit an officer to compose and dispatch an email or SMS communication to a selected segment of the membership, where a segment may be defined by tier, status, batch or event participation, and shall record the channel, recipient member, delivery status, scope and time.	C, I	S
FR-29a	The system shall let a member view only the member's own paginated outbound communication history, including channel, status, scope, message detail and delivery time.	I	S
FR-29b	The system shall record unavailable and failed provider outcomes without exposing failure detail on successful history rows.	I	S
FR-30	The system shall deliver in-application notifications for events material to a member, including application decision, payment verification, event promotion from the waiting list, and notices.	I	S
FR-31	The system shall provide member-to-member and member-to-officer messaging in real time, subject to the privacy settings of FR-03.	I, C	C

FR-27 is a constitutional requirement wearing the clothes of an editorial policy. A notice is an official act of the Association under Article IV; a news item is content. Allowing a member to publish something indistinguishable from an official notice would misrepresent the Association's voice, so the refusal is enforced at the point of submission and is not merely a permission on a button.

FR-29a and FR-29b are the specification for the member communication visibility work recorded in `docs/specs/004-pending-work-packages/spec.md` (items 81.1-81.3): a member's own outbound communication history, distinguishing queued, sent, failed and unavailable provider states, without exposing provider failure detail on a row that succeeded. The access-control property this depends on, that a member can retrieve only rows addressed to himself, is stated as NFR-S9 and its implementation is described in §8.4.

3.3.6 Governance

ID	Requirement	Source	Pri
FR-32	The system shall publish the constitution to members and to the public, presenting always the version in force by effective date, and shall not require any page or link to name a version.	D (Art. VIII), I	M
FR-33	The system shall retain every ratified version of the constitution with its version identifier, effective date and summary of change, shall present that history, and shall supersede rather than remove a version when a later one takes effect.	D (Art. VIII)	M
FR-34	The system shall maintain the Executive Committee record, comprising the fifteen positions, the holder of each, the term start and end, and whether the position is elected or ex officio, and shall retain past committees.	D (Art. IV)	M
FR-35	The system shall permit an authorised officer to open an amendment proposal for voting, recording the proposing route, the circulation date and the closing date, and shall enforce a minimum circulation period of fourteen days before the vote opens.	D (Art. VIII)	M
FR-36	The system shall admit a vote on an amendment proposal only from a member who is a voting member in good standing at the time of voting, shall admit exactly one vote per eligible member, and shall record the eligibility determination together with the vote.	D (Art. III §B, Art. V)	M
FR-37	The system shall report an amendment vote as the count of votes cast for and against, the number of eligible voters, and whether the two-thirds threshold was met, and shall not itself declare the amendment adopted.	D (Art. VIII)	M
FR-38	The system shall maintain the roll of members eligible to vote at a forthcoming election, derived from tier and standing, and shall make that roll available to the Election Commission in a form that can be published and challenged.	D (Art. V), election regulations	M
FR-39	The system shall support a persisted election workflow for authorized administration, including a frozen voter roll, nominations, secret ballots, counting and declaration, while detailed legal form wording and institutional adoption remain governed by the Election Commission's election documents.	D (Art. V), election regulations, I	M
FR-40	The system shall support non-binding member polls, open to a configurable eligibility set, reporting counts only.	I	C

The relationship between FR-36, FR-37 and FR-39 is the substance of this project's answer to RQ3. The system decides eligibility, which is a question of fact traceable to Article III Section B and to the standing computation of FR-24, and it records the count. It does not declare adoption, and it does not touch the ballot for office at all. The reasoning is in §2.7, the mechanism in §5.5, and the defence in §8.8.

3.3.7 Administration, Configuration and Site Content

ID	Requirement	Source	Pri
FR-41	The system shall provide an administrative console covering membership, payments, events, posts, communication, governance and configuration, with functions available according to the acting user's role.	I	M
FR-42	The system shall hold operational parameters as configuration rather than in code, including fee amounts, payment channel particulars and instructions, the subscription anniversary, rate limits and contact details, and shall permit an authorised administrator to change them without redeployment.	I, O	M
FR-43	The system shall record every administrative act that changes a member's status, verifies a payment, or amends configuration, together with the acting user and the time.	O, D (Art. VI)	M
FR-44	The system shall implement archival deletion, such that a record marked archived is excluded from ordinary retrieval while remaining recoverable, and shall not permit destructive deletion of membership or financial records through the interface.	D (Art. VI), I	M
FR-45	The system shall permit an administrator to edit the text and images of public informational pages without redeployment.	I	S
FR-46	The system shall present a public site giving the Association's purpose, its office holders, published news and notices, the constitution, election information, a gallery, and a route to apply for membership.	I, C	M

ID	Requirement	Source	Pri
FR-47	The system shall permit an administrator to select the presentation theme from a configured set, applying to the public site and to the member portal.	I	C

3.3.8 Mobile Application Requirements

ID	Requirement	Source	Pri
FR-48	The mobile application shall provide sign-in, profile maintenance, the directory, events, the identity card, payment declaration and notices, from a single codebase for both major platforms.	I, C	M
FR-49	The mobile application shall render the identity card and its machine-readable code without a network connection once the member has signed in at least once.	I	S
FR-50	The mobile application shall capture and compress a photograph or evidence image on the device before upload, to the size limit stated in NFR-P4.	O	M
FR-51	The mobile application shall discourage capture of the identity card and profile screens by screen recording or screenshot where the platform provides the means.	I	C
FR-52	The mobile application shall report an unavailable or failed network operation to the member in terms of what to do next, and shall not lose entered data on failure.	O	M

FR-49 exists because members present the card at events held in venues where mobile data is unreliable, which was observed rather than reported. FR-52 is stated as a functional requirement rather than a usability aspiration because the failure it addresses, namely a form cleared by a dropped connection, was the single most common complaint about the interim tooling the Association had tried.

3.3.9 Job Board

ID	Requirement	Source	Pri
FR-54	The system shall permit a member to post a job or career opportunity, and shall hold the posting unpublished until an administrator approves or rejects it, with a recorded reason on rejection.	R	C

FR-54 came after the elicitation of §3.1 had closed. Its provenance is a dated stakeholder request logged in the project tracker: `docs/TODO.md`, Work Package 40, "Member albums with admin approval, job-posting approval, events without registration (raised by user 2026-08-26)", which is also the origin of FR-53. That is a weaker record than an interview transcript, and it is marked R rather than I for that reason: what survives is the request and the date, not the discussion behind it. Both requirements were specified from the request and then implemented, rather than reconstructed from the shipped code afterwards.

3.4 Non-Functional Requirements and Quality-Attribute Scenarios

Classified by ISO/IEC 25010:2011 characteristic [6], with the FURPS+ category in brackets for cross-reference [23]; Figure 3.11 shows the classification as a tree, and Figure 3.12 the goal model from which the quality requirements were derived.

The edition matters, and the choice is deliberate. ISO/IEC 25010:2023 [74] supersedes the 2011 edition and is not a cosmetic revision: it renames usability as interaction capability and portability as flexibility, adds safety as a ninth characteristic, and reorganises several subcharacteristics. This requirement set was classified against the 2011 edition before that revision was consulted, and the NFR identifiers, the utility tree of Figure 3.10 and the evaluation plan of §4.5 all carry that structure. Reclassifying against the 2023 edition would change identifiers used throughout the book for no gain in the argument, so the 2011 edition is retained and named explicitly wherever it is cited. A reader working from the 2023 edition should read NFR-U* as interaction capability and NFR-Po* as flexibility; nothing in this dissertation makes a safety claim, which is the one characteristic the older model has no place for. Every entry has a measurable acceptance criterion; entries that could not be given one were rejected rather than weakened, which is why there is no requirement here about the system being "intuitive".

Functional suitability *(Functionality)*

ID	Requirement and acceptance criterion
NFR-F1	Every functional requirement of priority M shall have at least one automated test asserting its principal behaviour. Criterion: coverage demonstrated in Table 3.4 with no M-priority requirement unmapped.
NFR-F2	No enforced constitutional rule shall exist in the implementation without a corresponding DC entry, and no DC entry of priority M shall be unimplemented. Criterion: the DC column of Table 3.4 is complete in both directions.

Performance efficiency *(Performance)*

ID	Requirement and acceptance criterion
NFR-P1	Read endpoints shall respond within 500 ms at the 95th percentile under a load of 50 concurrent users on the production tier.
NFR-P2	Write endpoints shall respond within 1,000 ms at the 95th percentile under the same load.
NFR-P3	The web client's initial route shall become interactive within 3 s on a mid-range Android handset over a 3G-class connection, with an initial transfer budget of 500 KB compressed.
NFR-P4	Uploaded images shall be reduced to at most 350 KB before storage, without visible loss at the sizes at which they are displayed.
NFR-P5	Directory search over the full membership shall return the first page within 800 ms at the 95th percentile.

Compatibility *(Supportability)*

ID	Requirement and acceptance criterion
NFR-C1	The web client shall function on the current and preceding major versions of Chrome, Firefox, Safari and Edge.
NFR-C2	The mobile application shall support Android 8.0 and above and iOS 13 and above.
NFR-C3	The persistence layer shall be provider-neutral to the extent that the system runs against PostgreSQL in production and against a file-based engine for development and test without source change.

Usability *(Usability)*

ID	Requirement and acceptance criterion
NFR-U1	The system shall meet WCAG 2.1 level AA for all member-facing pages [24]. Criterion: automated audit clean, plus manual keyboard and contrast checks recorded in §9.9.
NFR-U2	Registration shall be completable on a handset in a single sitting by a member without assistance. Criterion: task completion rate and time reported in §12.4.
NFR-U3	The system shall obtain a System Usability Scale score of at least 68, the published average, from member participants [33].
NFR-U4	Every error presented to a user shall state what happened and what to do next, and shall not expose an internal identifier, stack trace or provider message.
NFR-U5	All member-facing text shall support Bengali content and shall render Bengali script correctly in all clients, including generated documents.

Reliability *(Reliability)*

ID	Requirement and acceptance criterion
NFR-R1	Monthly availability of the production service shall be at least 99 percent, excluding announced maintenance.
NFR-R2	A daily backup of the production database shall be taken and its restoration verified at least quarterly. Criterion: restoration drill recorded in §10.5.
NFR-R3	No single user action shall be capable of destroying a membership or financial record; archival deletion per FR-44 shall be the only deletion available through the interface.
NFR-R4	A failed external dependency, being mail delivery or file storage, shall degrade the affected function only and shall not prevent authentication or read access.
NFR-R5	A vote cast by an eligible voter shall be recorded at most once, even where the casting request is submitted more than once for the same voter and election. Criterion: the whole ballot is written in one serializable transaction that first checks the voter has no seat-vote row, and a unique index on the seat-vote row stops a second write that races past that check, so a second submission for the same voter is rejected with an already-voted error; verified in `ElectionServiceTests.CastBallotAsync_RejectsReplayAfterTheFirstVote`, which exercises a first-then-second sequential resubmission rather than two literally simultaneous requests.
NFR-R6	The mobile client shall retain a rotating on-device log of recent API errors and notable app events, so that a crash or failed request can be diagnosed from the device that produced it. Criterion: the log is capped at 256 KB, dropping the oldest half of its content once the cap is crossed rather than truncating silently, and its tail is attached to the in-app error report sent to the administrator.

Security *(Functionality, security sub-characteristic)*

ID	Requirement and acceptance criterion
NFR-S1	The system shall satisfy OWASP ASVS level 2 for the control families applicable to it [7], with every non-conformance recorded rather than omitted (§8.4).
NFR-S2	Passwords shall be stored using an adaptive hash with a cost parameter reviewed annually [49].
NFR-S3	Authentication attempts shall be limited to 5 per minute per source, registration to 10 per 5 minutes, and general API access to 100 per minute.
NFR-S4	National identity number, mobile number and email address shall each be unique across members, enforced in the database and not only in application code.
NFR-S5	A change of a member's credentials, roles or status shall invalidate all outstanding sessions for that member within one request.
NFR-S6	No resource shall be retrievable by a user not entitled to it by role or ownership. Criterion: the authorisation test suite of §9.7 exercises each protected endpoint with an unentitled principal.
NFR-S7	Uploaded files shall be validated by declared type, actual content signature and size, stored outside the web root, and served only through an authorising endpoint.
NFR-S8	Member-supplied rich text shall be sanitised on the server before storage.
NFR-S9	A member's request for his own communication history shall be scoped to that member's identity as carried in his own authenticated token, never by a client-supplied member identifier, and the equivalent view of another member's history shall be reachable only under the administrative authorisation policy.

Maintainability *(Supportability)*

ID	Requirement and acceptance criterion
NFR-M1	The dependency rule shall hold: no compile-time dependency shall point from an inner layer to an outer one. Criterion: architecture test in §9.4 fails the build on violation.
NFR-M2	A routine change of the kind catalogued in §9.12, being the addition of a field to an existing entity end to end, shall be completable by the maintainer within one working session.
NFR-M3	The automated suite shall run to completion in under 10 minutes on the integration runner, so that it is actually run.
NFR-M4	The system shall be operable by one maintainer with no on-call obligation, and shall require no scheduled manual intervention more often than monthly.

Portability *(Supportability)*

ID	Requirement and acceptance criterion
NFR-Po1	The system shall be deployable to any container host, with no dependency on a proprietary platform service that lacks a substitutable equivalent.
NFR-Po2	A complete environment shall be reproducible from the repository and a documented set of configuration values, with no undocumented manual step.
NFR-Po3	The Association's data shall be exportable in full in an open format on demand.

The requirements above are tested through quality-attribute scenarios, each stated in the six-part form of Bass, Clements and Kazman: source, stimulus, artefact, environment, response, response measure [13]. These are the scenarios carried into the utility tree of Figure 3.10 and into the design verification of §6.10. Eleven are given here; the remainder are in `docs/SRS.md`.

ID	NFR	Source	Stimulus	Artefact	Environment	Response	Response measure
QAS-01	NFR-P1	Member	Opens the directory listing	API and database	Production, 50 concurrent users	Returns the first page	≤ 500 ms at p95
QAS-02	NFR-P3	Prospective member	Requests the public home page on a mid-range handset over 3G	Web client	Cold cache	Page becomes interactive	≤ 3 s; ≤ 500 KB transferred

ID	NFR	Source	Stimulus	Artefact	Environment	Response	Response measure
QAS-03	NFR-S5	Administrator	Terminates a membership while that member holds a valid access token	API, session store	Normal operation	The member's next request is rejected	Within one request; no successful authenticated call after the act
QAS-04	NFR-S6	Authenticated member	Requests another member's payment record by identifier	API authorisation	Normal operation	Access refused, attempt recorded	0 unauthorised disclosures across the enumerated protected endpoints
QAS-05	NFR-R4	Mail provider	Becomes unavailable for 30 minutes	Notification component	Production	Authentication and reads continue; queued mail is retried or reported	0 failed sign-ins attributable to the outage
QAS-06	NFR-M2	Maintainer	Adds one field to an existing entity through to both clients	Whole stack	Development	Change completed with the suite green	≤ 1 working session; file-touch count reported in §9.12
QAS-07	NFR-M1	Maintainer	Introduces a dependency from the domain layer onto the persistence layer	Build	Continuous integration	Build fails with the violation named	Detected on the first run, before merge
QAS-08	NFR-F2	Examiner	Asks which code enforces the constitutional restriction on who may vote	Source and specification	Review	A single traced location is produced	Trace from Art. III §B to DC-03 to FR-36 to a named test
QAS-09	NFR-R2	Maintainer	Loses the production database	Backup and restore procedure	Disaster	Service restored from backup	≤ 4 hours; ≤ 24 hours of data lost
QAS-10	NFR-U1	Member using a screen reader	Completes registration	Web client	Assistive technology	Registration completed unaided	No level AA violation on the registration route
QAS-11	NFR-R5	Voter	Resubmits a vote for an election already voted in	Election API, voter roll	Production	Second submission is rejected; the first vote stands unchanged	0 ballots with more than one recorded vote per voter per election

QAS-08 is not a conventional quality scenario and is included deliberately. Auditability of the rule trace is treated here as a first-class quality attribute, because it is the property that distinguishes this artefact from the configured alternatives assessed in §2.10.

3.5 Use Cases, User Stories and the Definition of Done

The system-level model is Figure 3.1, decomposed by subsystem into membership (Figure 3.2), events (Figure 3.3), payments (Figure 3.4), governance (Figure 3.5) and administration (Figure 3.6). The actor hierarchy in Figure 3.7 matters because it is the model the authorisation policy implements: Guest is unauthenticated; Member is an authenticated member in any tier; Voting Member is the subset admitted by DC-03; EC Member holds a position under Article IV; Admin exercises administrative functions; SuperAdmin holds configuration and role assignment. Generalisation here is genuine inheritance of capability, with one deliberate exception noted in §5.3: an EC Member's administrative capability derives from an assigned role and not from the position itself, so that a committee position can exist in the record without conferring system access.

Table 3.3 gives the ten highest-value use cases in full. The remainder are in `docs/SRS.md`.

Alongside the use cases, requirements were carried into delivery as user stories in the form Cohn describes [45], with acceptance criteria in given-when-then form. The stories do not replace this specification; they decompose it, and each cites the requirement it serves. Two representative examples:

> **US-14** *(FR-22, FR-23)* As a member who has paid his subscription by bKash, I want to declare the payment with the transaction reference and a screenshot, so that my standing is corrected without my having to find an officer.
>
> Given an outstanding obligation and a valid reference and image, when I submit the declaration, then the declaration is recorded as unverified, appears in the administrative queue, and my standing shows the payment as pending verification rather than as settled.

> **US-31** *(FR-36, DC-03)* As the General Secretary I want the system to admit amendment votes only from voting members in good standing, so that the count I report cannot be challenged on eligibility.
>
> Given an open proposal, when a member of Associate, Honorary or Advisory tier attempts to vote, or a voting member in arrears attempts to vote, then the vote is refused with the reason stated, and when an eligible member votes twice, then the second attempt is refused.

Definition of done, applied uniformly and reported against in §11.10, the record of quality-assurance activities: the acceptance criteria pass; automated tests exist at the appropriate level and the whole suite is green; the architecture test of NFR-M1 passes; authorisation is asserted for every new endpoint; input validation and, where applicable, sanitisation are present; the traceability matrix is updated; user-facing text meets NFR-U4 and NFR-U5; the change is deployed to the pre-production environment and exercised there; and the documentation affected is updated in the same change rather than later.

3.6 Requirements Prioritisation and Traceability

Table 3.5 records the prioritisation and what was negotiated to reach it. The counts are over FR-01 to FR-55 as they stand at the time of writing:

Priority	Count	Basis
Must	38	Either constitutionally mandated, or the Association cannot operate without it
Should	10	Substantial value; deferrable by one increment without operational failure
Could	7	Desirable; first to be dropped under schedule pressure
Won't (this release)	8	Explicitly excluded, recorded so that the exclusion is a decision rather than an omission

Constitutional mandate was treated as automatically Must, which removed a third of the prioritisation argument before it began. The Won't set comprises live gateway payment, binding election balloting, statutory accounting output, offline-first mobile synchronisation, a native desktop client, machine-learning recommendation, an external language-model assistant, and multi-organisation tenancy. Each is recorded with its reason in §1.7 or §13.2.

Two prioritisation decisions are worth exposing because they were contested. Real-time messaging (FR-31) was reduced from Should to Could, on the ground that members already have working messaging channels and the Association's need was for official communication rather than conversation. Gamification of engagement, requested with some enthusiasm, was excluded entirely: the engagement literature of §2.3 gives no support for points and badges as a driver of the behaviour the Association actually wants, and the maintenance cost is permanent.

Each prioritised requirement was also traced. Traceability here is a working obligation, not a document produced at the end. Gotel and Finkelstein established that the traceability problem is principally one of pre-requirements-specification traceability, meaning the ability to establish where a requirement came from and why [47]. In this project the answer is frequently a constitutional clause, which makes the trace unusually verifiable: a reviewer can read Article III Section B and follow it to DC-03, to FR-36, to the rule in §5.5, to the guard in the governance service, and to the named test.

Table 3.4 records requirement to use case to design element to implementation artefact to test case. It prints a representative extract; the full matrix is Appendix B, and §12.2 closes it by reporting any requirement not traced to a passing test.

One limit of the chain as it currently stands has to be stated, because it bears on how far the traceability claim can be pressed. The trace is complete in one direction only. From a constitutional clause a reader can follow the matrix to the requirement, the rule and the test that pins it, but the tests themselves carry no requirement or constraint identifier, so the same trace cannot be started from the code and read back. The matrix is therefore maintained by hand and could drift from the suite without anything failing. §9.4.5 and Table 9.9 close the governance half of that gap by naming the test for each domain constraint; making the whole matrix machine-checkable would need the tests tagged, which is recorded as future work in §13.2 rather than claimed here.

3.7 Domain Constraints

These are the rules the software may not violate, listed in Table 3.6. They derive from the constitution at version 4.2 and from the election documents, and each cites its source. Where the wording below abbreviates the clause, the clause governs.

Each constraint also carries a class, and the classes matter more than the individual rules. Writing the constitution into software forces a question of every clause that reading it never asks: what is the software allowed to do with this rule. Answering it sixteen times produced five answers, and they are not a spectrum of how much automation is possible but five different kinds of relationship between a rule and a program.

- **Deterministic.** The rule can be decided from data the system already holds, and deciding it is arithmetic or a lookup. The software enforces it outright, and a human override would be an error rather than a discretion.
- **State.** The rule is a standing that follows from a history of recorded events. The software computes and displays it, and the computation is only as good as the recording, so the failure mode is a gap in the record rather than a wrong rule.
- **Evidence.** The rule turns on a judgement about evidence that has no computable predicate. The software collects the evidence, computes whatever part of the rule is computable, and presents the rest to a person. Automating the judgement would convert it into an arithmetic that the constitution never authorised.
- **Procedural.** The rule is a sequence with notice periods, thresholds and rights attached. The software supports the procedure by tracking its steps and its deadlines, and does not decide its outcome.
- **Authority.** The rule belongs to a body other than the platform's operator. The software may publish, inform and record around it, and must not perform it, because performing it would move authority to whoever administers the server.

The classification is used again in §12.6 to answer RQ3, where the boundary it draws is the answer rather than a preliminary to it. Its value is that the line between what software may do and what it must leave alone stops being a matter of the designer's restraint and becomes a property of the rule, readable from the constitution before any code is written.

ID	Constraint	Class	Source	Enforced by
DC-01	The Association's name, crest and motto shall be used only as the constitution prescribes, and the college's institutional identity shall not be represented as the Association's own.	Deterministic	Art. I §5	FR-46, §7.7
DC-02	The Association shall be non-political; no member shall use it for partisan purpose, and a Founding Member holding partisan office is suspended automatically.	Evidence	Art. I, Art. III, Art. VII	FR-24, §5.5
DC-03	Only Founding, Executive and General members may vote. Associate, Honorary and Advisory members may not.	Deterministic	Art. III §B	FR-36, FR-38
DC-04	Founding membership requires at least twenty years since higher secondary completion, a bachelor's degree, documented contribution to the Association or the college, no partisan office, and a signed non-political declaration.	Evidence	Art. III	Administrative, recorded; §5.5
DC-05	Executive membership requires at least ten years since completion, existing General membership in good standing, and a meeting-attendance record of at least sixty percent.	State	Art. III	Administrative, recorded; §5.5
DC-06	Every membership tier is assigned by the Association; an applicant may not select a tier, and Honorary membership requires a two-thirds decision of the Executive Committee.	Procedural	Art. III	FR-01, FR-41
DC-07	Membership shall be terminated only by the disciplinary procedure, with notice and a right of appeal exercisable within fifteen working days; false information in an application is ground for immediate termination under that procedure.	Procedural	Art. III §E, Art. X	FR-24, §3.2
DC-08	An application shall be verified within thirty days of submission.	State	Art. III §E	FR-23
DC-09	The Executive Committee comprises fifteen positions, thirteen elected and two ex officio without vote, for a term of three years.	Deterministic	Art. IV	FR-34
DC-10	For the first fifteen years, three elected positions, being President, General Secretary and Treasurer, are reserved for Founding Members.	Deterministic	Art. IV	FR-34, FR-38
DC-11	The Executive Committee shall meet at least half-yearly, with a quorum of fifty percent of elected members; Advisory members are excluded from the quorum computation.	State	Art. IV, Art. III	FR-34, §5.5
DC-12	Elections for all voting members in good standing shall be held every three years, conducted under the authority of Article V and the election documents.	Authority	Art. V	FR-38, FR-39
DC-13	An amendment may be proposed by the Executive Committee or by a petition of twenty percent of Voting Members, shall be circulated at least fourteen days before the Annual General Meeting, and requires a two-thirds majority at that meeting.	Procedural	Art. VIII	FR-35, FR-37
DC-14	Financial records shall be maintained and reported to the membership; the Treasurer is accountable for receipts and disbursements.	Deterministic	Art. VI	FR-21, FR-25, FR-44
DC-15	The conduct of a ballot, its sealing, counting and the declaration of result rest with the Election Commission under the election documents, and not with the platform.	Authority	Art. V, election regulations, ballot sealing certificate, counting authorisation	FR-39, §8.8
DC-16	The constitution in force is the latest ratified version by effective date; earlier versions are superseded and retained, not destroyed.	Deterministic	Art. VIII	FR-32, FR-33

DC-04 and DC-05 are the pair that shows why the classification is not a matter of degree. They read almost identically, both being lists of qualifying conditions for a membership tier, and they fall on opposite sides of the line. Every condition in DC-05 is recorded somewhere: years since completion, existing membership, an attendance figure. The system computes it, which makes it State. DC-04 carries the same computable years and adds "documented contribution to the Association", which is not a computable predicate at all, and one such term is enough to make the whole rule Evidence. The system records the evidence, computes the elapsed years, and presents the rest to a person. A system that scored the contribution would be converting a judgement into an arithmetic, which is the line §2.7 drew between eligibility as fact and adjudication.

DC-12 and DC-15 are the only two constraints in the Authority class, and they are the reason the platform stops where it does. DC-15 leaves the conduct of a ballot, its sealing, its counting and the declaration of the result with the Election Commission. The platform publishes the election documents and the timetable, records who is eligible under DC-03, and does nothing else. That is a deliberate refusal rather than an unimplemented feature: a system that also counted the votes would make the Association's most contested decision depend on the integrity of whoever holds the server credentials, which in this deployment is one unpaid volunteer. Section 8.8 states the same boundary as a security position, and §12.6 treats it as the substantive answer to RQ3.

DC-16 is the reason no page in this system names a constitution version. The reader always resolves the version in force, so a page cannot become stale by pointing at a superseded document, which was one of the observed failures of the Facebook-attachment era.

3.8 Feasibility Analysis and Requirements Validation

Table 3.7 summarises the four dimensions assessed below.

Technical. The maintainer already held working competence in the server platform, the web framework and the mobile framework selected, which was itself a selection criterion rather than a coincidence, and is recorded as such in §6.2. Nothing in the requirement set demands a capability outside a conventional transactional web system: the heaviest single computation is a paginated directory search, and the largest single artefact is a generated PDF. Feasible.

Economic. Development cost is the maintainer's unpaid time. Operating cost comprises a managed database, a container host, a domain name and outbound mail, for which §10.6 gives the model and the actual figures; the total is of the order of tens of dollars a year, against a commercial alternative whose annual licence exceeds the Association's annual income. The economic case is not marginal. Feasible.

Operational. The binding risk is not the software but the administrative workload the manual payment path creates, and the availability of officers to discharge it. Section 12.4 measures that workload rather than assuming it away, and §13.2 identifies it as the principal candidate for future reduction. Feasible with a stated and quantified reservation.

Schedule. Delivery was incremental with a working system after each increment, so schedule risk manifests as reduced scope rather than as no system, and the Could set of §3.6 is the declared shock absorber. Chapter 11 reports what was actually delivered against plan, including the two increments that slipped. Feasible.

Legal and ethical. The system holds national identity numbers, dates of birth, photographs and financial evidence for real people. Bangladesh's data protection statute was in draft at the time of writing, so the project cannot claim compliance with it and does not; instead §8.9 records the principles applied, being purpose limitation, minimisation, default non-disclosure and stated retention, and the front-matter declaration records the basis on which live data was handled during development. Use of the college's name and crest is governed by DC-01. Feasible, with the declarations made explicitly rather than implied.

Once judged feasible, the requirements were validated by four means, each aimed at a defect class the others miss.

Constitutional cross-check. Every DC entry was checked back against the clause it claims to derive from, and every article of the constitution was checked forward for software consequences not yet captured. The second direction is the one that found defects: two requirements, FR-27 on notice authorship and DC-11 on the exclusion of Advisory members from the quorum, exist because the forward pass found clauses the earlier analysis had classified as having no software consequence and which in fact had one.

Formal technical review in the sense Fagan established [48], conducted as a structured walkthrough of the specification with an officer of the Association acting as domain reviewer. Defect classes sought were ambiguity, omission, inconsistency, untestability and over-specification. One session was held, on 3 July 2026, working through the specification module by module against the implementation of the day. The record is the findings log kept in the repository at `docs/BUSINESS_FINDINGS.md`, supplemented by the consolidation report at `docs/BACKEND_REVIEW_2026-07-03.md`. The log has not grown since: its row count is unchanged in every commit that has touched the file after 3 July, which is itself the evidence that no later session added findings to it. Most entries belong to the implementation rather than to the specification, and are reported in that character in Chapter 11. Five are attributable to the specification itself, and those are the ones this section is answerable for.

Table 3.8 gives them by defect class. The pattern is worth naming: not one of the five was an outright error of fact. Four were places where the specification was silent on something a reader had to decide anyway, and the reader — the author, writing the code weeks later — decided differently each time. That is the failure mode a walkthrough catches and a reading does not.

Three outcomes changed the specification materially: the automatic termination requirement was reduced as recorded in §3.2; the membership number was moved from allocation at application to allocation at approval; and the directory default was inverted from disclose to withhold.

Testability audit. Each non-functional requirement was read with the single question of what measurement would falsify it. Four candidate requirements failed that test and were either restated measurably, which produced NFR-P3 and NFR-U3, or removed. Nothing survives in §3.4 that cannot be checked.

Prototype validation. The registration wizard and the payment declaration path were built early and put in front of members, on the reasoning that these are the two flows where a requirement error is most expensive and least likely to be caught by inspection. The staged-save behaviour in FR-01 and the plain-language failure requirement in FR-52 both come from that exercise rather than from the interviews.

One finding was recorded and deliberately not resolved. Entry P3-F2 observes that both clients enforce a ten-minute idle timeout while the API enforces none, so a captured token remains valid until it expires on its own schedule. The specification never stated an idle-expiry requirement, and adding one at this stage would have meant server-side session state that the stateless token design of §8.3 was chosen to avoid. It is carried as a known gap rather than closed, and it is the reason NFR-S5 is worded around credential change rather than around inactivity.

Residual risks carried forward: the participant sample is small and purposive, so the requirement set reflects the officers' view of the membership more strongly than the membership's own; the Association has not yet run an election under the new constitution, so the election-adjacent requirements FR-38 and FR-39 are validated

against documents rather than against practice; and the volume assumptions behind NFR-P1 and NFR-P5 are estimates, since the true membership size was unknown at specification time, which was itself one of the problems the project set out to solve.

3.9 Specification-Driven Development and the Spec Kit Artefacts

The requirements in §3.3 and §3.4 are written at the level an examiner or an officer of the Association reads. A second, finer specification sits beside them in the repository under `docs/specs/`, written with GitHub's Spec Kit [77]. Spec Kit makes the specification the main artefact of development. A feature starts as a written spec, is clarified, becomes a plan and a task list, and is then implemented, and the spec stays in the repository as the statement the code is checked against. This section records how the approach was used here, what it produced, and where the use falls short of the method.

The toolkit is installed at version 1.0.9.dev0 under `docs/specs/.specify`, with the commands specify, clarify, plan, tasks, analyze, checklist, implement, converge, constitution and taskstoissues. Its governing file is a specification constitution at `docs/specs/.specify/memory/constitution.md`. This is not the Association's Constitution v4.2 of §3.7, where the domain constraints come from. It is a short set of five engineering principles that every spec must respect: preserve the existing product contract, trace shared behaviour across all clients, keep boundaries clear, treat security and privacy as requirements, and verify with repository checks. The second and fifth principles shape the specs most. A requirement that touches shared data has to name its effect on the API, the web client and the mobile client, and a claim about behaviour has to point at a file or a test rather than at an intention.

There are twenty-one spec folders under twenty numbers. Number 010 was issued twice, which is tracked as TODO 84.31 rather than quietly renumbered. The folders fall into three groups. Spec 001 is the index: it keeps the cross-layer requirements and measurable outcomes, and points each user story at the spec that now owns it. Specs 002 to 011 each cover one feature or change, among them the idempotent member import (008), campaigns, scholarships and the archive (009), ad-hoc reporting (010) and the election module redesign (011). Specs 012 to 020 are as-built domain specs, one per domain, and between them they place each of the 45 API controllers in exactly one spec. Table 3.9 lists them.

Each domain spec follows the Spec Kit template: prioritised user stories with acceptance scenarios, edge cases, functional requirements in the "shall" form numbered locally from FR-001, key entities and measurable success criteria. Two sections are added to the template. An Evidence table maps each requirement to its controller, service, Angular route, Flutter screen and test. A Gaps section lists every requirement with no test, and any behaviour the code leaves unclear is marked `[NEEDS CLARIFICATION]` rather than guessed. The local FR numbers are not the FR-1 to FR-55 of Table 3.1. The book's catalogue states what the platform must do, and each subsection of §3.3 corresponds to one or more domain specs that state the same ground as requirements small enough to be tested one at a time.

The limit has to be stated plainly. Spec Kit was adopted late. The first specs were committed on 21 September 2026 and the domain specs on 26 September 2026, after most of the platform had been built. For most of the system the order the method prescribes was reversed: the code came first, and the spec was reverse-engineered from it in a fixed reading order of entities, DTOs and validators, services, controllers, clients and tests. The domain specs are therefore a record of what the platform does, not evidence that a specification drove its construction, and this dissertation does not claim the second. What the late adoption does give is the trace of §3.6 started from the other end. An Evidence row begins at a requirement and names the code and the test, and a Gaps entry makes an untested requirement visible instead of leaving it to be found. Writing the specs also found real defects. Seven were fixed and tested on 26 September 2026 under TODO 84.15, 84.24, 84.26, 84.30, 84.32, 84.33 and 84.34, among them a race that let a concurrent duplicate vote surface as a server error and a gallery upload that refused system administrators. An eighth, TODO 84.36, is still open: the FluentValidation validators are registered but never run on a request, so the rules they state are not enforced at the boundary.

3.10 Summary

Fifty-five functional requirements, thirty-seven non-functional requirements across the eight ISO/IEC 25010 characteristics, sixteen domain constraints traced to constitutional and electoral sources, and eleven quality-attribute scenarios carried forward for architectural evaluation. Thirty-eight requirements are Must, of which a substantial proportion are constitutionally mandated and were therefore not open to negotiation. Three stakeholder conflicts were resolved and recorded, in each case against the convenient option and in favour of the constitution or the member. The line the specification draws between what the software decides and what a human must decide, visible in DC-04, DC-07 and DC-15, is the specification's principal contribution to the answer to RQ3, and Chapter 5 turns it into modelled rules.

Beside that catalogue, nine as-built domain specs under `docs/specs/` restate the platform as 312 smaller requirements, each with its evidence or a recorded gap, as §3.9 describes. They were written after the code rather than before it, and are reported as a record of the built system.

Figures and Tables

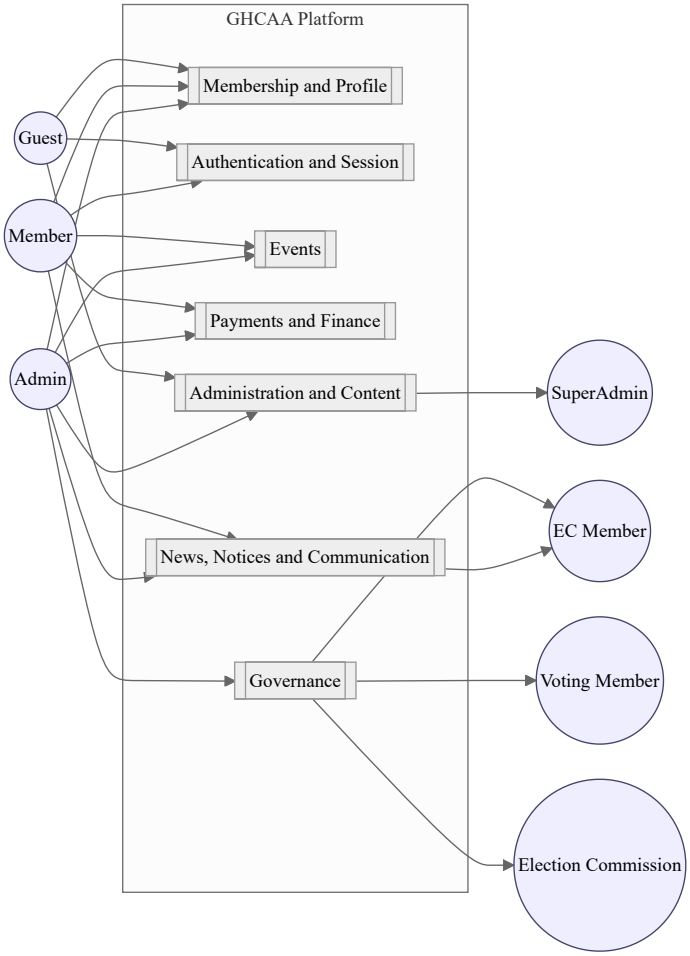


Fig. 3.1. System-level use-case diagram, packaged

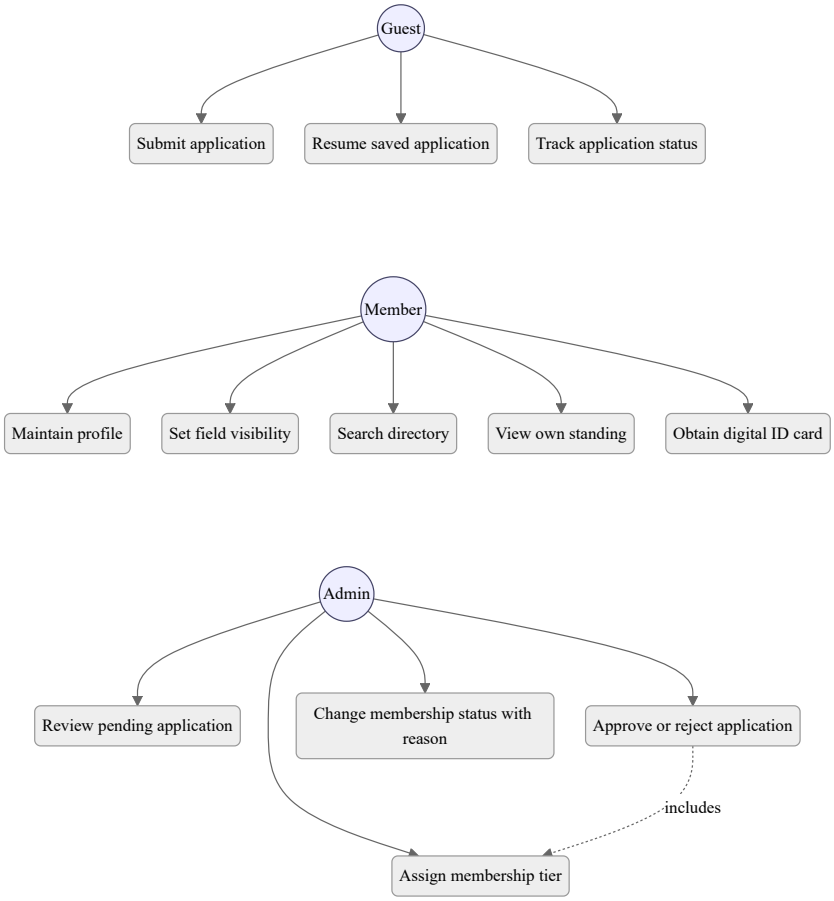


Fig. 3.2. Membership subsystem use cases

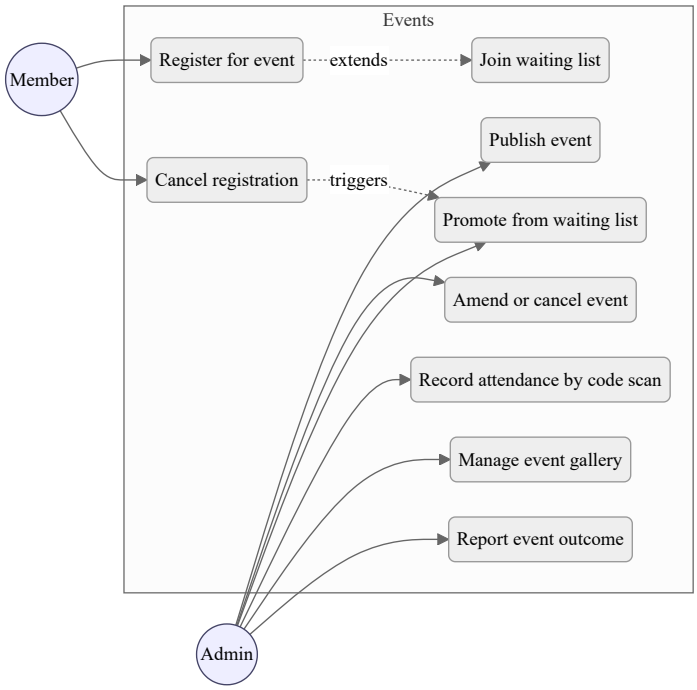


Fig. 3.3. Events subsystem use cases

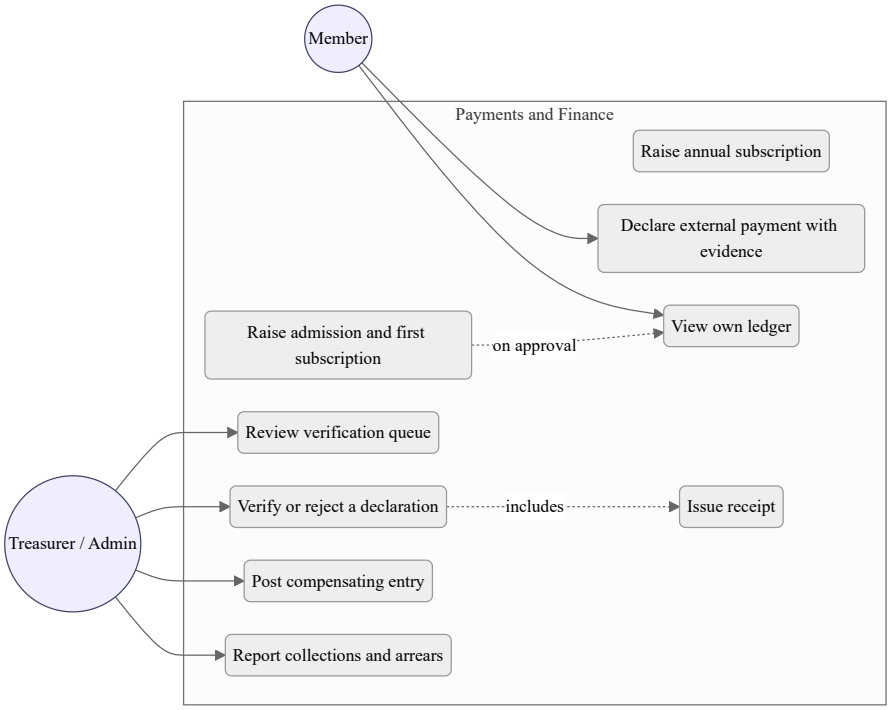


Fig. 3.4. Payments subsystem use cases

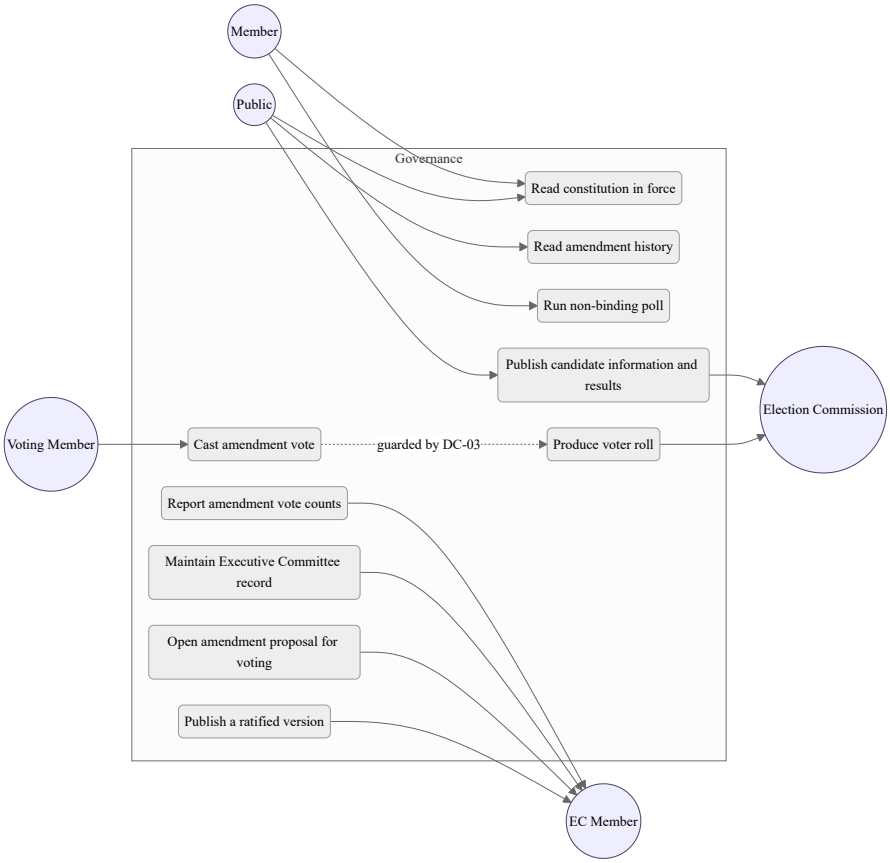


Fig. 3.5. Governance subsystem use cases

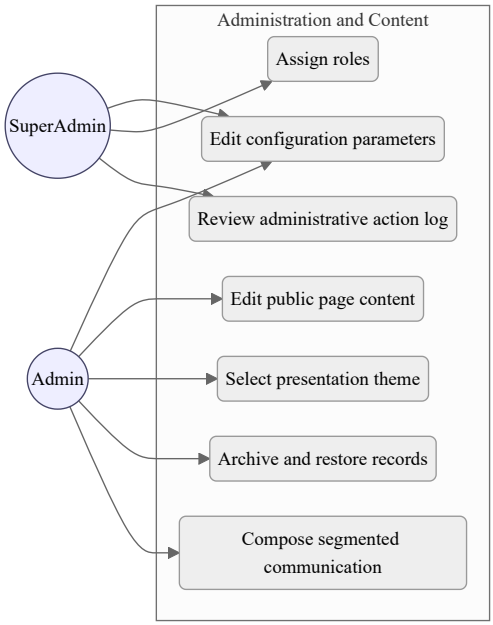


Fig. 3.6. Administration subsystem use cases

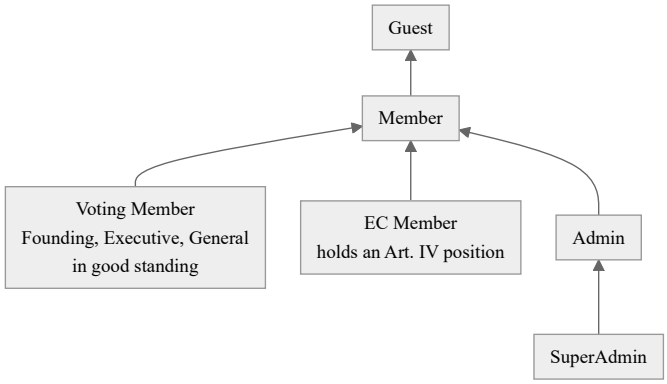


Fig. 3.7. Actor generalisation hierarchy

Administrative capability attaches to Admin by role assignment, not to EC Member by virtue of position. The two are drawn as separate specialisations of Member for exactly that reason.

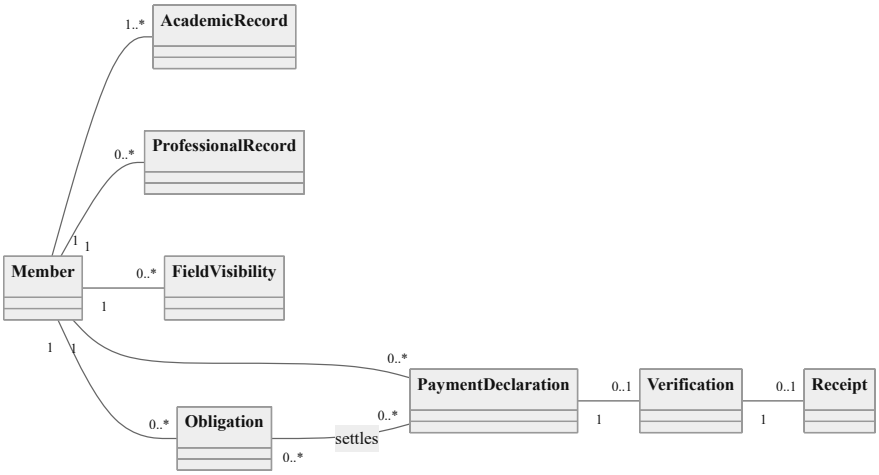


Fig. 3.8. Domain model, analysis level: membership, obligations and payment

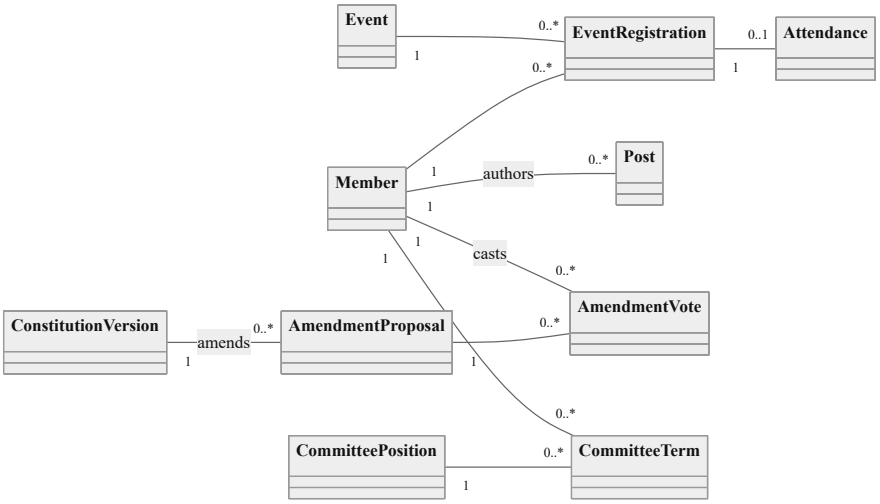


Fig. 3.9. Domain model, analysis level: participation, governance and content

Attributes are omitted from both halves so the classes and their multiplicities print legibly; they are given per class in the data dictionary of Table 6.2 and, for the classes that survive into the design model, in Figure 6.10.

This is an analysis model and deliberately omits identity, authentication, notification, configuration and content concerns, all of which appear in the design class diagram of Chapter 6. Two modelling decisions are worth stating. `Obligation` and `PaymentDeclaration` are separate classes related many to many, because a single payment may settle more than one obligation and an obligation may be settled in parts, and collapsing them, as the paper ledger effectively did, is what made arrears unanswerable. `AmendmentVote` carries `eligibilityBasis`, recording why the voter was admitted at the moment of voting, so that a later change in the member's standing cannot retroactively call the count into question.

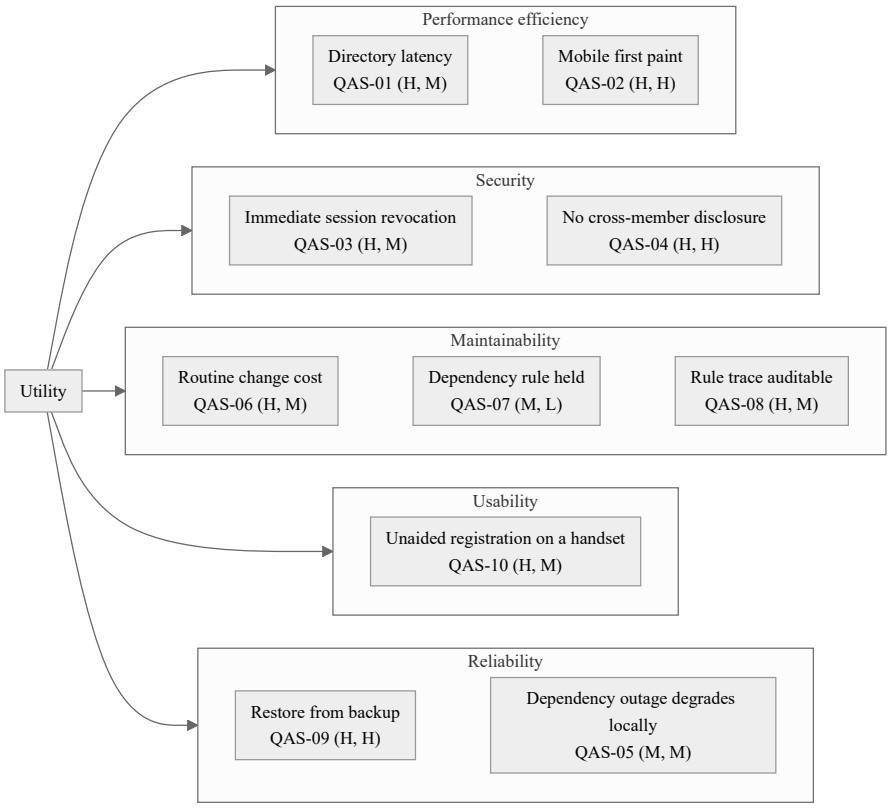


Fig. 3.10. Quality-attribute utility tree

Annotations are (business value, technical risk) on a high, medium, low scale, in the ATAM convention [34]. The four (H, H) and (H, M) items with highest combined weight, being QAS-02, QAS-04, QAS-08 and QAS-09, are the scenarios the architectural evaluation of §6.9 examines in detail.

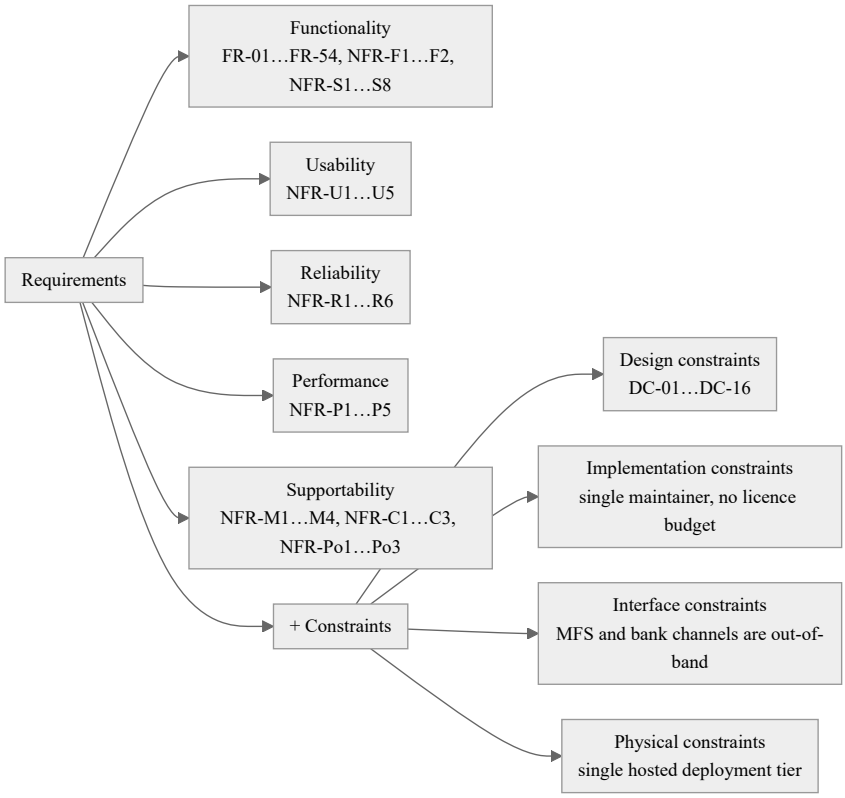


Fig. 3.11. Requirements classification, FURPS+

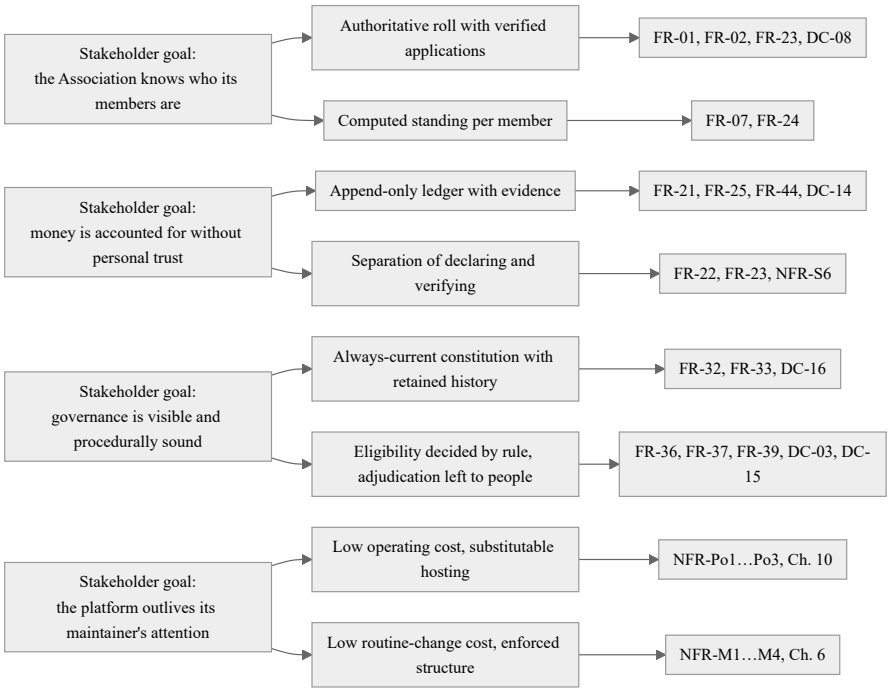


Fig. 3.12. Goal model

Table 3.1 Functional requirement catalogue

The catalogue is given in full in the tables of §3.3, which carry the ID, statement, source and priority for each of FR-01 to FR-55 grouped by subsystem, and is not repeated here. The RQ linkage is as follows: FR-01 to FR-31 and FR-41 to FR-55 answer RQ1 as the characterisation of the requirement set; FR-19 to FR-25 additionally bear on RQ2 through the manual-payment design position; FR-32 to FR-40 answer RQ3.

Table 3.2 Non-functional requirement catalogue

Given in full in §3.4, organised by ISO/IEC 25010 characteristic with the measurable acceptance criterion stated for each of the thirty-four entries.

Table 3.3 Use-case descriptions, ten highest-value cases

UC-01 Submit membership application — *Actor:* Guest. *Priority:* M. *Requirements:* FR-01, FR-02, FR-04. *Preconditions:* none. *Main flow:* the applicant opens the registration wizard; supplies personal particulars; supplies at least one college academic record; supplies at least one professional record; uploads a photograph; reviews a summary; submits. The system validates each step on advance, creates the application in status Applied, records the submission time, and acknowledges with a reference the applicant can use to track status. *Alternates:* the applicant saves and resumes later, with entered data retained. *Exceptions:* a duplicate national identity number, mobile number or email address is refused with the reason stated; a missing mandatory particular blocks advance from the step concerned; an oversized image is compressed or refused per FR-50. *Postconditions:* an application exists in Applied status and appears in the administrative queue of FR-23.

UC-02 Approve or reject an application — *Actor:* Admin. *Priority:* M. *Requirements:* FR-19, FR-23, DC-06, DC-08. *Preconditions:* an application in Applied status. *Main flow:* the administrator opens the queue, ordered by age with items older than thirty days flagged; examines the particulars and the uploaded evidence; assigns a membership tier; approves. The system allocates the next membership number in `GHC-[Year]-[Serial]` form, sets status Active, raises the admission fee and the first annual subscription as obligations, records the acting administrator and the time, and notifies the applicant. *Alternates:* rejection with a recorded reason and notification. *Exceptions:* a tier requiring a committee decision, per DC-06, cannot be assigned by a single administrator. *Postconditions:* an active member with a membership number and two outstanding obligations, or a rejected application, with the administrative act logged.

UC-03 Declare an external payment — *Actor:* Member. *Priority:* M. *Requirements:* FR-22, FR-21. *Preconditions:* an authenticated member with at least one outstanding obligation. *Main flow:* the member selects the obligation, sees the channel particulars and instructions drawn from configuration, makes the payment outside the platform, then records the amount, channel, date and transaction reference and uploads the evidence. The system records the declaration as unverified and places it in the verification queue. *Alternates:* declaration against more than one obligation. *Exceptions:* invalid or oversized evidence is refused with guidance; a lost connection does not discard entered data, per FR-52. *Postconditions:* an unverified declaration exists; the member's standing shows the payment as pending.

UC-04 Verify a payment declaration — *Actor:* Treasurer or Admin. *Priority:* M. *Requirements:* FR-21, FR-23, FR-25, DC-14. *Preconditions:* an unverified declaration. *Main flow:* the officer opens the queue, examines the evidence and the reference against the channel's own record, and verifies. The system settles the obligation to the extent of the amount, records the verifying officer and the time, generates a receipt, and notifies the member. *Alternates:* rejection with a reason, leaving the obligation outstanding. *Exceptions:* an attempt to amend or delete a verified record is refused; a correction requires a compensating entry. *Postconditions:* the obligation is settled or part-settled, a receipt exists, and the verification is attributable.

UC-05 Register for an event with a waiting list — *Actor:* Member. *Priority:* M. *Requirements:* FR-14, FR-15. *Preconditions:* a published event before its deadline. *Main flow:* the member registers; if capacity remains, the registration is confirmed; otherwise the member is placed at the tail of the waiting list with his position shown. *Alternates:* cancellation, which promotes the head of the waiting list and notifies that member. *Exceptions:* the deadline has passed; the member is already registered. *Postconditions:* the registration is confirmed or waitlisted, and the counts of FR-18 reflect it.

UC-06 Record attendance — *Actor:* Admin. *Priority:* S. *Requirements:* FR-16. *Preconditions:* an event in progress and a member holding a valid registration. *Main flow:* the administrator scans the member's identity code; the system resolves it to the member, confirms a valid registration for that event, records attendance and confirms. *Exceptions:* no registration for this event; already recorded; unresolvable code. *Postconditions:* attendance recorded once.

UC-07 Read the constitution in force — *Actor:* Public or Member. *Priority:* M. *Requirements:* FR-32, DC-16. *Preconditions:* at least one ratified version exists. *Main flow:* the reader opens the constitution page; the system resolves the version in force by effective date and presents it with its version identifier and effective date, together with a link to the retained history and to the authoritative document file. *Exceptions:* the document file is unavailable, in which case the stored text is presented and the condition is reported. *Postconditions:* none; the read is stateless and no page names a version.

UC-08 Open an amendment proposal for voting — *Actor:* EC Member. *Priority:* M. *Requirements:* FR-35, DC-13. *Preconditions:* a proposal originating from the Executive Committee or from a petition of twenty percent of voting members. *Main flow:* the officer records the proposal, its originating route and its circulation date, and sets the voting window. The system refuses a window opening less than fourteen days after circulation, and on opening makes the proposal visible to eligible voters. *Exceptions:* insufficient circulation period; an overlapping open proposal on the same article, which is warned rather than refused. *Postconditions:* an open proposal with a recorded circulation date and window.

UC-09 Cast an amendment vote — *Actor:* Voting Member. *Priority:* M. *Requirements:* FR-36, DC-03. *Preconditions:* an open proposal within its window. *Main flow:* the member opens the proposal; the system evaluates eligibility from tier and standing at that moment; an eligible member records for or against; the system stores the vote together with the basis on which eligibility was established. *Exceptions:* a non-voting tier is refused with the constitutional ground stated; a member in arrears is refused with the standing stated; a second vote by the same member is refused. *Postconditions:* exactly one vote per eligible member, each carrying its eligibility basis.

UC-10 Produce the voter roll — *Actor:* Election Commission. *Priority:* M. *Requirements:* FR-38, DC-03, DC-12, DC-15. *Preconditions:* an election has been called. *Main flow:* the Commission requests the roll as at a stated date; the system derives it from tier and standing, including for each entry the membership number, name, tier and the basis of inclusion, and produces it in a form that can be published and challenged. *Alternates:* a challenge is resolved by correcting the underlying membership or payment record and reissuing the roll, with both versions retained. *Exceptions:* none. *Postconditions:* a published roll; the ballot itself proceeds under the election documents and outside this system.

The remaining use cases are in `docs/SRS.md`.

Table 3.4
Requirements traceability matrix

The columns are requirement, constitutional source where applicable, use case, design element, implementation artefact and test case. A representative extract follows; the full matrix appears as Appendix B and is closed in §12.2. The first five columns are the analyst's reading of each requirement and are maintained by hand, with the consequence stated in §3.6. The Test column is regenerated from the `[Category("FR-NN")]` / `[Category("DC-NN")]` tags actually present in GHCAA.Tests by `docs/book/build/traceability.py`, whose `--check` mode fails the build if this table is edited out of step with those tags.

Req	DC / clause	Use case	Design element (Ch. 5–6)	Implementation artefact (Ch. 7)	Test (Ch. 9)
FR-01	—	UC-01	Registration sequence, §5.4	Registration endpoint and application service	RegisterAsync_WithValidData_ShouldCreateMemberAndPaymentHistory (+1 more)
FR-02	DC-08	UC-01, UC-02	Membership state machine, §5.3	Membership status transitions	GetStatusAsync_WithInvalidMemberId_ShouldThrowException (+2 more)

Req	DC / clause	Use case	Design element (Ch. 5–6)	Implementation artefact (Ch. 7)	Test (Ch. 9)
FR-03	—	UC-04 (profile)	Masking projection, §8.6	Field-visibility model and directory projection	GetProfileAsync_WithNonPrivilegedAccess_ShouldReturnMaskedProfile (+2 more)
FR-19	DC-14	UC-02	Approval flow, §5.4	Obligation raising on approval	no tagged test (gap)
FR-21	DC-14	UC-04	Ledger model, §5.3	Payment and verification records	RecordPaymentAsync_ShouldAddPaymentAndReturnDto
FR-25	DC-14	UC-04	Append-only rule, §5.5	Receipt generation; amendment refusal	DeletePaymentAsync_ShouldRemovePaymentAndLogActivity (+3 more)
FR-32	DC-16	UC-07	Version resolution, §5.5	Always-current reader	GetActiveConstitutionAsync_ReturnsLatestByEffectiveDate_NotInsertionOrder
FR-33	DC-16	UC-07	Supersede-not-delete rule, §5.5	Version synchronisation at start-up	ActivateConstitutionAsync_SupersedesPreviousVersion_WithoutDeletingIt
FR-36	DC-03	UC-09	Eligibility rule, §5.5	Voting eligibility guard	VoteOnConstitutionAsync_AcceptsVotingTierMember (+1 more)
FR-37	DC-13	UC-08, UC-09	Threshold reporting, §5.5	Count and threshold report	no tagged test (gap)
FR-44	DC-14	UC-02, UC-04	Archival deletion, §6.7	Global filter on archived records	DeletePaymentAsync_ShouldRemovePaymentAndLogActivity (+5 more)
NFR-S5	DC-07	UC-02	Session invalidation, §8.5	Security-stamp check per request	RotateRefreshToken_ReplayOfARotatedToken_RevokesWholeFamilyAndRotatesStamp (+1 more)

Table 3.5 MoSCoW prioritisation and negotiation outcome

Given in §3.6, with the count per priority band, the basis on which the band was assigned, the enumerated Won't set, and the two contested reclassifications recorded with their reasons. The three negotiated conflicts and their resolutions are in §3.2.

Table 3.6 Domain constraints, their class and their constitutional article

Given in §3.7, with the sixteen DC entries, each stating the constraint, its source article and section, and the requirement or design element by which it is enforced.

Table 3.7
Feasibility summary

Dimension	Principal finding	Verdict	Residual concern	Treated in
Technical	No requirement exceeds a conventional transactional web system; the maintainer already held the necessary platform competence	Feasible	None material	§6.2
Economic	Operating cost of the order of tens of dollars a year against a commercial licence exceeding the Association's annual income	Feasible	Cost is borne personally until the Association holds an account	§10.6
Operational	The binding constraint is officer time spent verifying payments, not the software	Feasible with reservation	Permanent administrative burden, quantified rather than assumed	§12.4, §13.2
Schedule	Incremental delivery, so schedule risk reduces scope rather than removing the system	Feasible	Two increments slipped	Ch. 11
Legal	Bangladesh's data protection statute was in draft; no compliance claim is made	Feasible	Obligations may change on enactment	§8.9
Ethical	Live personal data handled under a stated declaration; college identity used per DC-01	Feasible	Sole-maintainer access to production data is an unresolved structural risk	Front matter, §8.9, §13.2

Table 3.8
Specification defects found by the formal technical review

Log identifiers refer to the findings record at `docs/BUSINESS_FINDINGS.md`. Severity as recorded there. Classes follow the set stated in §3.8.

Log ID	Class	Defect in the specification	Disposition
COV-002	Inconsistency	FR-04 requires a Govt. Haraganga College record before submission but did not say at which layer the rule binds. The rule was implemented in the application service and omitted from the request validator, so a violating submission surfaced as an internal error rather than a rejected field	FR-04 restated to make the rule a submission precondition, binding at the boundary; validator gap logged as still open
COV-001	Omission	The same rule was specified for the profile-update path only. Nothing in the specification said whether it also governed the registration path, and no falsifying condition had been stated for that path	Scope of FR-04 made explicit across both paths; the missing test is recorded as an open coverage gap, not as a passing check
COV-003	Omission	FR-01's thirteen mandatory particulars were specified as a completeness rule, but the specification did not state what should happen on an attempt to approve an incomplete application	Approval precondition stated explicitly; negative case remains untested and is logged as such
COV-004	Omission	The registration fee was specified as payable before approval without stating the effect of its absence at the moment of approval	Same treatment as COV-003; gate specified, negative case logged as open
P3-F2	Omission	Idle-session expiry was implemented in both clients and in neither the specification nor the API. No requirement stated whether inactivity is a client concern or a server one	Not resolved. Carried as a known gap for the reason given in §3.8

Table 3.9
As-built domain specifications under docs/specs

Counts taken on 26 September 2026 from each `spec.md`: distinct FR identifiers, `### User Story` headings, and lines carrying a `NEEDS CLARIFICATION` marker.

Spec	Domain	User stories	Functional requirements	Open clarifications
012	Membership lifecycle and authentication	8	33	2
013	Profile, family and identity files	6	22	4
014	Events, news, gallery and content	5	40	2
015	Payments and finance	5	66	1
016	Networking and careers	3	28	1
017	Communication	6	31	3
018	Governance, elections and polls	5	40	2
019	Assistant	2	7	2
020	Platform and operations	7	45	3
	Total	47	312	20

PART II — METHOD AND DESIGN

Chapter 4 — Research Methodology

This chapter does two things a conventional methodology chapter often does not. It states the method, and it commits in advance to what will count as success, so that Chapter 12 has no room to choose its evidence after the fact. Everything measured later in this document is measured against a criterion fixed here.

4.1 Research Paradigm and Philosophical Position

The work sits in the pragmatist tradition rather than the positivist or interpretivist ones. The question that drove it was not "what is universally true of alumni platforms" but "what artefact, built and evaluated in this setting, resolves the problem stated in §1.3, and what does building it teach". Pragmatism measures an idea's worth by what it makes possible, which is the right test for a piece of software: a requirement, an architectural choice or a business rule is warranted here if it produces a working, checkable consequence, not because it follows from a prior theory of alumni engagement.

This has a direct consequence for how claims in this dissertation are read. A statement such as "the manual payment path is the correct design for this institution" is not offered as a general proposition about payment systems. It is offered as a proposition that held, for this Association, under the constraints of §1.7, and that is defended by the evidence Chapter 12 assembles. Where the literature review of Chapter 2 disagrees with a choice made here, §12.7 says so rather than quietly picking a side.

The researcher and the builder are the same person, which pragmatism does not treat as a defect to apologise for but as a fact to disclose and manage. §4.9 and §12.9 return to what that dual role costs the evaluation.

4.2 Design Science Research as the Governing Method

Design science research treats the artefact itself as the unit of contribution: a design science study must produce something built, not only something argued, and must evaluate what was built against stated criteria [4]. Hevner et al. organise this around three cycles that must all close before the work is complete [4].

The **relevance cycle** connects the environment, meaning the Association, its constitution and its current practice, to the research. It supplies the requirements of Chapter 3 and receives the artefact back for field testing; Figure 4.1 labels the three cycles with what each carried in this project, and Figure 4.2 shows the process as it was actually executed rather than as the model prescribes it; the formal technical review session of 3 July 2026, recorded in §3.8 with an officer of the Association acting as domain reviewer, is this cycle closing partway through the project rather than only at the end. The three implementation reviews in `docs/materials/IMPLEMENTATION_REVIEW_1.md` through `IMPLEMENTATION_REVIEW_3.md` supplement that session with the same cycle read at finer grain: each reconstructs one real instruction from `docs/TODO.md` against what was actually built, covering error handling and logging, the security audit, and the member-albums/job-approval feature together with a pair of minor UI fixes.

The **rigour cycle** connects the research to the existing knowledge base: the architecture, requirements-engineering, security and governance literature surveyed in Chapter 2, and the standards listed in the front matter. Every architectural and design choice in Chapter 6 is checked against this literature before it is treated as settled, and §6.2 records where the literature's default advice was rejected and why.

The **design cycle** is the inner loop of building and evaluating, repeated across the increments Chapter 7 reports and gated, in every increment, by the verification discipline described in §4.4. This is the cycle that ran most often. It is also the cycle for which the repository itself is the clearest evidence, since every one of the three hundred and seventy-nine commits between 9 February and 31 October 2026 is a turn of it.

4.3 Mapping Design Science Activities to the Work Performed

Peffers et al. resolve the design cycle into six activities, and the table below states, for each one, what was actually done and where it is reported [5]. The sixth activity, communication, is this document.

DSR activity	What was done	Evidence and chapter
Problem identification and motivation	Observation of the paper-based application, cash-and-ledger collection and Facebook-circulated constitution described in §1.2	§1.2, §1.3
Definition of objectives for a solution	The eight objectives of §1.6, each traced to a research question	§1.6
Design and development	Eighty-eight numbered work packages recorded as they were opened, most of them triggered by a stakeholder request rather than by the author's own plan; seventy-one mapped entities, forty-five API controllers and two client applications delivered across them	`docs/TODO.md`; Chs. 5–7
Demonstration	The artefact running against a seeded database, exercised in the formal technical review sessions of §3.8 and deployed to the pre-production environment of Chapter 10	§3.8, Ch. 10
Evaluation	Executed against the plan declared in §4.5	Ch. 9, Ch. 12
Communication	This dissertation, and the documentation corpus in `docs/` that a successor maintainer would read first	Whole document

The "design and development" row understates nothing by omission and nothing by exaggeration: the ninety-eight work packages are `docs/TODO.md`'s own numbering, several of them opened explicitly "raised by user" on a dated request, which is the clearest documentary evidence available that the relevance cycle kept running throughout construction rather than only at the requirements stage. This chapter and Chapter 11 count the same project by five different units — commit, tracker task, work package, WBS activity, feature — and a number that looks inconsistent between them usually means the unit changed, not the fact; §11.2, where the schedule and effort figures are built, defines all five once so neither chapter has to repeat the definition.

4.4 Software Process Model and its Justification

Three conventional alternatives were considered and rejected, each for a reason specific to this project's constraints rather than as a general criticism of the alternative. Figure 4.3 sets out the research design that resulted, phase by phase, with its inputs, outputs and evaluation points.

Waterfall requires a requirements specification to be frozen before design begins. Section 3.8 shows that the specification itself changed on formal technical review, after code already existed for the flows concerned; a waterfall commitment would have forced a choice between reopening a closed phase or shipping a specification known to be wrong. Given a single maintainer working unpaid and irregular hours, a long single-pass cycle also carries a schedule risk waterfall does not price well: any interruption of months, which the maintainer's other obligations made a real possibility, would have left no working system at all.

Spiral development [Boehm's risk-driven prototyping, 31] fits a setting with the analytic capacity to run a formal risk assessment before each cycle. That apparatus is disproportionate to a single-developer project; the risk identification that spiral development would have scheduled as a distinct activity happened here as an ordinary part of reading the constitution and testing the software, and is reported honestly as such in §4.7 rather than dressed up as a formal spiral.

Named agile methods such as Scrum presuppose a team, and most of their machinery, being sprint ceremonies, a product owner distinct from the developer, and story-point estimation for the purpose of team capacity planning, has no referent when the team is one person who is also the sole stakeholder for every technical decision. What survived from the agile literature was the principle behind it rather than its ceremonies: working software after every increment, and requirements that change in response to what the working software reveals [53], [54].

What was actually followed is incremental and evolutionary delivery in the sense Lehman describes software's continuing growth and increasing complexity as inherent to a live system rather than as a process failure [55]: ninety-eight work packages, opened as problems were found or requested, each closed against a gate rather than against a date. The gate is stated plainly in `docs/TODO.md`'s own verification standard: "no task marked done until its test passes", and the phased remediation plans in `docs/FORUM_PLAN_2026-05.md` end each phase with an explicit gate line, for example `dotnet test` (API), `flutter analyze` and `ng build` all green before Phase 1 of the July 2026 review remediation could be considered closed. That is a borrowed piece of the V-model, verification tied to the unit of work that produced the thing being verified, grafted onto an otherwise incremental lifecycle. Figure 4.4 draws the loop as it actually ran.

The rest of the project's software-engineering standards, being architectural conformance, coding convention and definition of done, are stated once in §3.5 and enforced at every increment rather than only at a phase boundary; this is what the incremental choice cost in process overhead and what it bought in resilience to interruption.

Specification-driven development in the Spec Kit sense came late to this lifecycle. The first spec was committed on 21 September 2026, when most of the platform already existed, so for most work packages the spec was written from the code rather than the code from the spec. §3.9, on the Spec Kit artefacts, states what that produced and why it is reported as a record of the built system rather than as the process that built it.

4.5 Evaluation Strategy and Metrics

This is the point of the chapter. Everything below is a commitment made before any measurement in Chapters 8 and 12 was taken, so that a result reported later can be checked against what was promised here rather than against a criterion chosen to fit the result. Table 4.3 collects the commitments in one place: instrument, what it measures, and where the result is reported.

Two of those commitments were later reduced, and the reduction is recorded here rather than removed. The performance commitment was written as a concurrent-load measurement and is delivered as a single-client latency measurement, for the reason given in §4.5.3 below, which is the shared pre-production tier. The security commitment was written as conformance against OWASP ASVS and is delivered as a self-assessment against it, for the reason given in §4.5.4 below, which is that no independent assessor was available. A third, mutation testing, was planned and is reported as not run in §9.12.5, the test-adequacy section. Editing this section to promise only what was achieved would have made the chapter tidier and would have destroyed the property that makes it worth writing, which is that a reader can hold the plan against the outcome. The gap between them is a result and is discussed in §12.9, threats to validity.

4.5.1 Functional evaluation

Requirement coverage is read off the traceability matrix established in §3.6 as Table 3.4. A requirement of priority Must is scored as covered only if it traces to a passing automated test; a requirement traced to code with no test is reported as uncovered, not as covered by inspection. The matrix is closed, meaning every requirement given a final verdict, in §12.2.

4.5.2 Quality evaluation

Quality is read against ISO/IEC 25010's characteristics [6], each evidenced by a static product metric or a test result defined in §4.5, computed over the delivered solution and reported per layer rather than as a single blended figure, because a blended figure would hide exactly the domain-versus-presentation distinction the risk-weighted coverage targets of §4.5 depend on.

4.5.3 Performance evaluation

The workload model is the one implied by the quality-attribute scenarios QAS-01 and QAS-02 of §3.4: concurrent reads against the directory and event listings, and a cold page load on a mid-range handset profile. What is measured against that model is less than the model describes, and the reduction is stated here rather than left for the reader to notice in Chapter 12.

Latency is measured from a single client against the pre-production deployment described in Chapter 10, which runs the same container image as production on a smaller instance tier, and is reported per endpoint at the ninety-fifth percentile over a fixed count of sequential requests. That is a latency measurement and not a load test. It cannot show how the platform behaves under the concurrent use QAS-01 assumes, §9.9, the performance-testing section, reports it under that limit, and §12.9, threats to validity, carries what follows from it. The reason is the deployment rather than the instrument: the pre-production tier is a shared instance whose throughput is set by the hosting plan, so a concurrency figure taken from it would be a measurement of the plan.

4.5.4 Security evaluation

Two instruments, run together rather than as alternatives. The threat model of Chapter 8, built by STRIDE walk-through of the data-flow diagrams of §5.2, gives coverage of *attack classes*. The OWASP ASVS level 2 checklist [7] gives coverage of *control families*. The edition used is 4.0.3 of 2021, which is the edition the security work of Chapter 8 was carried out against; ASVS 5.0.0 was released in May 2025 [75] and restructures the standard into seventeen chapters, so a conformance claim under 4.0.3 does not transfer to it and is not presented as though it did. Section 9.8 reports the testing and §8.11 the control-by-control assessment, and a control is scored conformant only where a specific code location or configuration enforces it, in the same style as the design-principle evidence table of §6.9.

The assessment is the author's own, carried out against the checklist by the person who wrote the code. It is not an audit, no independent assessor saw it, and no penetration test was commissioned. Self-assessment is weakest exactly where it matters most, on the controls whose absence the author never considered, so the claim made in §8.11 is conformance as assessed rather than conformance verified, and §12.9, threats to validity, records the difference.

4.5.5 Usability evaluation

Three instruments: task-based testing against the completion criterion of NFR-U2, the System Usability Scale against the published benchmark of 68 [33] per NFR-U3, and a heuristic walkthrough against Nielsen's heuristics [35] for the flows the task sessions do not reach.

The task script is not one script. The Association's work divides by office, so there are four: one for ordinary members covering registration, profile, payment declaration, event registration, the directory and the constitution; and one each for the Treasurer, the General Secretary and the President, built from what those offices actually do rather than from the platform's menus. Each session ends with the same SUS form and a set of open questions for the office, which is where the stakeholder evidence of §12.5 comes from. The instruments are delivered beside this dissertation, in `docs/book/instruments/`, so that the criterion can be checked against what participants saw.

Two properties of the design are stated here because they bound what Chapter 12 may claim. Sessions are run by the author, who is known to every participant and is the platform's sole maintainer, which is an acquiescence pressure the procedure reduces but does not remove. There is no control condition and no comparison system, so the study can report whether people completed a task and what obstructed them, and cannot report that the platform is better than any alternative.

4.5.6 Expert and stakeholder evaluation

This is not a separate activity invented for the evaluation chapter; it is the formal technical review already reported in §3.8, and its findings log at `docs/BUSINESS_FINDINGS.md` is read a second time in §12.5 for what it says about the delivered system rather than about the specification. The reviewer's role, an officer of the Association acting as domain reviewer rather than as a software professional, is unchanged between the two readings.

Table 4.1 gives each metric this strategy relies on, with its formula, the tool that computes it and the threshold it will be compared against, so that §9.12 and Chapter 12 apply rather than choose these numbers.

TABLE 4.1
Metric definitions

Metric	Formula / method	Tool	Target
Statement and branch coverage	Lines and branches exercised ÷ total, per test run	`coverlet.collector` 10.1.0 via `dotnet test`, per NUnit 4.2.2 project	Risk-weighted by module criticality; see §9.12.5
Mutation score	Mutants killed ÷ mutants generated	Planned, not run; §9.12.5 states why	Not reported
Cyclomatic complexity	McCabe's independent-path count per method [57]	Static analysis over the solution	Flagged above 10 per method
Coupling between objects (CBO), afferent/efferent coupling, instability	Chidamber and Kemerer's suite [56]; instability = efferent ÷ (afferent + efferent)	Static analysis over the solution	Plotted against Martin's main sequence, §9.12.3
Maintainability index	Oman and Hagemeister's composite of volume, complexity and comment ratio [58]	Static analysis over the solution	No fixed target; trended across the increments recorded in `docs/TODO.md`
Response latency	Wall-clock time from request to first byte, 95th percentile	Sequential requests from one client against pre-production, §9.9; not a load test	Per NFR-P1, NFR-P2, NFR-P5
System Usability Scale	Ten-item questionnaire, Brooke's scoring [33]	Form in `docs/book/instruments/`, one per participant after their tasks	≥ 68
ASVS conformance	Control satisfied / not satisfied / not applicable, by control family	Author's self-assessment against ASVS 4.0.3 [7]; not an audit	Full level 2 coverage of applicable families
Requirement coverage	Requirements traced to a passing test ÷ total Must requirements	Traceability matrix, Table 3.4	100% of Must

4.6 Data Collection and Analysis Procedures

Elicitation is reported here rather than in Chapter 3 because it is method: Chapter 3 states what the requirements are, and this section states how they were obtained and how each kind of data gathered during the project was analysed.

4.6.1 Elicitation techniques

Four techniques were used, in the order given, each chosen for what the previous one could not reach.

Document analysis came first and carried the most weight, which is unusual and is a consequence of the domain. The Association had already written down most of its rules before any software existed. The corpus analysed comprised the constitution at version 4.2, effective 1 July 2026, consisting of twelve articles; the seven documents of the election code, namely the election regulations, the operational manual, the code of conduct, the manual balloting procedure, the forms set, the ballot-box sealing and poll-integrity certificate, and the counting authorisation; and such records of prior practice as existed, principally the treasurer's ledger pages and the paper application forms. Analysis proceeded clause by clause. Each normative clause was classified as imposing a domain constraint, implying a functional requirement, implying a quality requirement, or having no software consequence. That pass produced the DC set of §3.7 and seeded roughly a third of the functional requirements directly.

Stakeholder interviews were used second, to establish current practice, pain and priority, since the documents state what should happen rather than what does. Interviews were semi-structured, with an instrument organised around the respondent's own tasks rather than around candidate features, on the reasoning that asking a volunteer treasurer what functions he wants tends to produce a description of whatever software he has last used.

Observation of current practice followed, because two of the most consequential findings could not have been obtained by asking. The first was that application forms were routinely accepted with the certificate field unfilled, on the strength of an officer recognising the applicant, which is the recognition gateway visible in Figure 2.3 and the origin of the thirty-day verification requirement in FR-23. The second was that the treasurer's ledger recorded receipts but not the identity of the receiving officer, so a discrepancy could not be attributed. That produced FR-21 and the separation-of-duty property in §8.7.

Competitor analysis, reported in §2.9, closed the set by surfacing capabilities nobody thought to ask for, of which the waitlist behaviour in FR-15 and the bulk communication segmentation in FR-29 are the two that survived prioritisation.

4.6.2 Participants, sampling and instruments

Sampling was purposive rather than representative. The population of people who hold operational knowledge of the Association's processes is small and largely identified by office, so the sampling frame was the office holders themselves plus a convenience sample of ordinary members drawn from those attending the reunion at which registration was being collected.

Ten people were interviewed. They included all three of the officers whose work the platform most directly changes, being the President, the Treasurer and the General Secretary, with the remainder ordinary members. Participants are not named here. They are identifiable members of a small association, they agreed to take part verbally rather than in writing, and naming them would put identifiable personal information into a public document to no analytical purpose; §4.8 states the consent position in full.

The author is one of the ten. He is also the Association's Information and Technology Secretary and the platform's sole maintainer, so his own account of current practice is not independent evidence, and it is not treated as such: every requirement traced to interview in the Source column of §3.3 was stated by at least one participant other than the author.

Sessions ran thirty to forty minutes each. Recruitment was direct: every participant was already a member of the Association, and each was approached as one member to another rather than through any formal call for volunteers. That is the recruitment route a single-maintainer study in a small association actually has, and it carries the selection bias that goes with it, since the people easiest to approach are the people already engaged.

The period over which the sessions ran was not recorded, and is not reconstructed here. No participant-derived figure is quoted anywhere in this dissertation, so nothing downstream depends on it; saying that it was not recorded is preferable to producing a date range from memory. The interview guides per role are held with the evaluation instruments delivered beside this dissertation, not bound into it.

Instruments used: an interview guide per role, with an opening account of current task flow, a probe set on failure and workaround, and a closing prioritisation exercise; an observation checklist covering the application, payment and recording steps; and a document-analysis coding sheet mapping each constitutional clause to its classification and to the identifier it produced.

4.6.3 Analysis procedures

Four kinds of data are collected, and each has a different analysis treatment. Automated test results and coverage reports are quantitative and are aggregated directly; no sampling is involved since the population is the whole test run. Static-analysis output, being complexity and coupling figures, is quantitative and is analysed by threshold and by distribution, per §9.12.2 and §9.12.3, rather than by a single mean that would hide the worst offenders the maintainability argument actually depends on. Formal-technical-review findings, held in `docs/BUSINESS_FINDINGS.md`, are qualitative records classified by module, layer and severity as the log itself already tags them; §3.8 and §12.5 read that classification rather than re-coding it, so that the analysis is traceable to the artefact rather than to a scheme invented after the fact. Usability data, being task completion, time on task and SUS responses, is quantitative on a small purposive sample and is reported descriptively, with the sample size stated alongside every figure so that no percentage is read as more precise than the sample supports.

4.7 Risk Management: the RMMM Plan

The structure of this section, being risk identification, projection by probability and impact, mitigation, monitoring and management, follows Pressman and Maxim's RMMM plan [54], and Table 4.2 uses their columns.

Risk was not managed through a separate formal apparatus running alongside development. It was managed through the same mechanism that managed everything else: a numbered, dated work item in `docs/TODO.md`, closed against a test. That is disclosed here rather than dressed up, and Table 4.2 is built from the actual severity classifications the project used at the time, not reconstructed after the fact to look tidier than the record. Figure 4.5 plots the same risks by probability against impact. The two the project scored at high probability, date-format drift and stale schema and seed data, are also the two that went on to happen.

TABLE 4.2
RMMM table

Risk	Category	Probability	Impact	Mitigation	Monitoring signal	Outcome
Single maintainer unavailable for an extended period	Project	Medium	High	Documentation corpus (`PROJECT_MAP.md`, `TODO.md`, `ARCHITECTURE.md`) written to let a successor start without the author present	Elapsed time since last commit	Open; mitigated, not eliminated
Payment amount not verified against the originating record before crediting membership (TODO 29-B.2)	Technical, security	Medium	High	Removed the `amount > 0` short-circuit; callback amount always compared to the originating `PaymentHistory` amount	Payment and approval integration tests	Closed, Phase 2 of the July 2026 remediation
Administrative action attributed to a hardcoded admin identifier rather than the acting user (TODO 29-F.1)	Technical, audit integrity	Medium	High	Acting admin read from the JWT `MemberId` claim on every approval and rejection path	Code review; audit-log spot check	Closed, Phase 2
Event capacity exceeded under concurrent registration (TODO 29-A.4)	Technical	Medium	Medium	Occupying-status count corrected to include all statuses that hold a place, cap enforced even when waitlisting is off, insert guarded against concurrent overfill	Concurrent-registration test	Closed, Phase 1
Date format inconsistency between `dd-MM-yyyy` display and ISO-8601 wire format risking silent data corruption across clients (TODO Work Package 23, 29-F.3)	Technical	High (had already caused defects)	High	ISO-8601 fixed as the canonical wire format; `DateFormatConverter` and client parsers reconciled to it	`DateFormatConverterTests.cs`, 20 pinning tests	Closed and test-pinned, 2026-08-22

Risk	Category	Probability	Impact	Mitigation	Monitoring signal	Outcome
`Database.EnsureCreated()` no-op on a non-empty database leaving seeded data, and later the schema itself, stale after a change	Technical	High	High	Two mechanisms, added a year apart: `ConstitutionSeeder.SyncAsync` at boot for revisable data, and `MigrationBootstrapper.EnsureMigratedAsync` for the schema once the same no-op was found to have withheld twenty-one migrations from preprod	Boot log; constitution version shown in the public reader; HTTP 500 rate on newly shipped endpoints	Closed twice, §6.5.6. The impact rating was raised from Medium to High after 27 August 2026, when the schema half surfaced as 500s from `/api/jobs` and `/api/gallery`
Manual payment verification backlog exceeding officer capacity as membership grows	Operational	Medium	Medium	Administrative queue ordered by age with a thirty-day flag (FR-23, DC-08); workload quantified rather than assumed away	Age of oldest unverified item in the queue	Open; monitored, not solved, §12.4
Volunteer officer turnover losing institutional knowledge of platform operation	Organisational	Medium	Medium	Administration console designed to be operable without developer involvement (NFR-M4); documentation corpus	Handover interval, three-year committee term (DC-09)	Open; structural mitigation only

Two things about this table are worth stating plainly. First, every closed row closed because a test was written that would fail if the defect recurred, which is the definition-of-done requirement of §3.5 applied to risk rather than to features. Second, the three rows left open are left open honestly: none of them has a technical fix waiting to be applied, and each is a property of the Association's size and volunteer structure rather than of the software.

4.8 Research Ethics

Two distinct ethical questions arise in this project, and they are kept separate rather than answered by one paragraph that quietly covers both.

The first concerns the people who took part in the elicitation reported in §4.6. That study was not reviewed by a university ethics committee. Permission to conduct it, to use the Association's records and governing documents, and to work with its members was given by the Government Haraganga College Alumni Association itself. No written approval was issued, no reference number exists, and no date of approval was recorded. That is stated rather than presented as an equivalent to institutional review, because it is not one.

Consent from interview participants was verbal and was not documented. Participation was voluntary and unpaid, and participants were told what the material would be used for. No written consent form was signed, no consent record was retained, and no participant is named in this dissertation or its appendices. Consent from the members whose real records the live system holds was obtained the same way, verbally, at Association gatherings, with no written record and no formal notice.

Three consequences follow, and they are limitations of this work rather than features of it. A participant who wished to withdraw consent has no documented statement to withdraw. Because consent was never recorded, its scope cannot be demonstrated to a reader, only asserted by the author. And a future study reusing this material could not establish that the people concerned had agreed to that reuse. §4.9 carries this forward as a limitation of the method, and the recommendation in §13.2 that the Association adopt a written consent line in its registration form comes directly from this gap.

The second concerns the personal data of the Association's real members, which the platform holds in production and which the maintainer necessarily saw during development and testing. That question is answered, not deferred, in the front matter's Ethics Statement and Data-Protection Declaration and expanded in §8.9: the principles applied are purpose limitation, minimisation, default non-disclosure and a stated retention position, applied because Bangladesh's data-protection statute was in draft rather than in force at the time of writing, and because no third-party ethics body governs a single volunteer's handling of his own association's data. No production member data appears anywhere in this dissertation; every figure, screenshot and test fixture quoted from this point forward is synthesised. In the running system that data sits behind the access controls of §8.4.

4.9 Limitations of the Chosen Method

The dual role of researcher and sole developer is the limitation that most affects how the rest of this document should be read. Every finding in Chapter 9, and every judgement in §12.7 about whether this project's answer to RQ3 is the right one, was reached by the same person who wrote the code being judged. The formal technical review of §3.8 is the one point at which an independent party, an officer of the Association, checked the work against something other than the author's own standard, and its five specification defects and the pattern among them, that a walkthrough catches what a re-reading does not, are reported for that reason rather than suppressed as evidence the author's judgement was imperfect.

The incremental process, chosen for the reasons of §4.4, also means the requirement set of Chapter 3 and the architecture of Chapter 6 were not fixed before construction began in the sense a waterfall study would fix them; the version presented in this dissertation is the state reached by 28 August 2026, and §11.8 records the changes of scope that occurred on the way there rather than presenting the final state as though it had been the plan from the start.

Finally, this is a single-case design science study in Runeson and Höst's sense [10]. The evaluation plan of §4.5 is thorough for this case, but generalising its results to another association requires the argument of §12.11 about what transfers, not an assumption that it transfers automatically.

4.10 Summary

The method is design science research, run as Hevner's three cycles and Peffers' six activities, delivered through eighty-two incrementally opened work packages gated by an automated test rather than by a calendar date. The evaluation plan is fixed in this chapter across six dimensions, each with a named metric, tool and threshold, before Chapter 9 measures anything. Risk was managed through the same gated work-item mechanism as everything else, and the RMMM table of §4.7 is built from the project's own severity record rather than reconstructed for presentation. Two ethical questions are kept distinct, one of them still open and recorded as such. Chapter 5 now turns from method to the system itself, modelling its behaviour and extracting the business rules the constitution imposes.

Figures and Tables

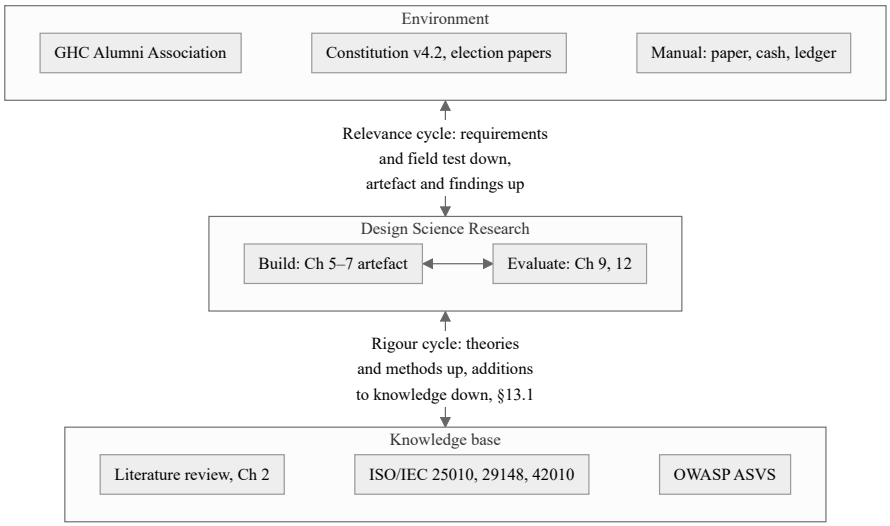


Fig. 4.1. Design Science Research framework with this project's instantiation labelled

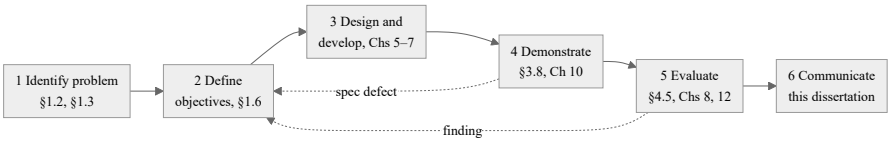


Fig. 4.2. Design Science process model as executed

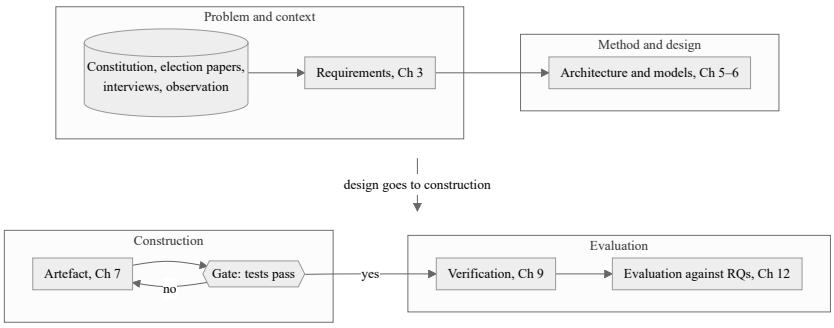


Fig. 4.3. Research design overview: phases, inputs, outputs, evaluation points

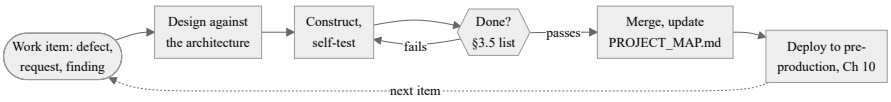


Fig. 4.4. Process model diagram of the adopted incremental lifecycle

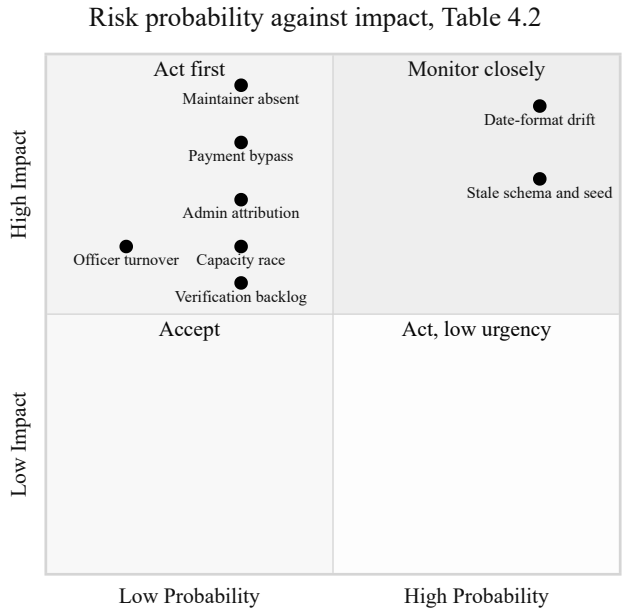


Fig. 4.5. Risk exposure matrix

TABLE 4.3
Evaluation plan

Research question	Criterion	Metric	Instrument	Threshold
RQ1	Requirement set fit for the domain	Requirement coverage; FTR defect count	Traceability matrix, Table 3.4; findings log	100% of Must traced; FTR defects resolved or recorded
RQ2	Architecture satisfies quality-attribute scenarios at bounded cost	Coupling, cohesion, maintainability index; operating cost	Static analysis, §9.12; cost model, §10.6	Thresholds of Table 4.1; cost within Association's stated means
RQ3	Governance rules encoded without loss of procedural legitimacy	DC-to-code trace completeness; ASVS conformance on governance endpoints	Table 3.4 DC column; §8.11	Every DC of priority M traced and enforced
RQ4	Measured quality against ISO/IEC 25010	All metrics of Table 4.1	As listed	As listed

Chapter 5 — System Analysis and Behavioural Modelling

Chapter 3 said what the system must do and what it must not violate. This chapter says how those statements turn into a model that can be built from. Two notations are used side by side rather than one being chosen over the other, and §5.1 explains why. The chapter's centre of gravity is §5.5: the business rules catalogue is where the constitution's clauses stop being prose and become named, locatable checks in the code, and it is the artefact Chapter 9 tests against and Chapter 8 audits.

5.1 Analysis Approach

Structured analysis, meaning data-flow diagrams and process specifications, answers a question object-oriented analysis answers badly: what moves through the system and where it is stored. A data-flow diagram of the payment process is the clearest way to show a member's declaration and its evidence arriving, sitting in a store, and leaving as a verified ledger entry, and §5.2 uses it for exactly that.

Object-oriented analysis answers a different question well: what the persistent things in the system are, what they are responsible for, and how they collaborate. A CRC card for `Member` or `GovernanceService` says something a data-flow diagram cannot, namely who else has to be involved before an operation can complete. The two views are kept complementary rather than merged into one notation, because the literature on this point is itself divided [46], and forcing a single diagram to answer both questions tends to produce a diagram that answers neither well.

A third view, state modelling, is needed because three of this system's central objects, being the member, the payment, and the constitution, are long-lived and pass through official states that the constitution itself names. Neither a data-flow diagram nor a class diagram makes a state machine explicit, so §5.4 draws it separately.

5.2 Structured Analysis: Data-Flow Modelling

5.2.1 Context level

Figure 5.1 is the same boundary as Figure 1.1, redrawn here because this chapter, not Chapter 1, is where the boundary starts to be decomposed.

5.2.2 Level 1 decomposition

Figure 5.2 decomposes the platform into seven processes, each corresponding to a subsystem named in the use-case model of §3.5, and four persistent data stores. The stores are drawn at the level a data-flow diagram uses them, which is coarser than the seventy-one mapped entities of §5.6; D1 Member Records, for instance, stands for the member, academic-record, professional-record and membership-history tables together, because at this level of analysis they are read and written as a unit.

5.2.3 Level 2 decompositions of the critical processes

Three processes are decomposed to Level 2 here: payment processing in Figure 5.3, membership approval in Figure 5.4, and constitution publication in Figure 5.5. These were chosen because each hides a decision inside it that the Level 1 diagram cannot show and that later chapters depend on. Authentication and OTP verification is represented instead as the sequence diagram of Figure 5.15, and the election-administration process as the BPMN diagram of Figure 5.10, because in both cases the property that matters is the ordering and timing of an interaction across roles rather than the transformation of data, which a data-flow diagram is not the right notation to carry. Table 5.3 gives the process specification for each numbered process in those three diagrams, and Table 5.4 the data stores they read and write.

5.3 Object-Oriented and Behavioural Analysis

5.3.1 Noun and verb analysis

Reading the requirement statements of §3.3 as sentences rather than as a specification produces a short list of recurring nouns: member, application, payment, event, post, constitution, committee, vote. Each survived as a persistence-bearing analysis class. Verbs attached to those nouns, being apply, approve, declare, verify, register, publish, vote and supersede, became the operations attached to the services of §5.3.2 rather than to the domain objects themselves in every case; a `Member` does not approve itself, an administrator does, acting through `MemberService`. That division, of state living on the domain object and the decision to change it living on a service, is the reason the design pattern discussion of §6.9.16 calls the model anaemic rather than rich, and §6.9.19 records that this was noticed and deliberately not corrected everywhere, because a richer domain model would have moved authorisation logic out of the layer where the API can enforce it uniformly.

5.3.2 CRC Modelling

Table 5.2 gives the CRC card set for the analysis classes with the widest collaboration surface. `Member` collaborates with almost everything, which is expected of the entity the whole domain orbits; `GovernanceService`'s collaborator list is the direct analysis-level evidence for the DC-03 enforcement discussed in §5.5.

5.3.3 Analysis class relationships

The relationships among these classes, at the level appropriate to analysis rather than to design, are drawn in Figure 3.8 already and are not repeated here; §6.3 and Figure 6.9 give the design-level version once persistence and DTO boundaries are added.

The behaviour is modelled in three activity diagrams, for the flows where a decision is taken by a person rather than by code: registration and administrative approval in Figure 5.6, payment declaration and verification in Figure 5.7, and event registration with its waitlist in Figure 5.8. Figure 5.9 redraws the amendment vote as a swimlane diagram, because there the interesting property is which role may act at which point rather than the sequence itself, and Figure 5.10 gives the election cycle in BPMN, whose audience is the Association's officers rather than engineers. Four sequence diagrams follow: login with one-time password and token issue in Figure 5.15, event registration in Figure 5.16, payment declaration and verification in Figure 5.17, and real-time notification over SignalR in Figure 5.18. Figure 5.19 draws token lifetime and the refresh window to scale. Each is discussed at the point it is first needed in §5.4 and §5.5 rather than in a separate narrative here, so that a diagram sits next to the rule it illustrates.

5.4 State Modelling of Long-Lived Entities

Four state machines are drawn, and all four are read directly from an enumeration already defined in the code rather than invented for the diagram; `Enums.cs` is authoritative and the diagrams follow it, not the reverse.

Member (Figure 5.11) moves through `MembershipStatus`: `Applied`, `Active`, `InactivePayment`, `InactiveResigned`, `Terminated`, `Rejected`. Every transition away from `Applied` or `Active` is guarded, and DC-07's disciplinary procedure means the transition to `Terminated` carries a reason recorded in `MembershipHistory`, whose `ChangedFrom`, `ChangedTo`, `ChangedByAdminId` and `Reason` fields exist for exactly this purpose. There is no transition directly from `Applied` to `Terminated` in the enumeration; a rejected application is `Rejected`, a discontinued membership is `Terminated`, and the two are kept distinct because DC-07's appeal right attaches to the second and not obviously to the first.

Event lifecycle and registration (Figure 5.14 draws the event's own states; the registration states below are modelled alongside because the transition rule is identical in shape) moves through `EventRegistrationStatus`: `Pending`, `Approved`, `Rejected`, `Waitlisted`. The waitlist promotion this diagram implies is FIFO by registration time, read directly from `EventService`'s `.OrderBy(r => r.RegisteredAt)` when a place is released, which is FR-15 as executable code rather than as a description of intended behaviour.

Constitution (Figure 5.13) has only two states in the persisted model, `IsActive = true` and `IsActive = false` with a `SupersededDate` set, but the invariant DC-16 depends on, that exactly one row is active at any time, is not a state a diagram can show by itself; it is enforced procedurally by `ConstitutionSeeder.SyncAsync` at boot, which supersedes rather than deletes a prior version so that its `AmendmentVote` rows survive. The diagram is drawn with three labelled states, Draft, Active and Superseded, because that is the vocabulary DC-16 and FR-33 use, and the mapping from that vocabulary to the two-column persisted representation is stated in the diagram's note rather than left implicit.

Payment moves through `PaymentStatus`: `Pending`, `Completed`, `Failed`, `Refunded`, which is the gateway-facing state; the member-facing declaration described in FR-22 is a separate, simpler unverified-to-verified transition recorded on `PaymentHistory` rather than on this enumeration, and Figure 5.12 draws both because the two are easy to conflate and the ledger rule of FR-25 depends on keeping them apart: an unverified declaration is not yet a `FinancialRecord`, and once a `FinancialRecord` exists it is not edited, only corrected by a compensating entry.

5.5 Business Rules Catalogue

Table 5.1 is the same sixteen domain constraints Table 3.6 stated, restated here with the actual enforcement location rather than the requirement identifier, because a rule catalogue that a reviewer cannot use to find the code is not doing the job the honesty rule in the front matter requires.

TABLE 5.1
Business rules catalogue

Rule	Statement	Source	Enforcement point
BR-01	Only Founding, Executive and General members may vote on an amendment	Art. III §K	`GovernanceService.VoteOnConstitutionAsync`, membership-type check
BR-02	A member may cast at most one vote per constitution version	Art. VIII	`GovernanceService.VoteOnConstitutionAsync`, `AmendmentVotes.AnyAsync` prior-vote check
BR-03	A notice is an official act and may not be created by a member submission	Art. IV	`NewsController`, member `submit` action returns `Forbid()` when `PostType == Notice`
BR-04	The constitution in force is always the latest ratified version; no page names a version	Art. VIII	`GovernanceService.GetActiveConstitutionAsync` reads the single `IsActive` row; `ConstitutionSeeder.SyncAsync` maintains the invariant at boot
BR-05	A superseded constitution version is retained, not deleted, so votes tied to it survive	Art. VIII	`ConstitutionSeeder.SyncAsync` supersedes by setting `SupersededDate`, never deletes a version with votes attached
BR-06	Membership status changes to Terminated require a recorded reason	Art. X	`MembershipHistory.ChangedFrom` / `ChangedTo` / `Reason`, written by `MemberService` alongside every status change
BR-07	A verified financial record cannot be amended or deleted; correction is by compensating entry	Art. VI	`FinancialRecord` exposed through no update or delete endpoint; `FinancialLedgerController` is append-only
BR-08	Event capacity is enforced across every status that occupies a place, not only Approved	Constitution silent; officer requirement, §3.2	`EventService`, occupying-status count corrected under TODO 29-A.4
BR-09	Waitlist promotion is first-in, first-out by registration time	Officer requirement, FR-15	`EventService`, `.OrderBy(r => r.RegisteredAt)`
BR-10	A public directory entry discloses a field only if the owning member has opted in	Officer/member negotiation, §3.2	`MemberService` masking projection; unmasked only through privileged `admin/members/{id}`
BR-11	A change of credentials, role or status invalidates every outstanding session immediately	Design position, §8.5	`SecurityStampMiddleware`
BR-12	Every mutating request is attributed to the acting user, not to a default identifier	Audit integrity, TODO 29-F.1	Acting admin id read from the JWT `MemberId` claim in every approval and rejection path
BR-13	A payment declaration is not credited until the callback or officer-verified amount matches the originating record	Financial integrity, TODO 29-B.2	Amount comparison against the originating `PaymentHistory` row before crediting
BR-14	Advisory members are excluded from the Executive Committee quorum computation	Art. IV, Art. III	Formal-technical-review finding, §3.8; enforcement recorded against `GovernanceService`'s committee-membership query
BR-15	Wire-format dates are ISO-8601; `dd-MM-yyyy` is display and input only	Data-integrity finding, TODO Work Package 23	`DateFormatConverter`; pinned by twenty tests in `DateFormatConverterTests.cs`
BR-16	The Association's name, crest and motto are used only as the constitution prescribes	Art. I §5	Public site content sourced from `OrgConfig.branding`, not hardcoded per environment

One rule in this table is enforced less completely than its source requirement states, and it is recorded here rather than silently corrected in the retelling. FR-36 requires a vote to be admitted only from "a voting member in good standing". `GovernanceService.VoteOnConstitutionAsync` checks `MembershipType` against the three eligible tiers and checks for a prior vote; it does not check arrears status before admitting the vote. A voting member with an outstanding subscription can therefore still vote on an amendment, which BR-01 as coded does not prevent. This is the same class of finding as the idle-timeout gap recorded in §3.8: a requirement the specification stated correctly and the implementation only partially closed. It is carried forward to §9.7 as an open item rather than corrected in this chapter, since correcting the prose here would misrepresent what the shipped code actually checks.

5.6 Data Modelling and Analysis Model Review

5.6.1 Conceptual to logical progression

The conceptual model is the domain class diagram of Figure 3.8; the logical model is the seventy-two `DbSet` properties on `ApplicationDbContext`, one per mapped entity, defined at analysis level in Table 5.4, which Chapter 6's data design and entity-relationship diagram take as their starting point. Nothing in that progression introduces a table that does not correspond to an analysis-level noun from §5.3.1; the nearest exceptions are the join and history tables, being `AmendmentVote`, `MembershipHistory`, `EventRegistration` and similar, which are relationships promoted to entities because they carry their own attributes, principally a timestamp and, in three cases, a reason.

5.6.2 Multiplicities worth stating explicitly

A `Member` has exactly one `User` account, zero or more `AcademicRecord` and `ProfessionalRecord` entries with at least one academic record required by FR-04, zero or more `ECMember` appointments across different `ECPeriod` rows, and zero or more `PaymentHistory` and `EventRegistration` entries. A `Constitution` has zero or more `AmendmentVote` rows, and exactly one `Constitution` row across the whole table has `IsActive = true`, which is an invariant enforced by procedure rather than by a database constraint, a decision revisited critically in §6.5.2.

The analysis model was checked the same way the requirement set was checked in §3.8: against the constitution clause by clause, and against the shipped code rule by rule. The clause-by-clause pass is the source of BR-14 above, an enforcement gap the specification review had already surfaced. The rule-by-rule pass against the code is the source of the FR-36 gap recorded in §5.5, which the specification review did not surface because §3.8 checked the specification's wording, not the implementation's behaviour; finding it required reading `GovernanceService` directly. Both gaps are carried forward rather than resolved in this document, and both are named again in §9.7 and §8.11 so that a reader working from either chapter alone still encounters them.

5.7 Summary

Structured and object-oriented analysis were used side by side because they answer different questions about the same system. Four state machines were drawn directly from the code's own enumerations rather than from a separate design exercise, and the business rules catalogue of §5.5 gives sixteen constitutional and operational rules a named location in the code, with one honestly reported as only partly enforced. Chapter 6 now takes this analysis model forward into the architecture and design that realise it.

Figures and Tables

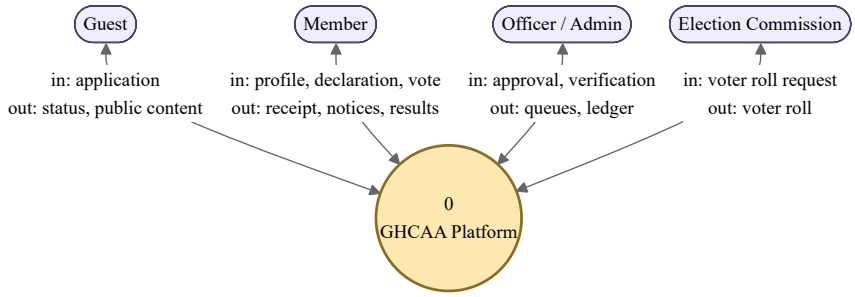


Fig. 5.1. DFD Level 0 (context). In and out are relative to the platform

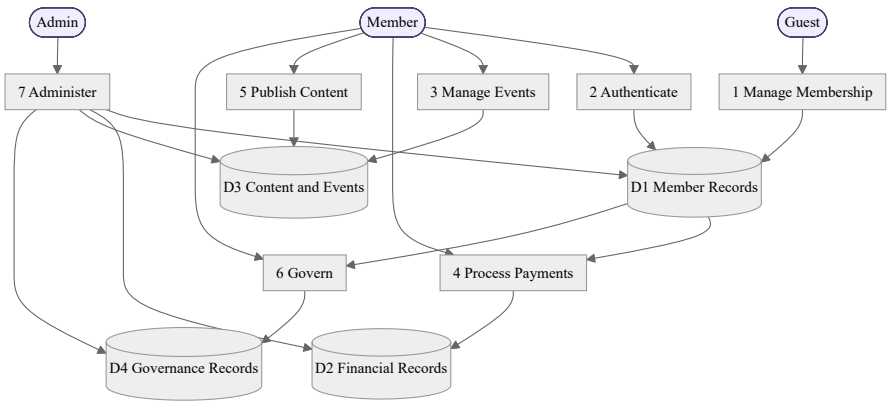


Fig. 5.2. DFD Level 1

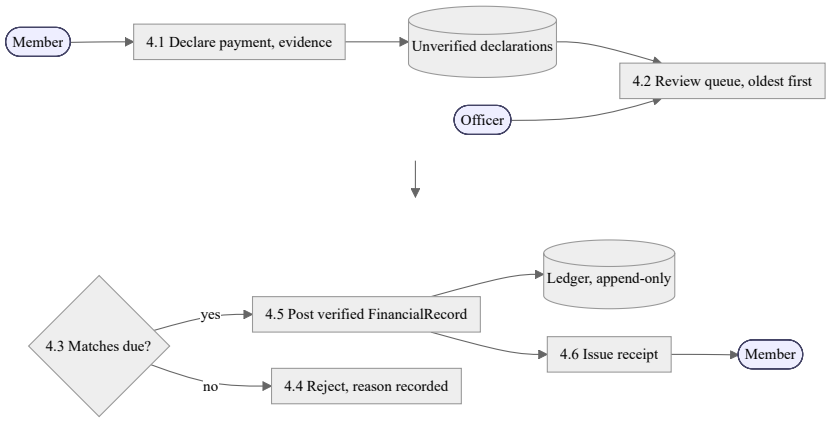


Fig. 5.3. DFD Level 2: Payment processing

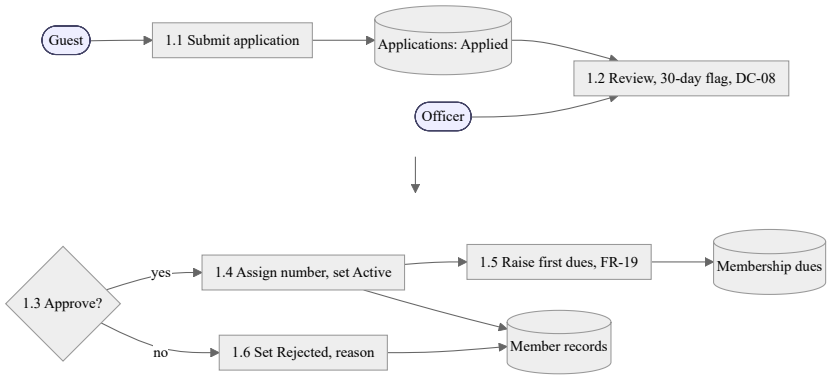


Fig. 5.4. DFD Level 2: Membership approval

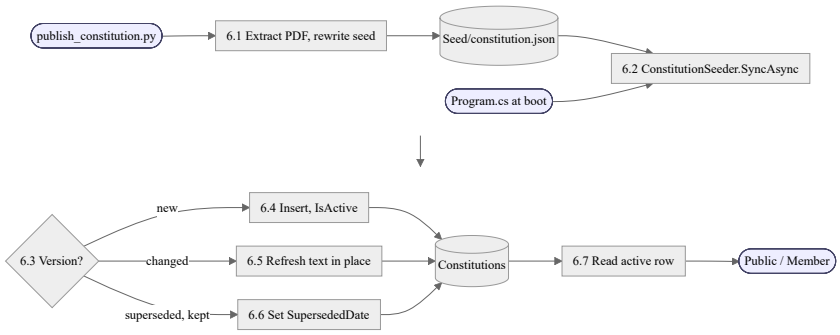


Fig. 5.5. DFD Level 2: Constitution publication

TABLE 5.2
CRC card set for the analysis classes with the widest collaboration surface

Class	Responsibilities	Collaborators
`Member`	Hold identity, standing and per-field disclosure settings	`User`, `AcademicRecord`, `ProfessionalRecord`, `PaymentHistory`, `ECMember`, `EventRegistration`
`MemberService`	Apply, approve, reject, and change status with a recorded reason	`Member`, `MembershipHistory`, `IFileStorageService`
`FinancialService`	Raise dues, record declarations, post verified entries	`PaymentHistory`, `FinancialRecord`, `Member`
`GovernanceService`	Publish the constitution, admit eligible votes, keep the committee record	`Constitution`, `AmendmentVote`, `ECMember`, `ECPeriod`, `Member`
`EventService`	Publish events, register, waitlist, promote from the waitlist, check in	`AlumniEvent`, `EventRegistration`, `Member`

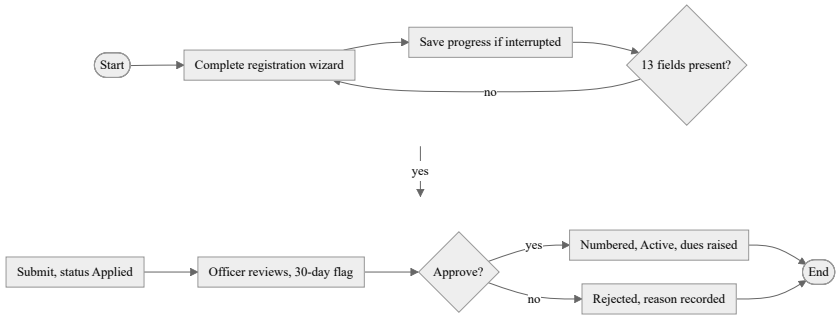


Fig. 5.6. Activity diagram: registration and administrative approval

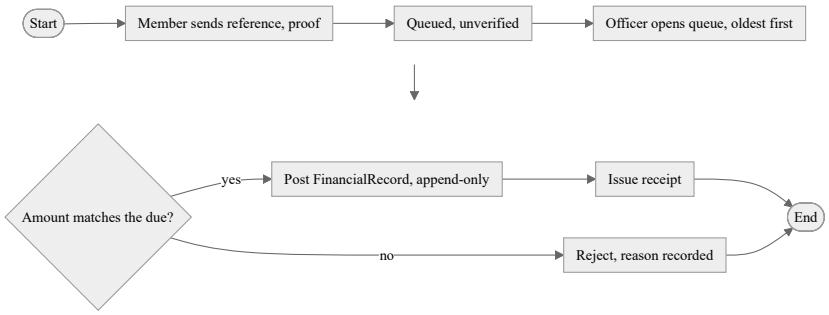


Fig. 5.7. Activity diagram: payment declaration and verification

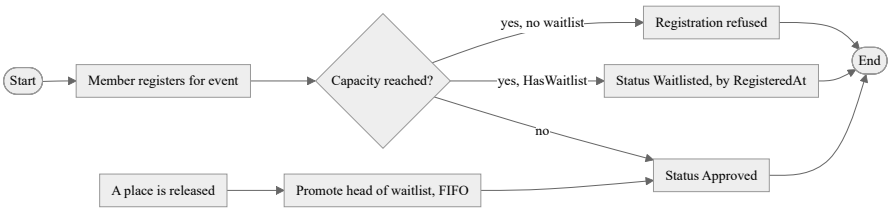


Fig. 5.8. Activity diagram: event registration with waitlist

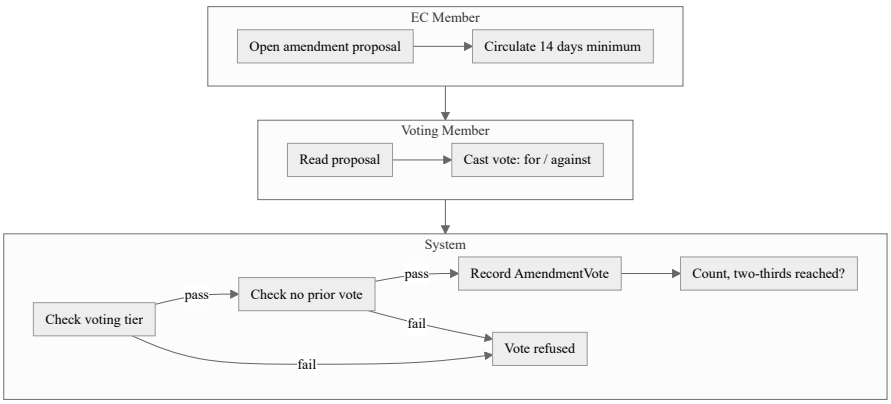


Fig. 5.9. Swimlane activity diagram: constitution amendment vote

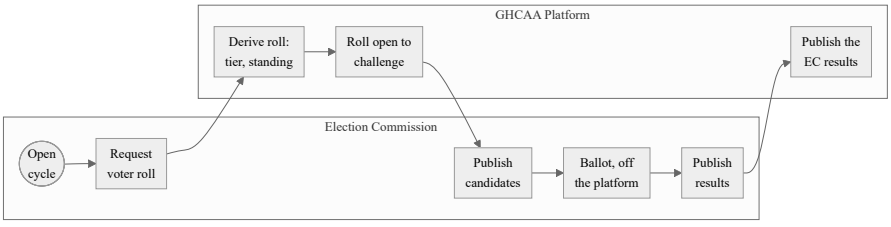


Fig. 5.10. BPMN process diagram of the election cycle

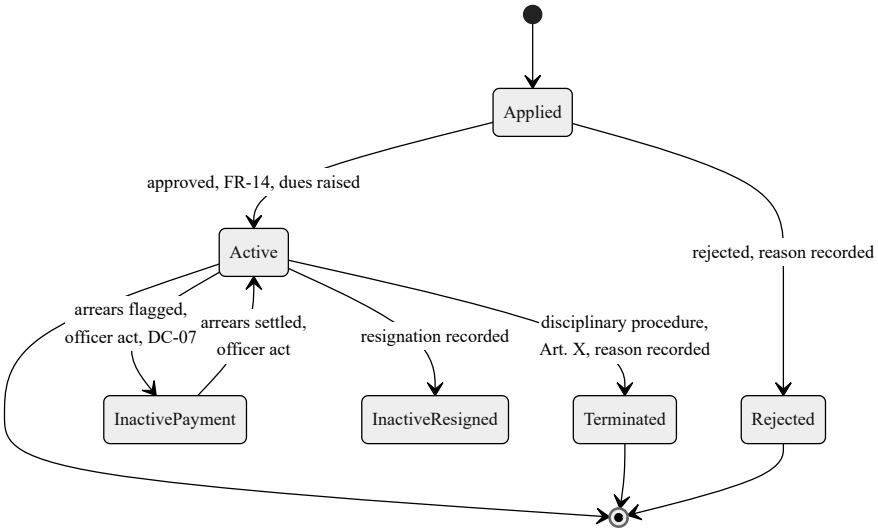


Fig. 5.11. State-machine diagram: member lifecycle

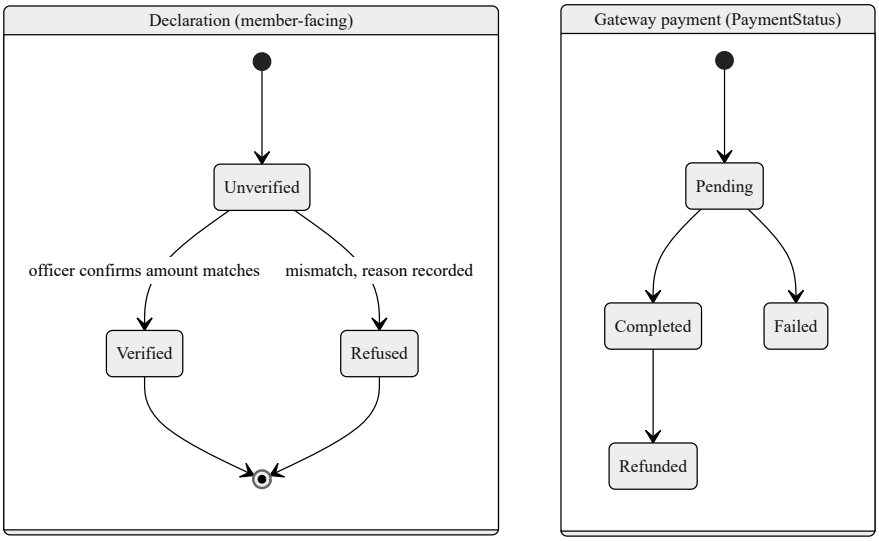


Fig. 5.12. State-machine diagram: payment and declaration

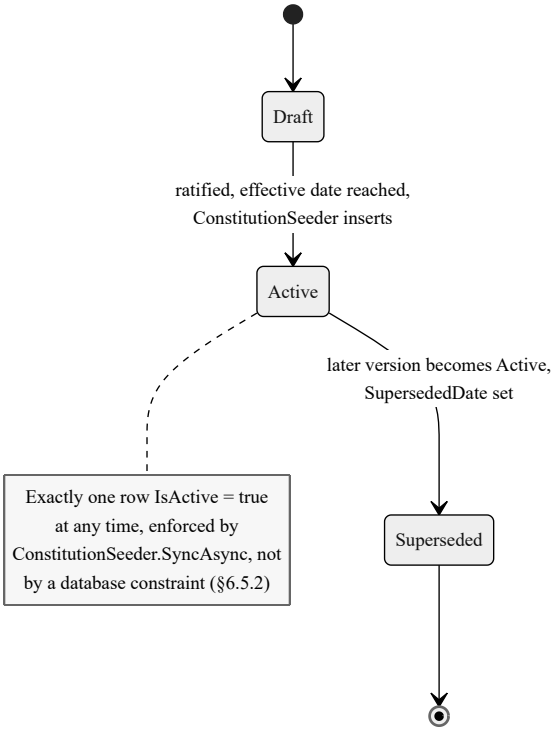


Fig. 5.13. State-machine diagram: constitution version

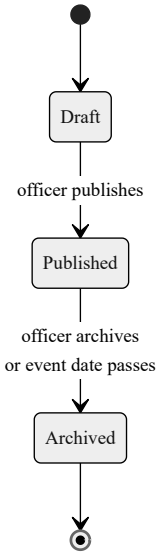


Fig. 5.14. State-machine diagram: event lifecycle

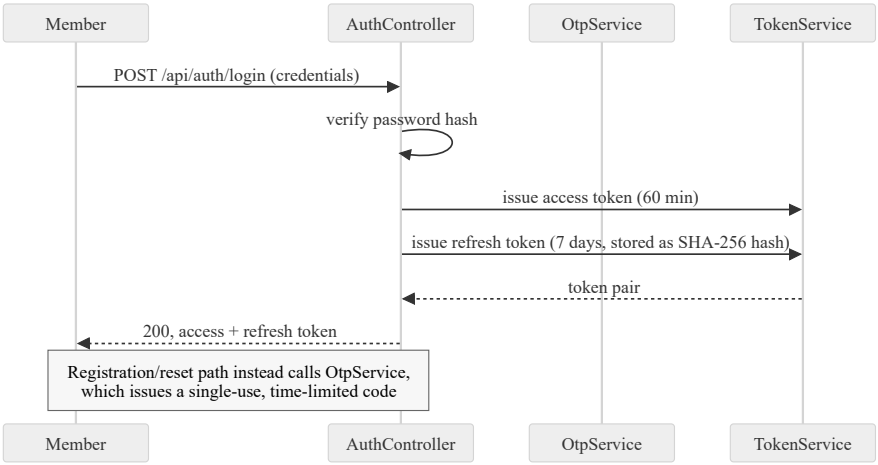


Fig. 5.15. Sequence diagram: login with OTP, token issue

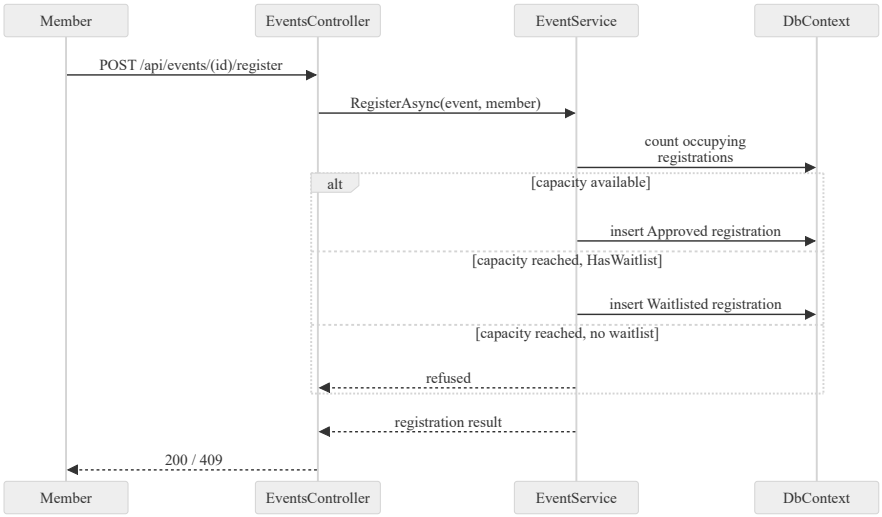


Fig. 5.16. Sequence diagram: event registration

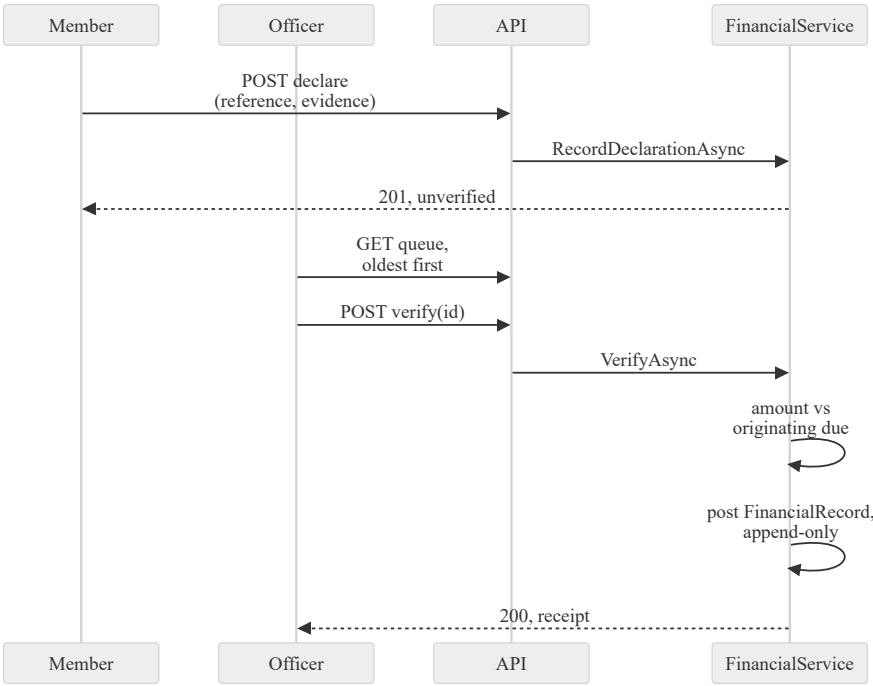


Fig. 5.17. Sequence diagram: payment declaration and verification

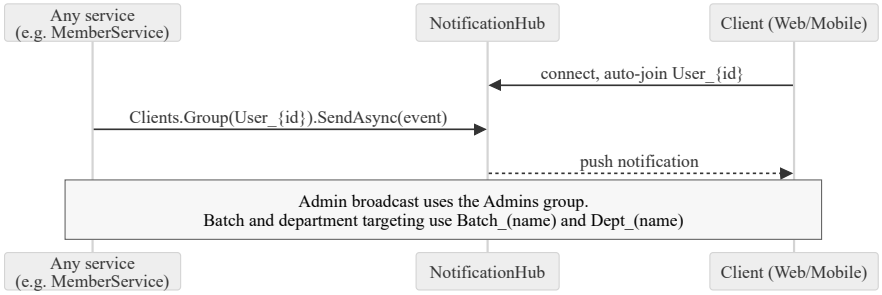


Fig. 5.18. Sequence diagram: real-time notification over SignalR

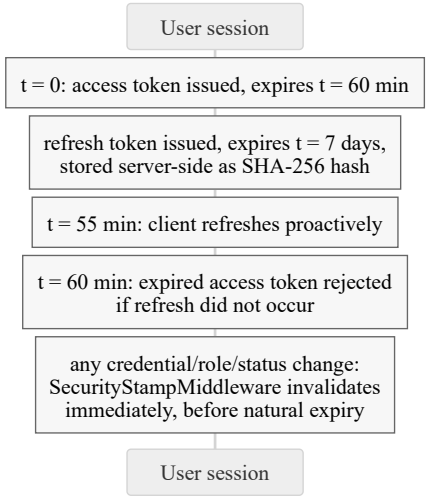


Fig. 5.19. Timing diagram: token lifetime and refresh window

TABLE 5.3
Process specifications for the Level-2 processes

Process	Input	Logic	Output
4.1 Declare payment	Reference, evidence file	Validate file type/size (NFR-P4); create unverified `PaymentHistory` row	Declaration in verification queue
4.3 Amount match	Declared amount, originating `MembershipDue`	Equality comparison; no short-circuit on `amount > 0` (BR-13)	Match / mismatch decision
1.2 Review application	Application record, `AppliedDate`	Flag if `AppliedDate` older than 30 days (DC-08)	Prioritised review queue
6.2 Sync constitution	`Seed/constitution.json`, current `Constitutions` table	Insert unknown version; refresh changed text; supersede prior	Exactly one `IsActive` row

TABLE 5.4
Data-store definitions (analysis level)

Store	Represents	Backing entities
D1 Member Records	Identity, standing, history	`Member`, `AcademicRecord`, `ProfessionalRecord`, `MembershipHistory`, `User`
D2 Financial Records	Dues, declarations, ledger	`MembershipDue`, `PaymentHistory`, `FinancialRecord`, `PaymentConfiguration`
D3 Content and Events	Public and member content	`AlumniEvent`, `EventRegistration`, `NewsPost`, `EventGallery`, `SiteContent`, `JobOpportunity`
D4 Governance Records	Constitution, committee, votes	`Constitution`, `AmendmentVote`, `ECPeriod`, `ECMember`, `Poll`, `PollVote`

Chapter 6 — System Architecture and Design

Chapter 5 modelled what the system does. This chapter records how it is built to do it, and, because §6.2 argues the point in earnest rather than as a formality, why it is built that way rather than one of the alternatives a reviewer would reasonably ask about. The chapter ends, in §6.9, by naming every design principle and pattern the codebase actually uses and giving each one a place a reader can go and check.

6.1 Design Goals, Principles and Constraints

Four goals governed every decision in this chapter, ranked in the order they were actually traded off against each other when they conflicted. The system must be operable by one volunteer with no on-call obligation (NFR-M4); it must run within an operating cost the Association can sustain indefinitely from subscription income (§10.6); it must satisfy the quality-attribute scenarios of §3.4, most of which concern correctness and auditability rather than throughput; and only after those three does raw performance matter, because the workload, being at most a few hundred members, does not stress a conventional web stack. Where a textbook recommendation would have improved a quality attribute the Association does not need at the cost of one it does, the textbook recommendation lost; §6.2 records the clearest instance of that trade.

6.2 Architectural Alternatives Considered and the Decision Taken

Figure 6.15 scores the four candidates below across the quality attributes that decided between them. Four alternatives were assessed against the quality-attribute scenarios of §3.4 and the sustainability constraint of §6.1: a conventional layered architecture without an explicit dependency rule, clean architecture with the dependency rule enforced [1], a modular monolith with service-style internal boundaries but a single deployable, and a microservices decomposition [17]. Full microservices was rejected first and least controversially: Newman's own case for the style rests on independent deployability and independent scaling paying for the operational cost of running several services [17], and neither benefit applies to a single maintainer deploying to a single small container. Fowler's advice to start monolithic and split later applies here in its strongest form, because there is no team boundary to relieve by splitting [16]. A conventional layered architecture without a dependency rule was rejected next, on the evidence of the DI registration comment recorded in §6.10 below: the project has already once let an infrastructure detail (a pooled `DbContext`'s runtime type) leak into a place that silently broke migrations across every environment, and an unenforced layering would have made that class of defect ordinary rather than exceptional.

Clean architecture, with the dependency rule enforced by project references so that a violation is a compile error rather than a code-review finding, was chosen over a modular monolith with only convention-based internal boundaries because NFR-M1 asks for exactly the property a compiler can check: "no compile-time dependency shall point from an inner layer to an outer one." A convention can be violated by mistake; a project reference that does not exist cannot be. The four assemblies of §6.3, `GHCAA.Domain`, `GHCAA.Application`, `GHCAA.Infrastructure` and `GHCAA.API`, are the mechanism, and §7.2 shows the reference graph that makes the rule enforceable rather than aspirational.

What this decision cost is stated rather than glossed. Clean architecture's ceremony, meaning an interface in `GHCAA.Application` for every service implemented in `GHCAA.Infrastructure`, is overhead when there is exactly one implementation and no plan for a second; §6.9.4 returns to this as the clearest instance of a principle applied further than its payoff in this project actually warrants.

6.3 Architectural Design — Clean Architecture

Figure 6.2 draws the four layers and the single direction dependencies are permitted to point.

6.3.1 Domain layer

`GHCAA.Domain` holds seventy-eight entity sets, which extend the analysis model of §5.6, the forty-one enumerations of `Enums.cs`, and constants. Figure 6.9 draws the membership, payment, event and governance classes at design level, with the attributes and multiplicities the analysis model of Figure 3.8 left out. It references nothing else in the solution, which is the dependency rule's starting point: if the domain depended on anything, the rule would already be broken at its centre.

6.3.2 Application layer

`GHCAA.Application` holds the service interfaces (sixty are declared, and the fifty-two implementations in `GHCAA.Infrastructure/Services` are bound to them by the assembly-scanning registration of §6.9.9) and the data transfer objects that cross the API boundary. It depends only on `GHCAA.Domain`.

6.3.3 Infrastructure layer

`GHCAA.Infrastructure` holds the service implementations, the two EF Core provider shims described in §6.5.4, the payment gateway adapters, and `ApplicationDbContext`. It depends on `GHCAA.Application` and `GHCAA.Domain`, and is the only layer that depends on Entity Framework Core, on `HttpClient`, and on the file system. Figure 6.10 draws the interface-to-implementation pairs across that boundary, which is where the dependency rule is either kept or broken.

6.3.4 API layer

`GHCAA.API` holds the thirty-eight controllers of Table 6.3, the two SignalR hubs, and the middleware pipeline of §6.4. It composes the other three layers at startup through `Program.cs` and the `AddInfrastructure` extension method, and depends on all of them.

One implementation lives here rather than in Infrastructure, and the reason is worth stating because it looks like a violation of §6.3.6 and is not. `IRealTimeService` is declared in the Application layer, and the services that push notifications — member, financial, family and notification — depend on that interface like any other. Its implementation, `RealTimeService`, needs `IHubContext<T>`, which is an ASP.NET Core hosting type, so it sits in `GHCAA.API` beside the hubs it drives. The dependency still points inwards: Infrastructure knows only the interface, and the arrow from the service implementations to the hubs in Figure 6.1 is a call at run time, not a compile-time reference.

6.3.5 Presentation layers

Two independent clients consume the API rather than sit inside the solution's dependency graph: `GHCAA.Web`, an Angular 21 single-page application serving the public site, the member portal and the admin console from one codebase with route guards separating the three, and `GHCAA.Mobile`, a Flutter application using Riverpod for

state and Dio for HTTP, targeting Android and iOS from one codebase. Neither client has a compile-time dependency on the API's implementation; both depend only on its HTTP and SignalR contract.

6.3.6 The dependency rule and the mechanism that enforces it

The rule is enforced by `.csproj` project references, which is a compiler-checked mechanism rather than a linter or a convention: `GHCAA.Domain.csproj` references no other project in the solution, and no `.csproj` in an inner layer references one in an outer layer. §7.2's dependency structure matrix is the evidence that this holds for the delivered solution, not only for its intended design.

6.4 Component and Interface Design

Figure 6.11 draws the request path as a component diagram: a controller depends on a service interface it does not implement, the concrete service is supplied by the DI container built in §6.9.9, and the service depends on `ApplicationDbContext` as its persistence port. The middleware pipeline, drawn in Figure 6.12, is the component boundary a request crosses before it reaches a controller at all, and its order is significant: `ExceptionMiddleware` wraps everything so that no unhandled exception below it reaches the client as anything other than a JSON error; rate limiting runs before authentication so that an unauthenticated flood is rejected before the cost of verifying a token is paid; `SecurityStampMiddleware` runs after authentication and before authorisation, which is the ordering that makes BR-11 hold, since a request must be authenticated before its security stamp can be checked, and must be checked before authorisation decides what it may do. `WwwRedirectMiddleware` sits just after `CorrelationIdMiddleware` and before `ExceptionMiddleware`. It answers a request for `www.` plus the host of `AppSettings:ClientUrl` with a redirect to that origin, a 301 for `GET` and `HEAD` and a 308 for other methods so that a `POST` stays a `POST`, and it does nothing when `ClientUrl` is unset, is a loopback or IP address, or is itself a `www` host. Figure 6.12 leaves it out, as it leaves out the correlation-ID stage and CORS, to keep the checks that decide access in view.

At the interface, the API resource model follows a `/api/[controller]` convention with sub-resources expressed as path segments, for example `/api/events/{id}/register`. Errors are returned as JSON with a consistent shape from `ExceptionMiddleware` rather than as provider stack traces, which is NFR-U4's requirement enforced at the one place that can guarantee it for every unhandled case. Table 6.3 catalogues every controller with its route and authorisation level; the full request and response shapes are in the generated OpenAPI document.

6.5 Data Design

6.5.1 Conceptual, logical and physical progression

The conceptual model is Figure 3.8; the logical model is the seventy-eight mapped entity sets that grew from §5.6, sixty-six of which have a Fluent API configuration class under `GHCAA.Infrastructure/Data/Configurations/` rather than attribute-only mapping, which keeps persistence concerns out of the domain classes themselves; the remaining twelve are mapped by EF Core's conventions alone, which is a gap rather than a decision and is recorded as such; the physical model is whichever of the two schemas in §6.5.4 the running environment selects. Figures 6.3 to 6.8 draw that schema as six sub-models, being identity and records, standing and money, events and participation, governance, election roll and nominations, and voting and the count, each with its keys and cardinalities. They are split this way rather than drawn as one diagram because one diagram of seventy-eight tables cannot be printed at a size anyone can read; the content, communication and configuration tables are in the generated schema documentation.

Figure 6.7 draws the election, its seats, the frozen voter roll, nominations and declared results. Figure 6.8 draws the voting tables, and it keeps who voted apart from what was voted. `SeatVote` records that a member voted for a seat and when. The choices go to `PendingBallot` in the same transaction, sealed with the election's public key, with no member column and no time column. At the count the sealed rows are opened with the returning officer's private key, shuffled, written to `Ballot` and `BallotVote` under random GUID keys, and deleted. No key joins a `SeatVote` row to a `Ballot` row, and `BallotReceipt` gives the voter a tracking code that is not linked to the ballot either. `ElectionAuditEntry` chains each entry to the hash of the one before it. The appointment and approval tables, which hold the officials and the two-person sign-off on each phase change, are left out of both figures to keep each to one page. §8.8, on governance integrity, sets this design against the attributable `PollVote` used for member polls. The design is checked by the test suite; it has not yet been used in a live election.

6.5.2 Normalisation and its deliberate exceptions

Most of the schema sits at third normal form: an `EventRegistration` row does not repeat event attributes, a `PaymentHistory` row does not repeat member attributes. Two exceptions were made deliberately rather than found by accident. `OrganizationConfig` stores its entire configuration document as a single JSON column, `ConfigJson`, rather than as a normalised table of key-value rows; the reasoning is that the configuration schema, given in full in `docs/CONFIG_DRIVEN_FRAMEWORK.md`, changes shape as features are added, and a normalised key-value table would have needed a migration for every new setting where a JSON column needs none, at the cost that the schema inside that column is enforced by the DTO layer rather than by the database. `NewsPost` merges what could have been two tables, news articles and official notices, into one, discriminated by a `PostType` column; this was chosen over two tables because the two kinds share every other attribute and the one place they differ, being who may create them, is exactly the business rule BR-03 already enforces in the controller rather than in the schema.

The constitution invariant of DC-16, that exactly one row of `Constitutions` has `IsActive = true`, is not a database constraint at all; it is a procedural invariant maintained by `ConstitutionSeeder.SyncAsync`. A partial unique index, in providers that support one, would enforce it at the database rather than relying on the seeder's discipline, and its absence is recorded here as a real gap rather than implied to be intentional hardening.

6.5.3 Indexing strategy

Every unique business constraint stated as a non-functional requirement is enforced by a database index, not only by application-level validation, which is what NFR-S4 explicitly demands for national identity number, mobile number and email address. `MemberConfiguration` places a unique index on `Email`, `NID` and `MobileNo` individually, plus a composite index on `(Status, IsArchived)` for the query the administrative queue of FR-23 runs most often. `AmendmentVoteConfiguration` places a unique composite index on `(ConstitutionId, MemberId)`, which is BR-02's one-vote rule enforced at the database rather than only in `GovernanceService`; the same shape, a composite unique index preventing a double insert, recurs in `EventRegistrationConfiguration` on `(EventId, MemberId)`, `PollVoteConfiguration` on `(PollOptionId, MemberId)`, and `RefreshTokenConfiguration` on `TokenHash`. `PaymentHistoryConfiguration` uniquely indexes both `TransactionId` and `GatewayPaymentId`, which is the database-level half of BR-13's amount-matching rule: it cannot prevent a wrong amount, but it can and does prevent the same gateway transaction being credited twice.

6.5.4 Multi-provider portability

`ApplicationDbContext` is abstract: two concrete subclasses, `PgSqlApplicationDbContext` and `SqliteApplicationDbContext`, are pooled behind it, selected at startup by a `DatabaseProvider` configuration value read in `GHCAA.Infrastructure.DependencyInjection`. Production runs PostgreSQL, the only provider with a migration tree; the test suite runs against the SQLite provider for a migration-free bootstrap path, and against EF Core's in-memory provider for unit-level service tests. A third subclass, `MySqlApplicationDbContext`, existed earlier and was removed once an audit found it advertised support it never had — see ADR-02 and `docs/adr/0006-drop-mysql-provider.md`. The cost of this portability is recorded in the source itself: the migrations assembly must be told which provider-specific shim type a migration is attributed to, and the DI registration comment explains, in the maintainer's own words, why pooling the base `ApplicationDbContext` type rather than the shim silently broke migration detection on every environment until it was found. That comment is treated in this dissertation as primary evidence, not paraphrased away, because it is a more honest record of the cost than a tidy retrospective claim would be.

6.5.5 Data dictionary

Table 6.2 gives a representative slice of the seventy-eight mapped entities; the full dictionary is generated from the schema and delivered beside this dissertation.

6.5.6 Seeding and runtime data-synchronisation strategy

Schema and reference data are handled by two separate mechanisms at boot, for reasons that only became clear after the first one failed in production.

Schema first. `Database.EnsureCreated()` creates a schema on an empty database and does nothing at all on a non-empty one, and preprod was built that way, so it had no `__EFMigrationsHistory` table and every migration added after its first successful boot was silently never applied. The failure was not visible as an error; it surfaced on 27 August 2026 as HTTP 500 from `/api/jobs` and `/api/gallery`, because the columns those endpoints read had never been created. Schema management is now `MigrationBootstrapper.EnsureMigratedAsync`, called from `Program.cs` before anything else touches the database, and `GHCAA.Infrastructure/Data/Migrations/PgSql` is the authority on the schema. The migration history was squashed on 8 September 2026 (`20260907193705_InitialBaseline`), so the folder now holds one migration carrying the full schema rather than the prior sequence built up migration by migration; the squash was rehearsed against a Postgres mirror of production before being applied, with a manual `__EFMigrationsHistory` insert marking the new baseline as already applied so existing databases are not re-run against it. On a database with no migration history it walks every migration in order and applies it for real; where the database reports that the object a migration creates already exists, that migration's effect predates migration tracking and it is marked applied without being re-run; any other failure aborts the boot rather than being swallowed. It also re-checks migrations already recorded as applied whose `Up()` mixes a schema change with a data seed, because EF runs a migration's operations in one transaction: a colliding seed insert rolls the schema change back with it, while the "object already exists" test still reports success, which is how the same 500s were reintroduced by the fix for them. Where a migration is recorded as applied but the column or table it creates is absent, its history row is dropped so the next `MigrateAsync` applies it properly.

Reference data second, and separately, because a migration is the wrong tool for content that is revised rather than structured. `ConstitutionSeeder.SyncAsync` runs at every boot, from `Program.cs`, and inserts a version it has not seen before, refreshes the text of a version whose content changed, and supersedes, without deleting, a version a newer one has replaced, so that a production database created months earlier still receives a constitution amendment shipped today. `HasData` would not do this: on a populated database it is applied only through a migration, and the amendment text would then be pinned to whichever migration happened to carry it.

A separate risk sits in `LoadSeed<T>`'s deserialisation step itself rather than in when it runs: `System.Text.Json.JsonSerializer.Deserialize<List<T>>` silently discards any JSON key that does not match a public property on `T`, with no exception and no log line. A field renamed on the entity without the same rename in its seed file therefore reaches production as a quietly wrong default — confirmed in practice, not hypothetically, when a stale `Category` key (the entity's real property had been renamed to `ArticleCategory`/`JobCategory`/`FinancialCategory`) left several thousand seeded records defaulted to the wrong enum value across three unrelated seed files. The mitigation is a reflection-based regression test (`SeedDataIntegrityTests`) that parses each seed file's raw JSON keys and asserts every one matches a real property on its target type, turning a silent runtime default into a build-time failure. A distinct failure mode surfaced later, on a correctly-named key: a path field holding punctuation-only placeholder text (`"..."`) instead of a real value, which deserialises without complaint and only surfaces as a broken image in a browser. The same test file was extended with a second, value-level assertion — every `*Path`/`*Url` string must contain at least one alphanumeric character — since a correct key name is not by itself evidence of a correct value.

6.6 Security Architecture

Summarised here as a design view; the threat model, control mapping and residual risk are Chapter 8's subject. Four mechanisms are worth naming as architecture rather than as detail, because each is a structural decision rather than a local check: `SecurityStampMiddleware`, which makes a credential or role change take effect within one request rather than at token expiry; the query-string token allowance in the middleware pipeline, which exists only to let a file download authenticate without a custom header and is scoped, in the pipeline order of Figure 6.12, to run before the ordinary authentication step rather than replacing it; the payment-gateway posture of §8.7, under which the platform never stores a payment credential of its own; and step-up authentication for destructive, financial and identity-changing admin actions, which re-verifies an already-authenticated Admin or SuperAdmin session by email OTP before it may reach one of five gated endpoints. The verification result is carried as a claim on the JWT rather than as server-side session state — the API has no session store to hold it in — so a claim's continued validity across the access token's routine hourly refresh is established by validating the outgoing token's signature and issuer before its step-up claim is trusted forward, not by re-running the OTP challenge on every refresh. The grace period since last verification was originally set to 30 days for admin usability, but a security review found this let the claim ride along on every hourly refresh for the full window, so a stolen or left-open session almost always already carried a valid one — defeating the control's own stated purpose. It was reduced to 30 minutes, the usability trade-off now resting on the claim surviving several refreshes within one working session rather than on a long calendar window.

A fifth mechanism belongs alongside these for the same reason: which scheme a request arrived over is not a property Kestrel can observe directly once TLS is terminated at the hosting platform's edge and the request is forwarded to the container over plain HTTP. `ForwardedHeadersOptions`, registered as the first middleware in the pipeline — ahead of exception handling, CORS and everything else — trusts the edge's `X-Forwarded-Proto` header so that `HttpContext.Request.IsHttps` reports correctly for every later stage: the HSTS header, the HTTPS redirect, and any authorization decision that might otherwise assume an unencrypted request. Placing this ahead of `UseHsts()`/`UseHttpsRedirection()` rather than treating it as an unrelated Dockerfile or platform concern is the structural point — three independent controls share one upstream dependency, and only one of them names it.

6.7 Web and Mobile User-Interface Design

6.7.1 Design principles and information architecture

The public site, the member portal and the admin console are three route trees within one Angular application, separated by `PublicLayoutComponent`, `PortalLayoutComponent` and `AdminLayoutComponent` and by the `authGuard` and `adminGuard`/`superAdminGuard` route guards listed in Table 6.3's companion, the route map of Figure 6.13. A visitor never crosses from the public tree into a guarded one without authenticating, and the admin tree is itself split by guard between `Admin` and `SuperAdmin`, matching the role distinction §6.6 sets out.

6.7.2 Design-token system, theming and the single-stylesheet decision

Presentation is governed by one stylesheet, `GHCAA.Web/src/styles.scss`, running to 3,363 lines on 7 September 2026. The decision to keep one file rather than a stylesheet per component was made for a reason specific to this project's constraint of one maintainer: a shared design-token set for colour, spacing and typography, resolved once and consumed everywhere, is the only way one person can change a brand colour in one place and have it apply to three route trees without hunting through the eighty components that declare one. §6.9.19 records what this decision cost as well as what it bought.

6.7.3 Shared control library and the duplication it eliminates, and where it does not yet

Some presentation is genuinely shared: `PaginationComponent`, `ToastComponent`, `FooterComponent` and similar are each written once under `src/app/common/` and used from every route tree that needs them. Member-specific record editing is not yet shared to the same degree, and this is recorded honestly rather than smoothed over. `docs/SHARED_PROFILE_COMPONENTS.md` is a design document, explicitly marked "design only — not implemented", written because the member-facing profile page and the admin member-detail modal render the same `Member` fields, being academic history, professional history, EC history, emergency contact and address, through two independent templates that had already drifted apart in section order and labelling by the time the document was written. The proposed remedy is a set of presentational sub-components, `AcademicHistoryEditor`, `ProfessionalHistoryEditor`, `EcHistoryView`, `EmergencyContactForm` and `AddressForm`, each taking the relevant model slice as input and emitting change events rather than making its own HTTP calls. As of this writing that extraction has not been built, and the two templates still drift independently; it is listed here as an identified but unresolved duplication rather than as a completed piece of design, and §13.4.2 carries it forward as a named item of technical debt.

6.7.4 Responsive design and accessibility strategy

Accessibility is targeted at WCAG 2.1 level AA per NFR-U1, verified by the audit reported in §9.10 rather than asserted here. Responsive layout follows Marcotte's approach of a fluid grid over fixed breakpoints [36], necessary because NFR-P3 and the assumption of §1.7 both treat a mobile browser, not a desktop one, as the primary surface for a member.

6.7.5 Web application design pyramid

Reading the site against Pressman and Maxim's design pyramid [54]: interface design is the guard- separated route trees of §6.7.1; aesthetic design is the token system of §6.7.2; content design is the admin-editable `SiteContent` blocks described in §5.2.3's public-content data flow, which let an officer change wording without a deployment (FR-45); navigation design is the route map of Figure 6.17 and the site map of Figure 6.14; architecture design is Figure 6.1; and component design is the shared control library of §6.7.3, with the gap just recorded.

On mobile, the Flutter client mirrors the web client's feature set (FR-48) rather than offering a reduced one, using `Dio` for HTTP against the same API and `flutter_riverpod` for state. Two platform-specific concerns are worth naming: the identity card is rendered from data cached at first sign-in so it remains visible without a network connection at the venue where it is presented (FR-49), and payment evidence images are compressed on the device before upload to the limit stated in NFR-P4, rather than relying on server-side compression, because the upload itself is the expensive step on the connections §1.7 assumes.

6.8 Configuration-Driven Design

Feature flags and organisation identity are design elements, not afterthoughts, because §1.7's delimitation to one association does not mean the code should hardcode that association's name. `OrganizationConfig.ConfigJson`, read through `IOrgConfigService` with a ten-minute in-memory cache and a fallback chain of cache, then database, then a built-in default so that a missing configuration row cannot crash the application, holds branding, contact details, currency, feature toggles such as `enableForum` and `enableMentorship`, and workflow settings such as the membership approval mode. `GET /api/config` is public and `PUT /api/config` is SuperAdmin-only, which is the interface design principle of §6.4 applied to configuration itself. One field group is a deliberate exception to the "database wins" rule. `Localization`, the UI copy strings such as the per-locale tagline, is overlaid from the built-in defaults on every read, whatever the stored config row holds. No admin screen edits it, so a stored row only carries it forward by accident: `UpdateConfigAsync` round-trips the whole DTO on any Branding or Workflow save. Trusting that stale copy would mean a text fix in the source never reaching production, which is what happened on 2026-08-31, when a corrected Bengali tagline kept serving the pre-fix value from an old row. At the time of writing the Angular and Flutter clients' consumption of this configuration is only partially complete, which `docs/CONFIG_DRIVEN_FRAMEWORK.md` itself records as Phase 2 and Phase 3, "TODO"; §13.4.2 carries the remaining wiring forward.

6.9 Design Principles and Patterns

6.9.1 Separation of concerns and the layer boundary

The four-assembly split of §6.3 is the mechanism; a controller containing a SQL query, or a domain class containing an HTTP call, would be the violation, and none exists in the solution.

6.9.2 Dependency inversion

The domain depends on abstractions only, in the strong sense that it depends on nothing at all; `GHCAA.Application` defines the interfaces `GHCAA.Infrastructure` implements. Enforced by the project reference graph of §6.3.6 and reported numerically by the dependency structure matrix planned for Chapter 7.

6.9.3 Single responsibility

Service decomposition follows the subsystem boundaries of §3.5: `MemberService` owns membership lifecycle, `FinancialService` and `FinancialLedgerService` are split apart from each other specifically so that raising and recording a due is a different responsibility from posting an append-only ledger entry, which is the

separation BR-07 depends on. The cohesion evidence for this claim is measured, not asserted, in §9.12.3.

6.9.4 Open/closed

Where it is achieved: a new payment gateway is added by implementing `IPaymentGatewayService` (directly, or through the shared `BasePaymentGateway` base class introduced after 2026-09-07 for the three gateways that share callback-verification logic) and registering it with `PaymentGatewayFactory`, without modifying any existing gateway, which is exactly how `DGePayGateway` was added alongside `SSLCommerzGateway`, `BkashGateway` and `NagadGateway`. `BkashGateway`, `NagadGateway` and `SSLCommerzGateway` now extend `BasePaymentGateway : IPaymentGatewayService`; `DGePayGateway` still implements the interface directly, since it does not share the callback-verification logic the base class factors out. Where it is not: `IFileStorageService` has exactly one implementation, `LocalFileStorageService`, so the interface's openness to a cloud-storage adapter is structural rather than demonstrated, and the abstraction's cost, an interface and a DI registration for a substitution that has never happened, is real overhead until that day comes.

6.9.5 Liskov substitution and interface segregation

Every one of the four `IPaymentGatewayService` implementations is substitutable through `PaymentGatewayFactory` without the caller testing which one it received, which is the property this principle names. Interface segregation is visible in the split between `IFileStorageService`, which callers depend on to store and retrieve a file, and `IFileUploadRepository`, which only `MemberImportController`'s bulk path and a narrow set of query callers depend on for metadata lookup; neither interface forces a caller to depend on methods it does not use.

6.9.6 Information hiding and encapsulation

A controller never queries `ApplicationDbContext` directly. Eight controllers did until 2026-09-06 — `AdminSocialAuthController`, `AuthController`, `FinancialsController`, `GatewaysController`, `GovernanceController`, `HealthController`, `PaymentConfigController` and `SecureFilesController` — each named rather than hidden while the exception stood, because a principle applied with undisclosed exceptions is not a principle a reader can trust elsewhere in the same table. The cleanup moved every one of those reads/writes behind a new or extended Application-layer interface (`ISocialAuthConfigService`, `IPaymentConfigService`, `IDatabaseHealthService`, and new methods on `IAuthService`, `IFinancialService`, `IEventService`, `IMemberService`, `IFileUploadRepository`); `GovernanceController`'s own field turned out to be dead — injected but never read. The principle now holds without exception.

6.9.7 Coupling and cohesion

Measured in §9.12.3 as CBO, afferent and efferent coupling and instability, and plotted against Martin's main sequence; this section states the claim, that layer boundaries keep coupling directional, and §9.12.3 is where the claim is checked rather than assumed.

6.9.8 Elimination of duplication

The single stylesheet of §6.7.2 and the assembly-scanning registration of §6.9.9 both remove a class of duplication that would otherwise recur on every new service or every new component. The shared member-record editors of §6.7.3 are the case where duplication was identified and not yet removed, and it is counted as a limitation here rather than claimed as a success.

6.9.9 Convention over configuration

`GHCAA.Infrastructure.DependencyInjection.AddInfrastructure` scans its own assembly for every non-abstract class in a namespace containing `Services`, finds the interfaces it implements under `GHCAA.Application.Interfaces`, and registers each as scoped, automatically. A new service class that follows the naming convention needs no line added to `Program.cs` to become injectable; only the nine registrations that do not follow the convention, being file storage, theming, the file-upload repository, the payment-gateway factory and the four gateways themselves plus the SMS client, are listed by hand, and the code comments them as exactly that: manual registrations for non-standard services.

6.9.10 Principle of least astonishment

The membership-number format `GHC-[Year]-[Serial]`, the consistent `/api/[controller]` route shape, and the uniform JSON error envelope from `ExceptionMiddleware` are the three places this principle is most visible to someone outside the maintainer's own head: an admin who has seen one membership number can read any other, and a client developer who has called one endpoint successfully can guess the shape of the next one correctly more often than not.

6.9.11 GRASP

Information expert: `Member` exposes its own standing computation from data it holds. Creator: `MemberService` creates the `MembershipHistory` row alongside the status change that caused it, because the service that knows the change is the service best placed to create its record. Controller: each API controller is a thin GRASP controller delegating to a service, not a fat one absorbing business logic. Pure fabrication: `PaymentGatewayFactory` is a class with no analysis-level counterpart, invented purely to keep gateway selection out of every controller that needs a gateway. Indirection and protected variations: the `IPaymentGatewayService` interface is the seam that protects every caller from which concrete gateway is behind it.

6.9.12 Principles deliberately traded away

Deferred generality was chosen over speculative abstraction in the file-storage case of §6.9.4, where a second adapter is not built until a second requirement for one exists. The modular monolith alternative to full microservices, argued in §6.2, is itself a principle traded away deliberately: independent deployability was sacrificed for operability by one person. Data transfer objects at every API boundary were kept even where they duplicate a domain class field for field, because the alternative, serialising domain entities directly, would have coupled the wire contract to the persistence model in a way that failed the compatibility requirement of NFR-C3 the first time a provider-specific attribute needed adding.

6.9.13 Creational patterns

Table 6.4 lists the patterns applied, with the alternative rejected in each case; the subsections below give the reasoning for the entries where the choice was not obvious.

Factory. `PaymentGatewayFactory` resolves an `IPaymentGatewayService` by `Enums.PaymentGateway` key at the point a payment is initiated, so the caller never names a concrete gateway type. Forces: the number of gateways was known to grow (four exist; a fifth was added during the project without touching the factory's callers)

and callers should not need to change when it does. Alternative rejected: a switch statement in `GatewaysController` was the initial shape and was replaced once a second gateway made the duplication visible.

Singleton via container lifetime. The DI container's scoped and singleton lifetimes take the place of a hand-written Singleton; `IMemoryCache` behind `IOrgConfigService` is registered with container-managed lifetime rather than a static field, which is the idiomatic .NET substitute for the classic pattern.

6.9.14 Structural patterns

Adapter. Each payment gateway class adapts a foreign HTTP API, bKash's, Nagad's, SSLCommerz's or DGePay's own request and response shapes, to the single `IPaymentGatewayService` contract. Forces: four incompatible external interfaces, one internal contract. Consequence observed: the callback controller, `GatewaysController`, is written once against the internal contract and does not vary by gateway except in its routing.

Facade. The service layer as a whole is a facade over `ApplicationDbContext` and EF Core's change-tracking, so that a controller never composes a LINQ query itself; the exceptions are the four controllers named in §6.9.6.

Decorator. The middleware pipeline of Figure 6.12 is ASP.NET Core's own decorator chain: each middleware wraps the next and can act before and after it without the inner stages knowing the outer ones exist.

Proxy. EF Core's lazy navigation and the ten-minute `IMemoryCache` wrapper in `OrgConfigService` both act as a proxy standing in front of an expensive operation, a database round trip in one case, a configuration read in the other.

6.9.15 Behavioural patterns

Strategy. The four `IPaymentGatewayService` implementations are interchangeable strategies selected by `PaymentGatewayFactory`; the three `ApplicationDbContext` provider shims play the same role for persistence, selected once at startup rather than per request.

Observer. SignalR's group-based push, `Clients.Group(User_{id}).SendAsync(...)`, is publish-observe at the transport level; Angular's signal-based reactivity on the client is the same pattern applied to UI state.

Chain of Responsibility. The eleven-stage middleware pipeline of §6.4, each stage free to short- circuit the request before it reaches the next.

Command. Each controller action is, in effect, a command object implicit in ASP.NET Core's model binding: a request is bound to a typed DTO, dispatched to exactly one handler method, and the pattern is used implicitly rather than as an explicit `ICommand` hierarchy, which was judged unnecessary ceremony for a project of this size.

6.9.16 Enterprise application patterns

Repository. Applied narrowly and by name to exactly one case, `IFileUploadRepository`, rather than uniformly across every entity; every other service reaches `ApplicationDbContext` directly. **Unit of Work.** Not hand-written, because EF Core's `DbContext` already is one: `SaveChangesAsync` commits every tracked change in a single transaction, which is the property a hand-rolled Unit of Work would exist to provide. **Service Layer.** The forty-two services of Table 6.3's companion DI map are this pattern, named and applied consistently. **Data Transfer Object.** Applied at every controller boundary, discussed in §6.9.12. **Domain Model.** Present but anaemic in Fowler's sense [2], as §5.3.1 already noted: state lives on the entity, behaviour that changes it lives on the service. **Identity Map.** EF Core's change tracker provides this per `DbContext` instance; no separate implementation exists or is needed at this scale.

6.9.17 Architectural patterns

Layered/clean architecture (§6.3), dependency injection as the composition mechanism throughout, MVC on the API side and a comparable smart/presentational split on the Angular side (§6.9.18), publish–subscribe for SignalR, and the API itself as the single gateway boundary both clients pass through.

6.9.18 Angular and Flutter presentation patterns

Angular components split, informally, into smart components that own a service dependency and presentational ones that do not, `PaginationComponent` and `ToastComponent` being the clearest presentational examples; the profile and admin-member components of §6.7.3 are the case where that split has been designed but not yet built. Reactive state uses Angular signals on the web side and Riverpod providers on the mobile side, both observer-pattern variants. `authGuard` and its `adminGuard`/`superAdminGuard` companions are the guard pattern; an HTTP interceptor attaches the bearer token and reacts to a 401 by attempting the refresh flow of Figure 5.15 before failing.

6.9.19 Anti-patterns identified and remediated during development

Table 6.6 records each one as symptom, diagnosis and what was done. One is remediated, one is designed against but not yet built, and one is accepted as it stands with the reason given.

God service, avoided by decomposition rather than discovered as a defect. Splitting `FinancialService` from `FinancialLedgerService` (PROJECT_MAP.md records both as separate registrations) kept the finance service from growing into one that owned both a business workflow and its append-only record of consequence.

Anaemic domain drift, noticed and knowingly not corrected. §5.3.1 and §6.9.16 both record this: correcting it would have moved authorisation logic out of the service layer where every access-control test of §9.7 currently finds it uniformly.

Raw-control styling leak, partially remediated. The shared control library of §6.7.3 remediates this for cross-cutting UI elements; the member-record duplication in the same section is the instance where the anti-pattern is named, designed against, and still present in the shipped code.

6.10 Architecture Decision Records and Design Verification

Table 6.1 indexes the six decisions this chapter treats as architectural, meaning that reversing one would change the shape of the system rather than the contents of a file.

Table 6.1
ADR index

ADR	Decision	Context	Consequence
ADR-01	Adopt clean architecture with a compiler-enforced dependency rule	§6.2: layering by convention had already once failed silently	NFR-M1 becomes checkable; four-assembly ceremony for a single-maintainer project
ADR-02	Pool provider-specific `ApplicationDbContext` shim types, not the base type	EF's `IMigrationsAssembly` matches migrations to the pooled context's exact runtime type; pooling the base type made `GetMigrations()` return zero migrations on every provider, so the self-healing boot logic was a silent no-op everywhere	Migrations apply correctly on boot on PostgreSQL, the only provider with a migration tree; the MySQL shim was removed once an audit found it advertised support it never had (`docs/adr/0006-drop-mysql-provider.md`), and SQLite is kept only for a migration-free test-bootstrap path outside this boot logic
ADR-03	Never write live constitution publication through `HasData`	Seed data expressed as `HasData` reaches a populated database only through a migration, which pins revisable text to whichever migration carried it; the `EnsureCreated()`-built preprod database never received it at all	`ConstitutionSeeder.SyncAsync` runs at every boot; schema is handled separately by `MigrationBootstrapper`
ADR-04	Keep the manual payment path primary and leave gateway integration optional	§3.2, §8.7: the Association holds no merchant account and no gateway credentials	Permanent officer verification workload, quantified in §12.4, in exchange for holding no payment credential
ADR-05	Merge news and notices into one table discriminated by `PostType`	§6.5.2: identical shape apart from authorship rule	BR-03 enforced at the controller rather than by two schemas
ADR-06	Defer a second `IFileStorageService` implementation	§6.9.4: no second storage requirement exists yet	Interface segregation is structural, not yet demonstrated

Table 6.5 checks the architecture against the utility tree of Figure 3.10 by mapping each quality-attribute scenario to the tactic that addresses it: QAS-01's directory latency to indexed queries (§6.5.3) and the cache in front of configuration reads (§6.8); QAS-03's session invalidation to `SecurityStampMiddleware`; QAS-06's maintainability scenario to the dependency rule and the risk-weighted coverage of §9.12.5; QAS-08's auditability scenario to the business rules catalogue of §5.5 itself, which is as much an architectural artefact as a requirements one. Nothing in this verification is new measurement; it is a cross-check that the design of this chapter actually answers the scenarios Chapter 3 set, which Chapter 9 then measures.

6.11 Summary

Clean architecture was chosen over a modular monolith and over microservices for a reason specific to this project, a compiler-enforced rule where a convention had already failed once, not as a default best practice. The four layers, three persistence providers, four payment-gateway adapters and one configuration document give the system the substitutability §6.9.4 and §6.9.5 claim, in the one case that has actually been exercised and honestly not yet in the other. The pattern catalogue of §6.9 and the principle evidence of §6.9 are written so that a claim in this chapter can be checked against a file, not only against this chapter's own prose. Chapter 7 now reports how this design was actually built.

Figures and Tables

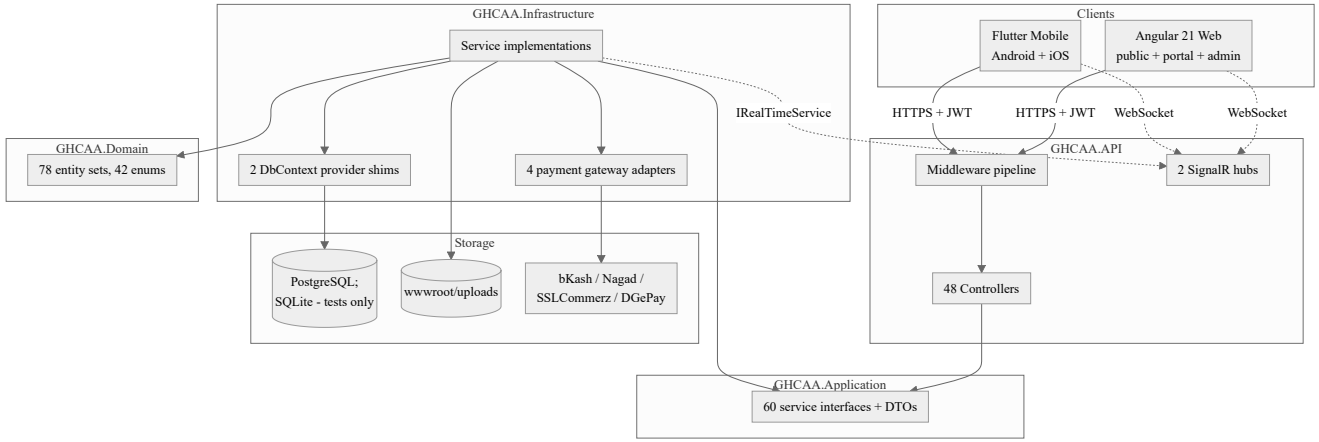


Fig. 6.1. High-level architecture diagram

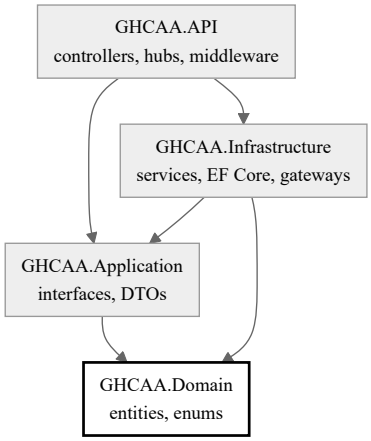


Fig. 6.2. Layered / clean architecture diagram with the dependency-inversion boundary marked

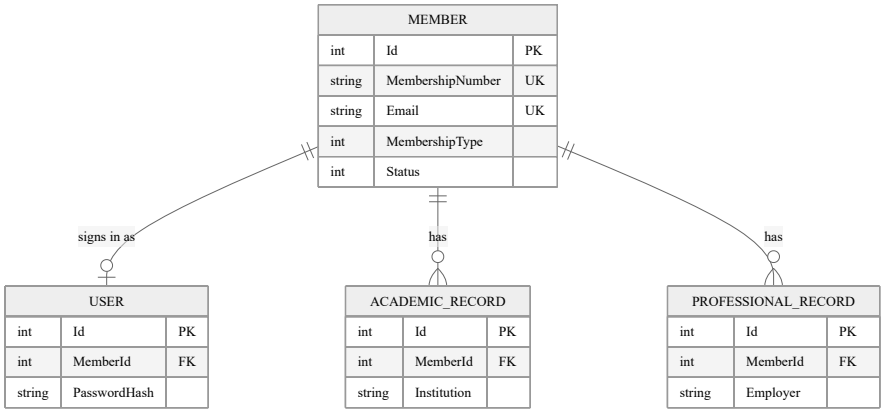


Fig. 6.3. Entity–relationship diagram, identity and records sub-model

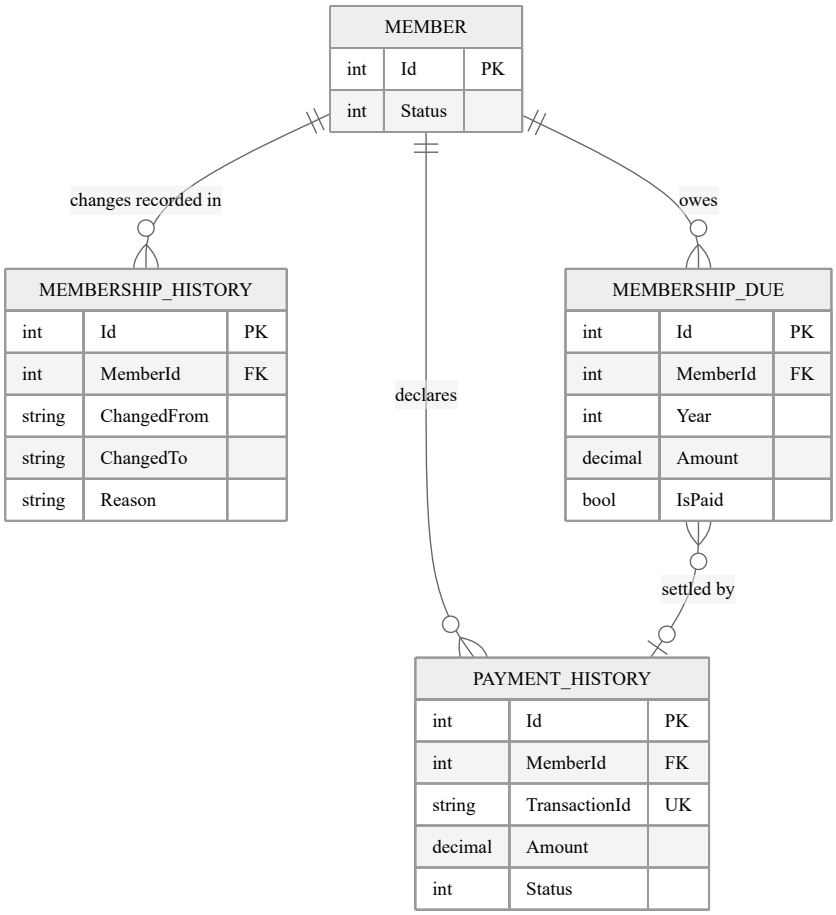


Fig. 6.4. Entity–relationship diagram, standing and money sub-model

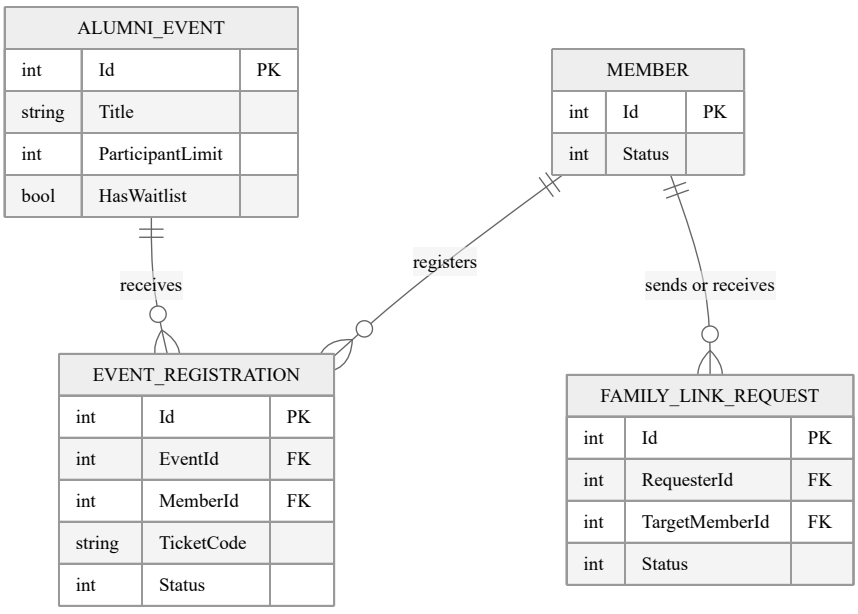


Fig. 6.5. Entity–relationship diagram, events and participation sub-model

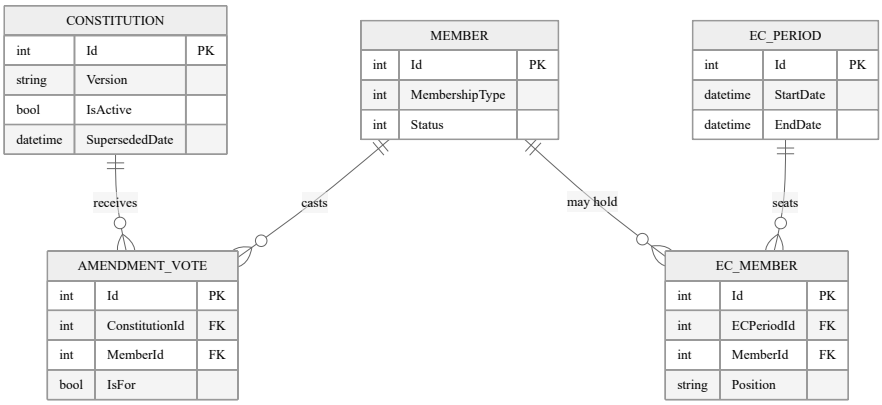


Fig. 6.6. Entity–relationship diagram, governance sub-model

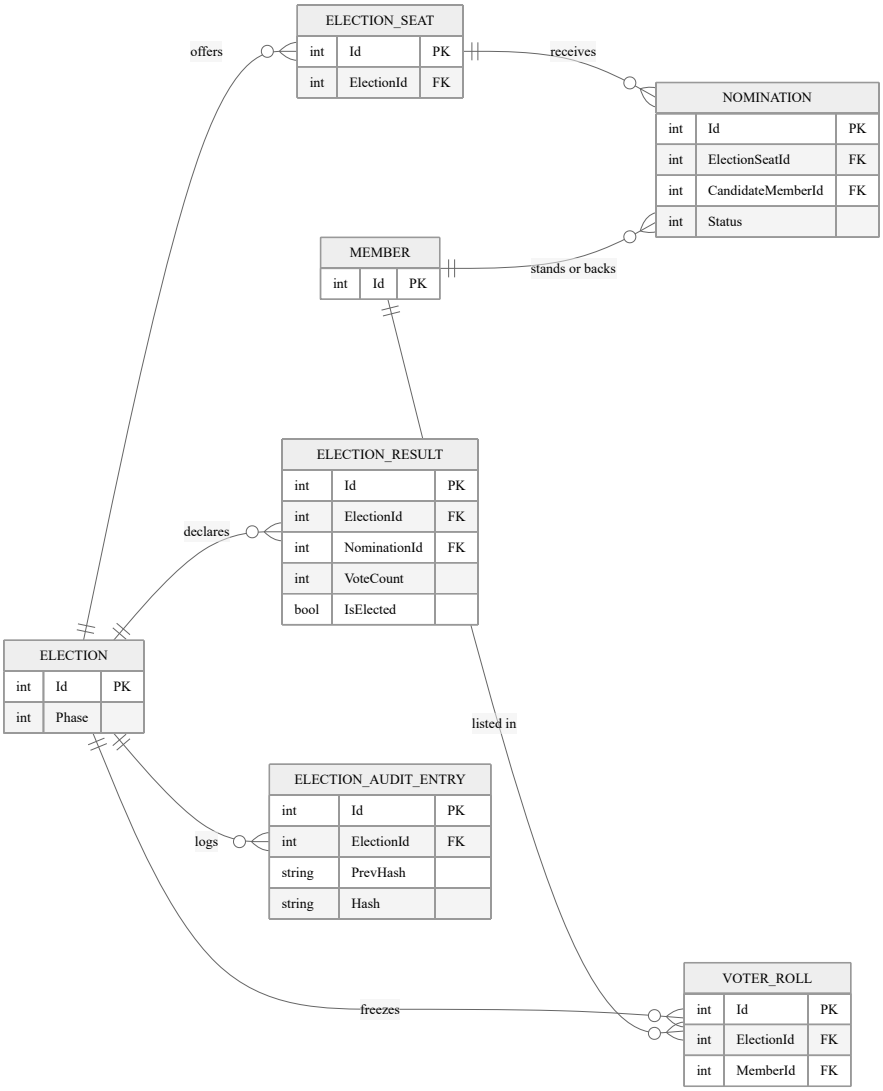


Fig. 6.7. Entity–relationship diagram, election roll and nominations sub-model

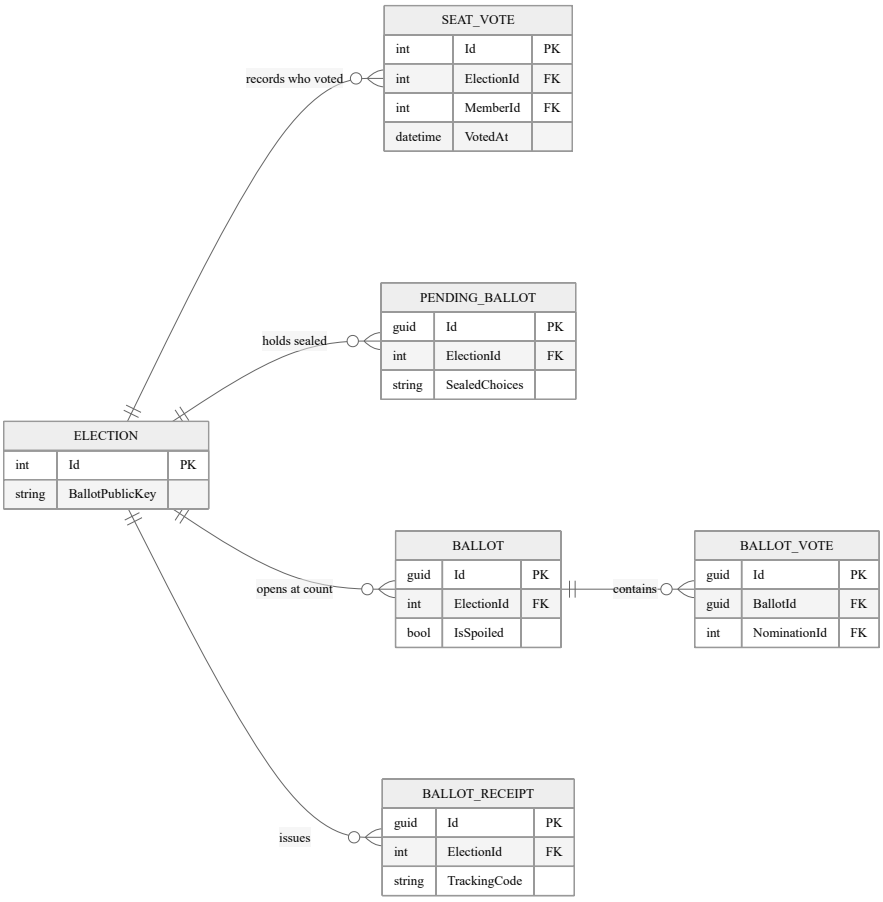


Fig. 6.8. Entity–relationship diagram, voting and count sub-model

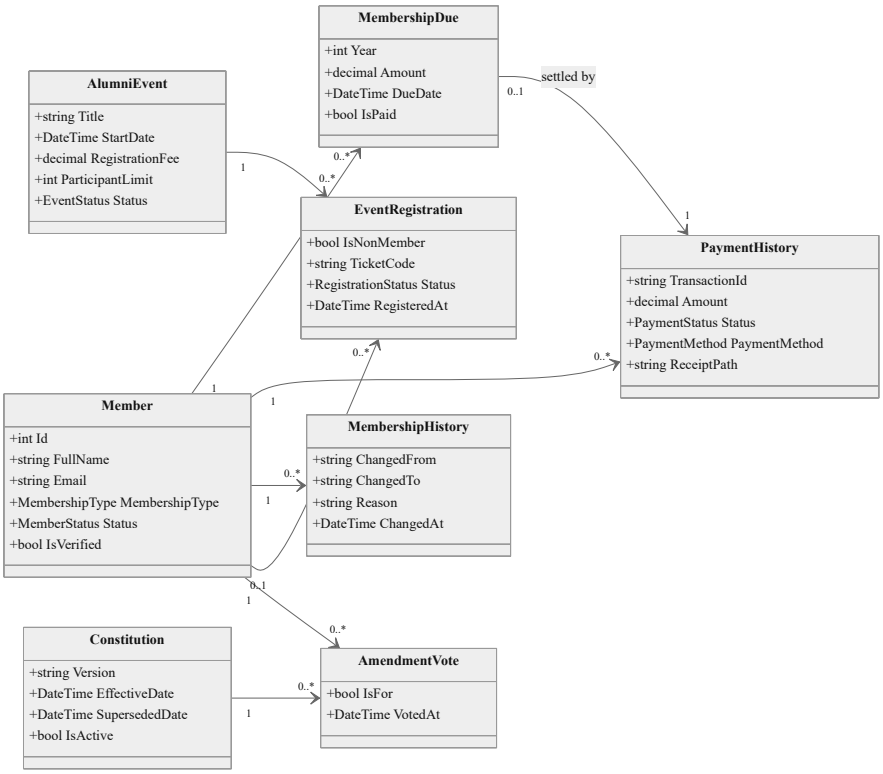


Fig. 6.9. Design class diagram: domain model

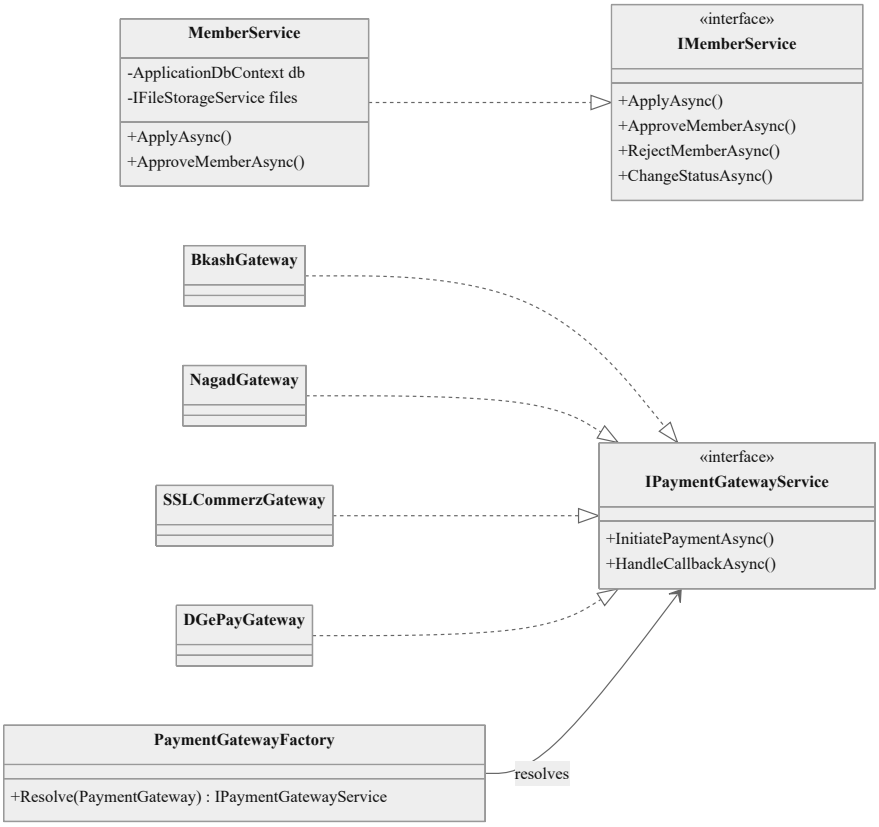


Fig. 6.10. Design class diagram: application interfaces and infrastructure services

The diagram shows the interface boundary all four gateways share; the concrete inheritance beneath it, after the 2026-09-08 refactor, is:

Gateway	Extends
`BkashGateway`	`BasePaymentGateway : IPaymentGatewayService`
`NagadGateway`	`BasePaymentGateway : IPaymentGatewayService`
`SSLCommerzGateway`	`BasePaymentGateway : IPaymentGatewayService`
`DGePayGateway`	`IPaymentGatewayService` directly

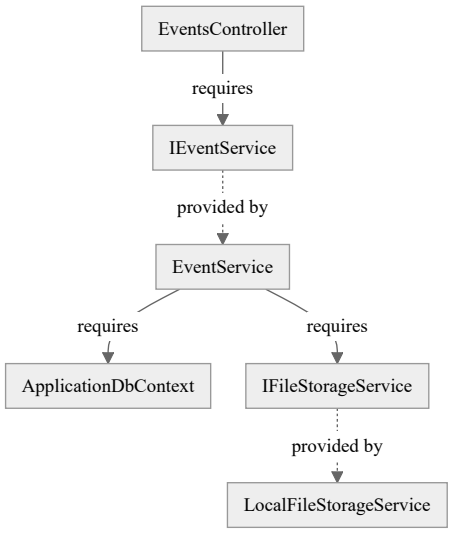


Fig. 6.11. Component diagram with provided and required interfaces

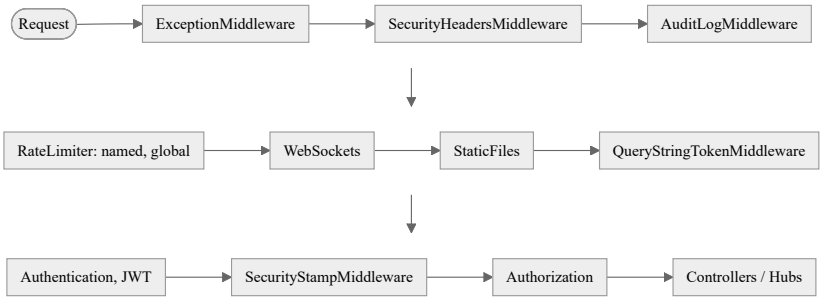


Fig. 6.12. Middleware pipeline diagram

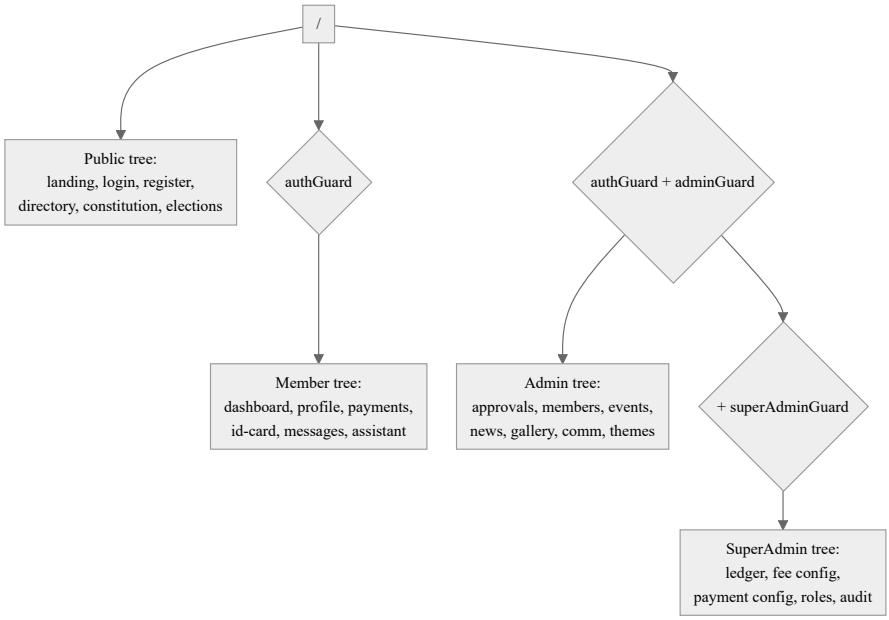


Fig. 6.13. Navigation and route map



Fig. 6.14. Site map and information architecture of the public site

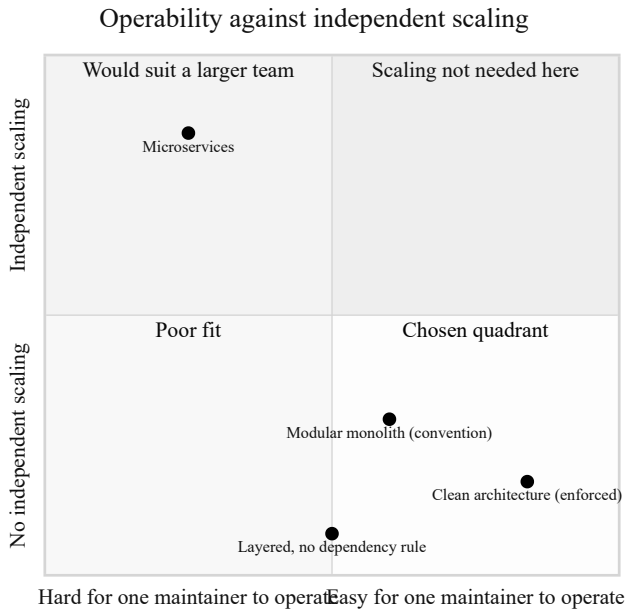


Fig. 6.15. Architectural trade-off radar

TABLE 6.2
Data dictionary (representative slice; the full dictionary is generated from the schema)

Table	Column	Type	Constraint	Description
Members	Email	string	Unique index	Login identifier alongside membership number
Members	NID	string	Unique index	National identity number, 10/13/17 digits
Members	MobileNo	string	Unique index	11-digit mobile number
Members	Status	MembershipStatus enum	Composite index with IsArchived	Drives the state machine of Figure 5.11
AmendmentVotes	(ConstitutionId, MemberId)	composite	Unique index	Enforces BR-02, one vote per member per version

Table	Column	Type	Constraint	Description
EventRegistrations	(EventId, MemberId)	composite	Unique index	Prevents duplicate registration
PaymentHistories	TransactionId	string	Unique index	Prevents duplicate crediting of one declared payment
PaymentHistories	GatewayPaymentId	string	Unique index	Prevents duplicate crediting of one gateway callback
RefreshTokens	TokenHash	string(64)	Unique index	SHA-256 hash; plaintext token never persisted
OrganizationConfigs	ConfigJson	text/JSONB	—	Whole configuration document, §6.5.2
Constitutions	IsActive	bool	Not database-enforced	Exactly-one invariant maintained procedurally, §6.5.2

TABLE 6.3
API endpoint catalogue (by controller; the full catalogue is the generated OpenAPI document)

Controller	Route	Authorisation	Notes
AuthController	/api/auth	Public	Login, social login
RegistrationController	/api/registration	Public, rate-limited	Registration wizard, OTP
MemberImportController	/api/admin/members/import	SuperAdmin	Bulk import
EventsController	/api/events	Public read, Auth register, Admin manage	Capacity and waitlist logic
FinancialLedgerController	/api/ledger and /api/financial/ledger	SuperAdmin	Append-only ledger; the controller carries both route attributes, with no comment recording why
GatewaysController	/api/gateways	Auth + public webhook	Callback verification
PaymentConfigController	/api/payment-config	SuperAdmin write, Auth read	Depends on `IPaymentConfigService` (2026-09-06; was `ApplicationDbContext` directly)
GovernanceController	/api/governance	Public read, Admin write	Constitution, EC record
AdminGovernanceController	/api/admin/governance	Admin	Committee and amendment administration
HealthController	/healthz	Public	Depends on `IDatabaseHealthService` (2026-09-06; was `ApplicationDbContext` directly)

TABLE 6.4
Design pattern catalogue (selected entries; full catalogue is §6.9 in full)

Pattern	Category	Problem and forces	Participants here	Alternative rejected
Factory	Creational	Growing set of interchangeable gateways; callers must not name a concrete type	PaymentGatewayFactory, IPaymentGatewayService	Switch statement in the controller
Strategy	Behavioural	Same operation, several incompatible implementations	The four gateway classes; the three DbContext shims	A single class with provider-conditional branches
Repository	Enterprise	Isolate persistence detail from a caller	IFileUploadRepository only	Uniform repository per entity, rejected as unneeded ceremony given EF Core's own abstraction

TABLE 6.5
Quality-attribute scenario to architectural tactic mapping

Scenario	Tactic
QAS-01 (directory latency)	Composite database index; ten-minute configuration cache keeps unrelated reads off the hot path
QAS-03 (session invalidation)	SecurityStampMiddleware, positioned after authentication and before authorisation
QAS-06 (maintainability, one field end to end)	Convention-based DI registration; single stylesheet; DTOs isolate the wire contract from schema change
QAS-07 (dependency-rule violation caught at build)	Project-reference enforcement of the dependency rule, §6.3.6
QAS-08 (auditability of the vote-eligibility rule)	Business rules catalogue, §5.5, naming the exact method for every constitutional rule

TABLE 6.6
Anti-patterns detected and remediated

Symptom	Diagnosis	Refactoring applied	Status
A service accreting both workflow and ledger-posting responsibility	God service	FinancialService split from FinancialLedgerService	Remediated
Two independent templates rendering the same member fields, drifting in order and labelling	Duplication across presentation layer	AcademicHistoryEditor, EcHistoryView and related sub-components designed	Designed, not yet built (§6.7.3)
Business state and behaviour split across entity and service	Anaemic domain model	Not corrected; kept for uniform authorisation, §6.9.16	Accepted, not remediated

PART III — CONSTRUCTION AND VALIDATION

Chapter 7 — Implementation

What this chapter owns. How the artefact was built: the mechanisms that were difficult, the alternatives weighed while writing them, and what the code does that the design did not anticipate.

What it must not repeat. Why the architecture is shaped this way, which is Chapter 6. Threats and controls, which are Chapter 8. How anything was tested, which is Chapter 9. Any measured result, which is Chapter 12. No section here is a walk through the source tree, and no listing appears unless the surrounding argument fails without it.

Unless a paragraph says otherwise, the counts in this chapter were taken against the working tree at commit `4c0f733f` on 16 September 2026, with the commands named beside each figure.

7.1 Development Environment, Toolchain and Reproducibility

The backend targets `net9` (`dotnet --version` reports SDK 9.0.310) with nullable reference types turned on in `GHCAA.Domain`, `GHCAA.Application` and `GHCAA.Infrastructure` (`<Nullable>enable</Nullable>` in each `.csproj`). No `global.json` pins that SDK version to the solution, so a contributor with a newer .NET 9 patch installed builds against whatever is on their machine rather than a version the repository states explicitly — a reproducibility gap that costs nothing today because only one developer has built this solution, but would need closing before a second one does.

The web client is Angular 21.2.22 with TypeScript 5.9.2 (`GHCAA.Web/package.json`). Its `npm run build` script is a chain rather than a single compiler invocation: `sync:docs`, `gen:org-config`, `gen:site-content`, `apply-brand`, `type-check`, then `ng build`, in that order. The middle three steps generate files `ng build` then reads as ordinary source — `apply-brand` is the institution-profile rewrite described in §7.14 — so a build run without `npm run build` (an `ng build` invoked directly, say) silently skips them and builds against whatever those generators last produced. There is no ESLint configuration in the project (`package.json` carries no `lint` script and no `eslint` dependency); the only static check the build chain runs is `type-check`, the TypeScript compiler in no-emit mode, so a style or dead-code problem that type-checks cleanly reaches `ng build` unflagged.

The mobile client targets Dart SDK `>=3.3.0 <4.0.0` (`GHCAA.Mobile/pubspec.yaml`) and is linted through `analysis_options.yaml`, which includes `package:flutter_lints/flutter.yaml` rather than a custom rule set.

Continuous integration runs from four GitHub Actions workflows in `.github/workflows/`: `ghcaa-ci-standard.yml` on pushes and pull requests against `dev`, `main` and `master`; `ghcaa-ci-preprod.yml` on the same events against `preprod`; `mobile_deployment.yml` for the Flutter build and store artefacts; and `neon_workflow.yml`, which creates and tears down a scoped Neon Postgres branch per pull request rather than pointing every PR build at one shared database.

7.2 Solution and Module Structure

The solution (`GHCAA.sln`) is nine projects rather than the classic four-layer split alone: `GHCAA.Domain`, `GHCAA.Application`, `GHCAA.Infrastructure` and `GHCAA.API` carry the layered backend Chapter 6 describes, and `GHCAA.Export` and `GHCAA.Tools` sit beside them as small, single-purpose projects rather than folders inside a bigger one — `GHCAA.Export` is one file (121 lines) and `GHCAA.Tools` three (107 lines), each kept separate because neither shares a reason to change with the layer it would otherwise have been folded into. `GHCAA.Web` (Angular) and `GHCAA.Mobile` (Flutter) are the two clients, and `GHCAA.Tests` holds the backend test suite; the clients carry their own test trees under their own roots rather than in `GHCAA.Tests`.

Table 7.1 gives file and line counts by project (`.cs` files only for the backend projects, counted with `find` and `wc -l`, excluding `bin/` and `obj/`):

Table 7.1
Size metrics by layer: files and lines of code

Project	Files	Lines
GHCAA.Domain	52	1,700
GHCAA.Application	96	2,924
GHCAA.Infrastructure (excluding EF migrations)	105	14,488
GHCAA.Infrastructure — EF migrations (`Data/Migrations`)	3	157,079
GHCAA.API	73	6,917
GHCAA.Export	1	121
GHCAA.Tools	3	107
GHCAA.Tests	87	15,723
GHCAA.Web (`src/**/*.ts`)	268	26,601
GHCAA.Mobile (`lib/**/*.dart`)	126	25,159

The migrations row is split out because it is not hand-written code: `20260907193705_InitialBaseline.cs`, its `.Designer.cs` and `ApplicationDbContextModelSnapshot.cs` together account for 157,079 of `GHCAA.Infrastructure`'s 171,567 lines, all of it EF Core's generated model-building code for the single squashed baseline migration referenced in §5.6 and §6.3.2. Reading the project's size from the combined figure without separating that row would overstate the hand-written backend by an order of magnitude.

7.3 Coding Standards, Conventions and Static Enforcement

Static enforcement differs by layer rather than following one house rule uniformly. The backend project files turn on `<Nullable>enable</Nullable>` in `GHCAA.Domain`, `GHCAA.Application` and `GHCAA.Infrastructure`, which converts a null-reference mistake into a compiler warning rather than a runtime `NullReferenceException`, but none of the four backend projects sets `TreatWarningsAsErrors` or references a general-purpose Roslyn analyzer package — the only analyzer in the solution is `NUnit.Analyzers` in `GHCAA.Tests.csproj`, which checks test-writing mistakes (a missing `await` on an assertion, for instance), not production code. A warning can therefore be introduced and merged without failing a build.

`GHCAA.Web/.editorconfig` sets formatting conventions (two-space indent, UTF-8, single quotes in TypeScript) that editors and `ng build`'s formatter honour, but nothing in CI runs a linter against them; the enforcement that does exist for the web client is the `type-check` step described in §7.1, which catches a type error, not a style one. `GHCAA.Mobile/analysis_options.yaml` is the one place a rule set is actually enforced at build time, through `flutter analyze` against `package:flutter_lints/flutter.yaml`, with no project-specific rule added on top of that default set at the time of writing.

7.4 Implementation of the Domain and Persistence Layers

`ApplicationDbContext` declares 72 `public DbSet<...>` properties (`grep -c "public DbSet<" GHCAA.Infrastructure/Data/ApplicationDbContext.cs`), 71 of them domain models and the 72nd `DataProtectionKeys`, a framework table `docs/adr/0007-data-protection-keys-in-database.md` records adding so that ASP.NET Core's data-protection keys survive a Render redeploy rather than being regenerated — and, before that fix, silently invalidating every session and every value the application had encrypted with the previous key. The schema's migration history is a single file: the prior 31-migration history was squashed into `20260907193705_InitialBaseline.cs` on 8 September 2026, discussed in §7.12 as the change that closed the build-time-out problem the old history had started causing.

Persistence is EF Core against PostgreSQL in production and SQLite in the test and local-development path, a split Chapter 6 sets out the reasoning for. The domain layer itself (`GHCAA.Domain`, 52 files, 1,700 lines) carries no EF Core reference: entities are plain classes, and every EF-specific concern — fluent configuration, converters such as the ISO-8601 `DateTime` converter §5.3 and §6.4 both describe, the `IsActive` global query filter on `AlumniEvent` — lives in `GHCAA.Infrastructure`, so a domain type can be read and reasoned about without also reading how it is persisted.

7.5 Implementation of the Application and Business Services

`GHCAA.Application` defines the interfaces the API layer depends on and `GHCAA.Infrastructure` implements them, the dependency direction Chapter 6 states as a rule. Grepping both projects for `public interface I*Service` finds 45 distinct service interface names declared across 44 files (some files declare more than one small interface alongside the one the file is named for); grepping `GHCAA.Infrastructure` for a class declared as `SomeService : ISomeService` on a single line — the common case, though a class implementing several interfaces or splitting its declaration across lines is undercounted by this pattern — finds 42 matching implementations. The two counts are close enough to confirm the interface-per-service convention holds through most of the layer without claiming an exact one-to-one mapping the grep pattern cannot actually prove.

Two conventions recur across the layer rather than being decided per service. First, a service method that performs more than one write opens its own `DbContext` transaction rather than relying on `SaveChangesAsync`'s implicit one — necessary wherever a single business operation spans several `SaveChangesAsync` calls that must succeed or fail together, and the subject of the transaction-scoping fix in §7.13. Second, a service that calls an external system (SMTP, the real-time hub, a payment provider) does so through its own interface (`IOtpService`, `IRealTimeService`) rather than a concrete client type, which is what let the fix in §7.13 move those calls out of the transaction: the transaction boundary and the external call were already two separate seams in the code, not one that had to be created for the fix.

The same boundary now carries the communication visibility work. `CommunicationService` resolves a template, substitutes member and organisation variables, selects the configured channel, and writes one `EmailLog` row for the attempted delivery. Email goes through `IEmailService`; SMS templates go through `ISmsService`. The log records the recipient member, channel, targeted or broadcast scope, status and timestamp. A missing mobile number or provider refusal is recorded as `Unavailable`, while an exception is recorded as `Failed`. The member history query takes its member ID from the caller's authenticated claims at the API boundary, so the client cannot substitute another member's ID. Direct OTP SMS still uses the provider interface directly and is not part of this history.

Work Package 37.1 uses the same application-service boundary for its persisted election workflow. The service enforces phase transitions, frozen voter-roll eligibility, officer permissions, candidate rules, secret-ballot separation and one-vote concurrency controls before writing election state. Generated election documents use the reusable document service and the active organisation profile. Election rules are frozen while any election is waiting for polling, polling or counting. A global filter, `ElectionRulesFrozenFilter`, backed by `ElectionFreezeService`, refuses a change to election settings, personas or appointments with a 409 and the code `ELECTION_RULES_FROZEN`, and audits the refusal. A SuperAdmin can lift the freeze for a set time through a route under `api/admin/elections`, 30 minutes by default and 60 at most, and must give a reason of 20 to 1,000 characters. The one change allowed during the freeze is an emergency revoke of an official, which needs a second active official holding Approve on that election and has no SuperAdmin bypass. A hosted service, `ExpiredRevokeSweep`, runs every 15 minutes and writes the audit row for any emergency revoke that expired unused. A unique `OpenKey` index on approval requests stops two identical open requests from being stored together. The appointment carries a Returning Officer flag, the approval row records who ran an approved count, and the organisation setting `CountRequesterOnly`, off by default, limits running an approved count to the official who asked for it, so anyone else is refused with `not-requester`. A user who holds election records cannot be hard deleted: the request is refused with a 409.

Work Package 37.2 carries the scholarship and student-aid programme through the same fund/call/ application/award shape the rest of the domain uses for a multi-stage workflow. A reviewer scoring an application through `ScholarshipService` never sees the applicant's name or member record — the review projection is built without that join, so blinding is a property of the query rather than a UI field left off the screen. Awarding a scholarship writes a `FinancialRecord` Grant entry through the same idempotency guard `FinancialLedgerController` already enforces elsewhere (§7.8), so a retried disbursement request cannot double-pay an award.

Work Package 37.6 is the oral-history and legacy archive: `ArchiveCollection` groups a set of `ArchiveItem` records, each one a transcript, recording or scanned document rather than a media file treated as an end in itself. The public `/legacy` page on the Angular client reuses the document-hero and document-prose shell §7.14's profile packs already style, rather than introducing a parallel public-page layout for archive content.

7.6 Implementation of the API Layer

`GHCAA.API/Controllers` holds 45 controllers exposing 333 endpoint action attributes (`grep -rhoE '\[(HttpGet|HttpPost|HttpPut|HttpDelete|HttpPatch)' GHCAA.API/Controllers --include=*.cs | wc -l`, re-run 23 September 2026), against 6,917 lines across 73 files in the project as a whole, which puts controllers themselves at roughly 16 lines per endpoint on average once routing attributes, `[Authorize]` and `[RequireStepUp]` decoration, model binding and the call into the corresponding service are counted — thin by design, since the layer's job is request/response translation and authorisation, not business logic. `ArchiveController`, `ScholarshipsController` and `CredentialVerificationController` (Work Packages 37.6, 37.2 and 37.8) follow the same shape as the rest of the layer: routing and authorisation only, with the archive lookup, blind-review scoring and short-code verification itself left to their respective application services. Two real-time hubs live in the same project rather than a separate one (`Hubs/ChatHub.cs`, `Hubs/NotificationHub.cs`), covered in §7.7.

`RequireStepUpAttribute` (§7.8) and the `SecurityStampMiddleware` check (also §7.8) are both applied at this layer rather than in `GHCAA.Application`, because both depend on the HTTP claims principal and the response status code convention, neither of which the application layer has a reason to know about.

7.7 Web Client, Mobile Client and Real-Time Features

`GHCAA.Web/src` is 268 TypeScript files and 26,601 lines (re-counted 23 September 2026), of which 102 components are declared `standalone: true` — Angular's module-free component style, used throughout rather than mixed with `NgModule`-declared components. The public `/legacy` and `/scholarships` pages (Work Packages 37.6 and 37.2) and their `ArchiveService`/`ScholarshipService` HTTP clients account for part of that growth. `styles.scss` is 3,468 lines, the shared token and utility layer §8.5's theme audit and the design-system skill both work against; component-scoped styles sit alongside it rather than replacing it, which is also the source of the scoping bugs recorded in §7.13 and the `:host-context()` pattern those bugs led to.

The build chain in §7.1 is what makes the client's institution-profile awareness (§7.14) a compile-time concern rather than a runtime one: `apply-brand` rewrites `index.html`, the sitemap and the favicon set before `ng build` runs, so the shipped bundle already carries the active profile's branding rather than fetching it after first paint.

`GHCAA.Mobile/lib` is 126 files and 25,159 lines, state managed with `flutter_riverpod` and routing with `go_router` (`pubspec.yaml`). Secure, on-device storage of the JWT and refresh token goes through a single `StorageService` (`GHCAA.Mobile/lib/core/storage/storage_service.dart`) wrapping `flutter_secure_storage`, rather than each screen touching the platform storage plugin directly — the wrapper is what made the fix in §7.13 a one-file change instead of a search-and-fix across every caller. `tool/apply_profile.dart` is the mobile equivalent of the web client's `apply-brand` script (§7.14): it rewrites the Android manifest label, the iOS display name and the launcher-icon/splash-screen generation inputs from the active profile before a platform build runs.

Real-time delivery is SignalR, wired in `GHCAA.API/Extensions/ServiceExtensions.cs` and `Program.cs` through two hubs: `ChatHub` and `NotificationHub` (`GHCAA.API/Hubs/`). `RealTimeService` (`GHCAA.API/Services/RealTimeService.cs`) is the single point application services call through to reach either hub — `MemberService`'s admin alert on a new registration (§7.8, §7.13) is one caller among several — rather than a controller or service pushing to a hub context directly, which keeps the transport detail (SignalR specifically, as opposed to a different push mechanism) behind one interface.

7.8 Security Implementation

This section describes three mechanisms and the difficulty each one answers. What each threatens and how it fits the wider control set is §8.3; this section is only the code and the reasoning behind it. Citations below were re-taken against commit `5f5ebfe4` on 15 September 2026.

Instant session revocation. A JSON Web Token is normally valid until it expires, so an admin who disables an account, a user who changes a compromised password, or a security response that needs a session killed immediately all have to wait out the token's remaining lifetime unless something else checks state on every request. `SecurityStampMiddleware` (`GHCAA.API/Middleware/SecurityStampMiddleware.cs`, lines 22-50) is that check: it reads the `SecurityStamp` claim carried in the token, looks up the current value stored against the user, and returns 401 the moment the two disagree. The stamp itself is a random value rotated whenever something should end every existing session for that user — `AuthService.cs:494` on an explicit revocation, `UserService.cs:122` and `:156` on account changes, and `MemberService_Approval.cs:108, 174, 255, 361` at points in the approval workflow where a member's standing changes enough that old tokens should no longer be trusted. The cost is a database read on every authenticated request; the alternative — a short-lived access token with no stamp check — was rejected because it trades an instant revocation for a window of up to the token's lifetime during which a disabled account keeps working. No unit test instantiates `SecurityStampMiddleware` directly; the behaviour is exercised indirectly through `AuthControllerMutationTests.cs` and, on the client side, `GHCAA.Mobile/test/session_manager_test.dart`. A middleware-level test is not yet written.

Refresh-token reuse detection. Rotating a refresh token on every use limits how long a stolen one stays useful, but rotation alone does not detect the theft: if an attacker captures a refresh token before its legitimate holder uses it, both parties now hold a valid-looking credential and the first one to redeem it invalidates the other's copy without anyone noticing which was which. `TokenService.RotateRefreshTokenAsync` (`GHCAA.Infrastructure/Services/TokenService.cs`, lines 136-180) closes that gap: if the presented token hashes to a row already marked revoked, that is not a stale request but a replay — the legitimate holder has already moved on to the token that replaced it — so the response is to revoke every refresh token belonging to that user (`RevokeAllRefreshTokensAsync`, lines 182-187) and rotate the security stamp in the same call, killing any access token already issued as well. The ticket that asked for this, 82.18, is named in the code comment at line 144. `docs/ARCHITECTURE_AUDIT_2026-09.md` still describes this as missing at its line 255-273; that finding predates the fix and should not be read alongside the current `TokenService.cs` as if both were still true. `TokenServiceTests.cs` covers the reuse path directly: `RotateRefreshToken_ReplayOfARotatedToken_RevokesWholeFamilyAndRotatesStamp` (line 136), alongside `StoreAndRotateRefreshToken_ShouldRotateCorrectly` (line 103) and `RevokeAllRefreshTokens_ShouldMarkAllRevoked` (line 170).

Step-up re-authentication for destructive actions. A session that is valid for browsing is not necessarily one that should be trusted to delete a user, change a role, or alter a financial record without asking again — an admin's browser left open at a shared desk is a real exposure the ordinary session model does not address. `RequireStepUpAttribute` (`GHCAA.API/Filters/RequireStepUpAttribute.cs`, lines 19-41) reads a `step_up_verified_at` claim (`StepUpClaim.cs`, line 15) set when the user last re-confirmed their identity, checks it against a time-to-live, and returns 403 with `Code = "STEP_UP_REQUIRED"` if the claim is missing or has expired. The filter is applied 15 times across six controllers: `RolesController` (`CreateAdmin`, `AssignRole`, `DeleteUser`, `DisableUser`, `EnableUser`, `ResetPasswordAdmin`), `AdminController` (`SyncMembers`, `ArchiveMember`, `BulkArchiveInactive`, `ResetPasswordAdmin`), `AdminGovernanceController` (`DeleteECMember`), `FinancialLedgerController` (`AddRecord`, `UpdateRecord`, `DeleteRecord`), and `PaymentConfigController` (`DeleteConfig`). The time-to-live was originally 30 days and was cut to 30 minutes (`StepUpClaim.cs`, lines 22-28) after a security review on 29 August 2026 concluded that a step-up claim good for a month gave away most of the protection a step-up check is meant to provide. `RequireStepUpAttributeTests.cs` covers the filter directly across six cases; `DestructiveStepUpActionsTests.cs` covers the controller/service contract for two of the delete actions it protects.

7.9 Document Generation

`IDCardService` (`GHCAA.Infrastructure/Services/IDCardService.cs`, 298 lines) generates every printable member artefact: an ID card, a membership certificate, and a PDF version of each. Its constructor (line 25) takes only `ApplicationDbContext` and `IOrgConfigService`, so a card carries no state of its own beyond what those two sources supply at the moment it is requested — there is no separate "card record" to keep in sync with the member row it describes.

Four public methods split along two axes: card or certificate, and data-URI (for on-screen preview in the Angular and Flutter clients) or PDF (for download and printing). `GenerateIDCardDataUriAsync` (line 67) and `GenerateCertificateDataUriAsync` (line 119) return an inline PNG; `GenerateIDCardPdfAsync` (line 152) and `GenerateCertificatePdfAsync` (line 210) build the same layout through QuestPDF's fluent API and return raw bytes. All four now call a shared private method first, `IssueCredentialAsync` (lines 42-65), rather than each building its own verification URL from the member's own data.

Work Package 37.8 replaced what that URL used to be. A card generated before this work package pointed its QR code at `/verify/{member.MembershipNumber}` — a value read straight off the member's own record, which meant it never expired, could not be revoked without changing the member's membership number, and was

guessable from any other document that carried the same number. `IssueCredentialAsync` instead generates a 10-character code (`CredentialCodeGenerator.Create()`, line 55), retries on collision against `IssuedCredentials` until it lands on a code nobody already holds (lines 49-58), and persists an `IssuedCredential` row — member, credential type, short code, issue timestamp — before returning a verify URL built from that code, not from the member row (`{PortalBaseUrl}/verify/{ShortCode}`, line 63). Generating a new card or certificate issues a new credential rather than reusing the last one, so an old printout's QR code and a freshly generated one point at different, independently revocable rows even for the same member.

`GET api/verify/{shortCode}` (`CredentialVerificationController.Verify`, lines 18-27) is the page that URL resolves to. It rejects anything that is not exactly 10 characters before touching the database, then calls `IDCardService.VerifyCredentialAsync` (`IDCardService.cs`, line 269), which looks the code up, checks it is not revoked and has not passed its expiry if one is set, and returns `CredentialVerificationDto { Valid, MemberName, MembershipType, IssuedOn, Status }` — nothing else from the member record. The endpoint carries `[AllowAnonymous]`, since the whole point is that someone checking a printed card has no account to log into, but the response never exposes an address, contact detail or raw member ID; anyone who fails validation, tries a code that never existed, or hits an expired or revoked one is told the same amount (`Valid: false`, or a 404 for an unrecognised code) as everyone else. A separate endpoint, `POST api/verify/{shortCode}/revoke` (`CredentialVerificationController.Revoke`, lines 29-36), lets an admin invalidate one credential with a required reason; it is not reachable without the admin policy the controller applies to it.

QR encoding itself is unchanged from before this work package: QRCoder at error-correction level Q (`QRCodeGenerator.ECCLevel.Q`), embedded in the artefact at the same four call sites. What changed is only what the code resolves to.

`IssueCredentialAsync` also closes part of the duplication the four generation methods used to carry independently — building the verify URL and issuing the underlying credential now happens once, rather than once per method. The remaining duplication (laying out the same header/footer per artefact type) has not been factored into a shared template as of this writing; it is a candidate for the deferred `GHCAA.Export` restructuring the outline does not require this chapter to resolve.

7.10 Constitution Publication Pipeline

The constitution reader (`docs/CONSTITUTION_PUBLISHING.md`) is built on one rule stated at the top of that document: the application always serves the latest ratified constitution, and no page, component or stored link may pin a specific version. Publishing a new version is a single command, `tools/constitution/publish_constitution.py` (309 lines, a documentation-side Python script using `pymupdf`, not an application dependency), run against the ratified PDF once it lands in `GHCAA.Web/public/assets/`. The script extracts the document's text into the paragraph/heading/list shape the Angular reader parses, rewrites `GHCAA.Infrastructure/Data/Seed/constitution.json` as a single active record, and repoints the one hardcoded fallback string (`CONSTITUTION_PDF_FALLBACK` in `constitution.ts`) at the new asset.

Editing that seed file is not what makes a new version live. `Program.cs` calls `Database.EnsureCreated()` at startup, which is a no-op once the tables already exist — on preprod and production the EF Core `HasData` seed step never runs again after first boot. Changing the JSON alone would sit there unread. `ConstitutionSeeder.SyncAsync`, called on every boot, closes that gap: it is idempotent, inserts a version it cannot find by matching on `Version`, refreshes a stored version in place when the seed text changes, and marks anything no longer in the seeded set as superseded rather than deleting it — a genuine prior version keeps its row, its `AmendmentVote` records and its own PDF link, because the Version History panel serves from that link and would break if the row disappeared. `PlaceholderVersions` (`ConstitutionSeeder.cs`, line 32; currently `{ "1.2.0" }`) marks the one exception: entries that only ever held internal placeholder text, which the seeder removes outright rather than preserving as history.

The rest of the client surface follows the active row rather than a file path: the reader renders whatever `GET /api/governance/constitution` returns, the landing page's "Read Constitution" link routes to that reader instead of a static file, and the version banner and history panel both read off the same query. `CONSTITUTION_PDF_FALLBACK` is the only version-bearing string left in application code, and the publishing script owns it exclusively — nothing else in the codebase is meant to hardcode a constitution PDF path.

`docs/CONSTITUTION_PUBLISHING.md` records three traps specific to the source PDFs that cost real diagnosis time and are not bugs in the extractor: the exported document's cover page can carry a stale version label (only the closing colophon is authoritative), the Google Docs export wraps every styled run in zero-width space characters that the script must interpret rather than strip outright (a doubled fence marks a swallowed word-break, a single fence does not), and a paragraph that crosses a page boundary arrives as two separate text blocks that the script has to rejoin. The v4.2 source document also carries one recorded defect of its own: Article V, Section C, item 6 still contains a leftover editing instruction ("TReplace the 21-day election notice rule with: …"), carried into the published text verbatim because the extractor's job is to transcribe the ratified document, not correct it.

7.11 Third-Party Libraries: selection criteria

Twenty-six distinct NuGet package references appear across `GHCAA.API`, `GHCAA.Application`, `GHCAA.Infrastructure` and `GHCAA.Export` (`grep -rh "PackageReference" ... | sort -u`, run against this tree). The web client's `package.json` lists 13 runtime dependencies; the mobile client's `pubspec.yaml` lists roughly 51 dependency entries once dev-only and transitive-only lines are included in the count. Most of these were adopted without a recorded alternatives comparison — `QuestPDF` (2026.2.3) for PDF generation, `QRCoder` (1.8.0) for the verification codes described in §7.9, `MailKit` (4.18.1) for outbound email, `ClosedXML` (0.105.1) for spreadsheet export, `Dapper` (2.1.72) alongside EF Core for the handful of read paths that favour a raw query over LINQ — and this chapter does not manufacture a selection rationale for choices the repository itself does not document.

One removal is documented in enough depth to describe honestly: `docs/adr/0006-drop-mysql-provider.md` records that `GHCAA.Infrastructure/DependencyInjection.cs` originally switched across three EF Core providers (SQLite, MySQL, PostgreSQL), but only the PostgreSQL migration tree (`Data/Migrations/PgSql/`) was ever complete, and nothing in the codebase actually called the MySQL path. The decision removed `Pomelo.EntityFrameworkCore.MySql` and its supporting classes (`MySqlApplicationDbContext`, `MySqlDesignTimeDbContextFactory`) entirely rather than leave a provider branch with no working migrations and no caller, and kept SQLite only as a fast, migration-free path for the test harness, not a deployment target. It is the one case in this repository where "why this library and not another" has a written answer rather than an inferred one.

7.12 Software Configuration Management

The repository carries 379 commits on `HEAD` and eight local branches: `dev`, `preprod`, `release-1`, `release-2`, `release-3_b4_generic_N_refactor`, `release-4_white_paper`, `prepod-election-refactoring` and `backup/2026-10-01-column-repair`, plus `dev-mobile` and `mobile_app` that exist only on the remote, as of 31 October 2026. `dev` is named as the integration branch, but `preprod` (the branch this chapter was written from) is ahead of it, not behind: `git rev-list --count dev..preprod` gives 124 and `preprod..dev` gives 0. Work has been landing on `preprod`, the branch that deploys to the staging environment described in §10, and `dev`

has not been brought up to it. The `release-N` branches are not conventional long-lived release trains — their names (`release-3_b4_generic_N_refactor`, `release-4_white_paper`) read as snapshots taken before a specific piece of work, not as a version-numbered release policy; no tag-based release identification scheme is in force as of this writing, and no `CHANGELOG.md` or equivalent exists at the repository root.

Four GitHub Actions workflows enforce change control mechanically rather than by review policy alone: `ghcaa-ci-standard.yml` (dev/main/master), `ghcaa-ci-preprod.yml` (preprod), a mobile build workflow (`mobile_deployment.yml`) and a per-pull-request Postgres branch provisioner (`neon_workflow.yml`) that gives every PR its own disposable database rather than a shared one. `docs/TODO.md` is the actual change-control ledger for this project — every planned or completed piece of work is an item there with a status, not a separate issue tracker — and §11 of this book covers its structure and the effort model built on top of it in full; this section does not repeat that.

The single squashed `InitialBaseline` migration discussed in §7.4 is itself a configuration-management decision worth naming here: it replaced a prior history of 31 incremental migrations with one file, trading the ability to replay each historical schema change step by step for a much smaller and faster-to-apply migration chain. §6.3.2 covers the reasoning in more depth; this section notes only that the choice was made deliberately, not as a side effect of losing history.

7.13 Notable Implementation Challenges and Their Resolution

Registration could commit a member row without ever sending the OTP that made the account usable. `MemberService.RegisterAsync` (`GHCAA.Infrastructure/Services/MemberService.cs`, line 78) writes the new member, an initial payment record and a set of uploaded files across several `SaveChangesAsync` calls before the registration is complete. The hypothesis was that any one of the two side effects that follow — sending the OTP (`_otp.GenerateAndSendOtpAsync`, line 290) and alerting admins over SignalR (`_realTimeService.SendAdminAlertAsync("NEW_REGISTRATION", ...)`, line 299) — could throw (a mail provider outage, a disconnected hub) partway through, and depending on where the write sat relative to that failure, the member row could end up committed with no OTP ever sent, leaving an account nobody could verify. The evidence was in `MemberServiceTests.cs`: `RegisterAsync_WhenOtpSendFails_ShouldStillCommitRegistration` (line 352) and `RegisterAsync_WhenAdminAlertFails_ShouldStillCommitRegistration` (line 367) exist specifically to pin the resolution. The fix wraps the whole registration in one explicit transaction (`_db.Database.BeginTransactionAsync(System.Data.IsolationLevel.Serializable, cancellationToken)`, line 80, committed at line 271) and moves both notification calls to after the commit, outside the transaction, so a failure in either one cannot roll back a registration that already succeeded and cannot leave a half-written member row either — the database write and the two best-effort notifications no longer share a failure path. `docs/TODO.md` item 83.1 records this as done on 16 September 2026; the file shows as modified in the current working tree, and this section describes that in-progress state, not a settled historical fact.

A locked file on Windows could abort a mobile logout before it reached the sign-in screen. `StorageService` (`GHCAA.Mobile/lib/core/storage/storage_service.dart`) wraps every `flutter_secure_storage` call the mobile app makes, including `clearAll()` (line 159), the method the logout flow on `dashboard_screen.dart` calls before navigating away. On Windows, the secure storage backend keeps its values in a DPAPI-encrypted file that can briefly be held open by the OS; deleting a key while that lock is held throws, and because `clearAll()` deleted each key directly, one exception during logout meant every delete after it in sequence was skipped and the navigation that should have followed never ran — the user stayed on the dashboard behind what looked like a frozen tap. The fix is `_safeDelete` (line 117), a private helper that wraps every delete in its own try/catch, used at every call site in the file (the JWT key at line 69, the refresh token at line 108, `clearAll()`'s two credential keys at lines 181-182). One locked file can now fail its own delete without blocking the ones after it or the navigation that follows. Because `StorageService` is the single wrapper every caller in the app already went through, the fix was contained to this one file rather than needing a change everywhere secure storage is used.

7.14 Institution Profile Packs and White-Label Configuration

Every string, seed row and asset described elsewhere in this chapter names Govt. Haraganga College by default, because the system was built for one association and had no reason to be otherwise until Work Package 62 asked what it would take to run the same codebase for a second one. The constraint that shaped the answer was not technical but organisational: the live deployment could not be touched. `ORG_PROFILE` is unset on every environment this system runs in today, and any change whose correctness depended on someone remembering to set it first was rejected on that basis alone, not because it was hard to build.

The mechanism is a profile pack: a folder under `profiles/<name>/` (`profiles/ghc/`, `profiles/default/`) holding `org-config.json`, an institution's own seed data, static page text and image assets. `IInstitutionProfileProvider` reads the `ORG_PROFILE` environment variable at boot, resolves it to a folder, and falls back file by file to `profiles/default/` for anything the named profile does not supply — the same defaulting rule `Path.Combine`-based seed resolution elsewhere in the codebase already used, extended rather than replaced. `OrgConfigService.BuildDefaults()` is the one place that decides whether the pack is trusted: it drives configuration only when `ProfileExplicitlySelected` is true, and falls back to the hardcoded values the service always had otherwise. An unset `ORG_PROFILE` therefore behaves exactly as it did before this work package, which is the property the constraint above demanded, and `OrgConfigGoldenSnapshotTests` freezes a snapshot of the pre-Work-Package-62 output specifically so a future change cannot alter that behaviour silently. The same guard shape recurs at every layer this work package touched: the ASP.NET Core data-protection application name, the Angular boot-time branding fallback, and the Flutter mobile application identifier are all read from the pack only once a profile is explicitly named, because each of them protects something that breaks if it changes under a live deployment — session cookies, first paint before the API responds, and the published app's store listing, respectively.

Seed data required a second decision, independent of the first: `docs/SEED_CLASSIFICATION.md` grades every seed file by whether its content is structural (every institution needs the same rows), shared-shape (every institution needs a row here, but not this one's), or historical (an empty table is the only correct starting state for a new institution). Moving the third class out of `Data/Seed/` and into `profiles/ghc/demo-data/` exposed two latent defects that had never been visible before, because the class of data being moved had always been present in the fallback path that both defects shared. `ApplicationDbContext.LoadSeed`'s test for whether it was safe to read a profile pack checked whether the EF Core design-time tooling assembly was loaded into the process, which is true for an ordinary test run as well as for an actual `dotnet ef` invocation, since the API project references that package for its own migration tooling; and the project file that copies seed fixtures into the test build output had never listed the `Data/Seed/Visual/` subfolder at all. Between them, every affected integration test had been silently reading the pre-migration seed file instead of the intended fixture set for as long as either defect existed, undetected because the fixture and the real data it stood in for happened to satisfy the same foreign keys. Replacing the assembly-scan check with the framework's own `EF.IsDesignTime` flag and adding the missing copy rule fixed both; the full backend suite (590 tests) is the evidence that nothing else depended on the accidental behaviour.

Two build-time generators keep the web and mobile clients honest to the same pack rather than a second, hand-copied one: `GHCAA.Web/scripts/apply-brand.mjs` rewrites `index.html`, the sitemap and the favicon set from the active profile before `ng build` runs, and `GHCAA.Mobile/tool/apply_profile.dart` does the equivalent for the Android manifest label, the iOS display name, and the launcher-icon/splash-screen generation inputs. Both are idempotent against the GHC profile specifically so that running them changes nothing observable in the current build — verified by diffing their output against the committed files rather than assumed.

`scripts/brand-lint.mjs` scans the three client codebases for the literal strings this work package removed and reports any that return, currently warn-only until Work Package 62.41's acceptance test exists to prove a `default`-profile build is actually clean before that check is allowed to fail a build.

Work Package 82.58 added the second layer the framework needed before it was useful to a development environment other than the GHC deployment: `profiles/default/demo-data/` now holds a minimal but fully-relational sample dataset — members, users, an EC history, news items, gallery entries, financials and membership dues — sized to exercise every foreign-key path in the schema without importing any real alumni record. `DatabaseBootstrapperExtensions` seeds this pack when `ORG_PROFILE` is unset and the environment is non-production, so a clean `dotnet run` in development reaches a usable state without any manual SQL. A companion `profiles/default/constitution.json` provides a placeholder constitution for the same reason: the `ConstitutionSeeder` would abort otherwise, because it enforces the always-latest invariant regardless of profile.

One path-resolution defect surfaced during this work: `ResolveProfilePackPath` had been matching file names by exact string, so a file stored as `ec_history.json` would not be found when the pack referenced it as `ec-history.json` (or vice versa). The fix normalises both the stored name and the lookup key to canonical low-dash delimiters before comparing, with a secondary hyphen-normalised pass, so the resolver is now format-agnostic. The change is covered by the `SeedDataIntegrityTests` suite, which was extended in the same work package to validate both the `ghc` and `default` profile packs: every referenced file must exist, every JSON document must parse, and every foreign-key reference within the pack must resolve against the schema — run for both profiles on every CI execution.

What this work package does not close is recorded rather than hidden: 631 real alumni records, their password hashes and their payment history are written as literal values into eight already-committed EF Core migrations, not only into the seed files this section describes moving, so a second institution's database would still not start empty until that separate problem (Work Package 62.31/82.31) is resolved. `docs/INSTITUTION_ONBOARDING.md` states this plainly to whoever deploys the system next, rather than letting a profile pack's existence imply a safety the migration chain does not yet provide.

7.15 Summary

The nine projects described in this chapter split cleanly by responsibility — domain, application interfaces, infrastructure implementations, API, two client front ends, and three small single-purpose helpers — but the enforcement of that split is uneven. Nullable reference types are on in the three backend layers that matter most for correctness; nothing stops a warning from being ignored, and no analyzer beyond `NUnit.Analyzers` runs across the solution. The web client checks types but not lint rules; the mobile client is the only place `flutter analyze` runs against a real rule set at build time. A pair of closed work-package items left a durable mark on how the code is written rather than just what it does: the transaction-and-notification-ordering fix in `MemberService.RegisterAsync` (§7.13) and the per-call error isolation added to `StorageService` (§7.13) both replaced an implicit assumption — that a secondary side effect cannot fail in a way that matters — with an explicit one. The institution-profile-pack mechanism (§7.14) and the security middleware (§7.8) are the two areas of this codebase with the most deliberate design behind them, both driven by a constraint that could not be worked around: a live deployment that could not be touched, and an authentication surface that had to survive a stolen token. Chapter 6 covers the architecture these choices sit inside; Chapter 9 covers how the resulting system was verified.

Figures and Tables

Table 7.2 lists a representative sample of the dependencies named across §7.9 and §7.11; the full backend package list is in the four `.csproj` files under `GHCAA.API`, `GHCAA.Application`, `GHCAA.Infrastructure` and `GHCAA.Export`, the full web list in `GHCAA.Web/package.json`, and the full mobile list in `GHCAA.Mobile/pubspec.yaml`. "Alternative considered" is left blank except where a repository record documents one.

Table 7.2
Selected third-party dependencies

Library	Version	Purpose	Alternative considered
QuestPDF	2026.2.3	ID card, certificate and financial document PDFs (§7.9)	*[Not documented]*
QRCoder	1.8.0	Verification QR codes embedded in ID cards and certificates (§7.9)	*[Not documented]*
Npgsql.EntityFrameworkCore.PostgreSQL	9.0.4	Production database provider	Pomelo MySQL provider — removed, ADR-0006
Microsoft.EntityFrameworkCore.Sqlite	9.0.20	Test/dev database provider only	—
MailKit	4.18.1	Outbound email (OTP, notifications)	*[Not documented]*
ClosedXML	0.105.1	Spreadsheet export	*[Not documented]*
@microsoft/signalr	^10.0.0	Web client hub connections (§7.7)	*[Not documented]*
flutter_secure_storage	^10.3.2	Mobile token/credential storage (§7.13)	*[Not documented]*

The outline specifies twelve artefacts for this chapter: Figures 7.1-7.8 (module structure, dependency matrix, web build pipeline, Git branching model, constitution publication flow, file upload flow, membership-due calculation flow, a control-flow graph with cyclomatic complexity), Listings 7.1-7.n, and Tables 7.2-7.1. Table 7.1 (size metrics by layer) is in §7.2. Table 7.2 is above. Table 7.2 (a module implementation-status matrix) and all eight figures are not built in this pass: `docs/TODO.md` item 67.3 defers the seventeen per-component diagrams and six chapter-level charts for Chapters 7-13 to a separate, later work item, and a status matrix for this chapter would need to cross-reference the WBS component mapping Chapter 11 already builds rather than duplicate it. Both are left as open work rather than filled with a diagram or table that does not carry real information.

Chapter 8 — Security, Privacy and Trust

What this chapter owns. The threat model, the controls that answer each threat, where each control is enforced in the code, and the risk left over.

What it must not repeat. The security architecture's rationale, which is §6.6. The implementation narrative of the security code, which is §7.8. The execution of security tests, which is §9.8. Every control here names the threat it mitigates; a control that names no threat does not belong in the chapter.

8.1 Security Objectives and Assumptions

The security work in this system answers to NFR-S1 through NFR-S8 of §3.4: ASVS level 2 conformance with every gap recorded rather than hidden (NFR-S1), adaptive password hashing (NFR-S2), rate limiting on authentication and API traffic (NFR-S3), uniqueness of national identity number, mobile number and email enforced at the database (NFR-S4), session invalidation within one request of a credential or status change (NFR-S5), no cross-member disclosure by role or ownership (NFR-S6), validated and isolated file uploads (NFR-S7), and server-side sanitisation of member-supplied rich text (NFR-S8). Chapter 9's security testing, §9.8, is what checks these were met; this chapter states what was built to meet them and what was not.

Three assumptions bound the threat model and are stated here so the rest of the chapter can be read against them rather than argued afresh in each section:

One maintainer, one production account. The system is operated by a single maintainer with administrative access to the database, the file store and the deployment pipeline. There is no separation of duties between the person who writes the code and the person who could, in principle, alter data directly. The security posture therefore protects the system against outside attackers and against a compromised member or officer account; it does not protect against the maintainer acting in bad faith, which is a governance question, not a control this chapter can supply.

The Association holds no payment-processor relationship. No merchant account, no gateway credential and no cardholder data touch the system in production. §8.7 gives the security reasoning this assumption rests on.

The platform is not the returning officer. §2.7 sets out why the system supports governance — publishing the constitution, maintaining the voter roll, running amendment votes and member polls — without conducting a binding election with the evidentiary guarantees Bernhard et al. require of one [27]. §8.8 restates the boundary this draws for security purposes specifically: what a poll result here is trusted to mean, and what it is not.

8.2 Threat Modelling (STRIDE)

The model in Table 8.1 follows Shostack's STRIDE decomposition [26]: each row names a threat category, the asset and entry point it applies to, and the control that answers it, with a pointer to where that control lives in the code. The assets are member personal data (NID, date of birth, contact details, photographs), financial records (dues, payment history, payment-proof images), governance records (the constitution, amendment votes, member polls), and session credentials (access and refresh tokens). The entry points are the public and authenticated HTTP API, the two SignalR hubs, the static file server, and the two upload roots described in §8.5.

Figure 8.1 draws the same boundaries as a data-flow diagram: the browser and mobile clients outside the trust boundary, the middleware pipeline as the checkpoint every request crosses before reaching a controller, and the database and file store as the two stores an attacker who defeats the pipeline would reach. Figure 8.5 later in this chapter shows one path through that pipeline in sequence form, for the specific case of a request that is rejected. Figure 8.2 traces the same threats to a concrete goal, an attacker after account takeover or a fraudulent payment credit, as an attack tree rather than a data-flow diagram. Figure 8.6 lays the controls answering these threats out as the layers a request must pass through in order, from the network up to the audit record.

Two entries in Table 8.1 are marked partial rather than closed. Repudiation of governance actions carried out through `GovernanceService` — ending a committee member's term, recording a poll result — relies on the acting admin id being recorded on the row (§8.10's Class A discipline), not on a cryptographic signature over the action, so a compromised admin account can repudiate less convincingly than it could forge. Denial of service against the two SignalR hubs has no rate limit of its own; the global rate limiter of §8.6 covers the HTTP surface, and the hubs are reachable only to an authenticated connection, but no per-connection message-rate cap exists at the time of writing. Both are carried into Table 8.5, the residual risk register.

8.3 Authentication and Session Security

Three threats against the session model are named here; §7.8 has the mechanism and the code for each.

A JWT normally stays valid until it expires, so the threat is a token that should be dead — because the account was disabled, the password changed, or a security response demanded it — but is not, simply because it has not yet reached its expiry. The control is `SecurityStampMiddleware` (§7.8), which checks a rotating stamp against the database on every request and rejects the token the moment the two disagree, so revocation does not wait on token lifetime.

A refresh token is redeemed once and replaced, but rotation alone does not tell the legitimate holder and a thief apart if the thief redeems a copy first. The threat is that theft going undetected while the stolen token is still used. The control, in `TokenService.RotateRefreshTokenAsync` (§7.8, ticket 82.18), treats a revoked token being presented again as the signal that a copy was stolen and revokes every refresh token belonging to that user, not only the one presented.

A session that authenticates a user for ordinary use is not the same guarantee that the person at the keyboard right now is still them, and an admin session left open at a shared desk is the concrete case. The threat is a destructive or financial action carried out on someone else's authority through a session that was never re-confirmed. The control is `RequireStepUpAttribute` (§7.8), applied to 15 actions across six controllers, which requires a re-authentication claim no older than thirty minutes before it lets the request through.

8.4 Authorisation Model and the Role–Permission Matrix

Authorisation has two independent layers, and NFR-S6 is met only where both agree: a request must carry a permitted role, and, for anything scoped to one member's own data, the caller must own the resource it names.

Role layer. `GHCAA.Domain/Constants.cs` (lines 7-9) declares three roles: `SuperAdmin`, `Admin` and `Member`. `ServiceExtensions.AddAppAuthorization` (`GHCAA.API/Extensions/ServiceExtensions.cs`, lines 73-91) registers three cumulative policies from them — `SuperAdminOnly` accepts only `SuperAdmin`; `AdminOnly` accepts `SuperAdmin` or `Admin`; `MemberOnly` accepts any of the three roles — so an officer inherits an ordinary member's access rather than the two being separate grants that could drift apart. The same method sets a `FallbackPolicy` requiring an authenticated user for any endpoint that names no policy of its own (ticket 3d), so

a controller action added without an explicit `[Authorize]` attribute is rejected by default rather than left open by omission. Figure 8.3 draws the three roles against the capability groups they reach; Table 8.2 gives the same mapping as a checklist.

Ownership layer. A role check alone would let any `Member` read any other member's payment history or profile fields, which NFR-S6 forbids. Endpoints that return or modify one member's data compare the resource's owning member id against the id carried in the caller's own token as well, independently of role, before either masking the response (§8.9) or refusing it outright. §9.7's authorisation test suite is what exercises this against an unentitled principal; this section states that the check exists and where the two roles it depends on are defined, not that it has been measured — that measurement belongs to Chapter 9.

Communication history is scoped the same way. `EmailLog` carries a `RecipientMemberId`, added alongside `Channel` and `DeliveryScope` in the migration `20260921032243_AddCommunicationVisibility.cs`, so that every logged email or SMS names the member it was sent to. `MemberCommunicationsController.GetMine` (`GHCAA.API/Controllers/MemberCommunicationsController.cs`, lines 13-21) never takes a member id from the request: it reads the caller's own id off the authenticated token through `CurrentMemberIdRaw()` and passes that value into `CommunicationService.GetLogsForMemberAsync` (`GHCAA.Infrastructure/Services/CommunicationService.cs`, lines 212-235), whose query filters `EmailLogs` on `RecipientMemberId == memberId` before it ever reaches the database. A member therefore has no parameter to alter to reach another member's communications, and the endpoint the mobile and web history screens call is `GET /api/communications/me`, not a member-id route. The one place a communication log can be fetched by an arbitrary member id is `CommunicationController.GetMemberLogs` (`GHCAA.API/Controllers/CommunicationController.cs`, lines 42-46), which sits under the controller's `[Authorize(Policy = Constants.Policies.AdminOnly)]` attribute and is unreachable by a plain `Member` token. The two facts together, own-identity scoping on the member route and role gating on the admin route, are what NFR-S9 requires and what makes a member's outbound-communication history a member-only view rather than a membership-wide one.

Design consequence. Two roles rather than a finer-grained permission table is a deliberate choice recorded as one of the anti-patterns considered and remediated in §6.9.19: a permission-per-feature matrix was judged unnecessary complexity for an association run by one maintainer, at the cost that a future need to grant a narrower administrative capability — treasurer access without member-approval access, for instance — would currently require a new role rather than a new permission flag.

8.5 Input Validation, Output Sanitisation and File Uploads

NFR-S8 requires member-supplied rich text to be sanitised on the server before storage, not only filtered on the client, because a client-side filter protects nothing against a request sent directly to the API. Three services hold a shared, statically-constructed `HtmlSanitizer` instance and run every rich-text field through it before the value reaches the database: `NewsService` (news article bodies), `SiteContentService` (editable site content blocks) and `ForumService` (forum posts and replies) — the three surfaces in the system where a member or officer supplies HTML rather than plain text. The sanitisation calls in `NewsService` and `SiteContentService` are tagged to ticket 24.42; the `ForumService` call is tagged "1d". A static, shared instance rather than one constructed per call was a deliberate choice: `HtmlSanitizer`'s allowed-tag configuration is fixed once and cannot drift between call sites.

This closes the stored-XSS threat for the three rich-text surfaces named above. It does not, by itself, cover every field in the system: plain-text fields (names, addresses, messages) are not passed through `HtmlSanitizer`, on the reasoning that a plain-text field rendered as plain text in Angular's templates is not a script-injection vector the same way rich HTML is — Angular escapes interpolated text by default. That reasoning has not been independently verified against every template in `GHCAA.Web` for this chapter, so it is recorded as a gap in Table 8.5 rather than assumed closed.

Uploaded files are input too. NFR-S7 requires an uploaded file to be validated by declared type, actual content, and size, and stored outside the web root, served only through an endpoint that authorises the request. Two services divide that work.

`FileValidationService` (`GHCAA.Infrastructure/Services/FileValidationService.cs`) runs four checks before a file is accepted: the file is non-empty and under the configured size limit; its declared `Content-Type` is on an allowlist that differs by category (images only for photographs, images plus PDF for documents, only the OOXML spreadsheet type for spreadsheets); its filename extension is on a matching allowlist, checked independently of the declared content type (lines 34-38 record the reason — a genuine JPEG uploaded as `x.html` with `Content-Type: image/jpeg` would otherwise pass every earlier check and be served back as `text/html` from the application's own origin, an HTML-injection route); and its first bytes match the magic-byte signature for the type it claims to be (JPEG, PNG, WEBP, PDF and the XLSX zip-container signature are each checked explicitly in `MatchesSignature`, lines 64-84). A file that passes the content-type and extension checks but whose bytes do not match either is rejected regardless of what its name or header claim.

`LocalFileStorageService` (`GHCAA.Infrastructure/Services/LocalFileStorageService.cs`) then decides where the accepted file is written. `IsSecureType` (lines 66-73, ticket 29B.6) routes `Certificate`, `PaymentProof` and `Signature` uploads to a separate secure root outside the publicly served uploads tree, on the reasoning that a certificate, a payment receipt or a signature image is either sensitive or forgeable and neither belongs behind an unauthenticated static-file route; every other upload type goes to the public root. The constructor (lines 36-55, ticket 82.51) checks at startup that the secure root does not resolve inside the web root and throws `InvalidOperationException` if it does, so a future configuration change that would silently expose Certificate/PaymentProof/Signature files through the static-files route fails loudly at boot rather than exposing them in production. Stored filenames are randomised with `Guid.NewGuid():N` rather than kept from the original upload (line 115), and files are nested by member id and upload type on disk.

`IsCompressibleImageType` (lines 79-84) recompresses only `Photo`, `GalleryPhoto` and `NewsImage` uploads; `Certificate`, `PaymentProof` and `Signature` are never re-encoded regardless of their actual file content, because a payment-proof image or a signature is evidence and legal record respectively, and re-encoding it would alter the file being kept as proof.

8.6 Transport Security, Browser Policy and Rate Limiting

`SecurityHeadersMiddleware` (`GHCAA.API/Middleware/SecurityHeadersMiddleware.cs`, lines 1-56) sets `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, a `Referrer-Policy`, and, only when the connection is HTTPS, `Strict-Transport-Security`. It also sets a Content-Security-Policy header, with an in-code comment recording why the policy allows script and style from `cdnjs.cloudflare.com` and fonts from Google Fonts specifically, rather than a blanket allowance. In `Program.cs` (lines 130-177) this middleware sits after HTTPS redirection is enforced and before the request reaches `AuditLogMiddleware` and the rate limiter, so a request that has not yet been upgraded to HTTPS in a non-development environment never reaches it.

Authentication cookies are set by a single shared helper, `AuthCookieExtensions.SetAuthCookie` (`GHCAA.API/Extensions/AuthCookieExtensions.cs`, lines 12-22, ticket 82.97), introduced after `AuthController` and `ProfileController` had each carried their own copy of the same cookie-setting logic. Every cookie it writes is `HttpOnly`, `Secure` outside Development, and `SameSite=Strict`. The CSRF-token cookie, `SetXsrfCookie` (lines 27-38), is deliberately not `HttpOnly` — Angular's `HttpClient` reads it and echoes it back as the `X-XSRF-TOKEN` header — and `XsrfMiddleware` (`GHCAA.API/Middleware/XsrfMiddleware.cs`) then checks the cookie value

against that header on every state-changing request, rejecting a mismatch. The check runs only when an `access_token` cookie is present, so an unauthenticated request carries no double-submit obligation, and safe methods (`GET`, `HEAD`, `OPTIONS`) are exempted from it.

CORS is configured as a single named policy, `AngularApp`, in `Program.cs` (lines 88-107): allowed origins come from configuration, credentials are permitted, and the application throws at startup if `AllowedOrigins` is empty outside Development — an empty allowlist in production is treated as a misconfiguration to fail on, not a default to fall back from. Outside Development the same startup check also refuses a list that contains `*`, an address that is not `https`, or an entry that is more than a scheme and host (a path, query, trailing slash or user info, which CORS would never match), and it refuses a `ClientUrl` that is not one of the listed origins. The rules are in `AllowedOriginsAudit` (`GHCAA.Infrastructure/Services/AllowedOriginsAudit.cs`) and are covered by `AllowedOriginsAuditTests`. The same class writes an activity-log row whenever the list differs from the one last recorded, because the list comes from an environment variable and the application cannot otherwise see who changed it. A development-only branch adds `localhost` origins so the Angular dev server can reach the API without a configuration change. A request for the `www` host is redirected to the `ClientUrl` origin by `WwwRedirectMiddleware` (§6.4, where the middleware pipeline is described). The redirect is not yet checked on the live site, because on 5 October 2026 the production and preprod domains did not resolve.

Rate limiting sits in `RateLimitingExtensions` (`GHCAA.API/Extensions/RateLimitingExtensions.cs`), on `System.Threading.RateLimiting`. It names six fixed-window policies. Five are keyed per source IP: `Auth` (one minute, 10 in production, 500 under test), `Refresh` (one minute, 20), `PasswordReset` (fifteen minutes, 5, ticket 80.16), `ScholarshipStatus` (fifteen minutes, 10) and `CredentialVerification` (fifteen minutes, 30). `Registration` is one shared window of five minutes and 10 requests. A global limiter of 100 requests a minute, with a queue of 2 served oldest-first, applies to every request as well. It used to be a named `Api` policy attached to `MapControllers()`, and ASP.NET Core resolves the last rate-limit attribute on an endpoint, so that policy silently replaced every narrower one, and the test for the shape of a 429 response could not pass. As a global limiter it now runs alongside a route's own policy (commit 44bd1c19), and that test passes. `UseRateLimiter` sits in the pipeline (`Program.cs`, line 161) after `AuditLogMiddleware` and before authentication, so a request over its limit is refused before the authentication and authorisation checks run. A comment dated 29 August 2026 in the same file records a prior audit finding: the relaxed test-environment limits must depend only on `Development`, never on the `ASP_SEED_PROFILE` variable alone, after a review found that condition could be satisfied unintentionally outside a development environment.

One figure does not match across the two sources. NFR-S3 in §3.4 states that authentication attempts "shall be limited to 5 per minute per source." The `Auth` policy actually enforces a `PermitLimit` of 10 per minute in production. The requirement and the implementation disagree, and this chapter records the disagreement rather than repeating the requirement's figure as fact: whichever number is correct, one of the two documents needs to change, and that correction is entered as an open item in Table 8.5 rather than silently resolved here.

8.7 Payment-Related Risk and the No-Gateway-Keys Posture

The system's payment model is manual by design: a member uploads proof of payment, and an officer verifies it against the Association's own bank or mobile-money record before the payment is recorded as received. This is the primary and only configured path in production. A gateway abstraction exists in code — `GHCAA.Domain/Enums.cs` declares a `PaymentGateway` enum including `Stripe`, `PayPal`, `SSLCommerz`, `BkashGateway`, `NagadGateway`, `RocketGateway`, `BankTransferGateway` and `DGePay` alongside `Manual`, and `GHCAA.Infrastructure/Gateways/` contains real, non-stub adapter classes (`BkashGateway`, `NagadGateway`, `SSLCommerzGateway`, `DGePayGateway`, built on a shared `BasePaymentGateway`, with `docs/PAYMENT_GATEWAY_WORKFLOW.md` documenting how a new one is wired in) — but none of these adapters is configured with live credentials, and none is reachable from the production system as it stands.

The security reasoning, recorded here per ADR-04 of §6.10, is that a gateway credential is itself an asset worth protecting, and the Association currently has none to protect: it holds no merchant account and cannot yet supply the banking relationship a gateway requires. Configuring a gateway without one would mean routing collections through an individual's personal account instead, which is the arrangement the manual, officer-verified path exists to end. The manual path therefore holds no gateway secret at all, at the accepted cost — quantified in §12.4 — of a permanent administrative workload of some minutes per payment for officer verification, rather than the workload disappearing into an unverified automated credit.

The residual risk is not in the manual path itself but in the schema that waits for the gateway path to be switched on. `docs/ARCHITECTURE_AUDIT_2026-09.md`, Finding 9, records that `PaymentConfiguration` carries `GatewayPublicKey` and `GatewaySecretKey` as plain `[MaxLength(500)]` string columns with no column-level encryption or protection marker. The finding's own disposition lowers its severity today — the columns are empty because no live gateway is configured — but states plainly that the schema invites a secret to be stored in plaintext the moment the Association does obtain gateway credentials. That item is carried into Table 8.5 rather than treated as closed, because the schema, not the current data, is what a future integration would inherit unless it is changed first.

8.8 Governance Integrity

§2.7 sets the boundary this section enforces in the code: the platform supports governance and now contains a persisted election workflow, but it does not claim that the software alone makes an institutional election legally binding. It maintains the authoritative voter roll, publishes the constitution and its amendment history, and runs constitutional amendment voting and non-binding member polls. Work Package 37.1 also stores election phases, seats, officers, nominations, secret ballots, counts and declarations. Legal form wording and institutional adoption remain under the Election Commission's election documents.

The code matches that stated scope rather than exceeding it. `PollVote` (`GHCAA.Domain/Models/PollVote.cs`) records a member id, a poll option id and a timestamp against each vote — the vote is attributable to the member who cast it, not anonymous, which is appropriate for an internal poll of sentiment but would not by itself meet the secrecy a binding ballot requires. Neither `Poll` nor `PollVote` carries a cryptographic commitment, a hash chain, or any other tamper-evidence mechanism over the vote record: a poll result rests on the same protection as any other Class A row (§8.10) — an authorised, audited administrative account rather than a mathematically verifiable count. Under the single-maintainer, single-database-account assumption of §8.1, that is a real limitation, and it is the reason §2.7 gives for keeping these features as sentiment and internal decision-making tools rather than presenting them as a secure-election system. The election engine adds phase checks, frozen-roll eligibility, conditional one-vote updates and serializable transaction handling. It still has no cryptographic receipt, independent tally verification, or proof that would make the software a secure-election system on its own.

Communication history follows the same minimisation rule. Member history is filtered from the authenticated member claim, while administrators use the protected per-member route. The log stores the intended recipient member, channel, scope and outcome so delivery can be audited. Successful rows do not return failure details. Direct OTP SMS remains a separate provider path and is not yet included in member communication history.

8.9 Personal Data: Lawful Basis, Minimisation, Consent, Retention and Subject Rights

§3.8 records that Bangladesh's data protection statute was in draft at the time of writing, so the system does not claim compliance with a specific law; instead it applies purpose limitation, minimisation, default non-disclosure and stated retention as design obligations, and this section records what that means concretely rather than leaving the claim abstract. Table 8.4 inventories the personal-data elements held, their purpose and their retention; Figure 8.4 traces the same elements from capture at registration through to the retention and disclosure points named below.

Minimisation and default non-disclosure. `Member` (`GHCAA.Domain/Models/Member.cs`, lines 41-45) carries five independent visibility flags — `IsMobilePublic`, `IsEmailPublic`, `IsAddressPublic`, `IsNIDPublic` and `IsFamilyPublic` — each defaulting to `false`. `MemberService_Search.cs` (lines 179-192) and `NetworkingService.cs` (lines 161-162, 203-204, 224-254, 300-303) apply these flags at the point a directory or networking result is built: a field is returned in full only if the owning member opted in or the caller holds a privileged role, and is otherwise returned masked (`MaskPii`, partially redacting the value) or replaced outright with the literal string `"Confidential"`, depending on the endpoint. `NetworkingService.cs` (line 52) carries a comment recording a specific enumeration risk that was checked for and avoided: a directory search that behaved differently for a public versus a masked field regardless of `IsEmailPublic` would let an unprivileged caller infer a member's contact details existed even when they were not disclosed, so the search predicate itself, not only the returned value, respects the visibility flag.

Work Package 37.2's scholarship review and Work Package 37.8's public credential check apply the same default-non-disclosure rule at the query level rather than in a view. `ScholarshipService`'s reviewer-facing projection does not join the applicant's member record, so a reviewer scoring an application has no member name, contact detail or membership history to see, blind by construction rather than by a field a screen happens not to render. `CredentialVerificationController.Verify` (§7.9) is reachable by anyone with a card's QR code and no account, so its response is built from the same minimisation discipline in the other direction: `CredentialVerificationDto` carries only `Valid`, `MemberName`, `MembershipType`, `IssuedOn` and `Status`, with no address, contact detail or member ID, whether the code is valid, expired, revoked or unrecognised.

Consent. `Member` also carries `HasAcceptedTerms`, `HasAcceptedGdpr` and `GdprAcceptedAt`. The public registration form (`GHCAA.Web/src/app/public/register/register.html`, lines 396-413) presents these as two required checkboxes before submission: acceptance of the Association's registration terms, and an explicit statement of consent to processing personal information under the "Data Privacy & GDPR" heading the form uses — a label chosen for the form's own wording rather than a claim that GDPR itself applies to the Association. Both are recorded with a timestamp rather than only a boolean, so a consent event has a date attached to it.

Retention and subject rights. No automated retention or erasure schedule exists in the codebase at the time of writing: a member's personal data persists for as long as the `Member` row exists, subject only to the archival (not deletion) discipline §8.10 describes for Class A entities. NFR-Po3 gives members a right to a full export of the Association's data on demand, but no corresponding right to erasure is implemented — deleting a Class A row is, by the same §8.10 discipline, deliberately not offered through the interface at all, which protects financial and governance integrity at the direct cost of a subject-erasure capability. This is recorded here as an open gap rather than an omission: a retention policy and an erasure path for personal data that is not evidentiary (a rejected applicant's record, for instance, which currently uses the same `IsArchived` mechanism as a settled financial record) is entered in Table 8.5.

8.10 Audit Logging and Non-Repudiation

Two separate mechanisms answer this, not one. `AuditLogMiddleware` (`GHCAA.API/Middleware/AuditLogMiddleware.cs`, lines 20-56) is a generic HTTP-level log: it records any non-GET request, and any request under `/api/admin`, as an activity row through `IActivityService.LogActivityAsync` once the response succeeds and the caller's user id can be read from the token. It does not know or care what kind of data a request touched — that distinction is made separately, per entity.

`docs/ARCHITECTURE.md` (lines 118-149) sets that distinction as Class A and Class B. An entity is Class A "if a row of it is evidence: money received or spent, a governance decision, a membership status, or anything a member could later dispute" — currently `FinancialRecord`, `PaymentHistory`, `MembershipDue`, `MembershipHistory`, `Member`, `User`, `ECMember`, `Constitution` and `Poll`. A Class A row is never hard-deleted: it carries `HasQueryFilter(x => !x.IsArchived)` so an ordinary read already excludes it once archived, a caller that genuinely needs the archived rows back asks for them explicitly with `IgnoreQueryFilters()`, and a service method that archives a Class A row takes the acting admin's id as a required argument — the controller returns `Unauthorized()` rather than record the act as done by nobody when it cannot identify the caller. Everything else is Class B: content and configuration that can be recreated if lost, which carries only a creation timestamp and can be hard-deleted without the same argument for keeping it. The `IsArchived` name was chosen deliberately: three Class A entities (`FinancialRecord`, `PaymentHistory`, `ECMember`) had used `IsDeleted` for the same soft-delete behaviour before the naming was unified under ticket 82.30.

Non-repudiation for a Class A row therefore rests on the row itself — who archived it and when, kept rather than erased — and audit logging rests on `AuditLogMiddleware`'s separate, undifferentiated request log. Neither is aware of the other, and no test exercises `AuditLogMiddleware` directly; its coverage today is incidental, through the controller tests that call the endpoints it wraps.

8.11 OWASP ASVS Conformance and Residual Risks

NFR-S1 commits the system to OWASP ASVS level 2 for the control families that apply to it [7], with every non-conformance recorded rather than omitted. The edition cited throughout this dissertation is ASVS 4.0.3, the version current when the work reported here was carried out; ASVS was revised to 5.0.0 in May 2025, after that point, and §2.8 states why a conformance claim is not carried across a revision it was not assessed against. No formal, independently-audited ASVS assessment has been performed on this codebase — what follows is a self-assessment mapping the controls found elsewhere in this chapter against ASVS 4.0.3's chapter structure, level 2, recorded honestly as a mapping rather than a certification. Table 8.3 gives the full mapping; the pattern is summarised here.

Authentication (V2) and session management (V3) are substantially met: adaptive password hashing with a documented username-enumeration countermeasure (§7.8, the dummy-verify timing equalisation at `AuthService.cs`, lines 88-89), rate-limited authentication endpoints (§8.6), and immediate session invalidation on a credential or status change (§8.3) are all present. Access control (V4) is met at the role level and, for owned resources, at the ownership level (§8.4), though not through a fine-grained permission model. Validation, sanitisation and encoding (V5) is met for the three rich-text surfaces identified in §8.5 and is an open question for plain-text fields, which is recorded rather than assumed. File and resource handling (V12) is met by the validation and storage-separation controls of §8.5. Communications security (V9) is met through the transport and header controls of §8.6. API and web-service security (V13) is met through the rate-limiting policies of §8.6. Malicious code (V10) and business-logic (V11) verification have not been separately assessed for this dissertation and are recorded as not conformed rather than assumed conformed.

After that assessment, Table 8.5 lists what remains open. The largest items, in descending order of how directly they touch an asset named in Table 8.1:

Gateway-credential schema. `PaymentConfiguration.GatewayPublicKey` and `GatewaySecretKey` are plain string columns with no encryption at rest (`docs/ARCHITECTURE_AUDIT_2026-09.md`, Finding 9). Empty today because no gateway is configured (§8.7), but the schema itself is the risk, and it should be closed before, not after, a gateway is switched on.

No retention or erasure path for non-evidentiary personal data. §8.9 records that a member's personal data has no automated retention schedule and no subject-erasure right distinct from the archival mechanism built for financial integrity. A rejected applicant's record is retained under the same discipline as a settled financial transaction, which was not the discipline's original purpose.

Governance records carry no tamper-evidence beyond the audit discipline of §8.10. §8.8 records that a poll or amendment vote result rests on an authorised administrative account, not on a cryptographic guarantee, which is why these features are scoped as internal governance tools rather than a secure-election system.

Supply-chain hardening is incomplete. `docs/ARCHITECTURE_AUDIT_2026-09.md` records that only one of five CI workflows pins its third-party actions by commit hash rather than by mutable tag (Finding 14, item 48.19), and that no automated dependency or container vulnerability scan runs in CI at all (Finding 16, item 82.26). Neither is a control this chapter has described as present, and both remain open.

The rate-limit figure mismatch of §8.6 between NFR-S3's five-per-minute figure and the `Auth` policy's actual ten-per-minute limit is an open documentation-versus-implementation disagreement, not yet resolved in either direction.

Denial of service against the SignalR hubs and **repudiation of governance actions by a compromised admin account**, both named in §8.2's Table 8.1 as partial rather than closed, remain open for the reasons given there.

None of these six items is disqualifying on its own; together they describe a system whose authentication, session and transport controls are comparatively mature and whose data-retention and supply-chain controls are the areas where the next security effort should go.

8.12 Summary

The controls in this chapter answer specific threats named against specific assets, not a generic security posture: session revocation and refresh-token reuse detection against a stale or stolen token (§8.3), role-and-ownership authorisation against cross-member disclosure (§8.4), server-side sanitisation against stored XSS in the three rich-text surfaces that carry it (§8.5), validated and separated file storage against a disguised or forged upload (§8.5), transport and header policy against network-level and browser-level attacks (§8.6), and rate limiting against credential-stuffing and abuse (§8.6). The payment model avoids holding a gateway credential at all by keeping manual verification primary (§8.7), and the governance features are scoped deliberately short of a binding election under a threat model that could not prove one honest (§8.8). Personal data is minimised and masked by default, with explicit, timestamped consent capture (§8.9), and Class A records are archived rather than erased so that a dispute always has a row to point to (§8.10). Measured against ASVS 4.0.3 level 2, the system is strong on authentication, session and transport, and has recorded rather than closed gaps in a fine-grained permission model, plain-text field sanitisation, malicious-code and business-logic verification, and — outside the ASVS mapping itself — retention, supply-chain hardening and the gateway-credential schema (§8.11). Chapter 9 measures the controls this chapter describes; Chapter 12 revisits, in §12.6, whether the governance boundary was drawn in the right place.

Figures and Tables

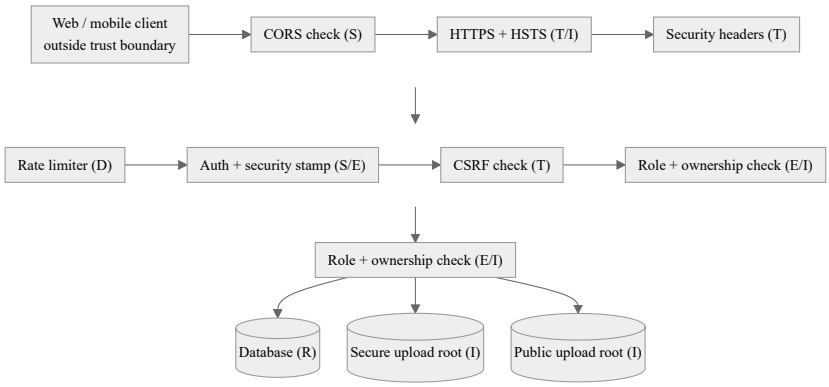


Fig. 8.1. Threat model data-flow diagram with trust boundaries, STRIDE-annotated

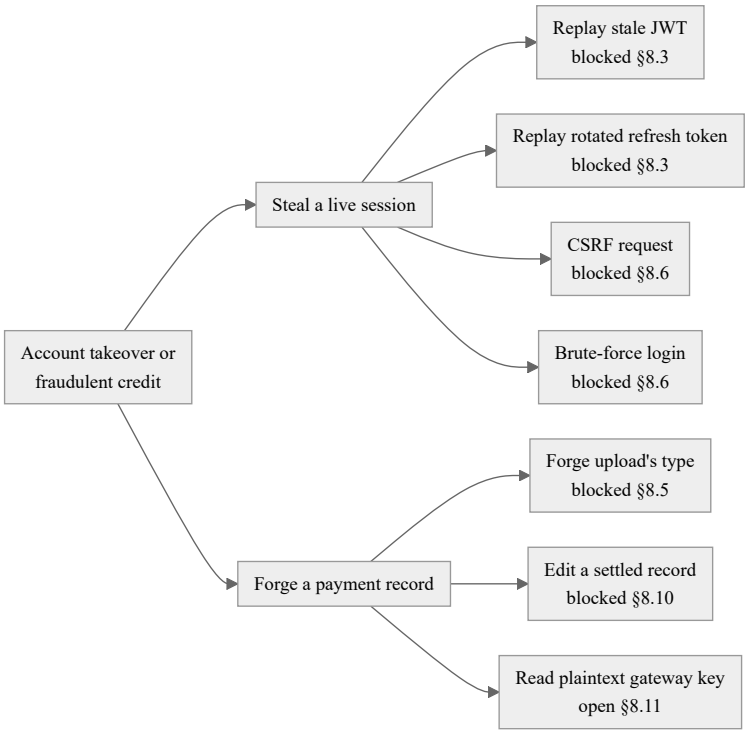


Fig. 8.2. Attack tree: member account takeover or fraudulent payment credit

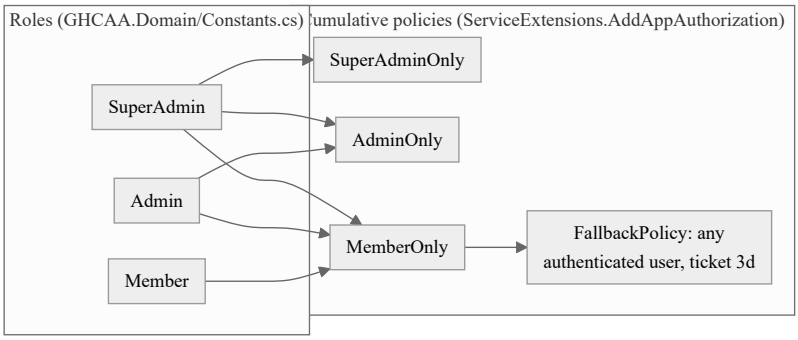


Fig. 8.3. Role–permission matrix diagram

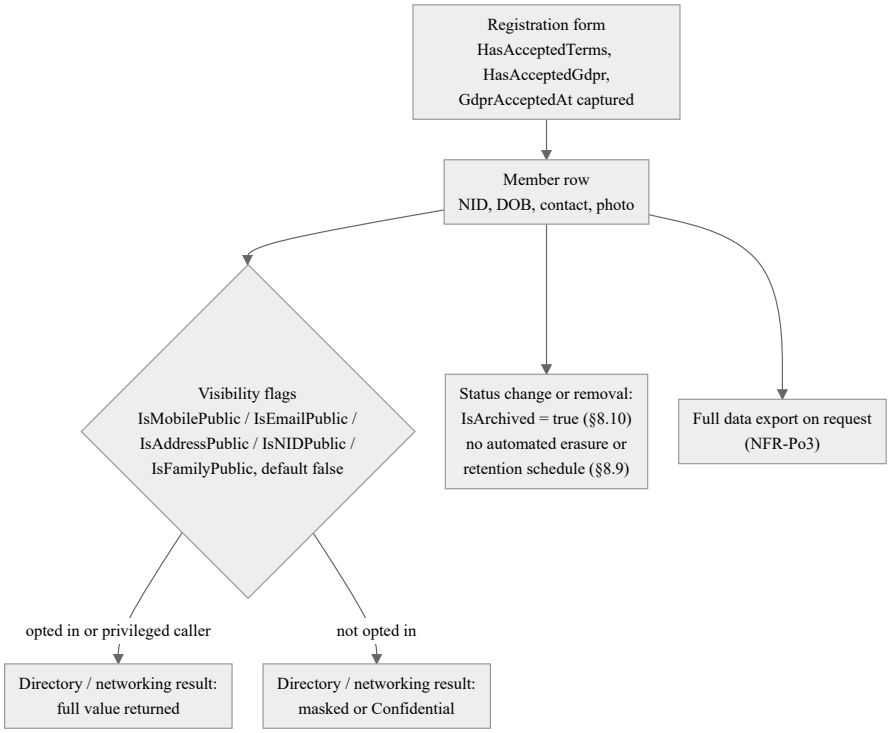


Fig. 8.4. Personal-data classification and flow diagram, with retention points

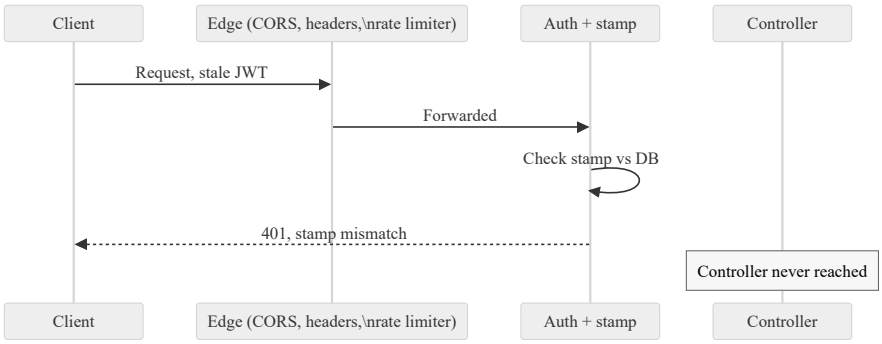


Fig. 8.5. Sequence diagram: an unauthorised request rejected through the middleware chain

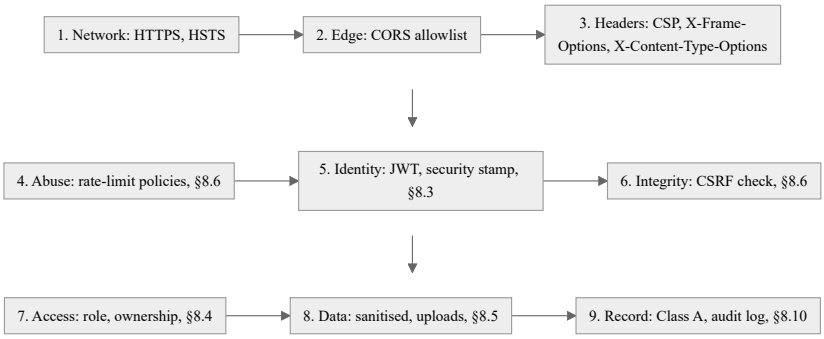


Fig. 8.6. Defence-in-depth layer diagram

TABLE 8.1
STRIDE threat enumeration with mitigations and their implementation location

STRIDE category	Threat	Asset / entry point	Mitigation	Implementation location
Spoofing	Stale or forged session token accepted	Session / API	Rotating security stamp checked every request	`SecurityStampMiddleware`, §8.3
Spoofing	Credential stuffing against login	Authentication endpoint	Per-IP rate limit	`Auth` policy, §8.6
Tampering	Cross-site request forgery on a state-changing request	Authenticated API	Double-submit cookie check	`XsrfMiddleware`, §8.6

STRIDE category	Threat	Asset / entry point	Mitigation	Implementation location
Tampering	Uploaded file's real content does not match its declared type	File upload endpoints	Extension + content-type + magic-byte check	`FileValidationService`, §8.5
Tampering	Stored XSS via rich-text fields	News, site content, forum posts	Server-side HTML sanitisation	Shared `HtmlSanitizer`, §8.5
Repudiation	Financial or membership record altered or deleted with no trace	Financial, membership, governance data	Archive-only deletion, admin id required	Class A discipline, §8.10
Repudiation	Governance action repudiated by a compromised admin account	Poll, Constitution, ECMember	Acting-admin id recorded on the row	§8.10; **partial** — no cryptographic signature, §8.2
Information disclosure	Member reads another member's contact or financial data	Directory, networking, financial endpoints	Ownership check independent of role; default-masked visibility flags	§8.4, §8.9
Information disclosure	Certificate/payment-proof/signature files served through the public static-file route	Secure file store	Separate secure root, fail-loud startup guard	`LocalFileStorageService`, §8.5
Denial of service	Stolen refresh token used repeatedly after theft	Refresh-token endpoint	Reuse detection revokes the token family	`TokenService.RotateRefreshTokenAsync`, §8.3
Denial of service	Excess connections or messages against the SignalR hubs	Real-time hubs	Authenticated connection required	**Partial** — no per-connection rate cap, §8.2
Elevation of privilege	Member-level account performs an admin-only action	Admin-scoped endpoints	Cumulative role policies, secure-by-default fallback policy	`ServiceExtensions.AddAppAuthorization`, §8.4

Table 8.2
Role × capability matrix

Capability	SuperAdmin	Admin	Member
Sign in, manage own profile, view masked directory	Yes	Yes	Yes
View unmasked directory fields (subject to owning member's opt-in)	Yes	Yes	No
Approve membership applications, record dues and payments	Yes	Yes	No
Archive a Class A record (financial, membership, governance)	Yes	Yes	No
Create or resolve constitutional amendments, EC membership, member polls	Yes	Yes	No
Manage other admin accounts	Yes	No	No
Any endpoint carrying no explicit policy	Rejected by `FallbackPolicy` unless authenticated as any role above		

Table 8.3
OWASP ASVS conformance checklist (self-assessment, ASVS 4.0.3, level 2)

ASVS chapter	Verdict	Evidence
V2 Authentication	Met	Adaptive hashing, rate-limited endpoints, timing-equalised verification (§7.8, §8.6)
V3 Session management	Met	Rotating security stamp, refresh-token reuse detection (§8.3)
V4 Access control	Met, coarse-grained	Role policies plus ownership checks; no fine-grained permission model (§8.4)
V5 Validation, sanitisation, encoding	Partial	Rich-text fields sanitised; plain-text field handling not independently assessed (§8.5)
V7 Error handling and logging	Not assessed	Outside this chapter's evidence base
V8 Data protection	Partial	Class A archival protects records from erasure; no encryption at rest for gateway-credential columns (§8.7, §8.10)
V9 Communications	Met	HTTPS enforced outside development, HSTS, security headers (§8.6)
V10 Malicious code	Not assessed	No dependency or container scanning in CI (`docs/ARCHITECTURE_AUDIT_2026-09.md`, Finding 16)
V11 Business logic	Not assessed	Outside this chapter's evidence base
V12 Files and resources	Met	Type, extension and content validation; secure/public root separation (§8.5)
V13 API and web service	Met	Named rate-limit policies on every endpoint class (§8.6)
V14 Configuration	Partial	CORS misconfiguration fails startup; supply-chain action pinning incomplete (§8.6, §8.11)

Table 8.4
Personal-data inventory: element, purpose, lawful basis, retention

Data element	Purpose	Lawful basis (design principle, §8.9)	Retention
National identity number (NID)	Uniqueness and identity verification (NFR-S4)	Necessary for membership eligibility	Held for the life of the `Member` row; no automated erasure
Date of birth, gender, blood group	Membership record, emergency use	Necessary for membership eligibility	Held for the life of the `Member` row
Mobile number, email	Contact, sign-in, directory	Consent for directory disclosure (`IsMobilePublic`/`IsEmailPublic`); necessary for account otherwise	Held for the life of the `Member` row
Present/permanent address	Directory, correspondence	Consent for disclosure (`IsAddressPublic`); necessary otherwise	Held for the life of the `Member` row
Photograph, signature	ID card, certificate, verification	Necessary for the credential it appears on	Held for the life of the `Member` row
Payment-proof image	Evidence of a payment claim	Necessary for financial verification (§8.5)	Archived, never hard-deleted (§8.10)
Terms/GDPR consent flags and timestamp	Record of consent given	Consent, recorded at capture	Held for the life of the `Member` row

Table 8.5
Residual risk register

ID	Risk	Evidence	Status
RR-1	`PaymentConfiguration` gateway-credential columns are plaintext with no encryption at rest	`docs/ARCHITECTURE_AUDIT_2026-09.md`, Finding 9	Open
RR-2	No automated retention schedule or subject-erasure path for non-evidentiary personal data	§8.9	Open
RR-3	Governance records (`Poll`, `Constitution`, `ECMember`) carry no cryptographic tamper-evidence beyond admin-id attribution	§8.8, §8.10	Open, scoped deliberately per §2.7

ID	Risk	Evidence	Status
RR-4	Four of five CI workflows reference third-party actions by mutable tag, not commit hash	`docs/ARCHITECTURE_AUDIT_2026-09.md`, Finding 14	Open
RR-5	No automated dependency or container vulnerability scan runs in CI	`docs/ARCHITECTURE_AUDIT_2026-09.md`, Finding 16	Open
RR-6	NFR-S3 (5 requests/minute) and the `Auth` rate-limit policy (10 requests/minute in production) disagree	§3.4, §8.6	Open
RR-7	No per-connection rate limit on the two SignalR hubs	§8.2	Open
RR-8	Plain-text fields outside the three rich-text surfaces have not been independently assessed for injection risk	§8.5	Open

Chapter 9 — Verification, Validation and Quality Assurance

> *[Chapter not written. The headings below are generated from `docs/DOCUMENTATION_BOOK_OUTLINE.md` and are kept in step with it: `build.py --strict` fails if a section exists in one and not the other.]*

What this chapter owns. How the system was verified: levels, techniques, test design, tooling, what was measured and what the measurements cannot show.

What it must not repeat. Results and their interpretation, which are Chapter 12 and appear nowhere here. This chapter says what was run and why that method was chosen; the chapter that says what came out of it is Chapter 12. The separation is the reason both chapters exist.

9.1 Verification and Validation Strategy and Test Levels

> *[Not written.]*

9.2 Software Quality Assurance Plan

> *[Not written. Brief: reviews, standards conformance, defect prevention, and the role of formal technical reviews in this project. Two review sessions to draw on, and they differ in kind: the specification review of 3 July 2026 (§3.8, output = the 5 specification defects of Table 3.8), and the architecture and engineering audit of 4 September 2026 (`docs/ARCHITECTURE_AUDIT_2026-09.md`, output = 15 tracker items, 82.14-82.28). The second is worth treating separately: it was run against an external brief (`docs/materials/REVIEW.md`) rather than an internal checklist, it found defects in the* implementation and its documentation* rather than in the specification, and two of its own research streams failed, which the report states rather than conceals. That last point is the more honest material for this section than the findings themselves: a review that records its own incompleteness is the behaviour §12.9's validity-threat discussion should be able to point at]*

9.3 Test-Case Design Techniques Applied

> *[Not written.]*

9.3.1 Equivalence partitioning

> *[Not written.]*

9.3.2 Boundary value analysis

> *[Not written.]*

9.3.3 Decision-table testing for the dues and eligibility rules

> *[Not written.]*

9.3.4 State-transition testing derived from the state machines of Chapter 5

> *[Not written.]*

9.3.5 Basis-path testing using the control-flow graph and cyclomatic complexity of §7.7

> *[Not written.]*

9.3.6 Use-case and scenario-based testing

> *[Not written.]*

9.3.7 Exploratory testing and its recorded charters

> *[Not written.]*

9.4 Unit and Component Testing

> *[Not written. Brief: Backend]*

9.4.1 Framework, runner and project layout

> *[Not written.]*

9.4.2 The definition of a unit in this system

> *[Not written. Brief: and the reasoning for testing at the service boundary rather than at the controller or the repository]*

9.4.3 Test doubles

> *[Not written. Brief: stubs, mocks, fakes and the in-memory provider; where each is appropriate and the fidelity each sacrifices]*

9.4.4 Test structure and naming: arrange–act–assert, given–when–then

> *[Not written.]*

9.4.5 Testing the business rules that carry constitutional force

> *[Not written. Brief: dues, eligibility, committee terms, voting rights — traced back to the rule catalogue of §5.5]*

9.4.6 Testing of failure and exception paths

> *[Not written.]*

9.4.7 Test independence

> *[Not written. Brief: determinism and the elimination of order dependence]*

> *[Not written. Brief: Web Client]*

9.4.8 Runner, harness and component-testing strategy

> *[Not written.]*

9.4.9 Testing signals, computed state and change propagation

> *[Not written.]*

9.4.10 Testing guards, interceptors and the token-refresh queue

> *[Not written.]*

9.4.11 HTTP mocking and contract fidelity against the live API

> *[Not written.]*

9.5 Widget and Golden Testing

> *[Not written. Brief: Mobile Client]*

9.5.1 Widget-test scope

> *[Not written.]*

9.5.2 Golden (snapshot) testing: what it catches

> *[Not written. Brief: what it cannot, and the platform-rendering problem that requires it to be skipped in continuous integration]*

9.6 Integration, System and End-to-End Testing

> *[Not written. Brief: and the reasoning for rejecting big-bang integration]*

> *[Not written.]*

9.7 Regression Testing and Test Selection

> *[Not written.]*

9.8 Security Testing

> *[Not written. Brief: mapped to the threat model of Chapter 8 and to OWASP ASVS]*

9.9 Performance and Load Testing

> *[Not written. Brief: workload model, environment, results]*

9.10 Usability and Accessibility Testing

> *[Not written. Brief: task success, time on task, System Usability Scale scores, WCAG audit]*

9.11 User Acceptance Testing

> *[Not written. Brief: participants, protocol, results, sign-off]*

9.12 Product Metrics, Static Analysis and Defect Analysis

> *[Not written.]*

9.12.1 Size

> *[Not written. Brief: lines of code and function points]*

9.12.2 Complexity

> *[Not written. Brief: cyclomatic complexity distribution and the worst offenders]*

9.12.3 Coupling and cohesion

> *[Not written. Brief: CBO, LCOM, afferent and efferent coupling, instability]*

9.12.4 Maintainability index and technical-debt estimate

> *[Not written.]*

9.12.5 Test adequacy: coverage

> *[Not written. Brief: and the argument coverage can and cannot support]*

> *[Not written. Brief: density, distribution, removal efficiency, root-cause categories]*

9.13 Summary

> *[Not written.]*

Figures and Tables

> *[Not drawn. The outline specifies 20 artefacts for this chapter. Each is added here with its caption as it is made, then `renumber.py --apply` is run.]*

Chapter 10 — Deployment and Operations

> *[Chapter not written. The headings below are generated from `docs/DOCUMENTATION_BOOK_OUTLINE.md` and are kept in step with it: `build.py --strict` fails if a section exists in one and not the other.]*

What this chapter owns. Whether the Association can run and keep running this platform after the project ends: deployment, pipeline, migration, secrets, observability, backup, cost and handover.

What it must not repeat. Architectural rationale (Chapter 6), the build of the pipeline as implementation narrative (§7.12), or measured operational results (Chapter 12). §10.6 is the chapter's centre because it carries the RQ2 evidence; containerisation is support, not subject, and no section here explains a tool to a reader who could read its manual.

10.1 Deployment Architecture, Environments and Containers

> *[Not written.]*

> *[Not written.]*

> *[Not written.]*

10.2 Continuous Integration and Continuous Deployment

> *[Not written.]*

10.2.1 Pipeline stages and the quality gates actually enforced

> *[Not written. Brief: formatting, type checking, static analysis, build and test, per workflow]*

10.2.2 Security in the pipeline

> *[Not written. Brief: what is automated and what is not. Dependabot raises grouped dependency updates against preprod for NuGet, npm, pub, Actions and Docker, so a published advisory against a library this project uses arrives as a pull request rather than waiting to be noticed. Nothing else is automated, and the reasons are separate rather than one excuse: static application security testing was scoped and rejected on cost, CodeQL being free only on a public repository and needing paid GitHub Advanced Security on a private one; dynamic testing has no environment to run against that is not either preprod or production; and the Flutter client would be out of reach of CodeQL in any case, Dart not being one of its languages. The security work of Chapter 8 was therefore done by review and by test, which is a weaker guarantee than a scan and is reported as one, with the remedy costed in §13.2]*

10.3 Database Provisioning, Configuration and Secrets

> *[Not written.]*

> *[Not written.]*

10.4 Observability

> *[Not written. Brief: logging, monitoring, alerting, error reporting]*

10.5 Backup, Recovery, Release and Rollback

> *[Not written.]*

> *[Not written.]*

10.6 Operating Cost, Maintenance and Handover

> *[Not written.]*

> *[Not written.]*

10.7 Summary

> *[Not written.]*

Figures and Tables

> *[Not drawn. The outline specifies 10 artefacts for this chapter. Each is added here with its caption as it is made, then `renumber.py --apply` is run.]*

Chapter 11 — Project Management

What this chapter owns. The project as it actually ran, against the plan: breakdown, schedule, estimate, effort, risk and what changed. Its figures are generated by `docs/book/build/wbs.py` from the repository and the task list rather than recalled.

What it must not repeat. Process-model justification, which is §4.4. Generic project-management theory has no place here at all; the chapter's value is the gap between plan and outcome and the reason for each gap.

11.0 The Four P's Applied

Pressman and Maxim's four P's are people, product, process and project. Each is stated here against the single-maintainer case this project actually is, rather than against the team the framework assumes.

People. One person held every role from requirements through deployment: developer, tester, reviewer and release manager. He came to the project with about two decades of professional development experience and a personal library of his own 2024 codebases on the same technology generation — ASP.NET Core, Angular, Flutter. §11.5 measures what that library contributed: 30,100 of the 172,880 lines in this codebase, 17.4%, are modules carried over from it rather than written for this project. That prior asset, not a larger team, is the reason the schedule of §11.3 held at all.

Product. A clean-architecture ASP.NET Core 9 API behind an Angular web client and a Flutter mobile client, delivering one alumni association's membership register, payments, elections, events, gallery, careers board, forum and messaging. §6.1 describes the architecture; §11.4 sizes it.

Process. §4.4 already argues the case for the incremental, test-gated process this project ran and why a plan-driven alternative was rejected; that argument is not repeated here. What this chapter adds is the evidence for how much of the work the process had to absorb without warning: §11.1 shows that most of it did not exist when the project began.

Project. Managed through one artefact rather than a change-control board or a project-management tool: `docs/TODO.md`, which is simultaneously the plan, the change log and the defect log. §11.8 reports what that cost and what it caught.

None of Pressman's four organisational paradigms — closed, random, open, synchronous — describes a team of one, because all four are ways of dividing work between people. A team of one has nobody to divide work with. That is the finding this section states rather than the label it withholds: below a certain team size, the paradigm question stops applying, and forcing this project into one of the four would misdescribe it.

11.1 Process Model in Practice and its Deviations from Plan

`docs/book/build/wbs.py`, run on 31 October 2026, classifies every one of the 98 work packages `docs/TODO.md` has raised, and every one of the 1,207 tracker tasks under them, by how it arrived: planned before the work started, or reactively, as stakeholder feedback, a review finding or a defect.

Counted by work package, 30 (31%) were planned and 68 (69%) were not: 41 (42%) arrived as feedback from a stakeholder, 16 (16%) as a defect, and 11 (11%) as a finding from one of the review sessions of §3.8 or the September 2026 architecture audit. Counted by task the reactive share is larger still: 961 of 1,207 tasks (80%) arrived after the work they belong to had already started, most of them as feedback (509 tasks, 42%) or as review findings (321 tasks, 27%), with defects a smaller share (130 tasks, 11%) than the package count alone suggests, because most defects closed in one or two tasks each while a handful of review sessions each seeded dozens.

A work breakdown structure drawn from the requirements of Chapter 3, before the first commit, would have covered less than a quarter of what the project went on to do. That is not a failure of planning; it is what an incremental process run against a living association, whose feedback and whose defects are only discovered by using the system, looks like in the record. A critical path computed over such a structure would describe work that mostly did not exist yet, which is why §11.3 builds the network retrospectively, from the components the finished codebase actually has, rather than from a plan this project never drew.

11.2 Work Breakdown Structure

Five units, one vocabulary. This chapter and Chapter 4 count the project five different ways, and the counts disagree because the units are different, not because one of them is wrong. Each is defined once, here, and used by name everywhere else in the book:

- **Commit.** One entry in the git history. `git rev-list --count HEAD` gives the total on any date. A commit is a unit of code change, not of planning — several commits usually close one tracker task.
- **Tracker task.** One numbered line in `docs/TODO.md`, such as `62.41`. This is the smallest unit the project plans, tracks and closes; it is what carries a `[TODO]`/`[DONE]` state and a priority.
- **Numbered work package.** A `# Work Package N` heading in `docs/TODO.md`, grouping the tracker tasks raised for one initiative (a feature request, an audit, a defect sweep). A work package closes when every task under it does.
- **WBS activity.** One of the eighteen components `docs/book/build/wbs.py` defines (`C1` … `C18`), each a named cluster of source paths — "Persistence and migrations", "Authentication and access" and so on — mapped to the tracker work packages that produced it. Activities exist only for this chapter's schedule and effort figures; the tracker itself has no concept of them.
- **Feature.** A capability described from the user's side, as `docs/FEATURES.md` or the SRS's functional requirements state it (for example, "digital ID cards"). A feature can span several work packages, and a work package can deliver only part of a feature — the two lists are not the same shape.

`docs/TODO.md` had 98 numbered work packages holding 1,207 tracker tasks between them, tied to 18 WBS activities and 379 commits, as of 31 October 2026 (`git rev-list --count HEAD`) — five different counts of one project, each answering a different question about it.

Five streams, one of them carrying no commit at all: the back-scheduled P1–P5 research and design work of §11.1, which git cannot date because it produced no commit, sits alongside the eighteen code components C1–C18 and the eight documentation deliverables D1–D8 that do. The activity list has one entry per component, each a cluster of source paths `wbs.py` groups by directory and rolls up to the tracker work packages that raised the changes inside it — Table 11.1 in §11.3.1 lists all eighteen with their tracker areas. C18, Ad-hoc reporting, is planned work with no commit yet. A component's place on the critical path is not the same thing as how long it took in the calendar: C1, Persistence and migrations, carries eleven working days of effort, yet its last commit landed on 25 October 2026, more

than eight months after its first; C17, Standards, docs and dissertation, carries fifteen days of effort spread across 145 working days between its first commit on 6 April and its last on 25 October. The activity list and the tracker are the same data read two ways — one groups by source path, the other by the work package a change was raised under — and `wbs.py --check` fails if a component has no tracker area, or has no commit although its files exist or its tracker items are done, which is what keeps the two lists from drifting apart.

11.3 Scheduling, Task Network and Critical Path

`wbs.py` builds an activity-on-node network by the precedence diagram method over the eighteen components of §11.2, dating each by its own commit history rather than by a plan drawn in advance. Every activity's early and late start and finish come from that network; there is no separate estimate to compare them against, because no network existed before the code did.

The critical path is C1 (Persistence and migrations) → C2 (Authentication and access) → C3 (Registry and membership) → C5 (Events and attendance) → C8 (Gallery and albums) → C13 (Web client) → C15 (Quality and testing) → C17 (Standards, docs and dissertation), 76 working days end to end. Against that, 104 working days are evidenced by a code commit somewhere in the project (§11.1), and the first commit to the last spans 9 February to 25 October 2026, 258 elapsed calendar days. Seventy-six planned days, a hundred and four worked days, two hundred and fifty-eight elapsed days: the three numbers diverge for different reasons. The gap between 76 and 104 is components running in parallel, which a single maintainer cannot actually do at the same moment, so total work exceeds the shortest possible path through it. The gap between 104 and 258 is availability rather than dependency — a volunteer project fits around a full-time job, and nothing in the network says a day off is a defect.

Two components do not sit on the critical path but were touched on far more days than their own duration would suggest, which is what Pressman calls a hammock activity: work stretched across the whole span rather than boxed at a fixed point in it. C12, Security and hardening, carries three days of effort and a float of twenty-seven, the widest of any component with commits behind it, yet it stayed open for 181 working days between its first commit on 12 February and its last on 23 October. C11, Configuration and white-label, carries four days of effort and a float of thirteen, and stayed open for 165 working days between 6 March and 22 October. Both are the kind of work a single maintainer keeps returning to as other components change under it — a new endpoint needs an authorisation check, a new setting needs a white-label default — rather than work with a start and a finish of its own. Drawing either as a short box at its early start would misrepresent it; the schedule that follows lists the float instead, which is where that difference actually shows.

11.3.1 CPM summary ordered by float

Table 11.1 lists all eighteen components ordered by float rather than by sequence, so the schedule reads by slack: the eight at float zero are the critical path above, and the rest are ordered by how much room the record shows they had.

Table 11.1
Component activity list, CPM summary ordered by float

Component	Effort (days)	ES	EF	LS	LF	Float	Predecessors	Tracker tasks	Done
C1 Persistence and migrations	11	0	11	0	11	0	—	23	21
C2 Authentication and access	4	11	15	11	15	0	C1	67	67
C3 Registry and membership	7	15	22	15	22	0	C1, C2	36	30
C5 Events and attendance	4	22	26	22	26	0	C3	6	6
C8 Gallery and albums	2	26	28	26	28	0	C5	29	27
C13 Web client	17	28	45	28	45	0	C3–C11	157	151
C15 Quality and testing	16	45	61	45	61	0	C12, C13	67	61
C17 Standards, docs and dissertation	15	61	76	61	76	0	C15, C16, C14	228	187
C4 Payments and finance	4	22	26	24	28	2	C3	9	9
C6 Governance and elections	3	22	25	25	28	3	C3	171	148
C10 Networking, forum and messaging	3	22	25	25	28	3	C3	25	25
C9 Careers and job board	2	22	24	26	28	4	C3	2	1
C7 Content and communication	4	15	19	24	28	9	C2	15	15
C16 Deployment and operations	5	45	50	56	61	11	C13	10	10
C11 Configuration and white-label	4	11	15	24	28	13	C1	74	63
C14 Mobile client	11	26	37	50	61	24	C3, C4, C5	41	37
C12 Security and hardening	3	15	18	42	45	27	C2	64	54
C18 Ad-hoc reporting	1	26	27	75	76	49	C3, C4	5	0

Source: `python docs/book/build/wbs.py`, run 31 October 2026, against `docs/TODO.md` and the git history as it stood that day. Effort is apportioned commit-days. ES/EF/LS/LF are in working days from the first commit. "Tracker tasks" counts the numbered items in the component's tracker areas, and "Done" the ones closed. C18 is planned work with no commit yet, so its effort is a stated one-day placeholder in the network, not a measurement.

Two things stand out reading it this way rather than in sequence. Float correlates with how self- contained a component is: C12, the widest among the built components, and C11, the widest after the mobile client, are both cross-cutting concerns that every other component depends on without depending on their timing — a security check or a white-label default can be added on any day and still be in place when the component that needs it ships. C18 has the widest float of all only because nothing has been built for it yet. And float does not correlate with the work raised: C12 has the widest float of any built component and 64 tracker tasks, more than all but six others, which is the hammock pattern of §11.3 stated as a number rather than an observation.

11.3.2 Crashing analysis

Classical crashing shortens a critical path by adding a resource to a task on it, trading cost for time. That lever does not exist here: the critical path runs through C1, C2, C3, C5, C8, C13, C15 and C17 in series, and every one of them had exactly one available resource, the same person, throughout the project. Assigning a second developer to compress C13 or C15 was never an option, so this section reports the honest result rather than a crashing table that would only formalise something that could not happen: nothing on the critical path was crashed, because nothing could be.

What did shorten the delivered schedule against what a from-scratch build would have taken is recorded elsewhere in this chapter and in §3.6, not here — reuse (§11.5) and scope deferral into the Won't-have set of the MoSCoW negotiation (Table 3.5) did the work that adding a resource would have done on a team with one to add. Deferring a requirement removes it from the network rather than compressing it, which is a different mechanism from crashing and is named as such.

11.4 Effort Estimation

Function points are counted here from the delivered system rather than from a specification written in advance, because no function-point count was made before the code existed. That makes this a size measurement of what was built, not an independent estimate of what it should have cost — a distinction §11.4.2 returns to.

`GHCAA.API/Controllers` carries 351 endpoint attributes across 47 controller files (`grep -rhoE '\[(HttpGet|HttpPost|HttpPut|HttpDelete|HttpPatch)' GHCAA.API/Controllers --include=*.cs`, 3 October 2026): 147 `HttpGet`, 148 `HttpPost`, 21 `HttpPut`, 29 `HttpDelete`, 6 `HttpPatch`. IFPUG counts external inputs (EI), outputs (EO) and inquiries (EQ) separately by whether a transaction changes stored data, derives new data or only retrieves it, which needs reading each endpoint's body to classify. That was not done here in this pass; instead every data-modifying verb (POST, PUT, PATCH, DELETE — 204 endpoints) is counted as an EI, and every GET (147 endpoints) as an EQ, treating none as a distinct EO. That is a simplification, stated as one: it undercounts wherever a GET performs a real calculation rather than a plain lookup, and it is the reason this section's function-point total is a lower bound on the standard method's own count, not the count itself.

`ApplicationDbContext.cs` declares 77 `DbSet<>` properties (`grep -c "public DbSet<" GHCAA.Infrastructure/Data/ApplicationDbContext.cs`), each an internal logical file (ILF). The system integrates seven external interfaces (EIF): four payment gateways (Bkash, DGePay, Nagad, SSLCommerz, confirmed via `GHCAA.Infrastructure/Gateways/`), one SMS gateway (`GreenwebSmsService.cs`), one email gateway (`GmailEmailService.cs`), and social sign-in, counted as one EIF for both providers it covers (`GoogleLoginAsync`, `FacebookLoginAsync` in `AuthService.cs`) rather than two, on the grounds that both are read through the one `SocialAuthConfigs` abstraction and neither adds a distinct interface contract of its own. Table 11.2 turns these four counts into an unadjusted function-point total.

Table 11.2
Unadjusted function-point count

Element type	Count	IFPUG average weight	Function points
External input (EI)	204	4	816
External inquiry (EQ)	147	4	588
Internal logical file (ILF)	77	10	770
External interface file (EIF)	7	7	49
Unadjusted function points (UFP)			**2,223**

No external output (EO) is counted, for the reason stated above. Every weight used is IFPUG's "average" complexity band rather than a low or high band assigned per element, because assigning complexity correctly needs the file and data-element counts (FTR, DET) that a full IFPUG count reads off each transaction and file — work this pass did not do. That is a second simplification on top of the first, and it pulls the total in the same direction: toward understating it.

The technical complexity factor is judged rather than measured — Pressman and Maxim's fourteen general system characteristics are properties of the whole system, not counts a grep can produce, so each is rated 0 (not present) to 5 (essential) by the author, against the delivered system as built. Table 11.3 lists all fourteen ratings and the total degree of influence they sum to.

Table 11.3
General system characteristics and technical complexity factor

#	Characteristic	Rating (0–5)	Basis
1	Data communications	4	Every client, web and mobile, talks to the API over HTTP only
2	Distributed data processing	2	One API process, one database; no distributed processing tier
3	Performance	3	No published SLA; ordinary interactive response expected throughout
4	Heavily used configuration	3	One hosting target (Render) with Postgres and SQLite both supported
5	Transaction rate	2	One association's membership, not a high-volume consumer system
6	Online data entry	5	Almost all data entry, member and admin, is through a form
7	End-user efficiency	4	Search, filters, breadcrumbs and a shared control set across screens
8	Online update	4	Most entities are updatable online, not batch-loaded
9	Complex processing	3	Election counting, payment reconciliation and access rules; not scientific computation
10	Reusability	4	A shared component and design-token library was built explicitly for reuse (§11.5)
11	Ease of installation	3	One documented deploy path; no installer or multi-environment rollout
12	Ease of operation	3	Administration console exists, but payment verification is still manual
13	Multiple sites	1	One deployed instance for one organisation; white-label config exists but is not live multi-tenant
14	Facilitate change	4	Organisation settings and provider adapters are configuration-driven
	Total degree of influence (TDI)	**45**	

VAF = 0.65 + 0.01 × TDI = 0.65 + 0.45 = 1.10. Adjusted function points, AFP = UFP × VAF = 2,223 × 1.10 ≈ 2,445.

Converting 2,445 AFP into an effort or cost figure needs a productivity factor — hours per function point for a comparable system and process — and this project has no independent source for one: it is a team of one working from a personal component library, which is not the population any published productivity table describes. *[A hours-per-function-point figure, cited to a source that applies to a single-developer, high-reuse process, is needed to carry this further; none has been found, so no effort or cost figure is derived from the count above]*. **COCOMO II is not attempted for the same reason, at one remove further out**: its five scale factors and seventeen effort multipliers are calibrated against team-based historical projects, and applying them to a process with one person and no comparable calibration data would produce a number with the appearance of rigour and none of its substance.

11.4.1 Two estimates compared

Table 11.1 carries two estimates of each component's size side by side: its effort in apportioned commit-days, and the number of tracker tasks raised against it. If tasks were a steady unit of work the two would rise together. They do not. Across the sixteen components with commits and more than a handful of tasks, tasks per effort-day run from under 2 to 57. Governance and elections (C6, 171 tasks on three days of effort), configuration (C11, 74 on four), authentication (C2, 67 on four),

standards and the dissertation (C17, 228 on fifteen) and security (C12, 64 on three) look heavy by task count, because the tracker holds many small items there. Persistence (C1, 23 tasks on eleven days), events (C5, 6 on four), deployment (C16, 10 on five), payments (C4, 9 on four) and the mobile client (C14, 41 on eleven) look heavy by effort, because a handful of items each took days. C9 (two tasks) is too small to read and C18 has no commit.

Task granularity varies by more than an order of magnitude between components, so any estimate built on task counts inherits that noise. That is the result of the comparison, and it is a stronger one than either estimate gives alone: neither column can be converted into the other with a single rate.

11.4.2 The estimate against the actual

The function-point count above (2,223 unadjusted, 2,445 adjusted) is a size figure: it describes how large the delivered system is by a standard method, counted from what actually exists. §11.1 and §11.5 report a different kind of figure: 200 days of effort the record can evidence, and 487 days the same work would represent built without this project's particular reuse. These are not directly comparable without the missing productivity factor named in §11.4, so no over-prediction ratio is computed here — one that could be, given only a placeholder rate, would be a fabricated number wearing a real one's shape, which is exactly what this chapter's sourcing rule exists to prevent.

What can be said without that missing rate is qualitative and still worth stating: a function-point count assumes a team-shaped cost structure — coordination overhead, a separate test or quality role, specification written before code — that this project did not have. Wherever that structure is assumed but absent, a size count converted through a team-calibrated productivity table would read as larger, in effort terms, than what one person actually spent building it. The direction of that gap is known; its size, honestly, is not, until a defensible productivity factor for this kind of project is found.

11.5 How the Implementation Time Was Optimised

A reader who reaches this point already has the scale: one unpaid maintainer, over eight months, a system of 351 endpoints, 76 entities, two clients and (§9's own count, cross-referenced there) several hundred automated tests. The honest answer to how that got built is not that the work was small — §11.4 puts its unadjusted size at 2,223 function points. It is that four things compounded: what the framework generated rather than a person writing it, what the project's own screens reused from each other, what the author's earlier projects supplied intact, and what tooling drafted for review rather than for authorship. This section prices each of the four instead of asserting that they mattered.

11.5.1 Sizing the delivered code

`wbs.py` counts the delivered source by kind, excluding generated migrations and the dissertation's own build scripts, and applies a production rate per kind rather than one blended rate across all of it, because a line of Dart and a line of stylesheet are not produced at the same speed: backend C# at 12 lines an hour, web TypeScript/templates/stylesheets at 25, Dart at 18, automated tests at 20. Table 11.4 gives the resulting hours by kind and their sum.

Table 11.4
Delivered size by kind, at a stated production rate

Kind	Lines	Rate (lines/hour)	Hours
Backend C# (excluding generated migrations)	38,820	12	3,235
Web client: TypeScript, templates, stylesheets	67,218	25	2,689
Mobile client: Dart	34,557	18	1,920
Automated tests	32,285	20	1,614
Total	**172,880**		**9,458**

Source: `python docs/book/build/wbs.py`, run 31 October 2026, against the working tree that day.

Nine thousand, four hundred and fifty-eight hours, at eight hours a working day, is 1,182 working days — what this codebase would cost built conventionally, one line at a time, with no reduction for anything reused, generated or assisted. That figure is the baseline the rest of this section reduces, not a claim about how the project actually ran.

11.5.2 The four reductions, each with its evidence

Four multiplicative reductions apply to the 1,182-day baseline, and they are stated in decreasing order of how directly each is evidenced rather than in the order they act during a build.

Reuse from the author's own earlier projects, ×0.87 — measured, not judged. `wbs.py` totals five carried-over modules against the tree: authentication, OTP, tokens and the middleware pipeline (2,041 lines); payment-gateway adapters (1,024); file handling, validation and storage (229); the front-end grid, shared controls, core services and layouts (22,814); and the design-token stylesheet (3,992) — 30,100 of 172,880 lines, 17.4% of the codebase. At roughly 78% of each module carried over intact from the author's 2024 projects rather than rewritten for this one, that is about 13.5% of the total build effectively already paid for, which is where the 0.87 factor comes from directly rather than by judgement.

Framework scaffolding and code generation, ×0.80. ASP.NET Core's Entity Framework tooling, model binding and dependency injection, and Angular's CLI-generated component and module boilerplate, produce working code from a declaration rather than from typed lines — this factor is judged, not measured line by line, but it is a conservative one against a codebase where the scaffolded proportion is visibly large: 57 service interfaces and their EF-backed implementations, 76 entities each with a generated migration, and every Angular component's routing and module wiring.

Reuse of shared components within the project, ×0.85. A screen built from an existing shared control (`app-page-header`, `app-search-bar`, `app-breadcrumb`, and the design-token system that backs all of them) costs less than one built from raw markup, because the control was priced once, in the line count above, and used again on dozens of screens without being priced again.

AI-assisted and rapid-development tooling, ×0.70. A draft generated by an assistant still needs review, correction and, where it changes behaviour, a test — none of that time is removed by this factor, which is why it is the smallest reduction of the four rather than the largest.

The four multiply rather than add, because they are not competing for the same effort: a screen built from an existing control, scaffolded by the framework and finished with assistant help is cheaper than any one of the three would make it alone, and the arithmetic has to say so.

11.5.3 The result and its sensitivity

0.80 × 0.85 × 0.87 × 0.70 ≈ 0.41. Applied to the 1,182-day baseline, that is 487 working days — what the delivered codebase represents as reuse-adjusted effort. Against the 200 days the commit and tracker record can actually evidence (§11.1), the ratio is 2.4×: for every day the record can prove, the delivered system represents a little over two days of conventionally-built equivalent. Both figures answer different questions and neither replaces the other — the 487-day figure is what the system represents as work if none of it had been reused or generated; the 200-day figure is a lower bound on what actually happened, because reading, debugging and design leave no commit to count.

The two factors that carry most of the 0.41 differ in kind, and the difference matters to how much the result should be trusted. Prior-project reuse (×0.87) is measured from the tree, line by line; the other three are judged. Of those, the tooling factor (×0.70) is the least certain, because "AI-assisted and rapid development tooling" was not logged as a separate activity at the time — it is a retrospective estimate of how much of the drafted code needed only review rather than composition. Moving that one factor from 0.70 to a less generous 0.85 would move the reuse-adjusted total from 487 to about 590 working days; the conclusion that reuse and generation, not raw effort, explain the delivered scope would still hold, but the specific multiplier should be read as the least firm number in this chain.

With 37 working days still outstanding (§11.6 and the "still to do" projection below), the completed project's reuse-adjusted total is about 524 working days, and its evidenced total about 237.

11.5.4 What this cost

Reuse buys speed and spends independence, and the largest single input behind the ×0.87 factor is a person, not a technique: the author's own 2024 codebases, which an association with no development budget cannot commission and cannot hire in the way a client with a budget could commission a contractor's reusable library. That dependency does not appear in a line count.

Three concrete costs sit behind the reduction figures rather than under them. The generated migration corpus is large enough on disk that the Render build has run out of memory during a deploy (§11.7's mitigation for that is operational, not architectural). The carried-over payment-gateway adapters brought a model shaped around a personal 2024 project's payment flows, not around GHCAA's own committee-approval workflow, which is part of why §6's gateway abstraction layer exists — to fit an inherited adapter to a process it was not written for. And assistant-drafted code is not free at the ×0.70 factor; it is priced at 70% of nominal specifically because the other 30% still has to be spent reading, correcting and, where behaviour changed, testing it. This section reports the trade, not only the saving it produced.

11.6 Progress Tracking and Earned Value

No weekly earned-value record was kept while this project ran, and this section does not pretend one was. BCWS, BCWP, BAC and ACWP are conventionally sampled on a calendar cadence against a resourced plan; this project had no resourced plan to sample against; §11.1 already establishes that most of the work did not exist as a plan before it was done.

What can be reconstructed, honestly labelled as a reconstruction rather than a contemporaneous record, is a single end-of-project snapshot rather than a time series. Taking the reuse-adjusted total for the finished project, 524 working days (§11.5.3), as a proxy for budget at completion (BAC), and the 237 evidenced working days at completion as a proxy for budgeted cost of work performed (BCWP) against that same total scope, gives a completion ratio of 237/524 ≈ 45%, as of 31 October 2026 — a little under half the reuse-adjusted work the finished project represents has commit or document evidence behind it so far, the rest being what §11.4 and the "still to do" projection below still owe.

SPI and CPI both need a time-phased baseline (planned value against a schedule, actual cost against a calendar) that this project did not record, so neither is computed. Stating a single-point completion ratio and declining to manufacture a schedule-performance index from data that cannot support one is the position this chapter takes throughout: a number not sourced from a real record is a placeholder, not an estimate dressed as one.

11.7 Team Structure and Responsibilities

One person held every role the four P's of §11.0 assign to separate people: developer, tester, reviewer, release manager and, for the elicitation of §4.6, the interviewer of the ten participants he also built the system for. That has one structural cost worth naming plainly: nobody reviewed a change before it shipped except the person who wrote it, and nobody but that same person decided whether a payment rule was correct and then wrote the test that checked it. §4.7's RMMM table already carries this as its first-listed risk, "single maintainer unavailable for an extended period", and §11.9 reports it with a risk-exposure score rather than only a description.

The mitigation actually used was mechanical rather than organisational, because there was no second person to mitigate the risk the ordinary way: `docs/TODO.md` recorded every change as a numbered, dated item before it counted as done, and the automated-test suite (§9) stood in for an independent reviewer by failing when a change broke something the tracker had already promised was fixed. Neither replaces a second pair of eyes on a design decision before it is made; §12.9 treats that gap as a validity threat to the project's own self-assessment, not only to the software, since a claim this report makes about its own numbers is subject to the same absence of independent review as the code.

11.8 Configuration and Change Management in Practice

There was no change-control board, no separate configuration-management tool, and no ticketing system distinct from the tracker. There was one tracker, split for length into two files on 15 September 2026: `docs/TODO.md` (open work packages, 2,320 lines) and `docs/TODO_ARCHIVE.md` (closed ones, 7,387 lines), 9,707 lines together as of 3 October 2026 (`wc -l docs/TODO.md docs/TODO_ARCHIVE.md`), simultaneously the project plan (the P1–P5 pre-development items and the planned work packages of §11.1), the change log (every reactively-arriving item, §11.1's 961 of 1,207 tasks), the defect log (the 130 tasks classified `defect` by `wbs.py`) and the decision record — a work package that changes direction mid-course states why in its own entry rather than in a separate minute.

That single file did the configuration-management job a formal board would have done in three respects a reader can check directly. Every change is numbered and traceable to the commits that closed it, because a work package's tasks stay open until the code and its test both exist. Every change is dated, which is what let `wbs.py` classify each one as planned or reactive in §11.1 without asking anyone to remember which it was. And nothing was silently dropped: an item that is no longer wanted is marked `ONHOLD` with a reason in place, rather than deleted, so the record of a decision not to do something survives as well as the record of what was done. What this arrangement could not do, because it needed a second person and never had one, is catch a decision that seemed sound to the one person who made it and reviewed it — the same gap §11.7 already names.

11.9 Risk Monitoring Record

Table 4.2 in §4.7 lists eight risks by category, probability, impact, mitigation, monitoring signal and outcome, in Pressman and Maxim's qualitative form. This section adds a numeric risk exposure to the same eight rows, RE = P × C, so they can be ranked rather than only read. Pressman's probability and impact bands are the qualitative Low/Medium/High of Table 4.2; converting each to a number needs a scale, and none was fixed at the time the risks were logged, so the mapping used here — Low = 1, Medium = 3, High = 5 on each axis — is an author-stated convention adopted now, for this table, not a measurement recovered from the project record. Table 11.5 applies it to all eight risks and ranks them by the resulting exposure.

Table 11.5
Risk exposure, computed from Table 4.2

Risk	P	C	RE = P × C	Outcome
Date format inconsistency between display and wire format	5 (High)	5 (High)	25	Closed and test-pinned, 2026-08-22
`EnsureCreated()` no-op leaving seeded data and schema stale	5 (High)	5 (High)	25	Closed twice, §6.5.6
Single maintainer unavailable for an extended period	3 (Medium)	5 (High)	15	Open; mitigated, not eliminated
Payment amount not verified against the originating record	3 (Medium)	5 (High)	15	Closed, Phase 2
Administrative action attributed to a hardcoded identifier	3 (Medium)	5 (High)	15	Closed, Phase 2
Event capacity exceeded under concurrent registration	3 (Medium)	3 (Medium)	9	Closed, Phase 1
Manual payment verification backlog exceeding officer capacity	3 (Medium)	3 (Medium)	9	Open; monitored, §12.4
Volunteer officer turnover losing institutional knowledge	3 (Medium)	3 (Medium)	9	Open; structural mitigation only

The two highest-exposure risks, both scored 25, are the same two §4.7 already names as the ones that went on to happen. That is not a coincidence the RE column manufactures — both were rated High on probability at the time, before either materialised, because each concerned a mechanism (a format crossing a client boundary, a startup no-op) the project had already seen behave unreliably elsewhere. The numeric score adds a ranking on top of a judgement §4.7 had already made correctly.

Risk information sheet — single maintainer unavailable for an extended period. Risk ID: matches Table 4.2, row 1. Category: project. Probability: Medium. Impact: High. RE: 15. Context: every role in §11.0's four P's is held by one person, and §11.7 states the resulting gap plainly — no independent review exists for any decision this project made. Mitigation in place: a documentation corpus (`PROJECT_MAP.md`, `docs/TODO.md`, `ARCHITECTURE.md`, and this dissertation itself) written so that a successor could orient without the author present, rather than a second developer, which the project's resourcing does not support. Monitoring signal: elapsed time since the last commit; a gap large enough to be visible in `git log` would itself be the indicator this risk had materialised. Current status: open. This is the one risk in the table with no closing action available — it is a property of the project's staffing, not a defect a commit can fix, and it is reported as open for that reason rather than left off the record because it cannot be closed.

11.10 Quality Assurance Activities Performed

Pressman's 40-20-40 allocation splits effort roughly evenly between up-front analysis and design, construction, and validation. This project's measured effort (§11.5.1's hour breakdown) does not match it: of the 8,413 nominal hours the delivered code represents, 1,340 are attributed to automated tests — about 16% of the total, well under the 40% the model prescribes for validation, with the remaining share split across backend, web and mobile construction and no separately-tracked analysis and design phase at all, because §4.4 already establishes that this project ran incrementally rather than in Pressman's staged form.

That gap is reported rather than explained away. A single maintainer with no separate QA role has to choose where review time goes, and the record shows it went disproportionately into fixing what testing found rather than into writing more of it: §11.1 shows 129 tasks classified as defects and 300 as review findings, both arriving after code existed to find something wrong with, which is a retrospective form of quality assurance rather than the 40% up-front and 40% back-end split Pressman's model assumes a resourced team can afford.

Defect removal efficiency, DRE = E / (E + D), can be computed from two evidenced counts rather than estimated. E, defects and issues found before delivery: the 5 specification defects of Table 3.8 (§4.7's U3) plus the 15 tracker items raised by the September 2026 architecture and engineering audit (82.14–82.28), 20 in total. D, defects found in production after delivery: the 2 confirmed, dated deployment incidents of §4.7's U5 (the `MigrationBootstrapper` legacy-database incident of 27 August 2026, and the stale-chunk-caching and `AuthService` NG0200 incidents of 28 August 2026, counted as one dated incident each per the corrected figure in §4.7). DRE = 20 / (20 + 2) ≈ 0.91.

Ninety-one per cent is a plausible figure for a system this size, but it rests on a review process and a production-incident record that were both run by the one person who also wrote the code — the same independent-review gap §11.7 names, applied here to the number that is supposed to measure how well defects were caught. Mean time to correction, MTTC, is what QAS-06 already measures (§3, quality attribute scenarios); its value is not restated here because §9.12, where it is measured against the delivered test suite, has not yet been written — this section names what QAS-06 measures rather than quoting a figure that does not yet exist.

11.11 Lessons in Project Management

Three things this project would do differently, stated plainly rather than as a list of virtues.

First, the tracker should have separated the plan from the change log from the start, even while both lived in the same file. §11.1 and §11.8 show that most of the tracker's 1,207 tasks arrived reactively; nothing about the file's structure distinguishes a planned work package from one raised by a defect except the prose in its own heading, which is why `wbs.py` has to classify each one by regex against wording rather than by a field that was there to record it at the time.

Second, a single maintainer needs the mechanical checks §11.7 describes — the tracker and the test suite — more than a resourced team does, not less, because they are the only substitute available for a second person's review. Where those checks were thin (before Chapter 9's test suite reached its current size, and before the seeding and migration bugs of §4.7's RMMM table were caught), the record shows the cost directly: two production incidents, both from mechanisms nothing was checking.

Third, effort estimation for a project like this one should be stated in the terms §11.4 and §11.5 actually use — a measured baseline size, named reduction factors with their evidence class stated, and an honest gap where a figure cannot be sourced — rather than in a single number presented as if it came from a method built for a resourced team. The function-point count of §11.4 is useful as a size measurement precisely because it does not claim to be an effort prediction it cannot support.

11.12 Summary

This chapter counted the same project five different ways and did not let the counts agree by force. The critical path is 76 working days; 104 are evidenced by a commit; 258 elapsed on the calendar between the first and the last. The delivered codebase, priced at conventional rates with no reduction, represents 1,182 working days; four reductions, three of them judged and one measured, bring that to 487; the record can evidence 200. Eighty per cent of the tracker's 1,207 tasks arrived after the work they belong to had already started. A function-point count of the delivered system, simplified and stated as such, comes to 2,223 unadjusted points, and could not be converted to an effort or cost figure without a productivity rate this project has no defensible source for — recorded here as an open gap rather than filled with an invented one. Defect removal efficiency, from an evidenced 20 pre-delivery findings against 2 post-delivery incidents, is about 0.91, computed by a process with no independent reviewer to check it. None of these figures individually proves the project was run well or badly; together, sourced and dated, they are what the plan-against-outcome question this chapter set out to answer actually looks like for one person building this system in a little over eight months.

Figures and Tables

Tables 11.1 through 11.5 are made and appear above, each generated from a command run against the tree or drawn directly from `docs/book/build/wbs.py`'s output, cited where it stands. The chapter's Mermaid diagrams — the seventeen per-component activity diagrams for the components with commits and the chapter-level dependency, WBS, arrival-profile, float-distribution, estimate-comparison and reduction-waterfall charts the outline lists — are TODO item 67.3's scope, not this one's, and are not drawn here. `renumber.py --apply` is run after this section settles, since table numbering was added in this pass.

PART IV — EVALUATION AND CLOSURE

Chapter 12 — Results, Evaluation and Discussion

> *[Chapter not written. The headings below are generated from `docs/DOCUMENTATION_BOOK_OUTLINE.md` and are kept in step with it: `build.py --strict` fails if a section exists in one and not the other.]*

What this chapter owns. What the evidence showed, and what it means. Every result in the dissertation is reported here and only here.

What it must not repeat. How anything was tested or measured, which is Chapter 9, and what was promised, which is §4.5. This chapter is read against Table 4.3 row by row, including the three commitments §4.5 records as reduced or not run. §12.6 answers the four research questions directly; RQ3's answer applies the five-class scheme of §3.7 to the delivered system.

12.1 Overview of the Delivered Artefact

> *[Not written.]*

12.2 Functional Evaluation

> *[Not written. Brief: requirement coverage and the closed traceability matrix]*

12.3 Quality Evaluation against ISO/IEC 25010

> *[Not written. Brief: characteristic by characteristic, with the evidence and the metric value for each]*

12.4 Performance, Security and Usability Results

> *[Not written.]*

> *[Not written.]*

> *[Not written.]*

12.5 Stakeholder and Expert Evaluation

> *[Not written.]*

12.6 Answering the Research Questions

> *[Not written. Brief: RQ1 through RQ4 answered explicitly, each with the evidence supporting the answer and the confidence that evidence warrants]*

12.7 Discussion

> *[Not written. Brief: interpretation, and comparison against the literature of Chapter 2: where this work agrees with prior findings and where it diverges]*

12.8 Comparison against the Existing Manual System

> *[Not written.]*

12.9 Threats to Validity

> *[Not written. Brief: the threats to the Chapter 9 test evaluation as well as the results; construct, internal, external and conclusion validity, each with the mitigation applied and the residual limitation acknowledged]*

12.10 Limitations of the Artefact

> *[Not written.]*

12.11 Reflection on the Design Science Contribution

> *[Not written. Brief: what transfers beyond this institution]*

12.12 Summary

> *[Not written.]*

Figures and Tables

> *[Not drawn. The outline specifies 9 artefacts for this chapter. Each is added here with its caption as it is made, then* `renumber.py --apply` *is run.]*

Chapter 13 — Conclusion and Future Work

> *[Chapter not written. The headings below are generated from `docs/DOCUMENTATION_BOOK_OUTLINE.md` and are kept in step with it: `build.py --strict` fails if a section exists in one and not the other.]*

What this chapter owns. What was done, what was learned, what it is worth and what should happen next.

What it must not repeat. No new evidence appears here. Anything that would need to be defended belongs in the chapter that carries its evidence, and is cited from here rather than restated.

13.1 Summary of the Work and its Contributions

The research questions are answered in §12.6, Answering the Research Questions, and the answers are not repeated here.

> *[Not written.]*

> *[Not written.]*

13.2 Practical Implications for Similar Institutions

> *[Not written.]*

13.3 Lessons Learned

> *[Not written. Brief: technical, methodological and organisational]*

13.4 Future Work

> *[Not written.]*

13.4.1 Near-term functional roadmap

> *[Not written.]*

13.4.2 Technical debt and reengineering priorities

> *[Not written.]*

13.4.3 Research directions opened by this work

> *[Not written.]*

13.5 Concluding Remarks

> *[Not written.]*

Figures and Tables

> *[Not drawn. The outline specifies 2 artefacts for this chapter. Each is added here with its caption as it is made, then `renumber.py --apply` is run.]*

References

Citations follow the IEEE numeric style. Numbers are assigned once for the whole book in order of first appearance, so that a marker means the same thing in every chapter; the list below is therefore the running bibliography and includes entries first cited after Part I. Standards are cited by designation and year of the edition consulted. Where an online source is cited, the access date is given.

Online sources carry the date they were accessed. Where a standard cited here has since been revised, the entry names the superseding edition and §3.4 or §4.5 states which edition the work was carried out against; no conformance claim is transferred across editions.

[1] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.

[2] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[3] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[4] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[5] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[6] *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*, ISO/IEC 25010:2011. Superseded by ISO/IEC 25010:2023 [74]; the 2011 edition is the one this dissertation's requirement classification was built against, as §3.4 states.

[7] OWASP Foundation, *OWASP Application Security Verification Standard*, version 4.0.3, 2021. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/. Accessed: 1 Sep. 2026. This is the edition the security work was carried out against; superseded by version 5.0.0 [75].

[8] J. Monks, "Patterns of giving to one's alma mater among young graduates from selective institutions," *Economics of Education Review*, vol. 22, no. 2, pp. 121–130, 2003.

[9] D. J. Weerts and J. M. Ronca, "Profiles of supportive alumni: Donors, volunteers, and those who do it all," *International Journal of Educational Advancement*, vol. 7, no. 1, pp. 20–34, 2007.

[10] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.

[11] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Keele Univ. and Univ. of Durham, EBSE Tech. Rep. EBSE-2007-01, 2007.

[12] M. J. Page *et al.*, "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," *BMJ*, vol. 372, art. n71, 2021.

[13] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA, USA: Addison-Wesley, 2021.

[14] A. Cockburn, "Hexagonal architecture," 2005. [Online]. Available: https://alistair.cockburn.us/hexagonal-architecture/. Accessed: 1 Sep. 2026.

[15] J. Palermo, "The onion architecture," 2008. [Online]. Available: https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/. Accessed: 1 Sep. 2026.

[16] M. Fowler, "MonolithFirst," 2015. [Online]. Available: https://martinfowler.com/bliki/MonolithFirst.html. Accessed: 1 Sep. 2026.

[17] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[18] N. Dragoni *et al.*, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, Cham, Switzerland: Springer, 2017, pp. 195–216.

[19] *Systems and Software Engineering — Architecture Description*, ISO/IEC/IEEE 42010:2011.

[20] *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*, IEEE Std 1016-2009.

[21] *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE 29148:2018.

[22] *Software and Systems Engineering — Software Testing*, ISO/IEC/IEEE 29119 (Parts 1–4), 2013–2015.

[23] R. B. Grady, *Practical Software Metrics for Project Management and Process Improvement*. Englewood Cliffs, NJ, USA: Prentice Hall, 1992.

[24] *Web Content Accessibility Guidelines (WCAG) 2.1*, W3C Recommendation, 5 June 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[25] OWASP Foundation, "OWASP Top Ten 2021." [Online]. Available: https://owasp.org/Top10/. Accessed: 1 Sep. 2026. Superseded by the 2025 edition [76].

[26] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis, IN, USA: Wiley, 2014.

[27] M. Bernhard *et al.*, "Public evidence from secret ballots," in *Electronic Voting (E-Vote-ID 2017)*, Cham, Switzerland: Springer, 2017, pp. 84–109.

[28] D. Springall *et al.*, "Security analysis of the Estonian internet voting system," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 703–715.

[29] J. A. Halderman and V. Teague, "The New South Wales iVote system: Security failures and verification flaws in a live online election," in *E-Voting and Identity (VoteID 2015)*, Cham, Switzerland: Springer, 2015, pp. 35–53.

[30] S. Park, M. Specter, N. Narula, and R. L. Rivest, "Going from bad to worse: From internet voting to blockchain voting," *Journal of Cybersecurity*, vol. 7, no. 1, art. tyaa025, 2021.

[31] B. W. Boehm *et al.*, *Software Cost Estimation with COCOMO II*. Upper Saddle River, NJ, USA: Prentice Hall, 2000.

[32] C. Wohlin *et al.*, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2012.

[33] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, P. W. Jordan *et al.*, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

[34] R. Kazman, M. Klein, and P. Clements, "ATAM: Method for architecture evaluation," Software Engineering Institute, Carnegie Mellon Univ., Tech. Rep. CMU/SEI-2000-TR-004, 2000.

[35] J. Nielsen, *Usability Engineering*. San Francisco, CA, USA: Morgan Kaufmann, 1993.

[36] E. Marcotte, *Responsive Web Design*, 2nd ed. New York, NY, USA: A Book Apart, 2014.

[37] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.

[38] S. Krug, *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability*, 3rd ed. Berkeley, CA, USA: New Riders, 2014.

[39] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 6th ed. Newtown Square, PA, USA: PMI, 2017.

[40] *IEEE Standard for Software Quality Assurance Processes*, IEEE Std 730-2014.

[41] A. Cavoukian, "Privacy by design: The 7 foundational principles," Information and Privacy Commissioner of Ontario, 2009, revised 2011.

[42] *IEEE Standard for Configuration Management in Systems and Software Engineering*, IEEE Std 828-2012.

[43] E. M. Rogers, *Diffusion of Innovations*, 5th ed. New York, NY, USA: Free Press, 2003.

[44] A. Cockburn, *Writing Effective Use Cases*. Boston, MA, USA: Addison-Wesley, 2001.

[45] M. Cohn, *User Stories Applied: For Agile Software Development*. Boston, MA, USA: Addison-Wesley, 2004.

[46] D. Clegg and R. Barker, *Case Method Fast-Track: A RAD Approach*. Wokingham, U.K.: Addison-Wesley, 1994.

[47] O. C. Z. Gotel and A. C. W. Finkelstein, "An analysis of the requirements traceability problem," in *Proc. 1st IEEE Int. Conf. on Requirements Engineering*, 1994, pp. 94–101.

[48] M. E. Fagan, "Design and code inspections to reduce errors in program development," *IBM Systems Journal*, vol. 15, no. 3, pp. 182–211, 1976.

[49] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conf.*, 1999, pp. 81–91.

[50] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[51] D. Hardt, Ed., "The OAuth 2.0 authorization framework," IETF RFC 6749, Oct. 2012.

[52] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-based one-time password algorithm," IETF RFC 6238, May 2011.

[53] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[54] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill, 2015.

[55] M. M. Lehman, "Programs, life cycles, and laws of software evolution," *Proceedings of the IEEE*, vol. 68, no. 9, pp. 1060–1076, 1980.

[56] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," *IEEE Transactions on Software Engineering*, vol. 20, no. 6, pp. 476–493, 1994.

[57] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.

[58] P. Oman and J. Hagemeister, "Metrics for assessing a software system's maintainability," in *Proc. Conf. on Software Maintenance*, 1992, pp. 337–344.

[59] K. Beck, *Test-Driven Development: By Example*. Boston, MA, USA: Addison-Wesley, 2003.

[60] G. Meszaros, *xUnit Test Patterns: Refactoring Test Code*. Boston, MA, USA: Addison-Wesley, 2007.

[61] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA, USA: Addison-Wesley, 2010.

[62] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.

[63] G. Kim, J. Humble, P. Debois, and J. Willis, *The DevOps Handbook*. Portland, OR, USA: IT Revolution Press, 2016.

[64] E. Ostrom, *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge, U.K.: Cambridge Univ. Press, 1990.

[65] A. Fung, "Varieties of participation in complex governance," *Public Administration Review*, vol. 66, no. s1, pp. 66–75, 2006.

[66] T. R. Tyler, "Procedural justice, legitimacy, and the effective rule of law," *Crime and Justice*, vol. 30, pp. 283–357, 2003.

[67] P. Dunleavy, H. Margetts, S. Bastow, and J. Tinkler, "New public management is dead — long live digital-era governance," *Journal of Public Administration Research and Theory*, vol. 16, no. 3, pp. 467–494, 2006.

[68] D. A. Norman, *The Design of Everyday Things*, revised and expanded ed. New York, NY, USA: Basic Books, 2013.

[69] Hivebrite, "Hivebrite plans." [Online]. Available: https://hivebrite.io/pricing/. Accessed: 1 Sep. 2026.

[70] Almabase, "Pricing." Quoted rather than published; no plan tiers or figures are given on the vendor's own pages. [Online]. Available: https://www.almabase.com/. Accessed: 1 Sep. 2026.

[71] Zoho Corporation, "Zoho CRM pricing." [Online]. Available: https://www.zoho.com/crm/pricing/. Accessed: 1 Sep. 2026.

[72] Salesforce, "Power of Us program." [Online]. Available: https://www.salesforce.com/company/power-of-us/. Accessed: 1 Sep. 2026.

[73] Paid Memberships Pro, "Pricing and plans." [Online]. Available: https://www.paidmembershipspro.com/pricing/. Accessed: 1 Sep. 2026.

[74] *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*, ISO/IEC 25010:2023.

[75] OWASP Foundation, *OWASP Application Security Verification Standard*, version 5.0.0, 2025. [Online]. Available: https://asvs.dev/. Accessed: 1 Sep. 2026.

[76] OWASP Foundation, "OWASP Top 10:2025." [Online]. Available: https://owasp.org/Top10/2025/. Accessed: 1 Sep. 2026.

[77] GitHub, "Spec Kit," version 1.0.9.dev0. [Online]. Available: https://github.com/github/spec-kit.

Governing documents of the Association

These are primary sources rather than literature, and are cited in the text by article and section rather than by reference number.

- Constitution of the Govt. Haraganga College Alumni Association, version 4.2, ratified and effective 1 July 2026, Articles I–XII.
- Election Regulations of the Association.
- Election Operational Manual.
- Code of Conduct for Candidates and Members.
- Manual Balloting Procedure.
- Election Forms Set.
- Ballot Box Sealing and Poll Integrity Certificate.
- Vote Counting Authorisation.

All are held in `docs/Elections/` and in the constitution store of the platform described in §5.5.